

计算机图形学

第二章：光栅图形学算法

随着光栅显示器的出现，为了在计算机上处理、显示图形，需要发展一套与之相适应的算法：

光栅图形学算法

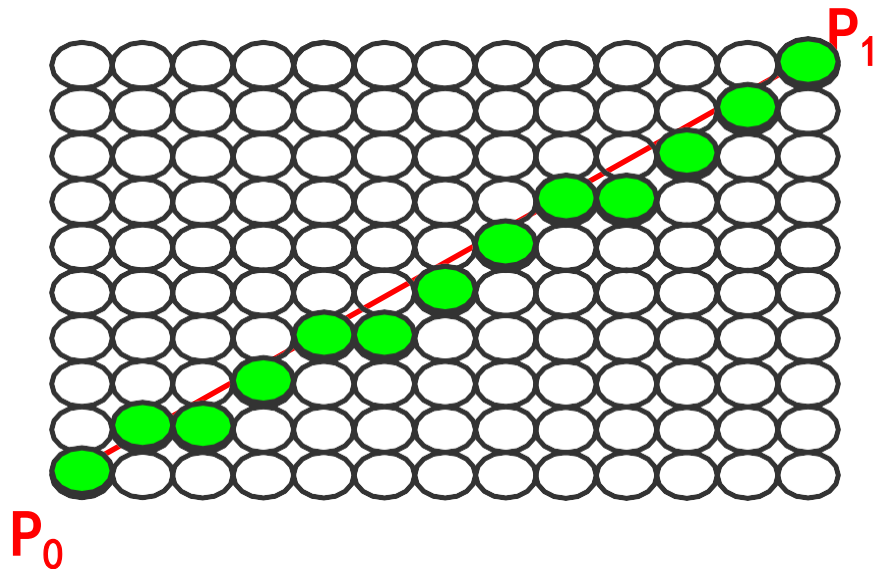
光栅图形算法多数属于计算机图形的底层算法，很多图形学的基本概念和思想都在这一章

光栅图形学算法的研究内容

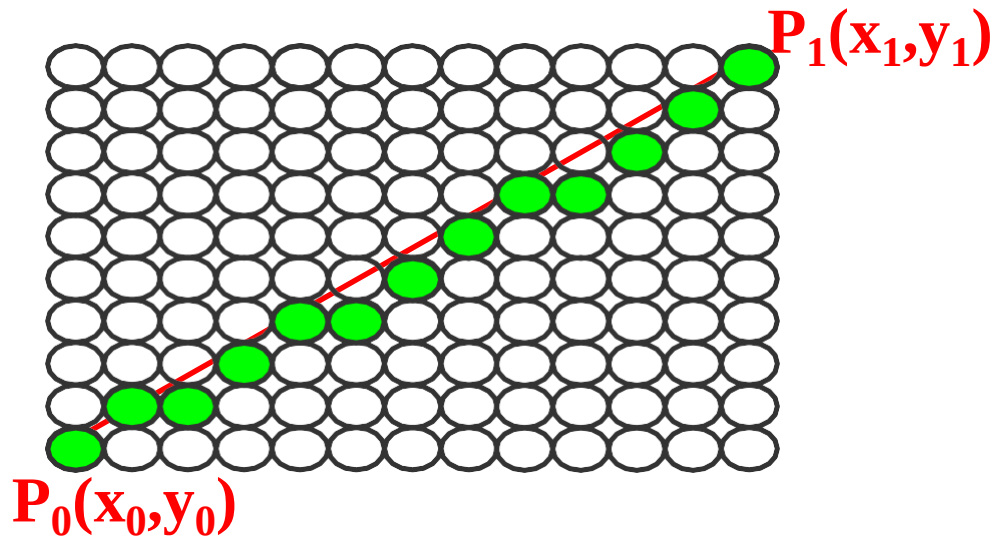
- 直线段的扫描转换算法
- 多边形的扫描转换与区域填充算法
- 裁剪算法
- 反走样算法
- 消隐算法

一、直线段的扫描转换算法

在数学上，直线上的点有无穷多个。但当在计算机光栅显示器屏幕上表示这条直线时需要做一些处理。



为了在光栅显示器上用这些离散的像素点逼近这条直线，需要知道这些像素点的 x , y 坐标。



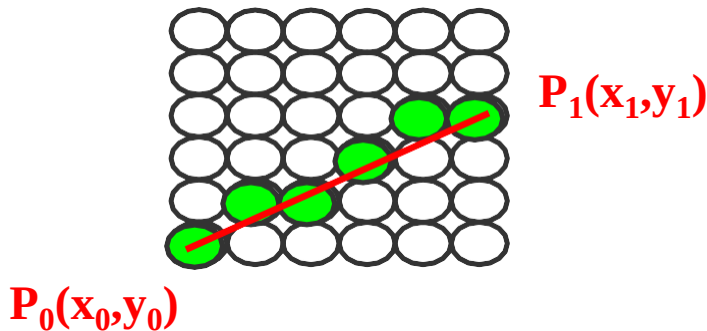
求出过 P_0 , P_1 的直线段方程：

$$y = kx + b$$

$$k = \frac{(y_1 - y_0)}{(x_1 - x_0)} \quad (x_1 \neq x_0)$$

$$y = kx + b \quad k = \frac{(y_1 - y_0)}{(x_1 - x_0)} \quad (x_1 \neq x_0)$$

假设x已知，即从x的起点 x_0 开始，沿x方向前进一个像素（步长= 1），可以计算出相应的y值。



因为像素的坐标是整数，所以y值还要进行取整处理

如何把数学上的一个点扫描转换一个屏幕像素点？

如： $p(1.7, 0.8)$ $\xrightarrow{\text{取整}}$ $p(1, 0)$

$p(1.7, 0.8)$ $\xrightarrow{+0.5}$ $p(2.2, 1.3)$

$p(2.2, 1.3)$ $\xrightarrow{\text{取整}}$ $p(2, 1)$

$$y = kx + b$$

直线是最基本的图形，一个动画或真实感图形往往需要调用成千上万次画线程序，因此直线算法的好坏与效率将直接影响图形的质量和显示速度。

回顾一下刚才的算法：

$$\underline{y = kx + b}$$

为了提高效率，把计算量减下来，关键问题就是如何把**乘法**取消？

二、直线绘制的三个著名的常用算法

1、数值微分法 (DDA)

2、中点画线法

3、Bresenham算法

1、数值微分DDA(Digital Differential Analyzer)法

引进图形学中一个很重要的思想——增量思想

$$y_i = kx_i + b$$

$$y_{i+1} = kx_{i+1} + b$$

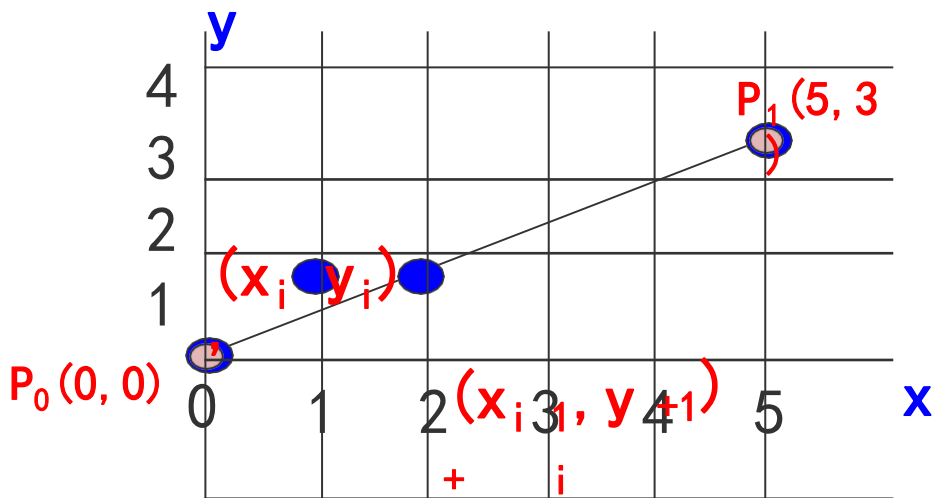
$$= k(x_i + 1) + b$$

$$= kx_i + k + b$$

$$= kx_i + b + k$$

$$= y_i + k$$

$$\underline{y_{i+1} = y_i + k}$$



$$\underline{y_{i+1} = y_i + k}$$

增量

这个式子的含义是：当前步的y值等于前一步的y值加上斜率k

这样就把原来一个乘法和加法变成了一个加法！

用DDA扫描转换连接两点 $P_0(0, 0)$ 和 $P_1(5, 3)$ 的直线段。

$$k = \frac{y_1 - y_0}{x_1 - x_0} = \frac{3 - 0}{5 - 0} = 0.6 < 1 \quad y_{i+1} = y_i + k$$

x	y	int(y+0.5)
---	---	------------

0	0	0
---	---	---

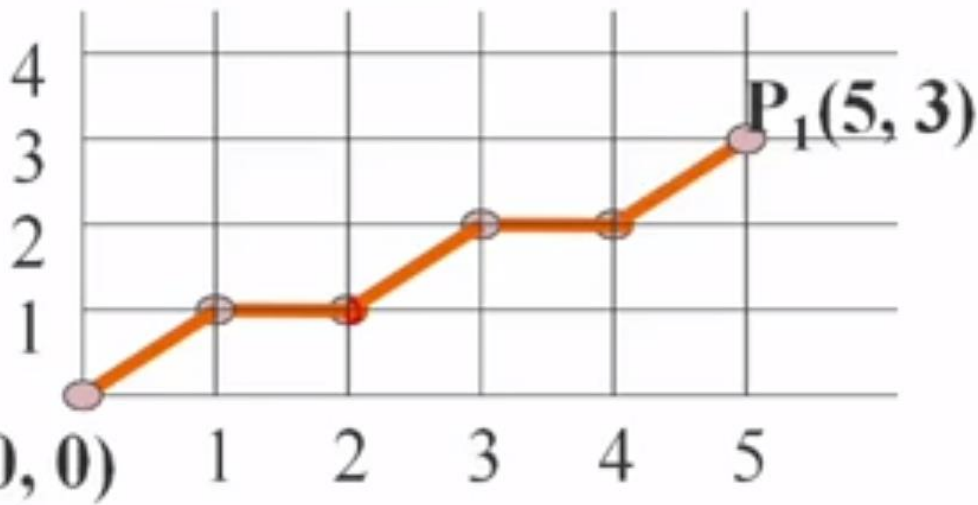
1	0+0.6	1
---	-------	---

2	0.6+0.6	1
---	---------	---

3	1.2+0.6	2
---	---------	---

4	1.8+0.6	2
---	---------	---

5	2.4+0.6	3
---	---------	---



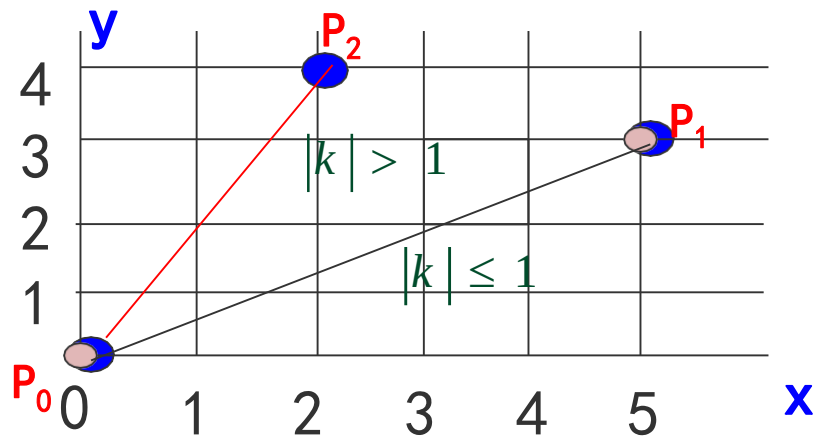
给大家留二个思考题

(1) DDA画直线算法：x每递增1，y递增斜率k。是否适合任意斜率的直线？

$$|k| \leq 1$$

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i + k$$

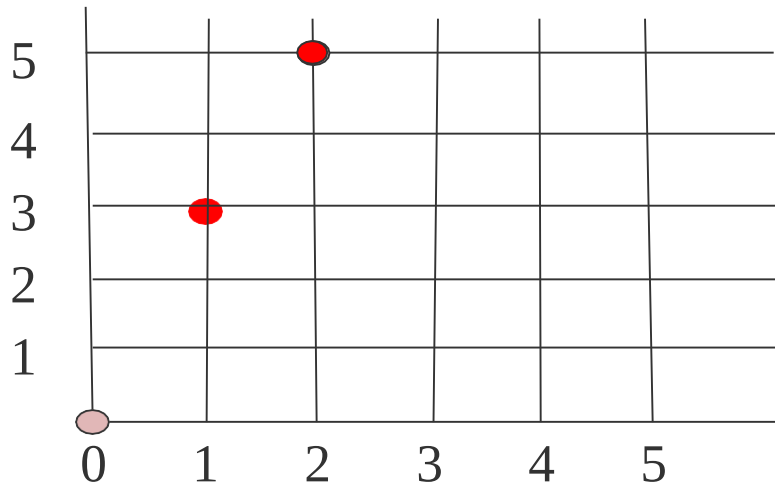


$$|k| > 1$$

用DDA方法画连接两点 $P_0(0, 0)$ 和 $P_1(2, 5)$ 的直线段

$$K=5/2=2.5 > 1 \quad y_{i+1} = y_i + k$$

x	y	int(y+0.5)
0	0	0
1	2.5	3
2	5	5



再比如直线点从 $(0, 0)$ 到 $(2, 100)$, 也只用3个点来表示

$$(1) \quad |k| > 1$$

$$x_{i+1} = x_i + ?$$

$$y_{i+1} = y_i + ?$$

(2) DDA画直线算法是否最优呢？若非，如何改进？

采用增量思想的DDA算法，直观、易实现，每计算一个像素坐标，只需计算一个加法。

$$\underline{y_{i+1} = y_i + k}$$

这个算法是否最优呢？若非最优，如何改进？

(1) 改进效率。这个算法每步只做一个加法，能否再提高效率？

$$\underline{y_{i+1} = y_i + k}$$

一般情况下k与y都是小数，而且每一步运算都要对y进行四舍五入后取整。

唯一改进的途径是把浮点运算变成整数加法！

(2) 第二个思路是从直线方程类型做文章

$$y = kx + b$$

而直线的方程有许多类型，如**两点式**、**一般式**等。
如用其它的直线方程来表示这条直线会不会有出人意料的效果？

直线绘制的三个著名的常用算法

1、数值微分法 (DDA)

2、中点画线法

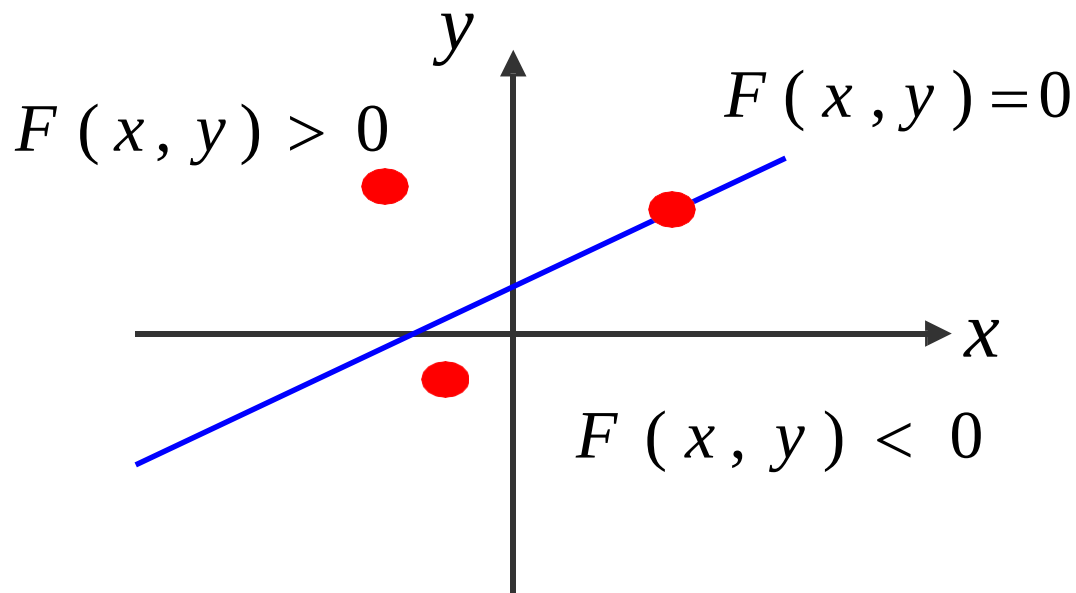
3、Bresenham算法

中点画线法

直线的一般式方程：

$$F(x, y) = 0$$

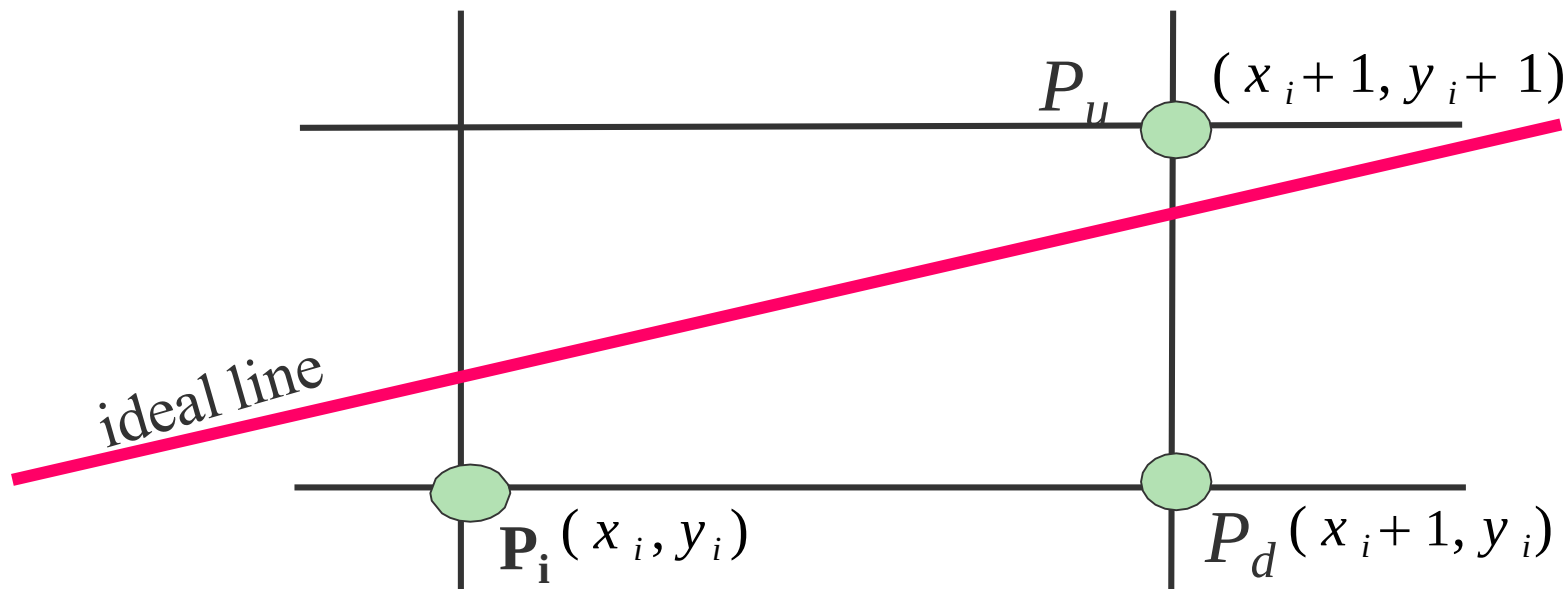
$$Ax + By + C = 0$$

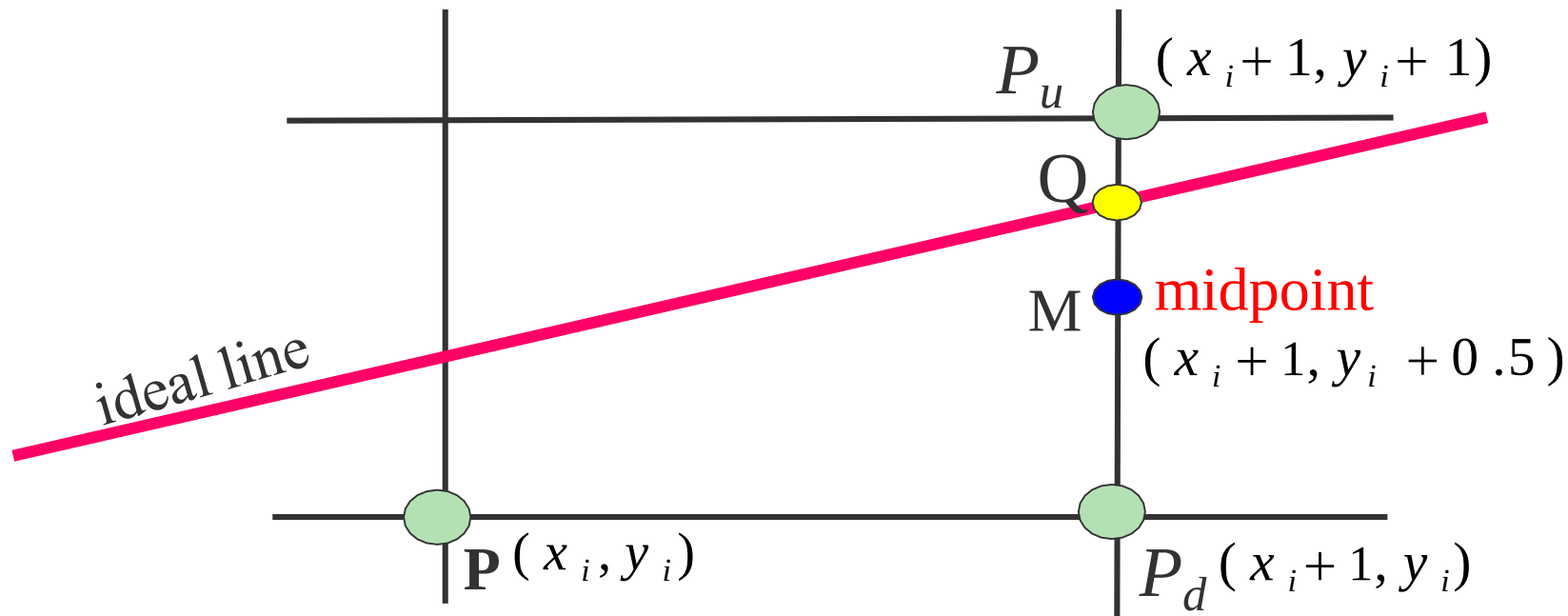


- 对于直线上方的点： $F(x, y) = 0$
- 对于直线上方的点： $F(x, y) > 0$
- 对于直线下方的点： $F(x, y) < 0$

每次在最大位移方向上走一步，而另一个方向是走步还是不走步要取决于中点误差项的判断。

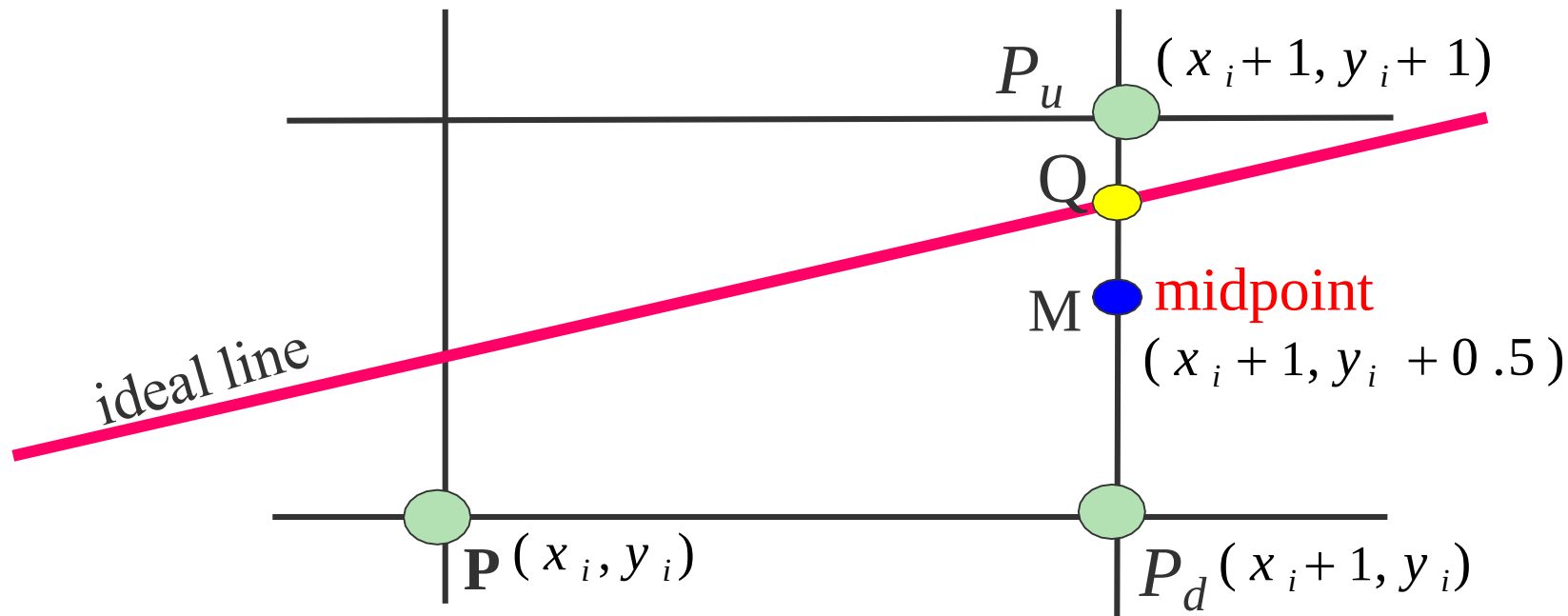
假定： $0 \leq |k| \leq 1$ 。因此，每次在x方向上加1，y方向上加1或不变需要判断。



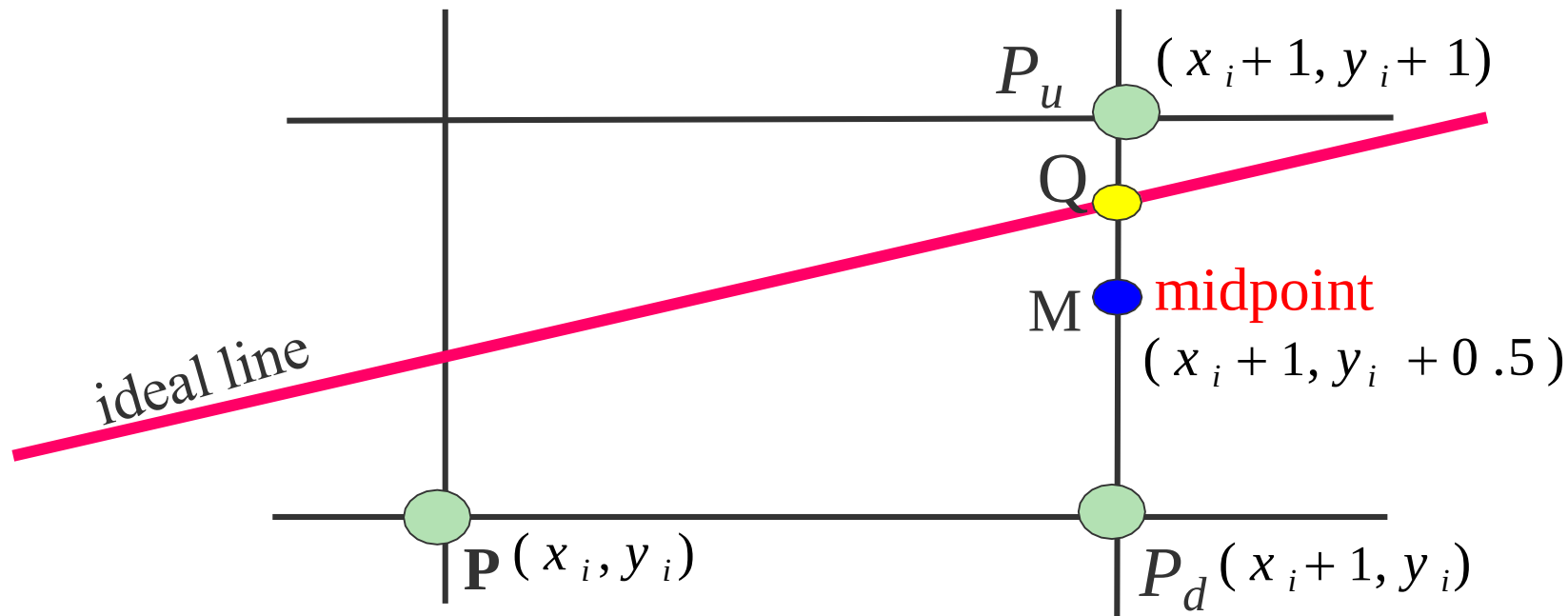


当M在Q的下方，则 P_u 离直线近，应为下一个像素点

当M在Q的上方，应取 P_d 为下一点。



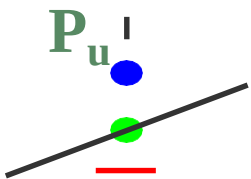
如何判断Q在M的上方还是下方？

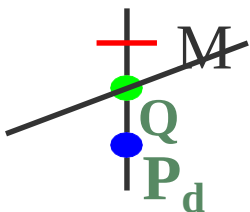


把M代入理想直线方程：

$$F(x_m, y_m) = Ax_m + By_m + C$$

$$d_i = F(x_m, y_m) = F(x_i + 1, y_i + 0.5) \\ = A(x_i + 1) + B(y_i + 0.5) + C$$

当 $d < 0$ 时:  M在Q下方, 应取 P_u

当 $d > 0$ 时:  M在Q上方, 应取 P_d

当 $d = 0$ 时: M在直线上, 选 p_d 或 p_u 均可。

$$y = \begin{cases} y + 1 & (d < 0) \\ y & (d \geq 0) \end{cases}$$

这就是中点画线法的基本原理

下面来分析一下中点画线算法的计算量？

$$y = \begin{cases} y + 1 & (d < 0) \\ y & (d \geq 0) \end{cases}$$

$$d_i = A(x_i + 1) + B(y_i + 0.5) + C$$

为了求出d值，需要两个乘法，四个加法

能否也采用增量计算，提高运算效率呢？

$$d_{i+1} = d_i + \text{?}$$

增量

分析一下中点画线算法的计算量

$$y = \begin{cases} y + 1 & (d < 0) \\ y & (d \geq 0) \end{cases}$$

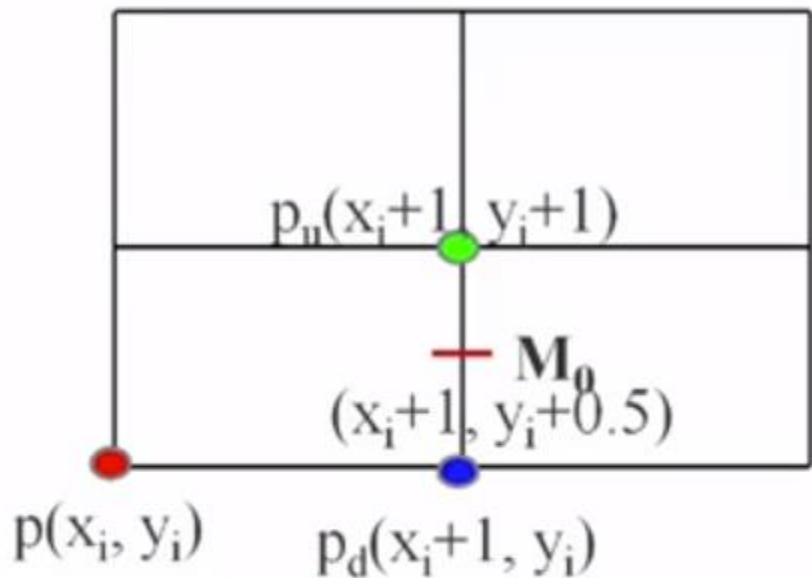
$$d_i = A(x_i + 1) + B(y_i + 0.5) + C$$

能否也采用增量计算，提高运算效率呢？

$$d_{i+1} = d_i + \text{增量}$$

d 是 x, y 的线性函数，采用增量计算是可行的

如何推导出d值的递推公式？



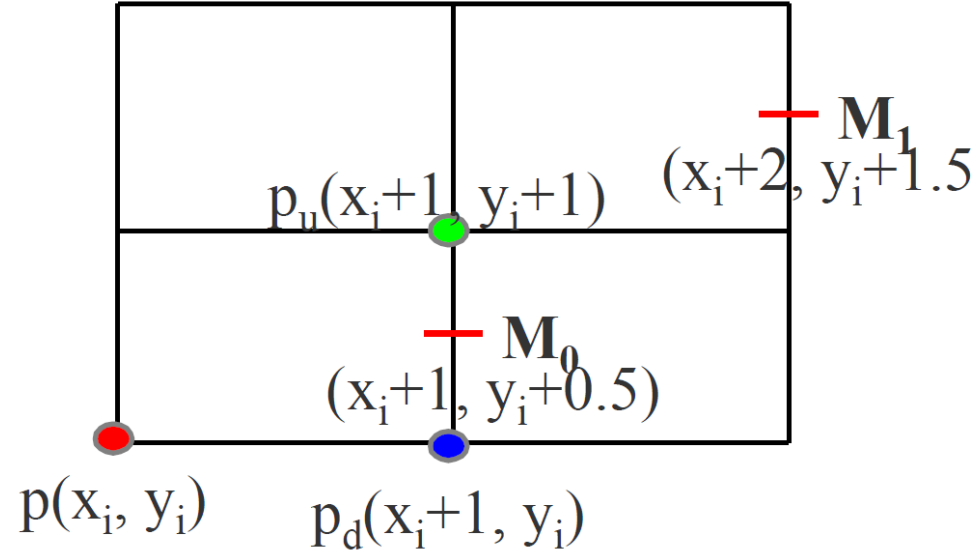
$$y = \begin{cases} y + 1 & (d < 0) \\ y & (d \geq 0) \end{cases}$$

$$d_0 = F(x_{m0}, y_{m0})$$

$$= F(x_i + 1, y_i + 0.5)$$

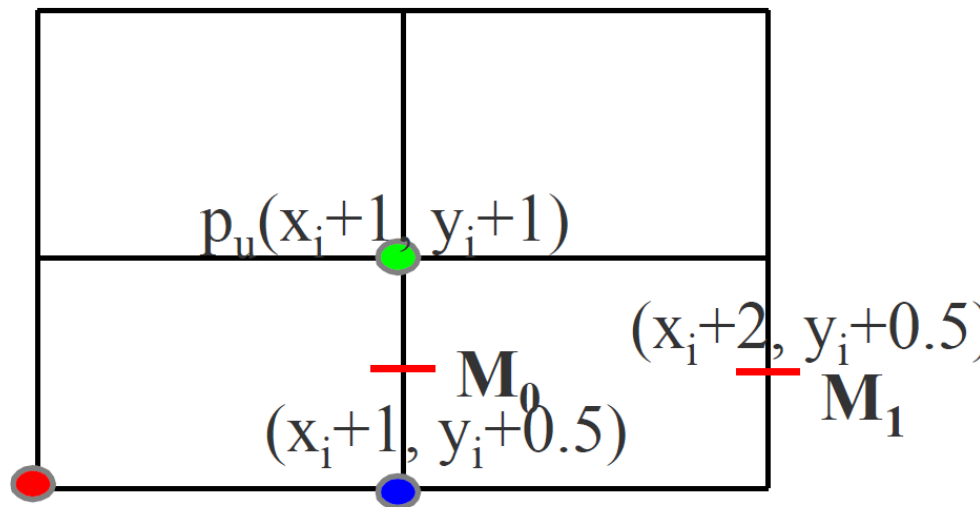
$$= A(x_i + 1) + B(y_i + 0.5) + C$$

$$\begin{aligned}
 d_0 &= F(x_{m0}, y_{m0}) \\
 &= F(x_i + 1, y_i + 0.5) \\
 &= A(x_i + 1) + B(y_i + 0.5) +
 \end{aligned}$$



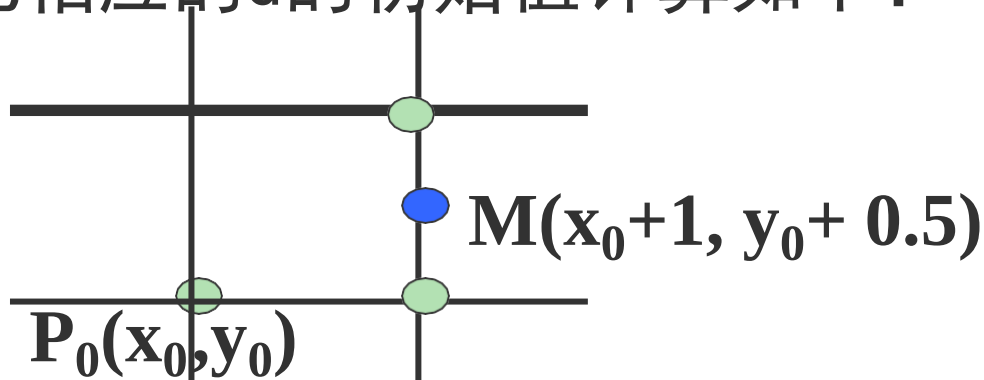
$$\begin{aligned}
 d_1 &= F(x_{m1}, y_{m1}) \\
 &= F(x_i + 2, y_i + 1.5) \\
 &= A(x_i + 2) + B(y_i + 1.5) + C \\
 &= A(x_i + 1) + B(y_i + 0.5) + C + A + B \\
 &= d_0 + A + B
 \end{aligned}$$

$$y = \begin{cases} y + 1 & (d < 0) \\ y & (d \geq 0) \end{cases}$$



$$\begin{aligned} d_1 &= F(x_i + 2, y_i + 0.5) - p(x_i, y_i) - p_d(x_i+1, y_i) \\ &= A(x_i + 2) + B(y_i + 0.5) + C \\ &= A(x_i + 1) + B(y_i + 0.5) + C + A \\ &= d_0 + A \end{aligned}$$

下面计算d的初始值 d_0 ，直线的第一个像素 $P_0(x_0, y_0)$ 在直线上，因此相应的d的初始值计算如下：



$$\begin{aligned}d_0 &= F(x_0 + 1, y_0 + 0.5) \\&= A(x_0 + 1) + B(y_0 + 0.5) + C \\&= Ax_0 + By_0 + C + A + 0.5B \\&= A + 0.5B\end{aligned}$$

$$d_{new} = \begin{cases} d_{old} + A + B & d < 0 \\ d_{old} + A & d \geq 0 \end{cases} \quad d_0 = A + 0.5B$$

至此，中点算法至少可以和DDA算法一样好！

可以用 $2d$ 代替 d 来摆脱浮点运算，写出仅包含**整数运算**的算法。

这样，中点生成直线的算法提高到整数加法，优于DDA算法。

小 结

(1) $Ax + By + C = 0$

(2) 通过判中点的符号，最终可以只进行整数加法

这个算法是否还有改进的余地？

中点画线法

$$y = \begin{cases} y + 1 & (d < 0) \\ y & (d \geq 0) \end{cases}$$

这个算法是否还有改进的余地？

能否提出一个算法，使这个算法不但能解决画直线，还能解决圆弧、抛物线甚至自由曲线的光栅化问题，使算法的覆盖域扩大。

直线绘制的三个著名的常用算法

1、数值微分法 (DDA)

2、中点画线法

3、Bresenham算法

DDA把算法效率提高到每步只做一个加法。

中点算法进一步把效率提高到每步只做一个整数加法

Bresenham提供了一个更一般的算法。该算法不仅有好的效率，而且有更广泛的适用范围



E. Jack Bresenham

Biography

[\[edit\]](#)

He retired from 27 years of service at [IBM](#) as a Senior Technical Staff Member in [1987](#). He taught for 16 years at [Winthrop University](#) and has nine [patents](#)^[1]. He has three children: Janet, Linda, and David.

[Bresenham's line algorithm](#), developed in [1962](#), is his most well-known innovation. It determines which points on a 2-dimensional [raster](#) should be plotted in order to form a straight line between two given points, and is commonly used to draw lines on a computer screen. It is one of the earliest algorithms discovered in the field of [computer graphics](#). The [Midpoint circle algorithm](#) shares some similarities to his line algorithm and is known as *Bresenham's circle algorithm*.

- [Ph.D., Stanford University, 1964](#)
- [MSIE, Stanford University, 1960](#)
- [BSEE, University of New Mexico, 1959](#)

Graphics and
Image Processing

W. Newman*
Editor

A Linear Algorithm for Incremental Digital Display of Circular Arcs

Jack Bresenham
IBM System Communications Division

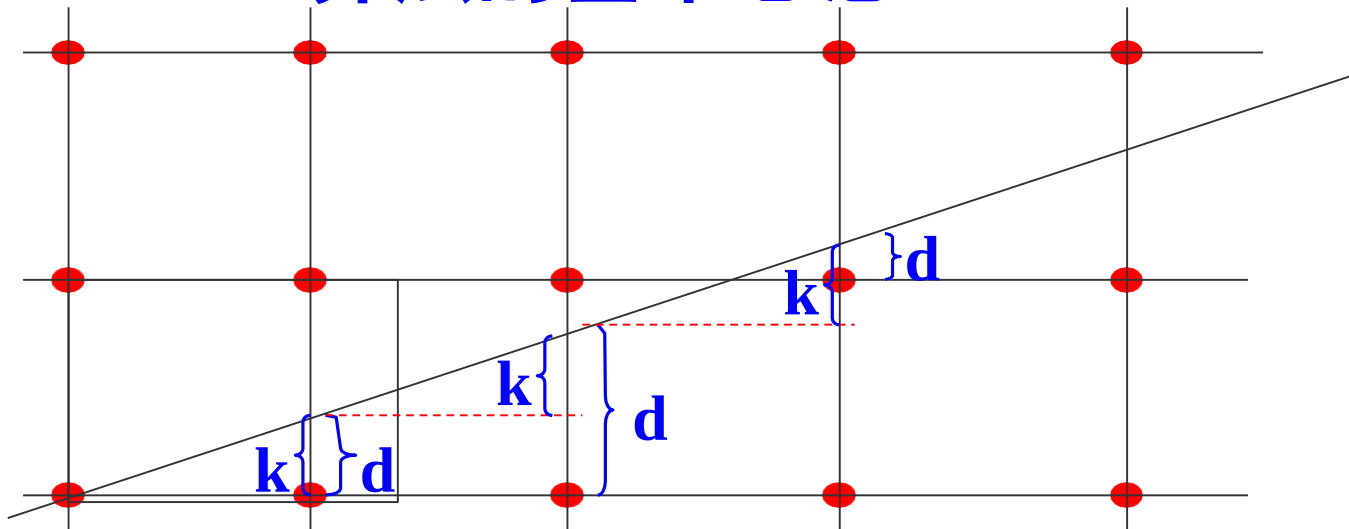
Circular arcs can be drawn on an incremental display device such as a cathode ray tube, digital plotter, or matrix printer using only sign testing and elementary addition and subtraction. This paper describes methodology for producing dot or step patterns closest to the true circle.

This paper describes an algorithm for circular arc mesh point selection using incremental display devices such as a cathode ray tube or digital plotter. Error criteria are explicitly specified and both squared and radial error minimization considered. The repetitive incremental stepping loop for point selection requires only simple addition/subtraction and sign testing; neither quadratic nor trigonometric evaluations are required. When a circle's center point and radius are integers, only integer calculations are required.

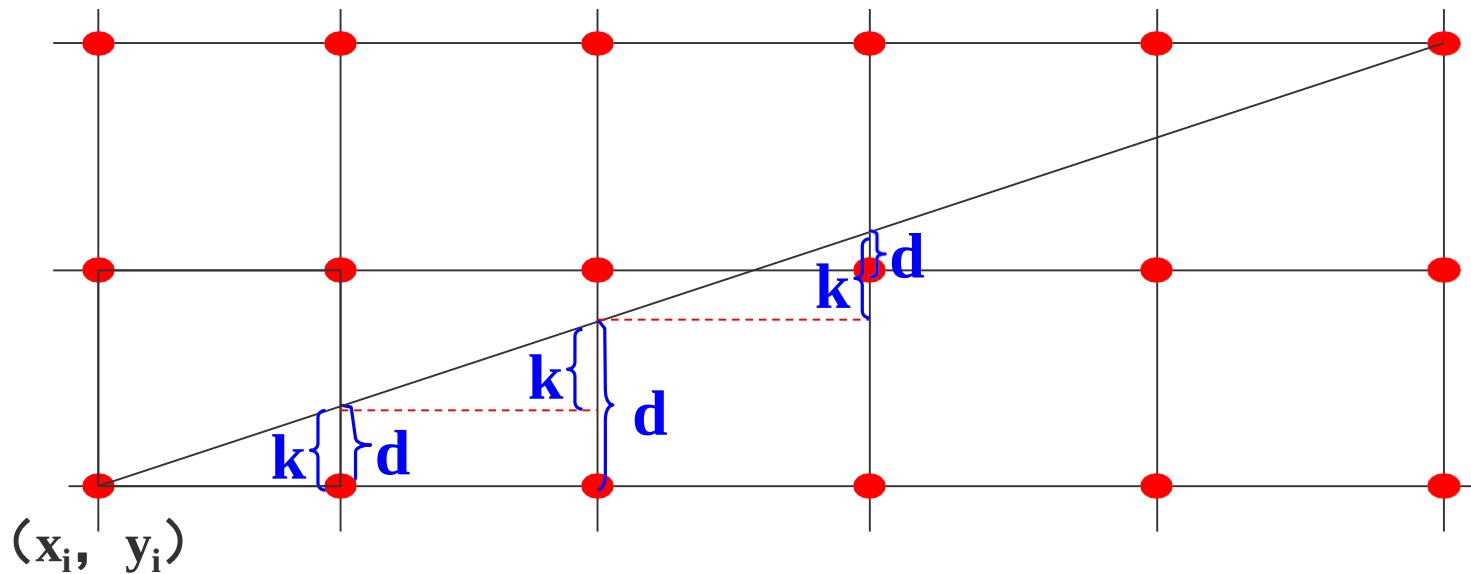
The circle algorithm complements an earlier line algorithm described in [1, 2]. The algorithm's minimum error point selection is appropriate for use in numerical control, drafting, or photo mask preparation applications where closeness of fit is a necessity. Its simplicity and use only of elementary addition/subtraction allow its use in small computers, programmable terminals, or direct hardware implementations where compactness and speed are desirable.

The display devices under consideration are capable of executing, in response to an appropriate pulse, any one of the eight linear movements shown in Figure 1. Thus incremental movement is from a point on a mesh to any of its eight adjacent points on the mesh.

Bresenham算法的基本思想



该算法的思想是通过各行、各列像素中心构造一组虚拟网格线，按照直线起点到终点的顺序，计算直线与各垂直网格线的交点，然后根据误差项的符号确定该列像素中与此交点最近的像素。

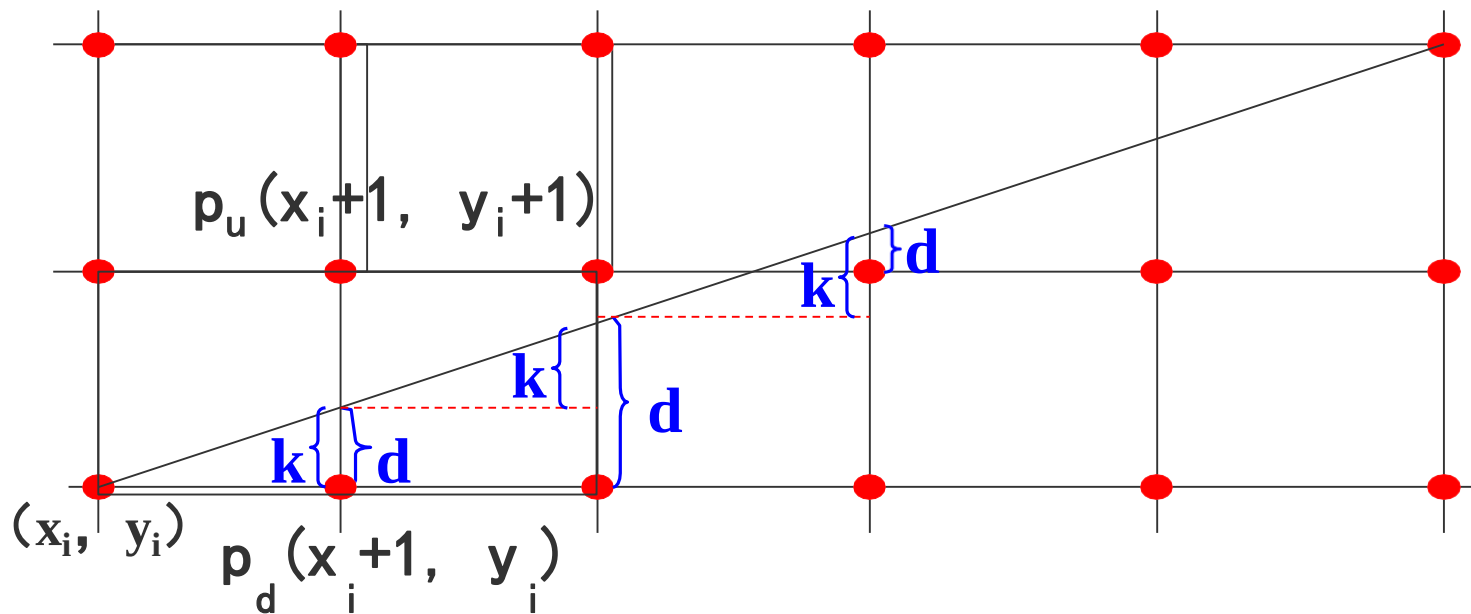


假设每次 $x+1$, y 的递增（减）量为0或1，它取决于实际直线与最近光栅网格点的距离，这个距离的最大误差为0.5。

误差项 d 的初值 $d_0=0$

$$d = d + k$$

一旦 $d \geq 1$ ，就把它减去1，保证 d 的相对性，且在0、1之间。



$$\begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 & (d > 0.5) \\ y_i & (d \leq 0.5) \end{cases} \end{cases}$$

关键是把这个算法的效率也搞到整数加法，否则就是失败。如何提高到整数加法？

$$\begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 \\ y_i \end{cases} \end{cases} \quad \begin{matrix} (d > 0.5) \\ (d \leq 0.5) \end{matrix}$$

如何把这个算法的效率也提高到整数加法？

改进1： 令 $e = d - 0.5$

$$\begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 \\ y_i \end{cases} \end{cases} \quad \begin{matrix} (e > 0) \\ (e \leq 0) \end{matrix}$$

$$\begin{cases} x_{i+1} = x_i + 1 \\ y_{i+1} = \begin{cases} y_i + 1 & (e > 0) \\ y_i & (e \leq 0) \end{cases} \end{cases}$$

$e > 0$, y 方向递增1; $e < 0$, y 方向不递增

$e = 0$ 时, 可任取上、下光栅点显示

- $e_{\text{初}} = -0.5$
- 每走一步有 $e = e + k$
- if ($e > 0$) then $e = e - 1$

$$e_{\text{初}} = -0.5 \quad k = \frac{dy}{dx}$$

改进2: 由于算法中只用到误差项的符号，于是可以用 $e*2*\Delta x$ 来替换 e 。

- $e_{\text{初}} = -\Delta x$,
- 每走一步有: $e = e + 2\Delta y$ 。
- if ($e > 0$) then $e = e - 2\Delta x$

算法步骤为：

1. 输入直线的两 endpoint $P_0(x_0, y_0)$ 和 $P_1(x_1, y_1)$ 。
2. 计算初始值 Δx 、 Δy 、 $e = -\Delta x$ 、 $x = x_0$ 、 $y = y_0$ 。
3. 绘制点 (x, y) 。
4. e 更新为 $e + 2\Delta y$ ，判断 e 的符号。若 $e > 0$ ，则 (x, y) 更新为 $(x+1, y+1)$ ，同时将 e 更新为 $e - 2\Delta x$ ；否则 (x, y) 更新为 $(x+1, y)$ 。
5. 当直线没有画完时，重复步骤3和4。否则结束。

Bresenham算法很像DDA算法，都是加斜率

但DDA算法是每次求出一个新的 y 以后取整来画；
而Bresenham算法是判符号来决定上下两个点。所以该算法集中了DDA和中点两个算法的优点，而且应用范围广泛

小 结

1、计算机科学问题的核心就是算法

把一个含有乘法和一个加法的普通直线算法，是如何通过改进和完善其性能，最终变成整数加法的一个精彩过程

2、领会算法中所蕴含的创新思想

改进和完善算法的过程中所体现出来的一些闪光的思想是我们所要认识和领会的

3、科学研究无止境，学术面前人人平等

学会研究性学习，对已有的算法提出质疑找出其不足。

1	Bresenham直线生成算法的改进	贾银高; 张焕春; 经亚枝	中国图象图 形学报	2008-01-15	期刊	21		470		
2	基于Bresenham画线算法的图像快速-高精度 旋转算法	石慎; 张 艳宁; 郝 润平; 郑 江滨	计算机辅助 设计与图形 学报	2007-11-15	期刊	13		211		
3	基于Bresenham算法的四步画直线算法	林笠	暨南大学学 报(自然科学 与医学版)	2003-10-30	期刊	16		477		
4	改进的Bresenham直线生成算法	刘晶; 李 俊; 孙通	计算机应用 与软件	2008-10-15	期刊	11		202		
5	改进的直线Bresenham算法	李高平; 檀结庆	合肥工业大 学学报(自然 科学版)	2003-10-28	期刊	10		343		
6	Bresenham画圆算法的改进	王志喜; 王润云	计算机工程	2004-06-20	期刊	11		435		

1. 标题: Bresenham Algorithm: Implementation and Analysis in Raster Shape

作者: Gaol, F.L.

来源出版物: Journal of Computers 卷: 8 期: 1 页: 69-78 DOI: 10.4304/jcp.8.1.69-78 出版年: Jan. 2013

被引频次: 0 (来自所有数据库)

[查看摘要]

2. 标题: Image Enhancement with Rotating Kernel Transformation Filter Generated by Bresenham's Algorithm

作者: Seung-Won Shin; Kyeong-Seop Kim; SeMin Lee; 等.

来源出版物: Transactions of the Korean Institute of Electrical Engineers 卷: 61 期: 6 页: 872-8 DOI: 10.5370/KIEE.2012.61.6.872 出版年: June 2012

被引频次: 0 (来自所有数据库)

[查看摘要]

计算机图形学

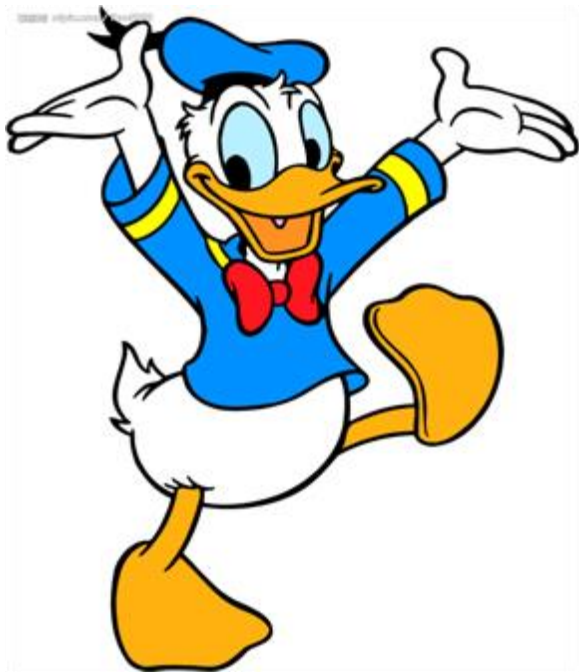
第二章：光栅图形学算法

光栅图形学算法的研究内容

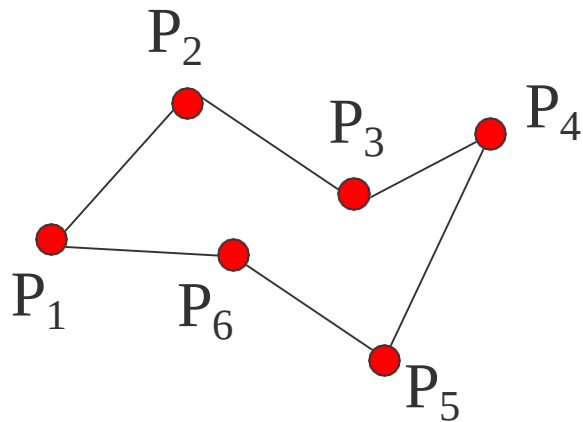
- 直线段的扫描转换算法
- 多边形的扫描转换与区域填充算法
- 裁剪算法
- 反走样算法
- 消隐算法

一、多边形的扫描转换

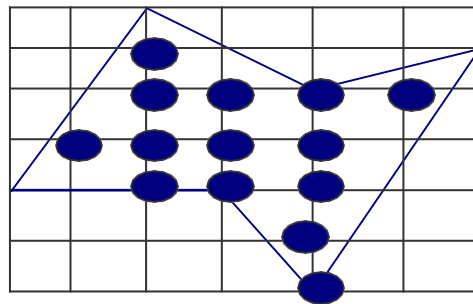
多边形的扫描转换和区域填充这个问题是怎么样在离散的像素集上表示一个连续的**二维图形**



多边形有两种重要的表示方法：**顶点**表示和**点阵**表示



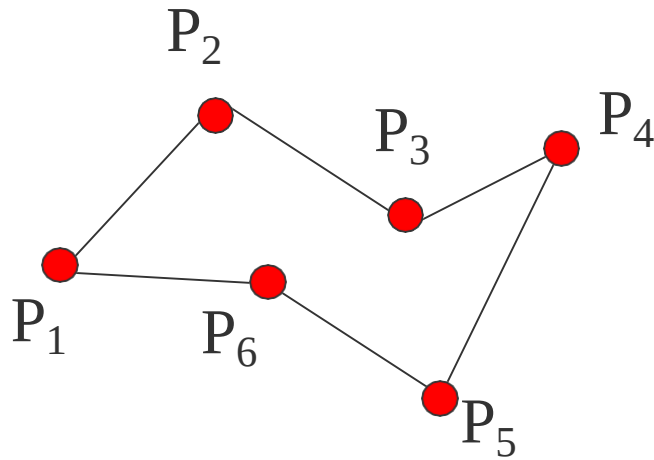
顶点表示



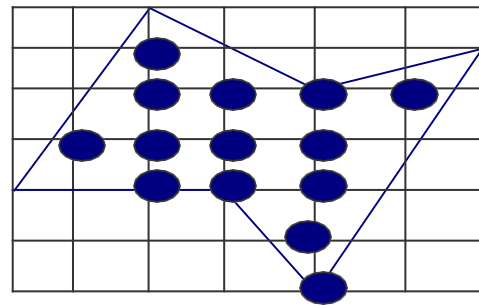
点阵表示

顶点表示是用多边形的顶点序列来表示多边形。这种表示直观、几何意义强、占内存少，易于进行几何变换

但由于它没有明确指出哪些像素在多边形内，故不能直接用于面着色

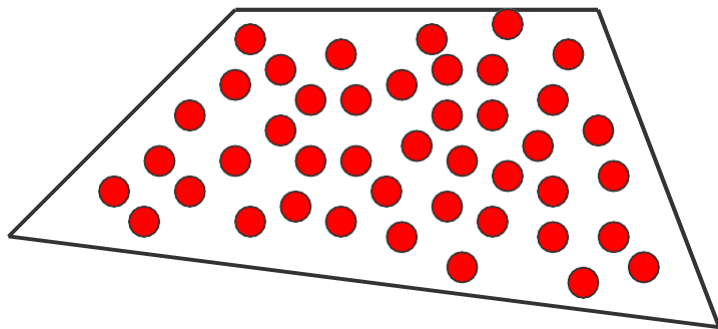


点阵表示是用位于多边形内的象
素集合来刻画多边形。这种表示
丢失了许多几何信息（如**边界**、
顶点等），但它却是光栅显示系
统显示时所需的表示形式。

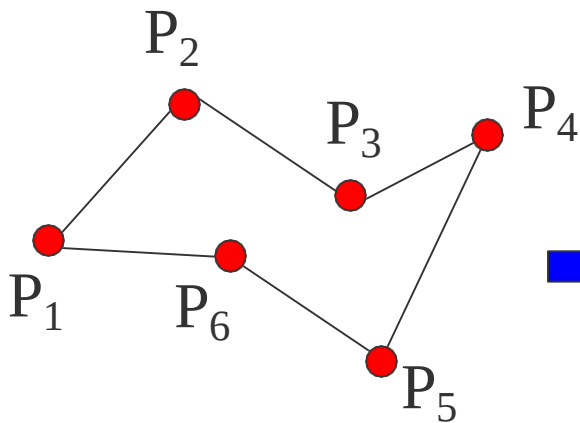


这涉及到两个问题：**第一个问题**是如果知道边界，能否求出**哪些像素**在多边形内？

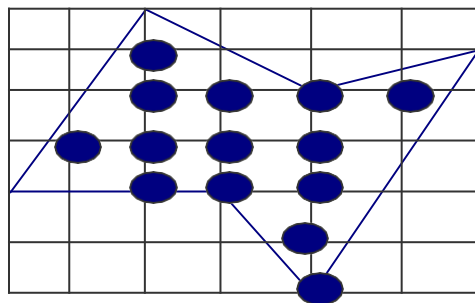
第二个问题是知道多边形内部的像素，反过来如何求多边形的边界？



光栅图形的一个基本问题是把多边形的**顶点**表示转换为**点阵**表示。这种转换称为**多边形的扫描转换**



顶点表示

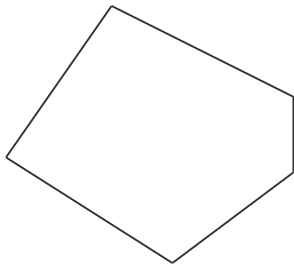


点阵表示

多边形分为凸多边形、凹多边形、含内环的多边形等：

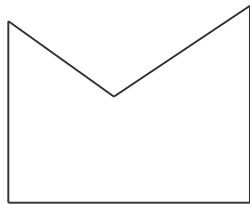
(1) 凸多边形

任意两顶点间的连线均在多边形内



(2) 凹多边形

任意两顶点间的连线有不在在多边形内

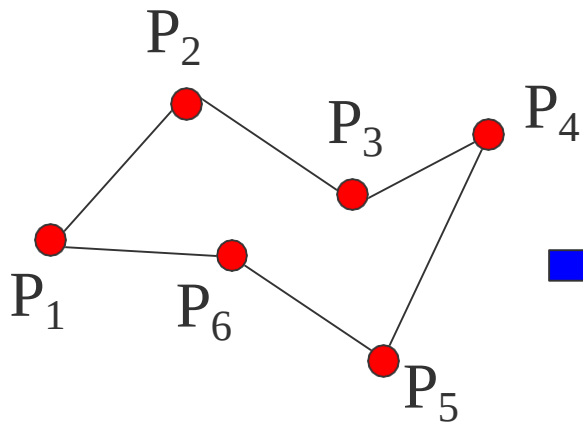


(3) 含内环的多边形

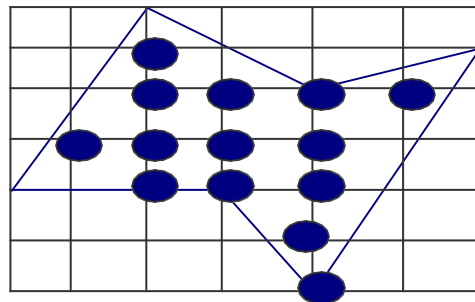
多边形内包含多边形



现在的问题是，知道多边形的边界，如何找到多边形内部的点，即把多边形内部涂上颜色



顶点表示

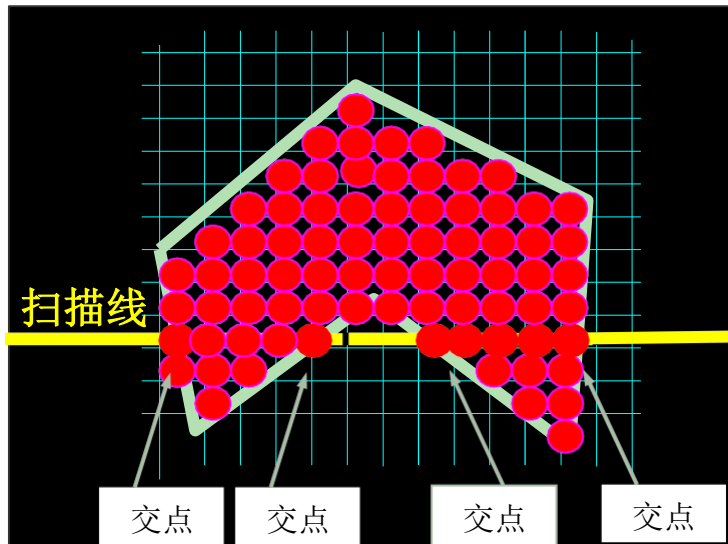


点阵表示

1、X-扫描线算法

X-扫描线算法填充多边形的基本思想是按扫描线顺序，计算扫描线与多边形的相交区间，再用要求的颜色显示这些区间的像素，即完成填充工作

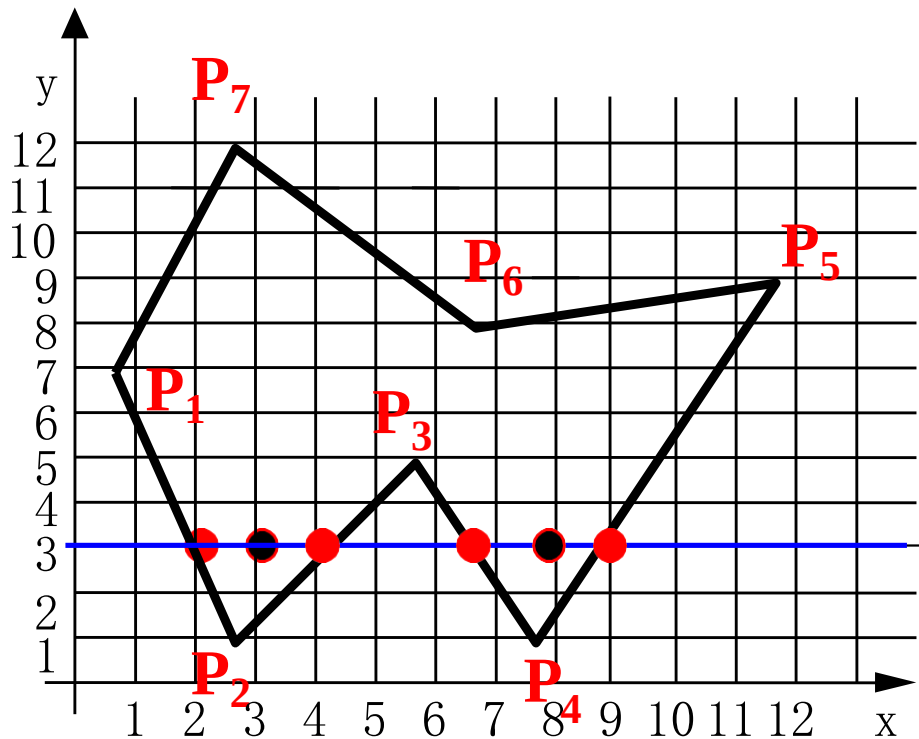
区间的端点可以通过计算扫描线与多边形边界线的交点获得



如扫描线 $y=3$ 与多边形的边界相交于4点：

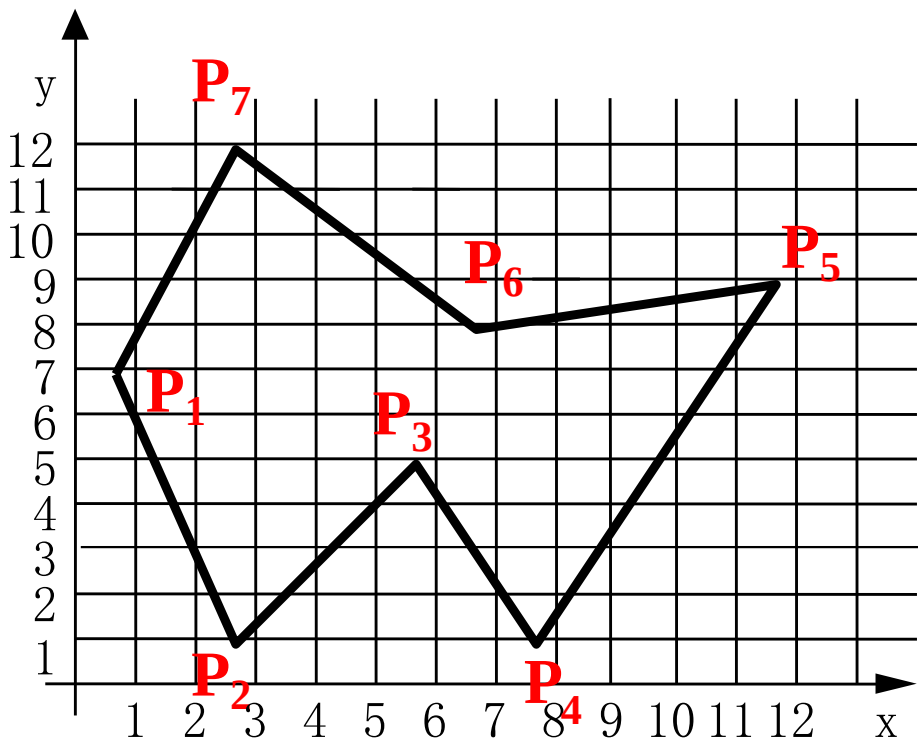
$(2, 3)$ 、 $(4, 3)$ 、 $(7, 3)$ 、 $(9, 3)$ 。

这四点定义了扫描线从 $x=2$ 到 $x=4$ ，从 $x=7$ 到 $x=9$ 两个落在多边形内的区间，该区间内的像素应取填充色



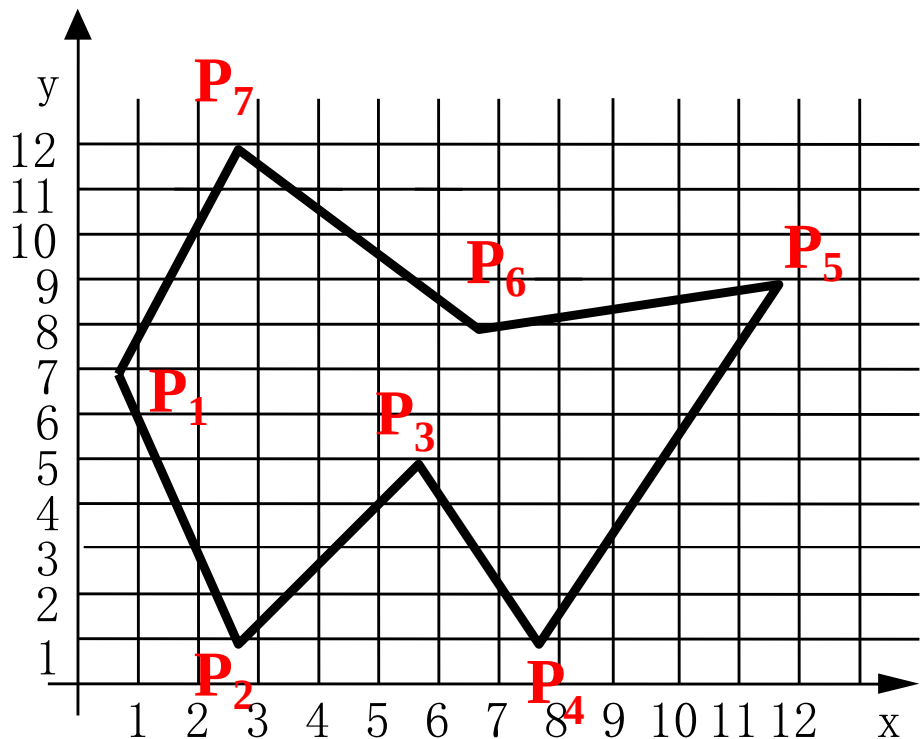
算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

(1) 确定多边形所占有的最大扫描线数，得到多边形顶点的最小和最大y值 ($y_{\min}$ 和 $y_{\max}$)



算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

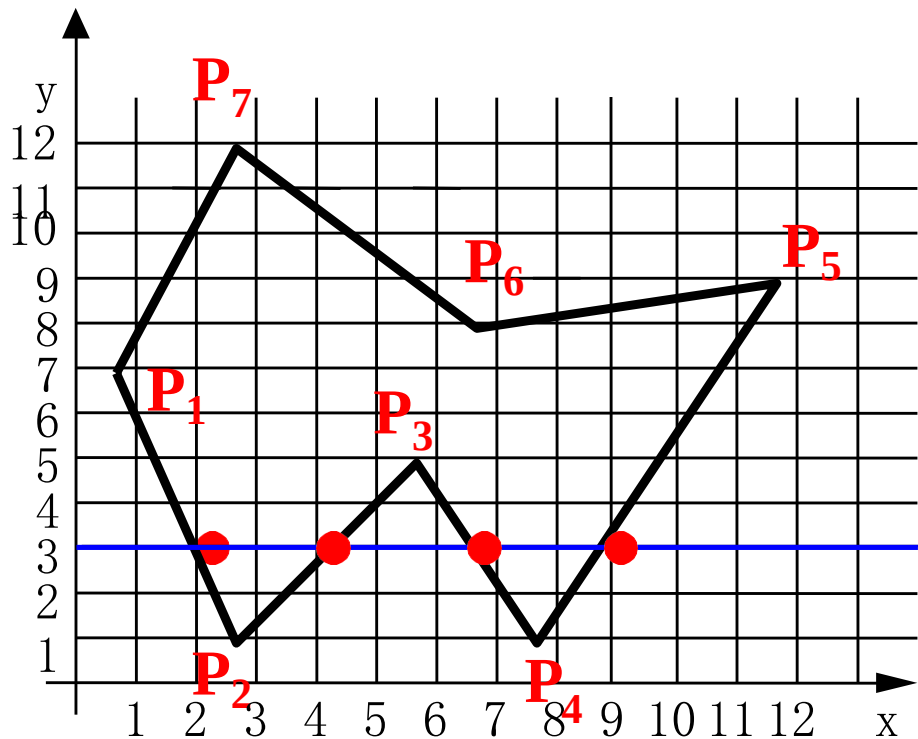
(2) 从 $y = y_{\min}$ 到 $y = y_{\max}$ ，
每次用一条扫描线进行填充

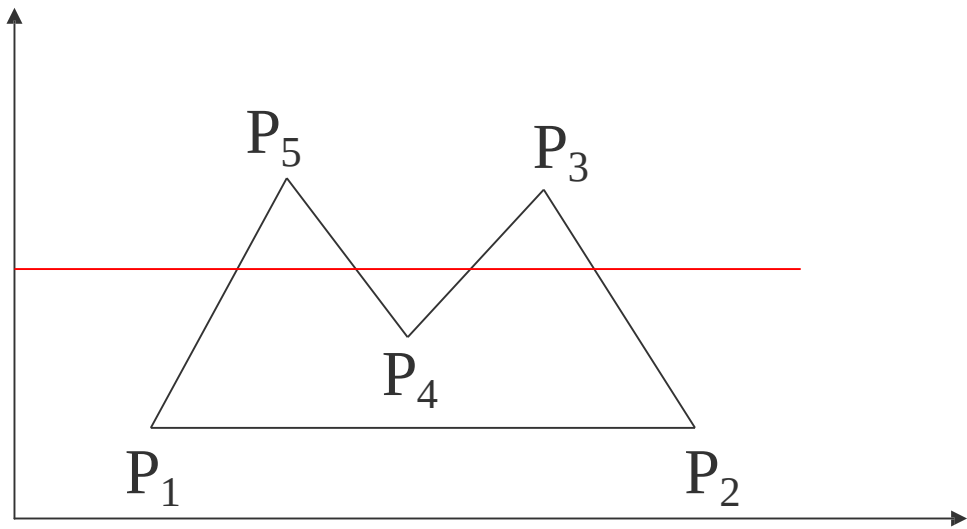


算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

(3) 对一条扫描线填充的过程可分为四个步骤：

- a、求交：计算扫描线与多边形各边的交点
- b、排序：把所有交点按递增顺序进行排序



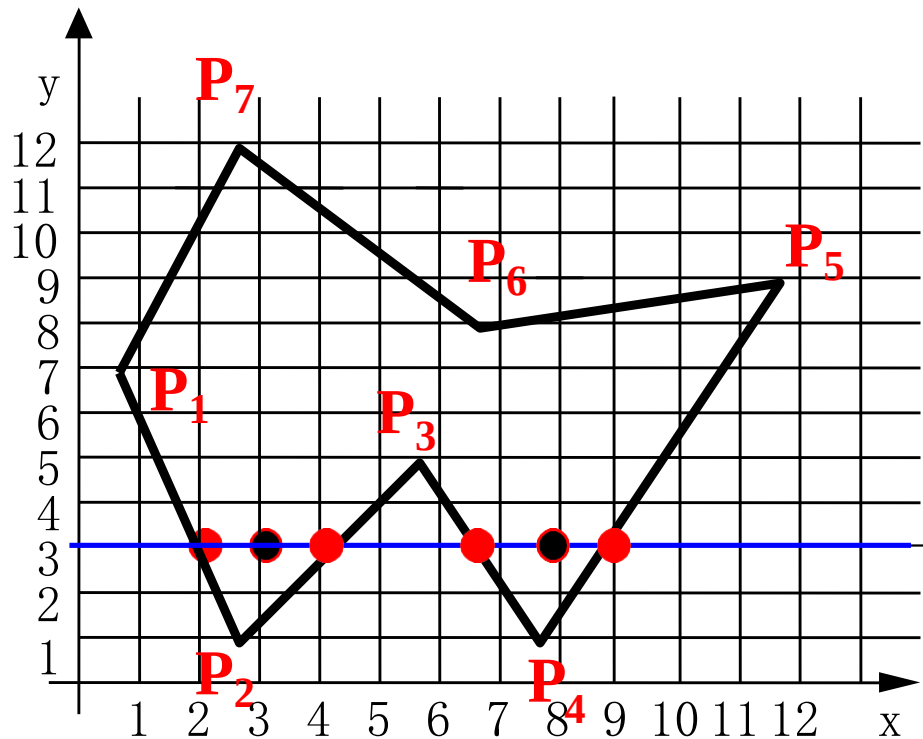


算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

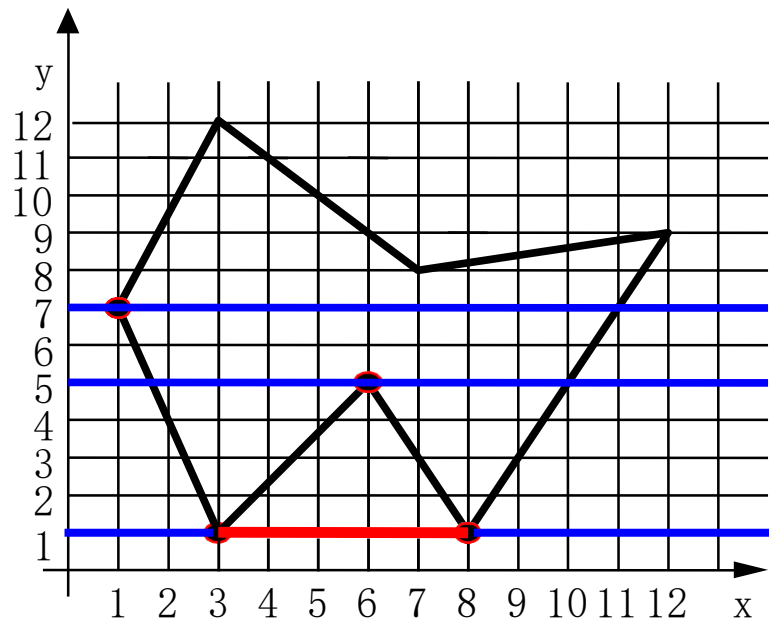
(3) 对一条扫描线填充的过程可分为四个步骤：

c、交点配对：第一个与第二个，第三个与第四个

d、区间填色：把这些相交区间内的像素置成不同于背景色的填充色



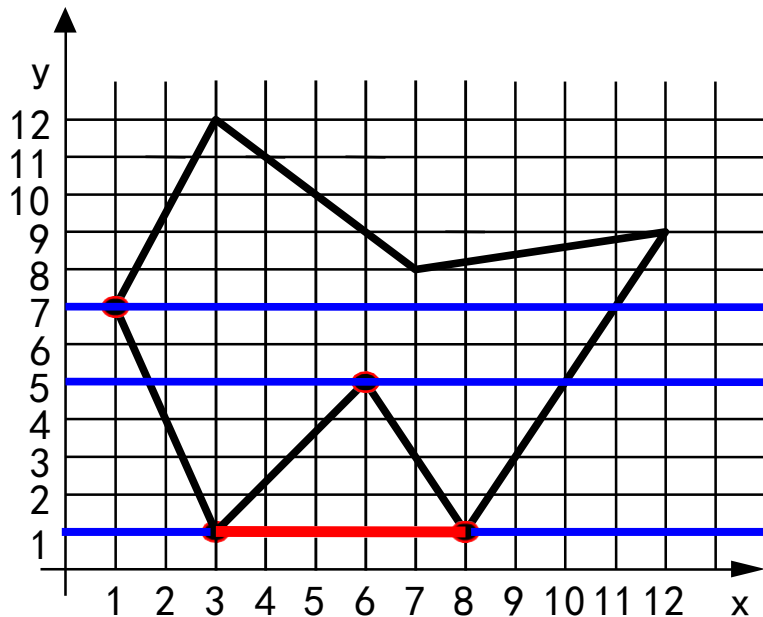
当扫描线与多边形顶点相交时，交点的取舍问题（交点的个数应保证为偶数个）



解决方案：

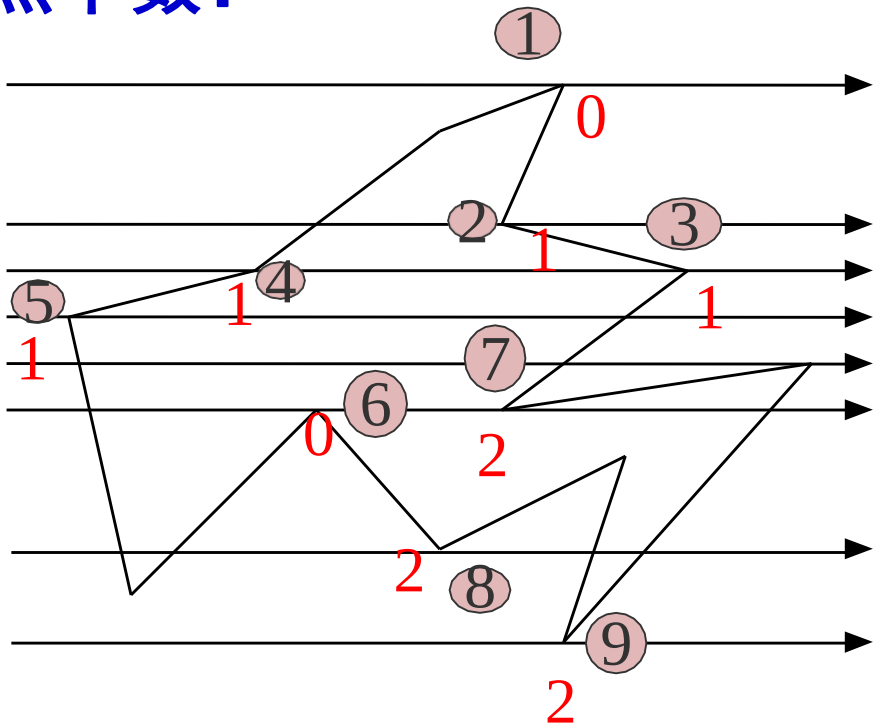
(1) 若共享顶点的两条边分别落在扫描线的两边，交点只算一个

(2) 若共享顶点的两条边在扫描线的同一边，这时交点作为
零个或两个



检查共享顶点的两条边的另外两个端点的y值，按这两个y值中大于交点y值的个数来决定交点数

举例计算交点个数：



为了计算每条扫描线与多边形各边的交点，最简单的方法是把多边形的所有边放在一个表中。在处理每条扫描线时，**按顺序从表中取出所有的边，分别与扫描线求交**

这个算法效率低，为什么？

关键问题是**求交**！而求交是很可怕的，求交的计算量是非常大的

1、X-扫描线算法

X-扫描线算法填充多边形的基本思想是按扫描线顺序，计算扫描线与多边形的相交区间，再用要求的颜色显示这些区间的像素，即完成填充工作

关键问题是**求交**！求交的计算量是非常大的

排序、配对、填色总是要的！

最理想的算法是**不求交**！

2、多边形的扫描转换算法的改进

扫描转换算法重要意义是提出了图形学里两个重要的思想：

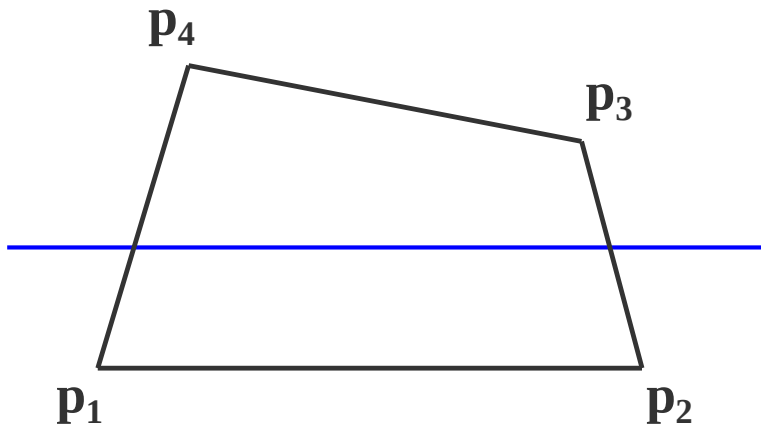
(1) **扫描线**：当处理图形图像时按一条条扫描线处理

(2) **增量**的思想

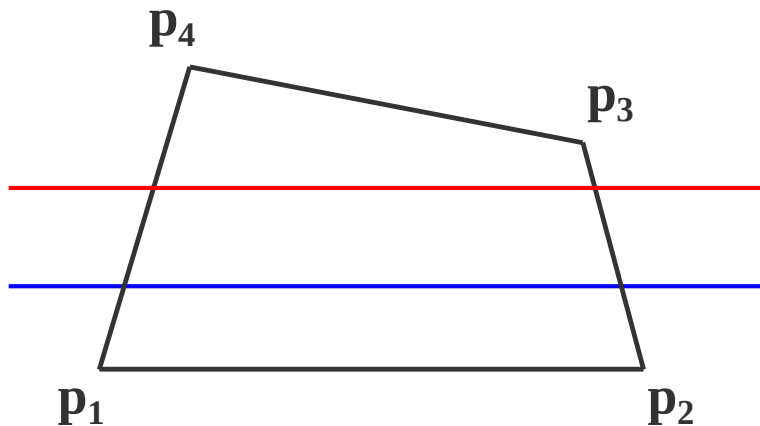
求交点的时候能不能也采取增量的方法？每条扫描线的 y 值都知道，关键是求 x 的值。 x 是什么？

可以从三方面考虑加以改进：

- (1) 在处理一条扫描线时，仅对与它相交的多边形的边（**有效边**）进行求交运算

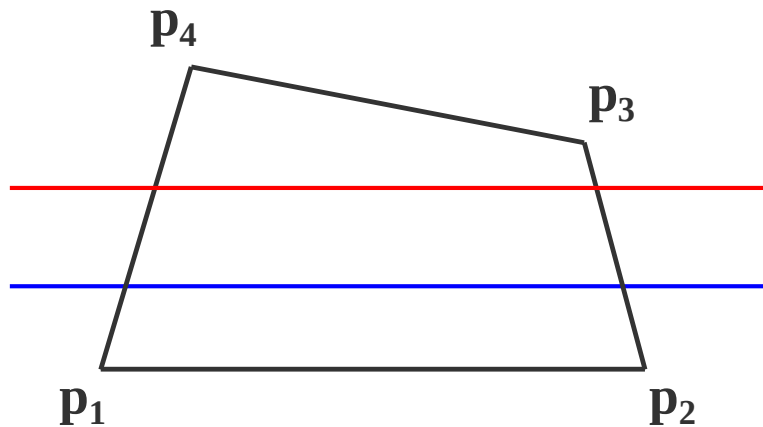


(2) 考虑扫描线的**连贯性**



即当前扫描线与各边的交点顺序与下一条扫描线与各边的交点顺序很可能相同或非常相似

(3) 最后考虑多边形的连贯性



即当某条边与当前扫描线相交时，它很可能也与下一条扫描线相交

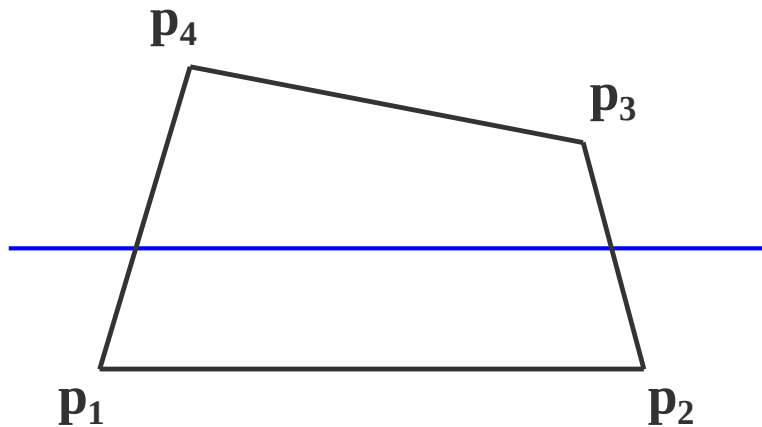
为了避免求交运算，需要引进一套
特殊的数据结构

2、多边形的扫描转换算法的改进

为了避免求交运算，需要引进一套特殊的**数据结构**

数据结构:

- (1) **活性边表** (AET): 把与当前扫描线相交的边称为活性边, 并把它们按与扫描线交点 x 坐标递增的顺序存放在一个链表中。



(2) **结点内容** (一个结点在数据结构里可用结构来表示)

x : 当前扫描线与边的交点坐标

Δx : 从当前扫描线到下一条扫描线间 x 的增量

$y_{\max}$: 该边所交的最高扫描线的坐标值 $y_{\max}$

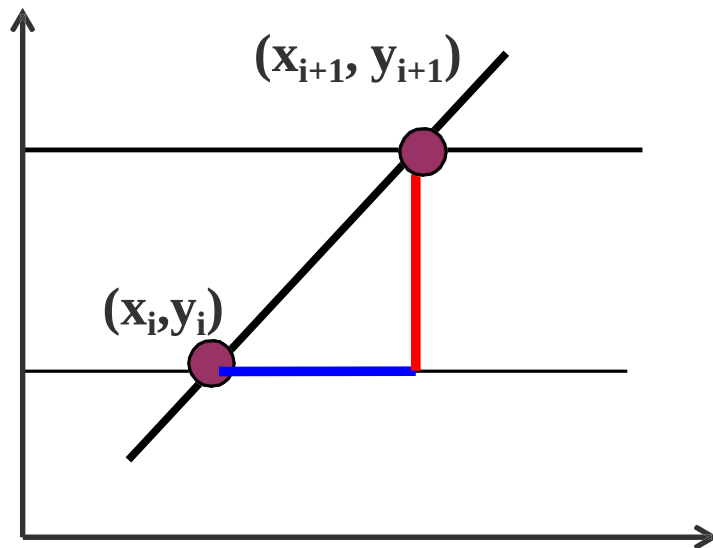
x	Δx	$y_{\max}$	next
-----	------------	------------	------

随着扫描线的移动，扫描线与多边形的交点和上一次交点相关：

设边的直线斜率为 k

$$k = \frac{\Delta y}{\Delta x} = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

$$x_{i+1} - x_i = \frac{1}{k} \quad \longrightarrow \quad x_{i+1} = x_i + \frac{1}{k}$$

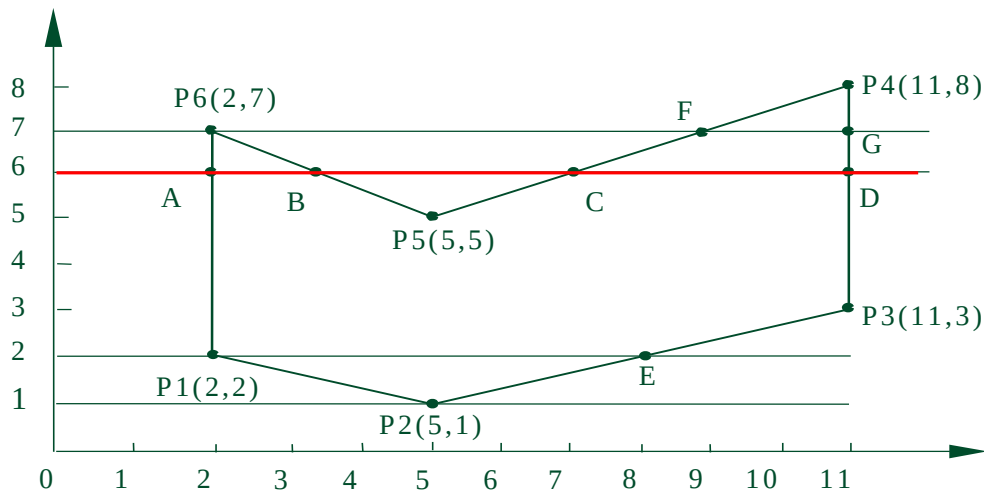


$$\text{即: } \Delta x = \frac{1}{k}$$

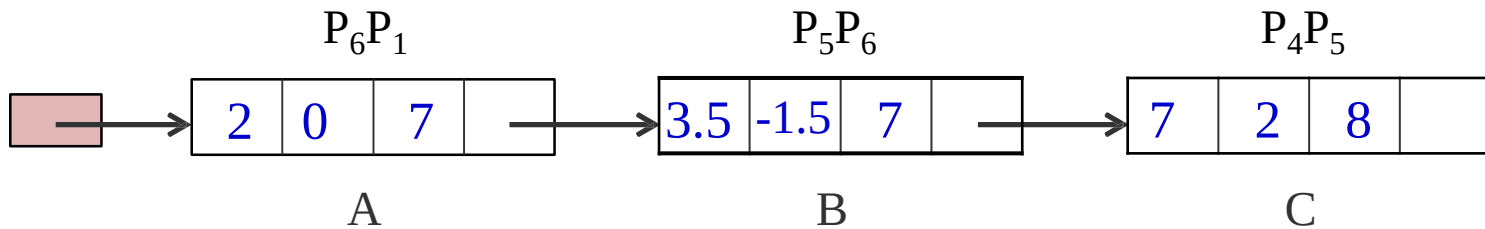
另外，需要知道一条边何时不再与下一条扫描线相交，以便及时把它从有效边表中删除出去，避免下一步进行无谓的计算

x	Δx	$y_{\max}$	next
-----	------------	------------	------

其中 x 为当前扫描线与边的交点， $y_{\max}$ 是边所在的最大扫描线值，通过它可以知道何时才能“**抛弃**”该边， Δx 表示从当前扫描线到下一条扫描线之间的 x 增量即斜率的倒数。 next 为指向下一条边的指针



X	ΔX	$y_{\max}$	next
---	------------	------------	------

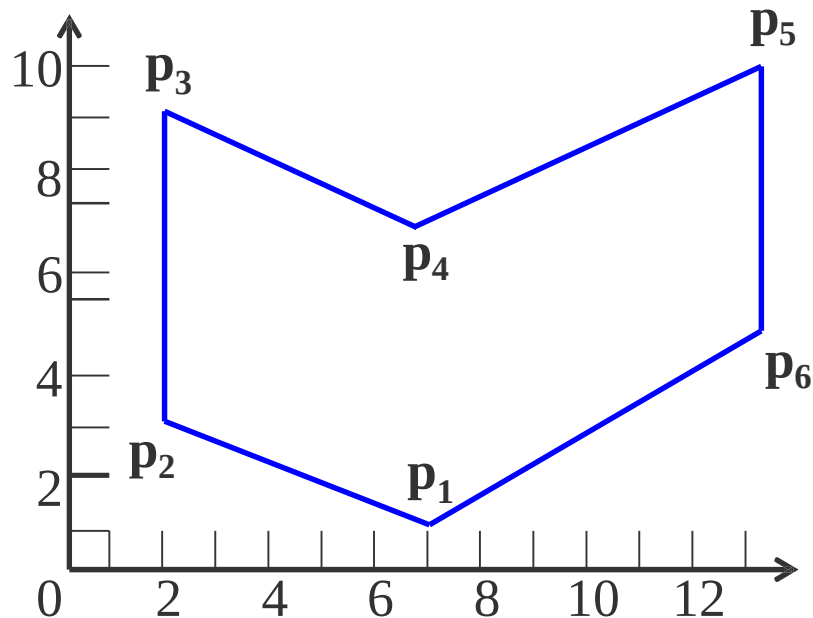


为了方便**活性边表**的建立与更新，需构造一个**新边表**（NET），用来存放多边形的边的信息，分为4个步骤：

- （1）首先构造一个纵向链表，链表的长度为多边形所占有的最大扫描线数，链表的每个结点，称为一个**吊桶**，对应多边形覆盖的每一条扫描线

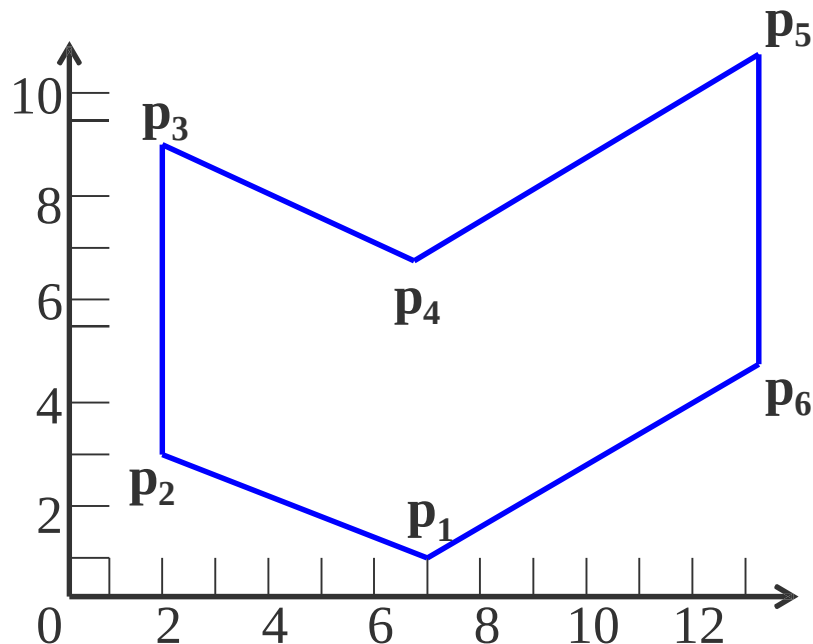
(1) 首先构造一个纵向链表，链表的长度为多边形所占有的最大扫描线数

10	
9	
8	
4	
3	
2	
1	
0	



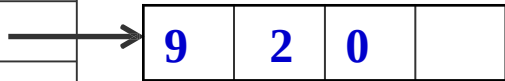
(2) NET挂在与该边低端
 y 值相同的扫描线桶中。
也就是说，存放在该扫描
线第一次出现的边

- 该边的 $y_{\max}$
- 该边较低点的 x 坐标值 $x_{\min}$
- 该边的斜率 $1/k$
- 指向下一条具有相同较低端 y 坐标的边的指针



$y_{\max}$	$x_{\min}$	$1/k$	next
------------	------------	-------	------

10
9
8
7
6
5
4
3
2
1
0



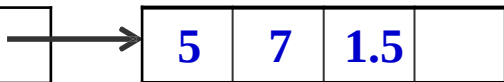
9	2	0	
---	---	---	--

P_2P_3



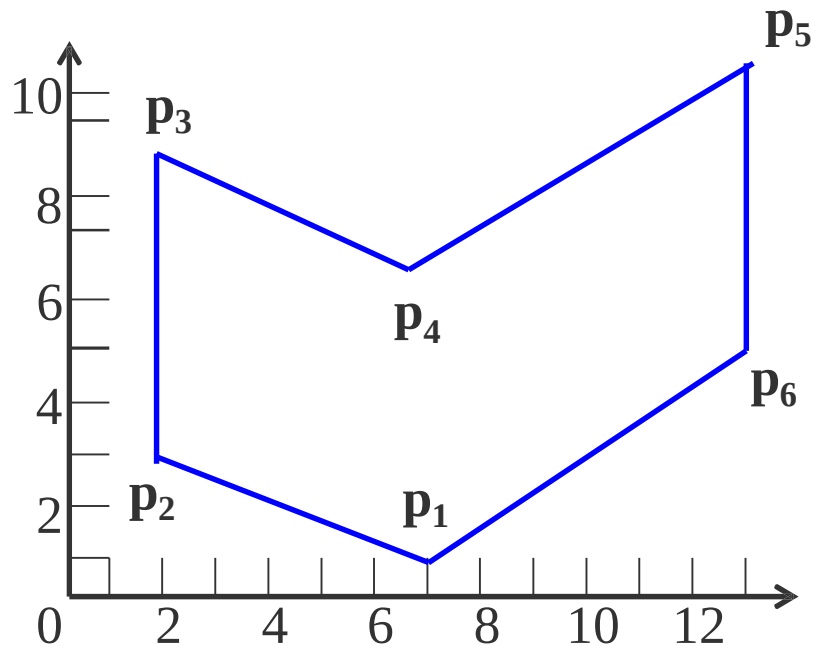
3	7	-2.5	
---	---	------	--

P_1P_2



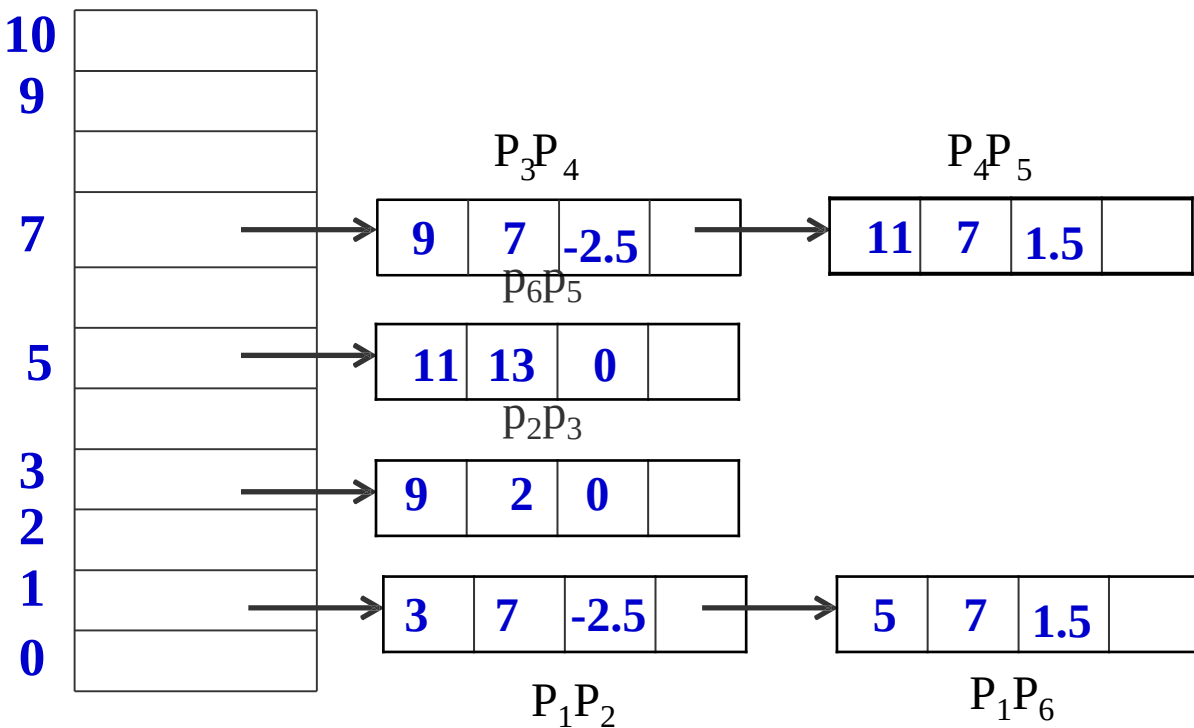
5	7	1.5	
---	---	-----	--

P_1P_6



从右边这个NET表里就知道多边形是从哪里开始的

在这个表里只有1、3、5、7处有边，从y=1开始做，而在1这条线上有两条边进来了，然后就把这两条边放进活性边表来处理



每做一次新的扫描线时，要对已有的边进行三个处理：

1、是否被去除掉；

2、如果不被去除，第二就要对它的数据进行更新。所谓更新数据就是要更新它的x值，即： $x+1/k$

3、看有没有新的边进来，新的边在NET里，可以插入排序插进来。

这个算法过程从来没有求交，这套数据结构使得你不用求交点！避免了求交运算。

```

void polyfill (polygon, color)
    int color; 多边形    polygon;
{   for (各条扫描线i )
    {   初始化新边表头指针NET[i];
        把 $y_{\min} = i$  的边放进边表NET[i];
    }
    y = 最低扫描线号;
    初始化活性边表AET为空;
    for (各条扫描线i )
    {
        把新边表NET[i] 中的边结点用插入排序法插入AET表,
        使之按x坐标递增顺序排列;
        遍历AET表, 把配对交点区间(左闭右开)上的像素(x, y)
        , 用putpixel(x, y, color) 改写像素颜色值;
        遍历AET表, 把 $y_{\max} = i$  的结点从AET表中删除, 并把 $y_{\max} > i$ 
        结点的x值递增 $\Delta x$ ;
        若允许多边形的边自相交, 则用冒泡排序法对AET表重新排序;
    }
}  /* polyfill */

```

多边形扫描转换算法小结

扫描线法可以实现已知任意多边形域边界的填充。该填充算法是按扫描线的顺序，计算扫描线与待填充区域的相交区间，再用要求的颜色显示这些区间的像素，即完成填充工作

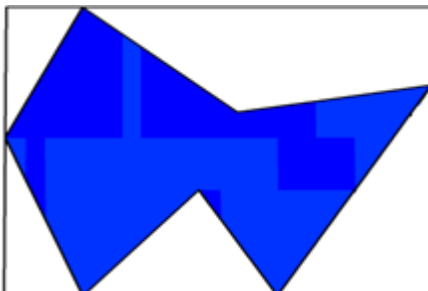
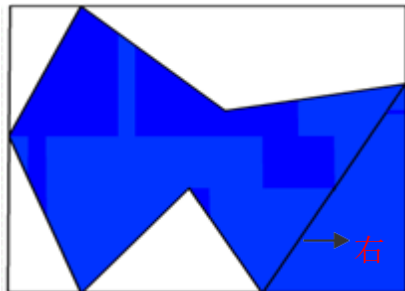
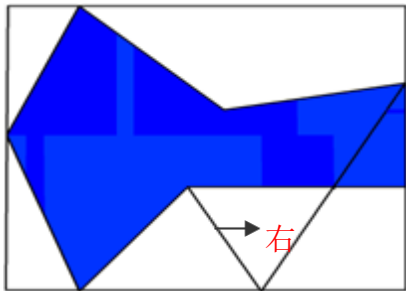
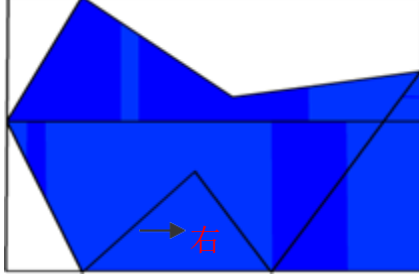
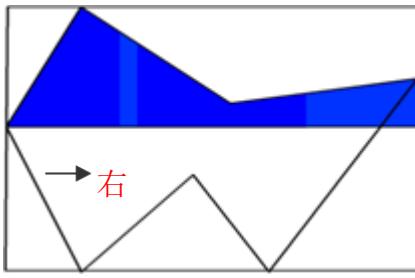
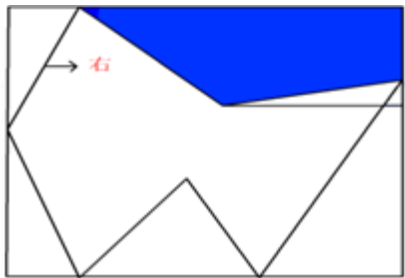
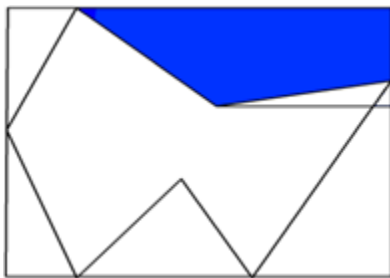
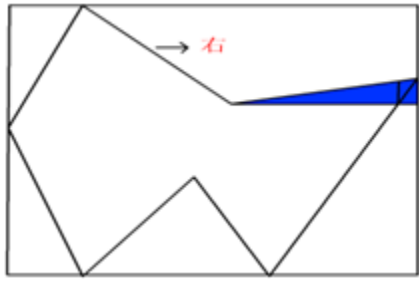
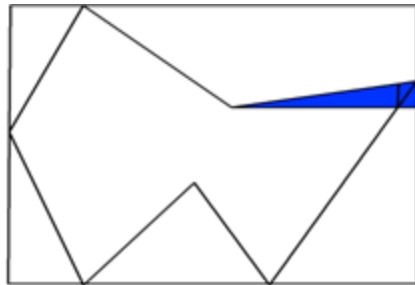
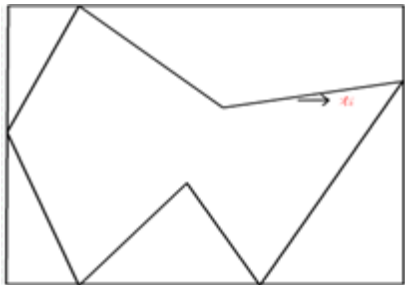
为了提高算法效率：

- (1) 增量的思想
- (2) 连贯性思想
- (3) 构建了一套特殊的数据结构

这里区间的端点通过计算扫描线与多边形边界的交点获得。所以待填充区域的边界线必须先知道，因此它的缺点是**无法实现对未知边界的区域填充**

3、边缘填充算法

其基本思想是按任意顺序处理多边形的每条边。在处理每条边时，首先求出该边与扫描线的交点，然后将每一条扫描线上交点**右方**的所有像素**取补**。多边形的所有边处理完毕之后，填充即完成。



算法简单，但对于复杂图型，每一像素可能被访问多次。输入和输出量比有效边算法大得多。

为了减少边缘填充法访问像素的次数，可采用栅栏填充算法

4、栅栏填充算法

栅栏指的是一条过多边形顶点且与扫描线垂直的直线。它把多边形分为两半。在处理每条边与扫描线的交点时，将交点与栅栏之间的像素取补

5、边界标志算法

帧缓冲器中对多边形的每条边进行直线扫描转换，亦即对多边形边界所经过的像素打上标志

然后再采用和扫描线算法类似的方法将位于多边形内的各个区段着上所需颜色

由于边界标志算法不必建立维护边表以及对它进行排序，所以边界标志算法更适合硬件实现，这时它的执行速度比有序边表算法快一至两个数量级。

第二章：光栅图形学算法

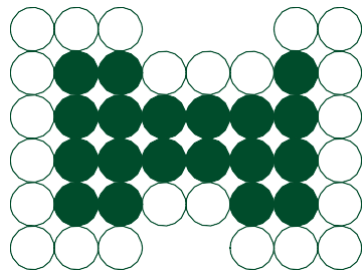
多边形的扫描转换与区域填充

二、区域填充

区域——指已经表示成点阵形式的填充图形，是象素的集合

区域填充是指将区域内的一点(常称**种子点**)赋予给定颜色,然后将这种颜色**扩展**到整个区域内的过程。

区域可采用**内点**表示和**边界**表示两种表示形式



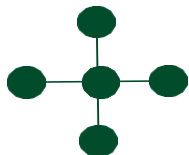
● 表示内点 ○ 表示边界点

内点表示：枚举出区域内部的所有像素，内部的所有像素着同一个颜色，边界像素着与内部像素不同的颜色

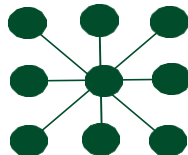
边界表示：枚举出边界上的所有像素，边界上的所有像素着同一个颜色，内部像素着与边界像素不同的颜色

区域填充算法要求区域是连通的，因为只有在连通区域中，才可能将种子点的颜色扩展到区域内的其它点。

区域可分为4向连通区域和8向连通区域



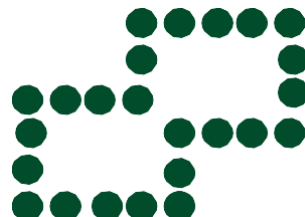
四个方向运动



八个方向运动



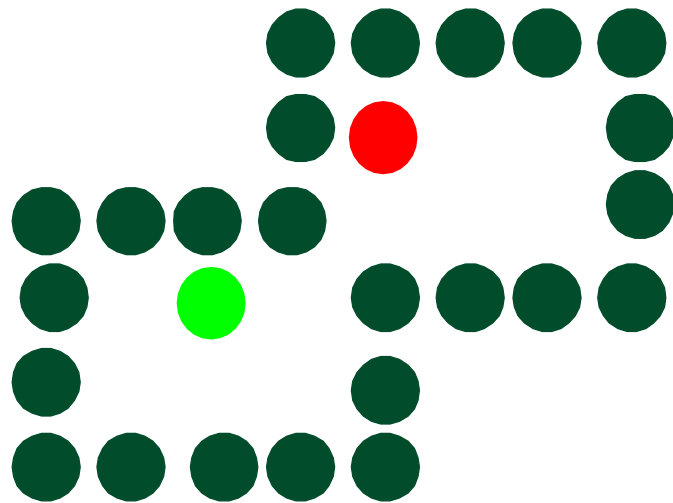
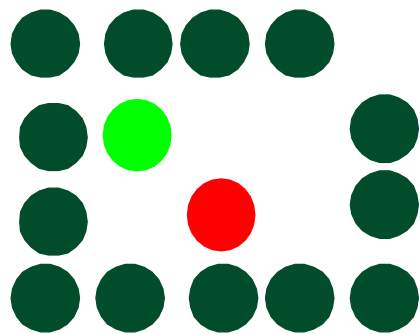
四连通区域



八连通区域

4向连通区域指的是从区域上一点出发，可通过四个方向，即上、下、左、右移动的组合，在不越出区域的前提下，到达区域内的任意象素

8向连通区域指的是从区域内每一象素出发，可通过八个方向，即上、下、左、右、左上、右上、左下、右下这八个方向的移动的组合来到达

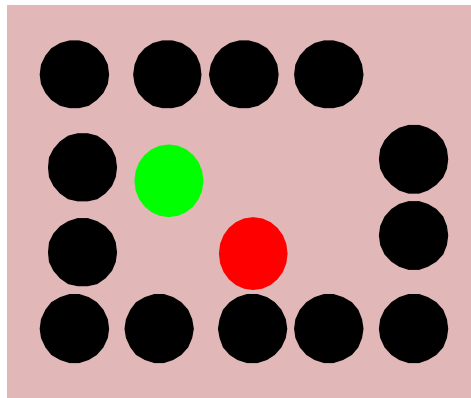


简单四连通种子填充算法（区域填充递归算法）

种子填充算法的原理是：假设在多边形区域内部有一像素已知，由此出发找到区域内的所有像素，用一定的颜色或灰度来填充

假设区域采用边界定义，即区域边界上所有像素均具有某个特定值，区域内部所有像素均不取这一特定值，而边界外的像素则可具有与边界相同的值

考虑区域的四向连通，即从区域上一点出发，可通过四个方向，即上、下、左、右移动的组合，在不越出区域的前提下，到达区域内的任意像素。

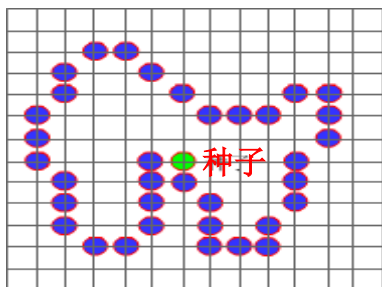
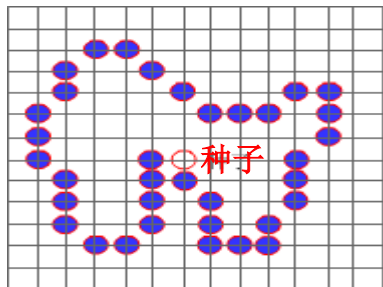


使用栈结构来实现简单的种子填充算法

算法原理如下：

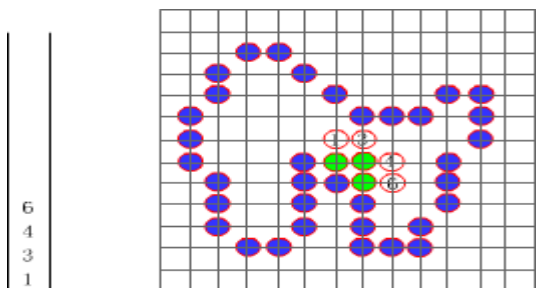
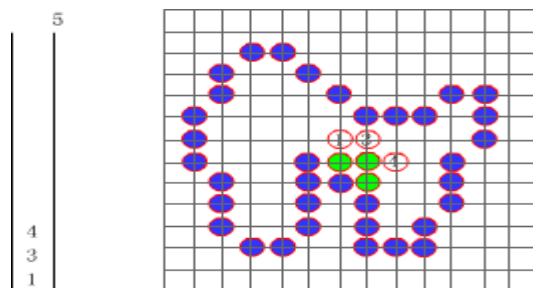
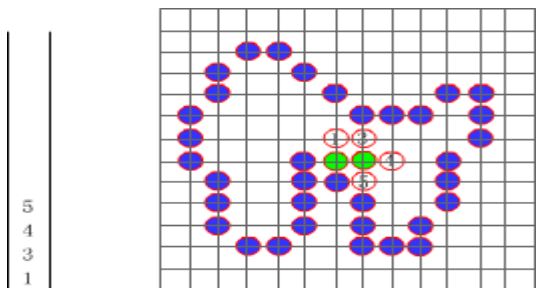
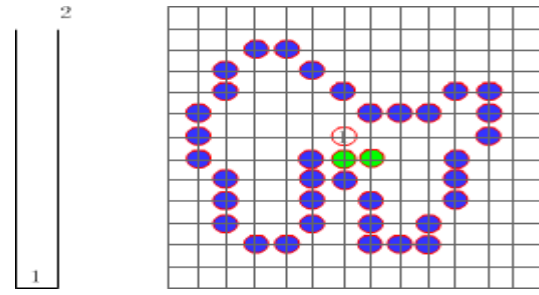
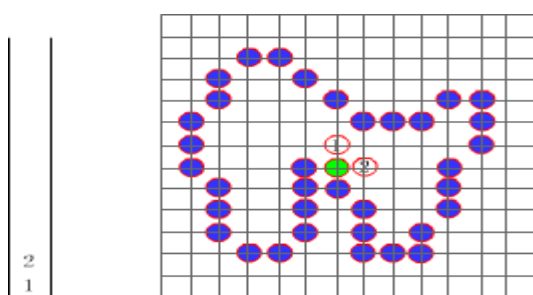
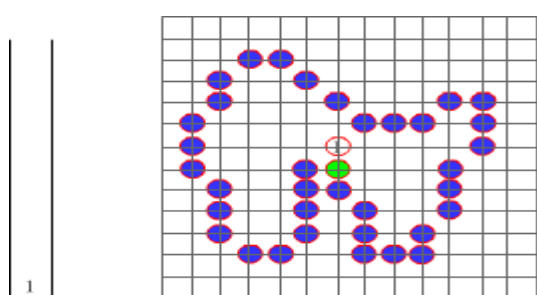
种子像素入栈，当栈非空时重复执行如下三步操作：

- (1) 栈顶像素出栈
- (2) 将出栈像素置成要填充色
- (3) 按左、上、右、下顺序检查与栈像素相邻的四个像素，若其中某个像素不在边界且未置成填充色，则把该像素入栈



种子像素入栈，栈非空时重复执行如下三步：

- (1) 栈顶像素出栈
- (2) 将出栈像素置成要填充色
- (3) 按左、上、右、下顺序检查与栈像素相邻的四个像素，若其中某个像素不在边界且未置成填充色，则把该像素入栈



种子填充算法的不足之处

- (1) 有些像素会入栈多次，降低算法效率；栈结构占空间
- (2) 递归执行，算法简单，但效率不高。区域内每一像素都引进一次递归，进/出栈，费时费内存
- (3) 改进算法，减少递归次数，提高效率

可以采用区域填充的扫描线算法

三、多边形的扫描转换与区域填充算法小结

- 基本思想不同

- 多边形扫描转换是指将多边形的顶点表示转化为点阵表示
- 区域填充只改变区域的填充颜色，不改变区域表示方法

- 基本条件不同

- 在区域填充算法中，要求给定区域内一点作为种子点，然后从这一点根据连通性将新的颜色扩散到整个区域
- 扫描转换多边形是从多边形的边界(顶点)信息出发，利用多种形式的连贯性进行填充的

扫描转换区域填充的核心是知道多边形的边界，要得到多边形内部的像素集，有多种方法。其中扫描线算法是利用一套特殊的数据结构，避免求交，然后一条条扫描线确定

区域填充条件更强一些，不但知道边界，而且还知道区域内的一点，可以利用四连通或八连通区域不断往外扩展

填充一个定义的区域的选择包括：

- 选择实区域颜色或图案填充方式
- 选择某种颜色和图案

这些填充选择可应用于多边形区域或用曲线边界定义的区域；此外，区域可用多种画笔、颜色和透明度参数来绘制

☐	题名	作者	来源	发表时间	数据库	被引	下载	预览	分享
☐ 1	多段扫描转换直线算法	祝建中	计算机辅助设计与图形学学报	2003-03-20	期刊	12	 140		
☐ 2	任意区域的边界扫描转换	余正生; 马利庄; 彭群生	工程图学报	1999-03-30	期刊	4	 66		
☐ 3	链码描述边界区域的扫描转换算法	陈建勋; 马恒太	武汉冶金科技大学学报	1998-12-30	期刊	3	 66		
☐ 4	并行扫描转换结构中的状态管理	殷诚信; 韩俊刚; 黄虎才	中国图象图形学报	2013-09-16	期刊		 17		
☐ 5	基于向量的多边形扫描转换方法	陈友军; 何洪英; 潘大志	智能计算机与应用	2012-10-01	期刊		 14		
☐ 6	改进的多边形扫描转换方法	张志刚; 刘小冬; 张志强; 张曰贤	计算机工程与应用	2009-02-01	期刊	5	 138		
☐ 7	一个基于扫描转换的图像格网处理通用算法	凌海滨; 吴兵	计算机辅助设计与图形学学报	2001-03-01	期刊	5	 71		

☐	题名	作者	来源	发表时间	数据库	被引	下载	预览	分享
☐ 1	区域填充极点判别算法	吴章文; 杨代伦; 勾成俊; 罗正明	计算机辅助设计与图形学学报	2003-08-20	期刊	25	225		
☐ 2	区域填充算法的研究	秦晓薇	赤峰学院学报(自然科学版)	2011-06-25	期刊	7	239		
☐ 3	扩充堆栈结构的种子点区域填充算法	倪玉山; 林德生	复旦学报(自然科学版)	2000-02-29	期刊	23	155		
☐ 4	压入区段端点的区域填充扫描线算法	柳朝阳; 李叔梁	计算机辅助设计与图形学学报	1996-12-25	期刊	29	142		
☐ 5	一种基于缝隙码的区域填充算法	陈优广; 顾国庆; 王玲	中国图象图形学报	2007-11-15	期刊	6	130		
☐ 6	新区入栈的区域填充扫描线算法	张荣国; 刘焜	计算机工程	2006-03-05	期刊	14	155		
☐ 7	在MapInfo中实现等值线图区域填充的快速算法	李井冈; 姚运生; 李贤华; 董曼; 李胜乐	计算机工程与设计	2009-04-16	期刊	6	180		

计算机图形学

第二章：光栅图形学算法

光栅图形学算法的研究内容

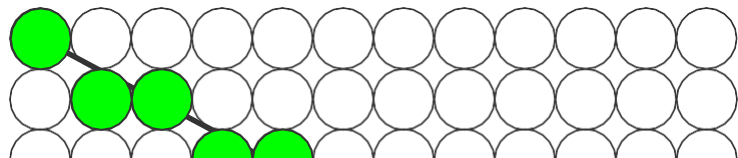
- 直线段的扫描转换算法
- 多边形的扫描转换与区域填充算法
- 直线裁剪算法
- 反走样算法
- 消隐算法

对直线、圆及椭圆这些最基本元素的生成速度和显示质量的改进，在图形处理系统中具有重要的应用价值

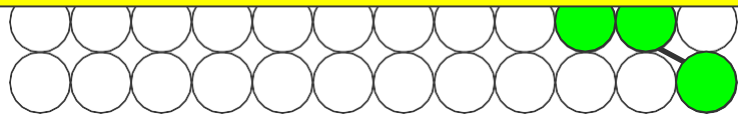
但它们生成的线条具有明显的“锯齿形”即会发生走样（Liasing）现象

一、走样

P_1



走样是数字化的必然产物!



P_0

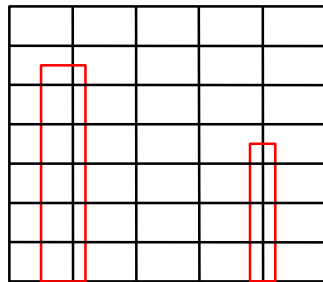
“锯齿”是“走样”（aliasing）的一种形式。而走样是光栅显示的一种固有性质。产生走样现象的原因是像素本质上的离散性

走样现象:

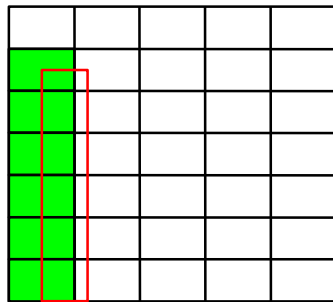
一是光栅图形产生的阶梯形（锯齿形）

二是图形中包含相对微小的物体时，
这些物体在静态图形中容易被丢弃或
忽略

小物体由于“走样”而消失

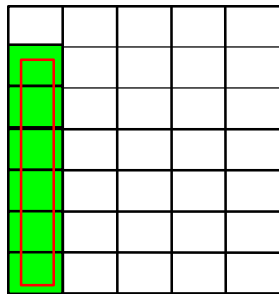


(a) 需显示的矩形

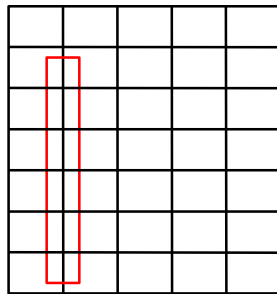


(b) 显示结果

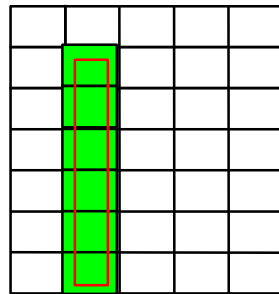
在动画序列中时隐时现，产生闪烁



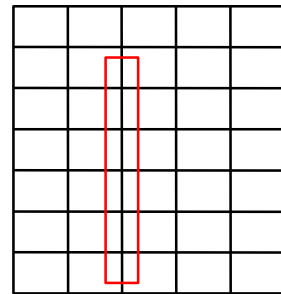
(a) 显示



(b) 不显示



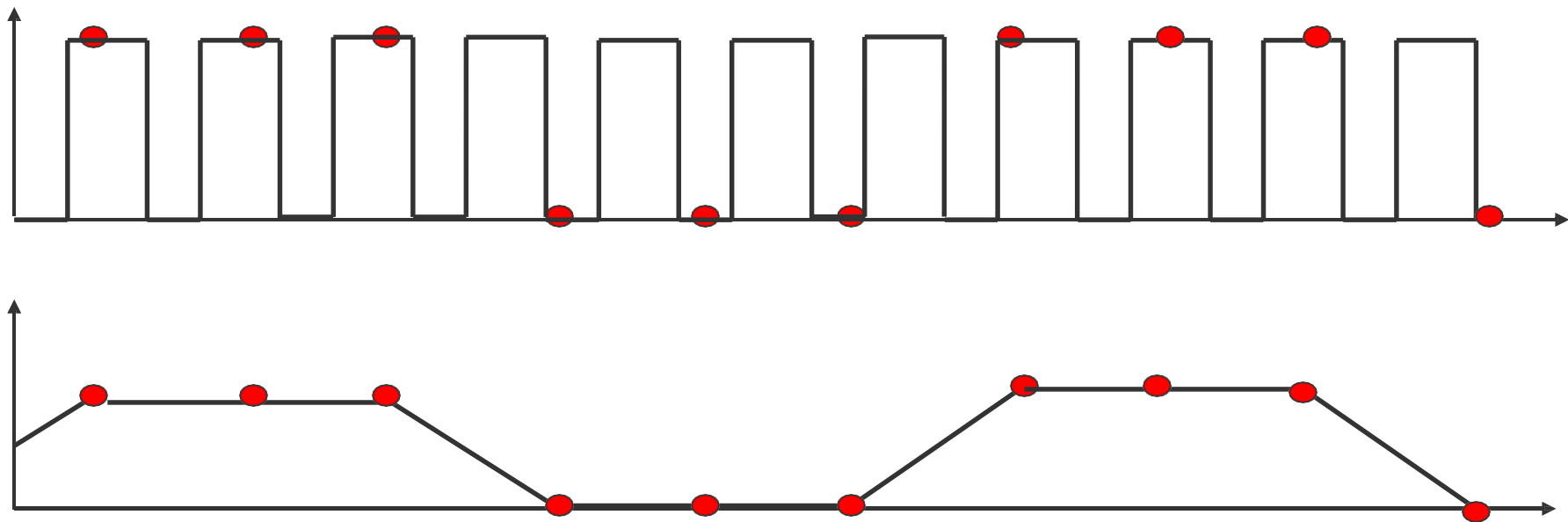
(c) 显示



(d) 不显示

矩形从左向右移动，当其覆盖某些像素中心时，矩形被显示出来，当没有覆盖像素中心时，矩形不被显示

简单地说，如果对一个快速变化的信号采样频率过低，所得样本表示的会是低频变化的信号：原始信号的频率看起来被较低的“**走样**”频率所代替



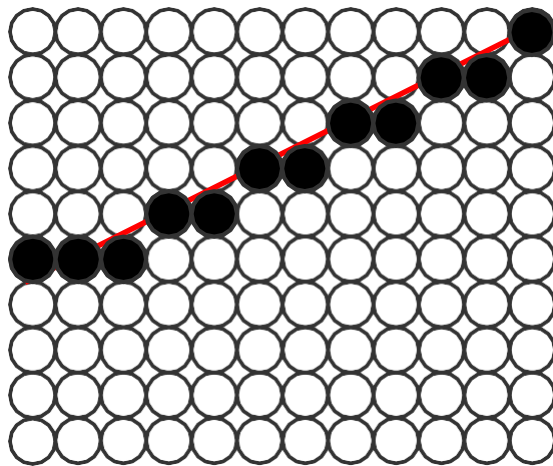
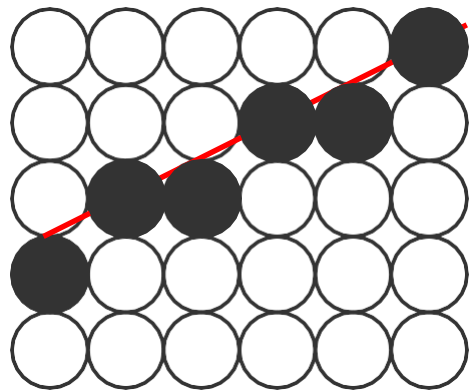
二、反走样技术

如何降低由于采样不足而产生的走样现象呢？

用于减少或消除走样效果的技术，称为反走样（Antialiasing）技术

由于图形的走样现象对图形的质量有很大影响，几乎所有图形处理系统都要对基本图形进行反走样处理

采用分辨率更高的显示设备，对解决走样现象有所帮助，
因为可以使锯齿相对物体会更小一些



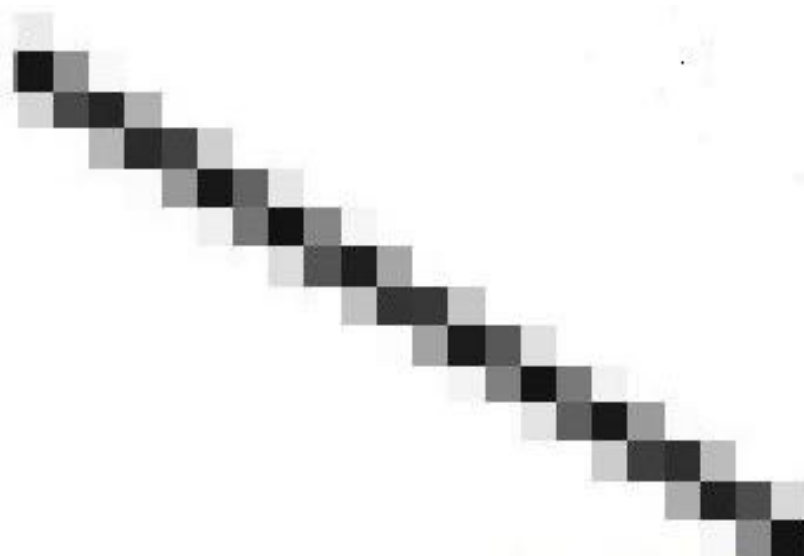
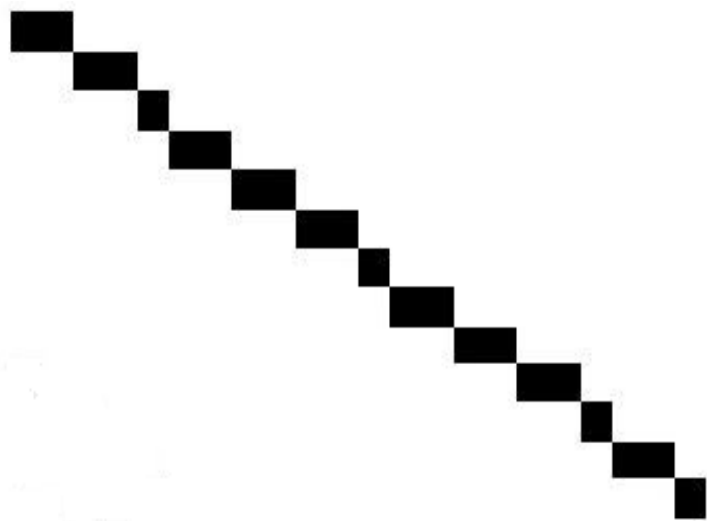
该反走样方法是以4倍的存储器代价和扫描转换时间获得的

为了稳定屏幕上的图象，电子枪至少要 $1/24$ 秒时间轰击屏幕上所有像素一次，如果像素提高一倍，电子枪就要快4倍！

反走样技术涉及到某种形式的“模糊”来产生更平滑的图像

对于在白色背景中的黑色矩形，通过在矩形的边界附近掺入一些灰色像素，可以柔化从黑到白的尖锐变化

从远处观察这幅图像时，人眼能够把这些缓和变化的暗影融合在一起，从而看到了更加平滑的边界

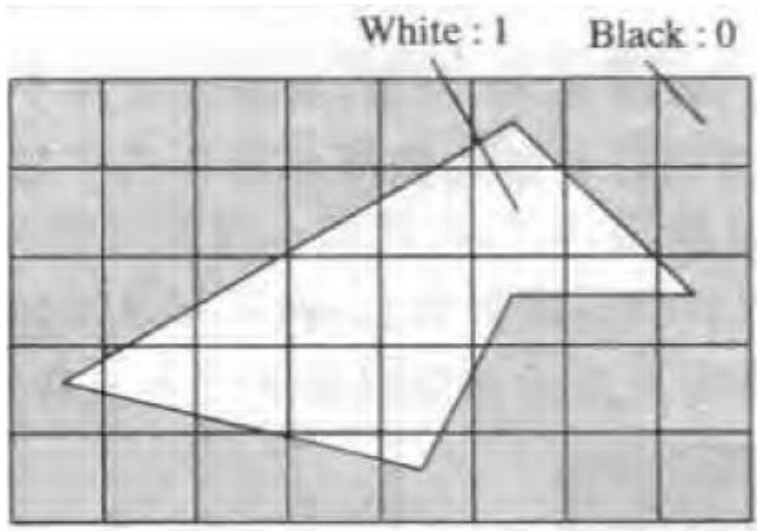


介绍两种反走样方法：

- 1、非加权区域采样方法
- 2、加权区域采样方法

1、非加权区域采样方法

根据物体的覆盖率（coverage）计算像素的颜色。覆盖率是指某个像素区域被物体覆盖的比例



0	0	0					
					15	8	
						7	

把这个多边形放在方格线中，其中每个正方形的中心对应显示器上一个像素中心。被多边形覆盖了一半的像素的亮度值赋为 $1/2$ ，覆盖三分之一的像素赋值为 $1/3$ ；以此类推

如果帧缓冲区的每个像素有4个比特位，那么0表示黑色，。。。15表示白色

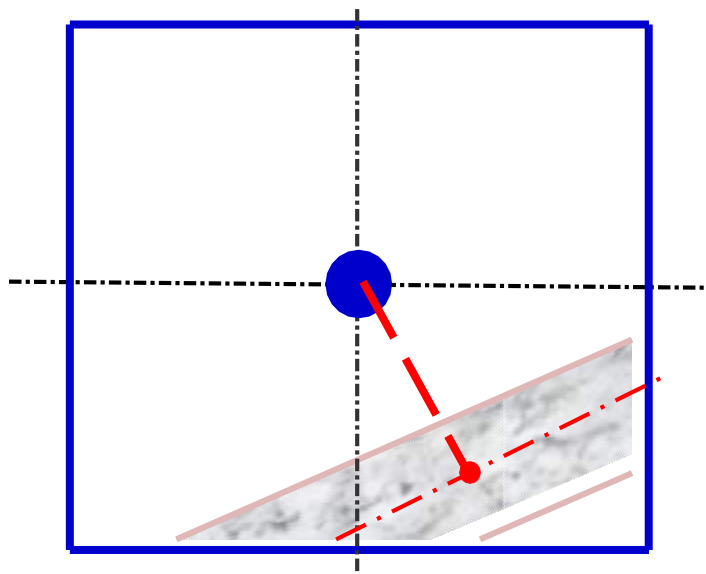
非加权区域采样方法有两个缺点：

- 1、像素的亮度与相交区域的面积成正比，而与相交区域落在像素内的位置无关，这仍然会导致锯齿效应
- 2、直线条上沿理想直线方向的相邻两个像素有时会有较大的灰度差

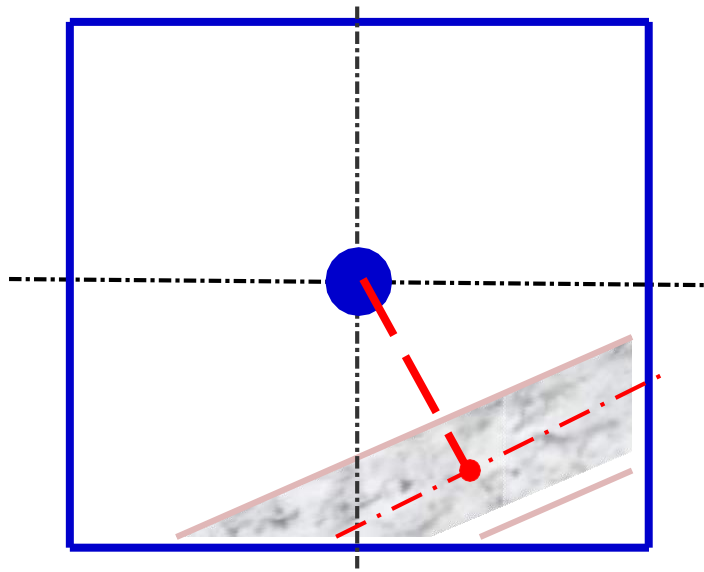
每个像素的权值是一样的，这是它的主要缺点。所以也称非加权区域采样方法

2、加权区域采样方法

这种方法更符合人视觉系统对图像信息的处理方式，反走样效果更好



将直线段看作是具有一定宽度的狭长矩形；当直线段与像素有交时，根据相交区域与像素中心的距离来决定其对象素亮度的贡献



直线段对一个像素亮度的贡献正比于相交区域与像素中心的距离

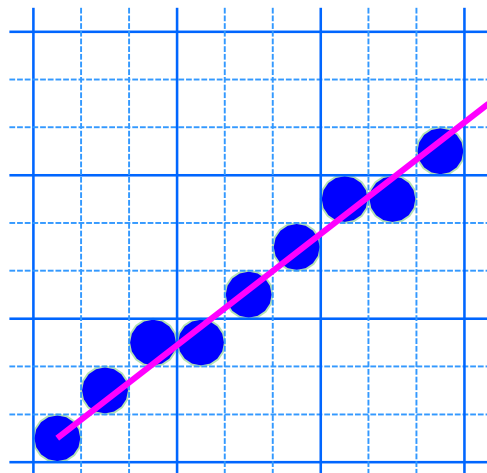
设置相交区域面积与像素中心距离的权函数（高斯函数）反映相交面积对整个像素亮度的贡献大小

利用权函数积分求相交区域面积，用它乘以像素可设置的最大亮度值，即可得到该像素实际显示的亮度值

可采用离散计算方法

将一个像素划分为 $n = 3 \times 3$ 个子像素，加权表可以取作：

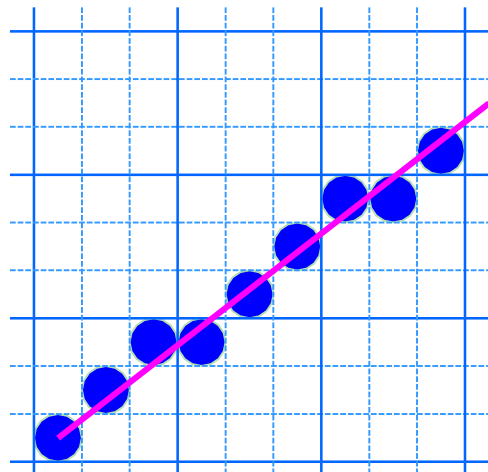
$$\begin{bmatrix} w1 & w2 & w3 \\ w4 & w5 & w6 \\ w7 & w8 & w9 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



每个像素含9个子像素

加权方案：中心子像素的加权是角子像素的4倍，是其它像素的2倍，对九个子像素的每个网格所计算出的亮度进行平均

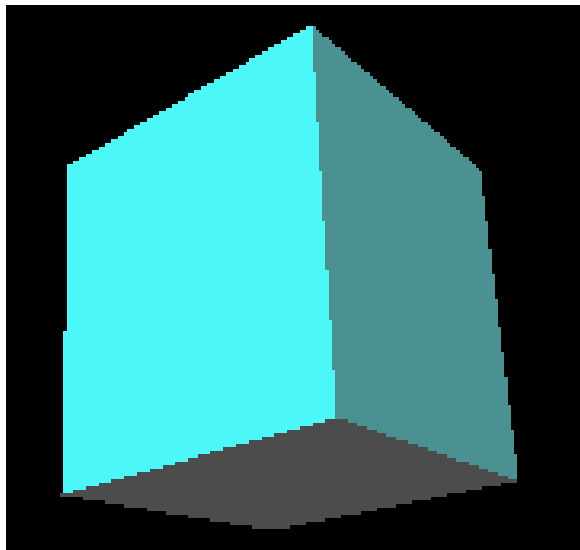
$$\begin{bmatrix} w1 & w2 & w3 \\ w4 & w5 & w6 \\ w7 & w8 & w9 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



每个像素含9个子像素

- 然后求出所有中心落于直线段内的子像素。
- 最后计算所有这些子像素对原像素亮度贡献之和

反走样是图形学中的一个根本问题，不可避免；是图形学中的一个永恒问题



原 图



反走样图

一种基于阴影图反走样的实时阴影绘制算法的研究	殷金	华中师范大学	2011-05-01	硕士	
基于OpenGL纹理映射反走样技术的研究	赵方; 张军和; 彭亚雄	电脑知识与技术	2011-06-15	期刊	
改进的实时反走样透视阴影图算法及实现	吴婷; 程亚奇; 张延军	计算机工程与设计	2011-10-16	期刊	
一种支持多线宽直线反走样算法	骆朝亮	计算机技术与发展	2010-09-10	期刊	
基于VAPS虚拟仪表反走样设计仿真实现	史志波	电子技术应用	2010-12-06	期刊	
嵌入式图像系统的改进Bresenham反走样算法的应用	张鹏; 王良	电子设计工程	2011-02-20	期刊	
基于亚像素精度的任意宽度直线反走样算法	桂丽娟; 申闫春	计算机仿真	2013-09-15	期刊	
基于光栅显示器的反走样图元生成算法研究	李晓楠; 王文义; 王春霞	郑州大学学报(工学版)	2010-09-10	期刊	

计算机图形学

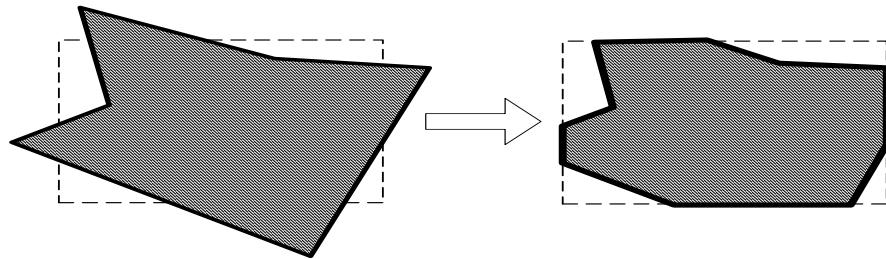
第二章：光栅图形学算法

光栅图形学算法的研究内容

- 直线段的扫描转换算法
- 多边形的扫描转换与区域填充算法
- 直线裁剪算法
- 反走样算法
- 消隐算法

一、裁剪

使用计算机处理图形信息时，计算机内部存储的图形往往比较大，而屏幕显示的只是图形的一部分。



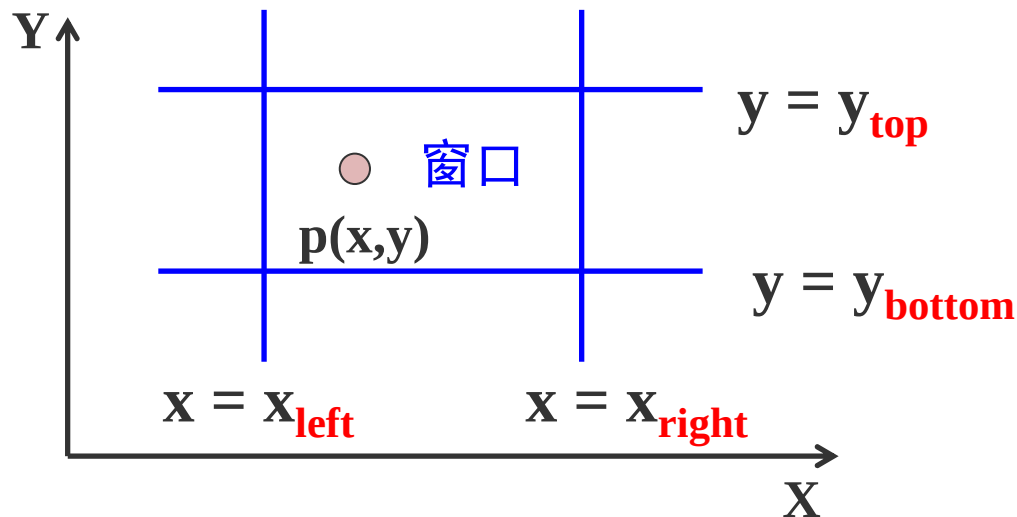
因此需要确定图形哪些部分落在显示区之内，哪些落在显示区之外。这个选择的过程就称为**裁剪**。

最简单的裁剪方法是把各种图形扫描转换为点之后，再判断点是否在窗口内。

1、点的裁剪

对于任意一点P (x, y) ,
若满足下列两对不等式:

$$\begin{cases} x_{left} \leq x \leq x_{right} \\ y_{bottom} \leq y \leq y_{top} \end{cases}$$



则点P在矩形窗口内；否则，点P在矩形窗口之外

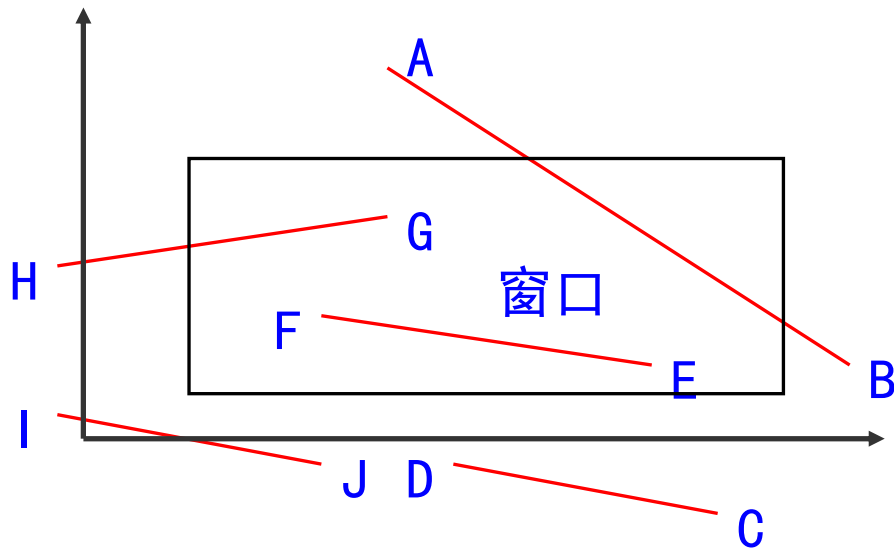
判断图形中每个点是否在窗口内，太费时，一般不可取

2、直线段的裁剪

直线段裁剪算法复杂图形裁剪的基础

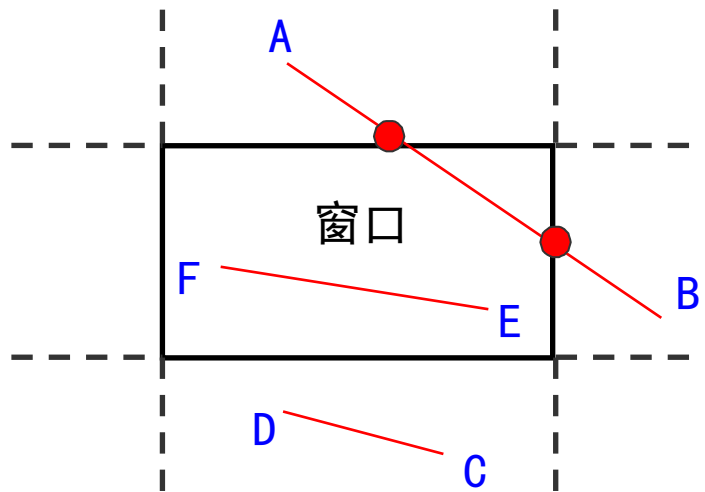
直线段和剪裁窗口的可能关系：

- 完全落在窗口内
- 完全落在窗口外
- 与窗口边界相交



要裁剪一条直线段，首先要判断：

- (1) 它是否完全落在裁剪窗口内？
- (2) 它是否完全在窗口外？
- (3) 如果不满足以上两个条件，则计算它与一个或多个裁剪边界的交点

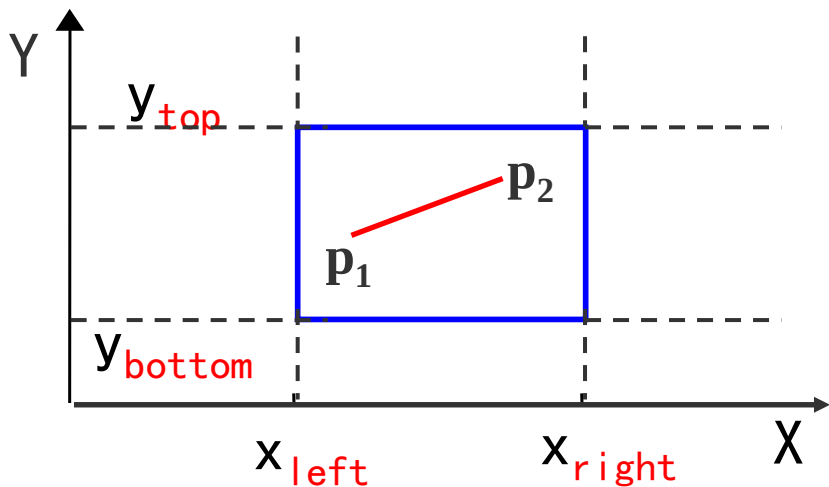


常用的裁剪算法有三种，即Cohen-Sutherland、中点分割法和Liang-Barsky裁剪算法

1、Cohen-Sutherland 算法

本算法又称为编码裁剪算法，算法的基本思想是对每条直线段分三种情况处理：

(1) 若点 p_1 和 p_2 完全在裁剪窗口内



“简取”之

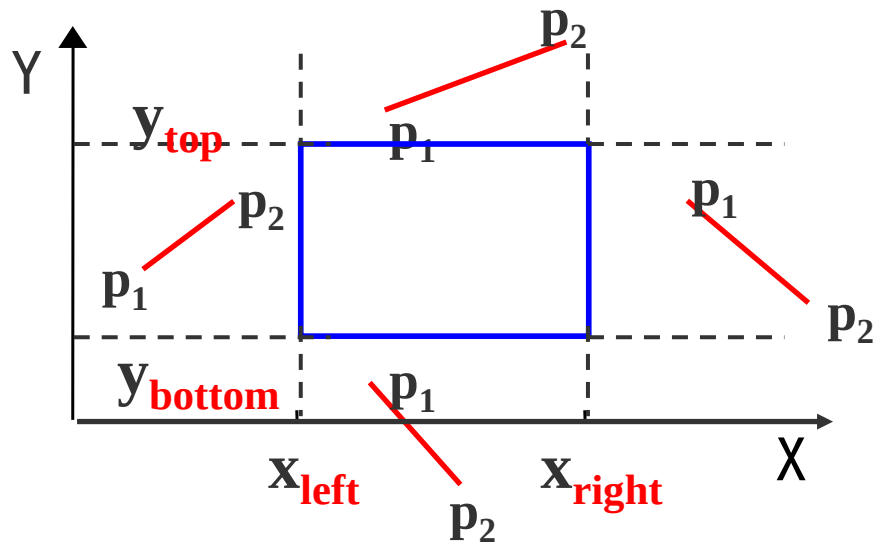
(2) 若点 $p_1(x_1, y_1)$ 和 $p_2(x_2, y_2)$ 均在窗口外，且满足下列四个条件之一：

$$x_1 < x_{left} \text{ 且 } x_2 < x_{left}$$

$$x_1 > x_{right} \text{ 且 } x_2 > x_{right}$$

$$y_1 < y_{bottom} \text{ 且 } y_2 < y_{bottom}$$

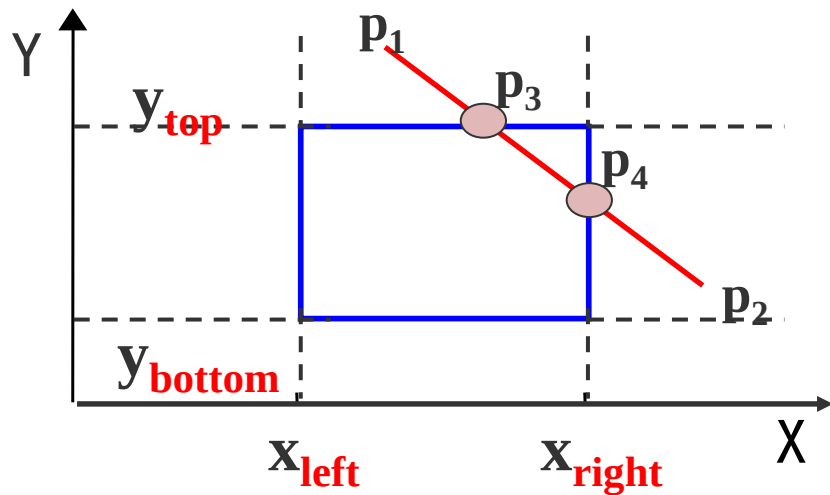
$$y_1 > y_{top} \text{ 且 } y_2 > y_{top}$$



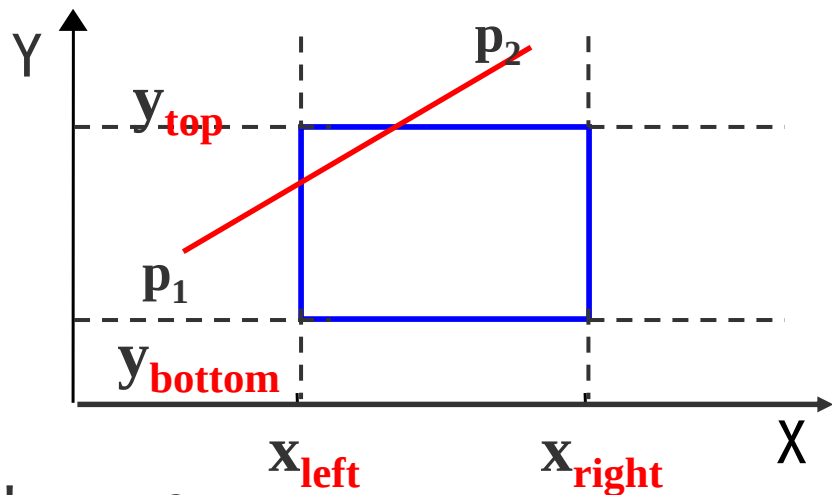
对这四种类型的直线，“简弃”之

(3) 如果直线段既不满足“简取”的条件，也不满足“简弃”的条件？

需要对直线段按交点进行分段，分段后判断直线是“简取”还是“简弃”。



每条线段的端点都赋以四位二进制码 $D_3D_2D_1D_0$ ，编码规则如下：



- 若 $x < x_{\text{left}}$ ，则 $D_0=1$ ，否则 $D_0=0$
- 若 $x > x_{\text{right}}$ ，则 $D_1=1$ ，否则 $D_1=0$
- 若 $y < y_{\text{bottom}}$ ，则 $D_2=1$ ，否则 $D_2=0$
- 若 $y > y_{\text{top}}$ ，则 $D_3=1$ ，否则 $D_3=0$

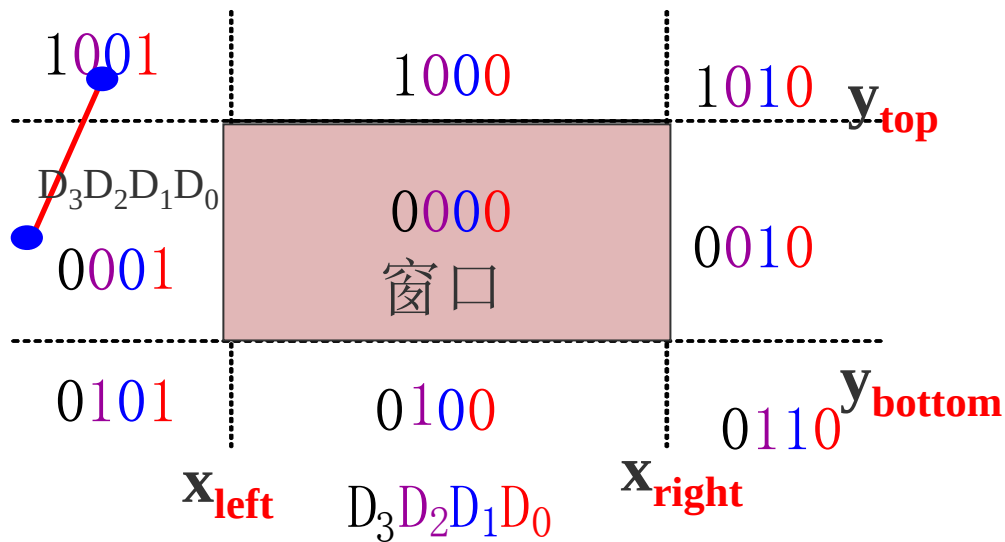
窗口及其延长线所构成了9个区域。根据该编码规则：

D_0 对应窗口左边界

D_1 对应窗口右边界

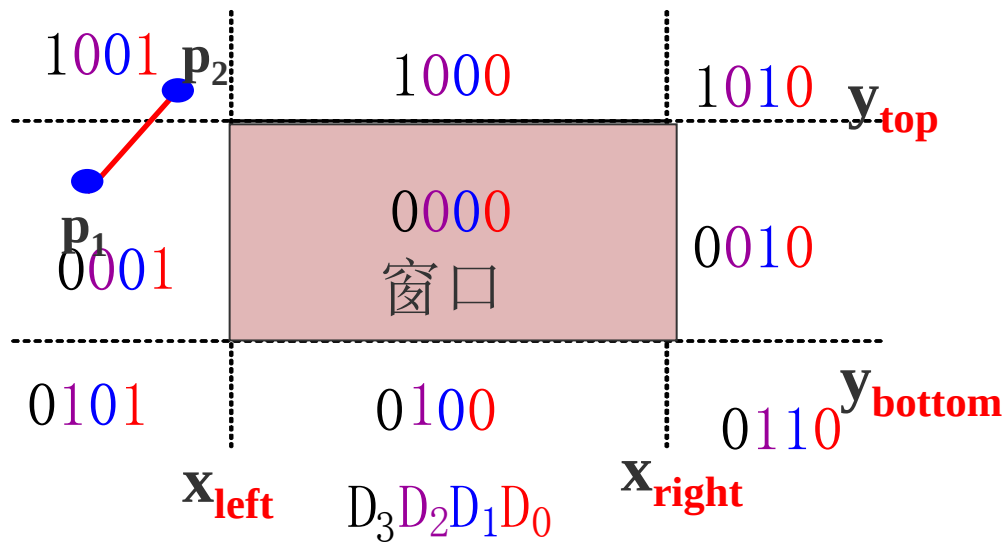
D_2 对应窗口下边界

D_3 对应窗口上边界



裁剪一条线段时，先
求出端点 p_1 和 p_2 的编
码 $code_1$ 和 $code_2$

然后进行二进制“或”
运算和“与”运算

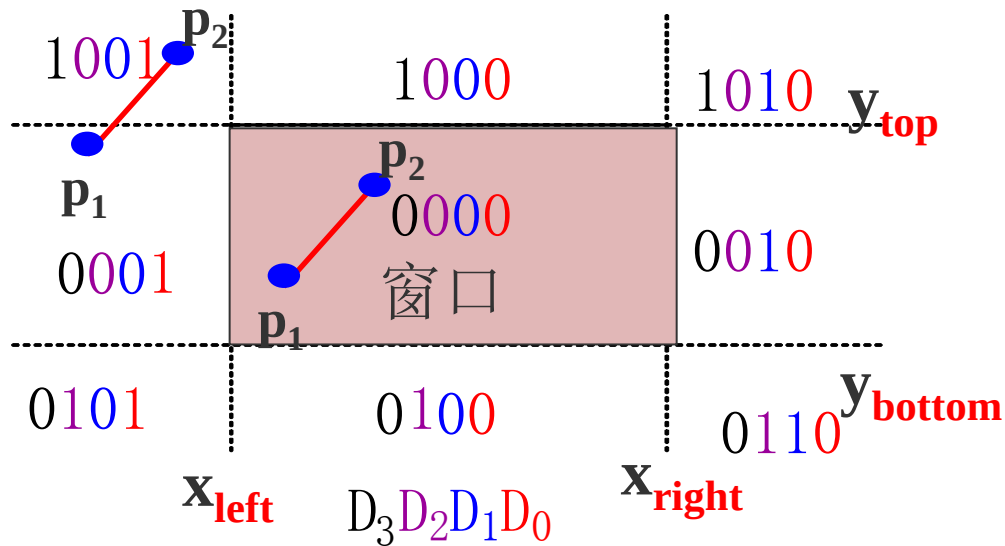


二进制运算

运算符	名称	例子	运算功能
~	位反	$\sim b$	求b的位反
&	与运算	$b \& c$	b和c位与
	或运算	$b c$	b和c位或
^	异或运算	$b \wedge c$	B和c位异或

(1) 若 $\text{code}_1 | \text{code}_2 = 0$
 , 对直线段应**简取**之

$$\begin{array}{r} \text{或} \quad 0000 \\ \quad 0000 \\ \hline 0000 \end{array}$$

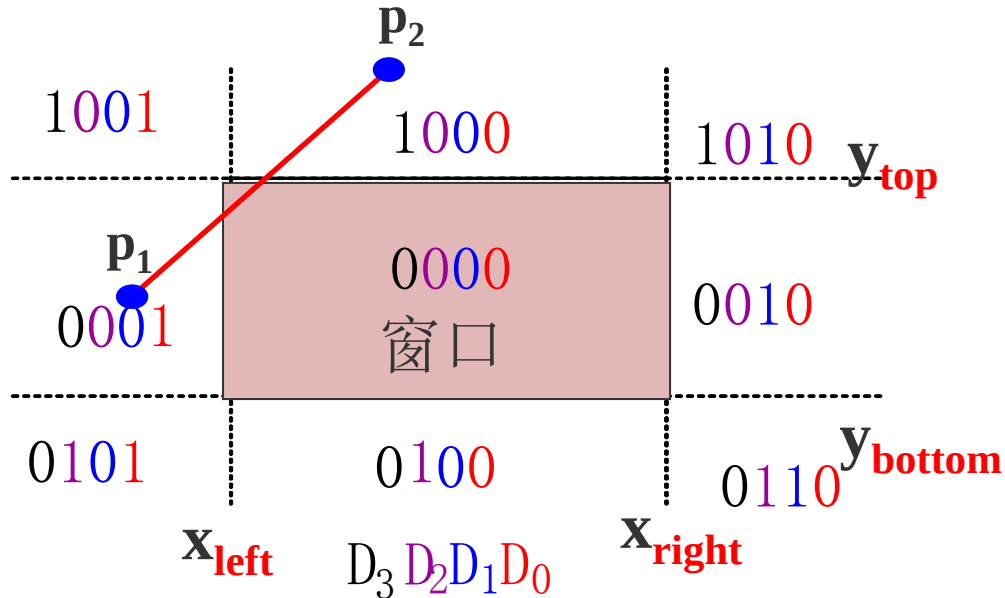


(2) 若 $\text{code}_1 \& \text{code}_2 \neq 0$, 对直线段可**简弃**之

$$\begin{array}{r} \text{与} \quad 1001 \\ \quad 0001 \\ \hline 0001 \end{array}$$

若上述两条件均不成立

或	0001	与	0001
	1000		1000
	1001		0000

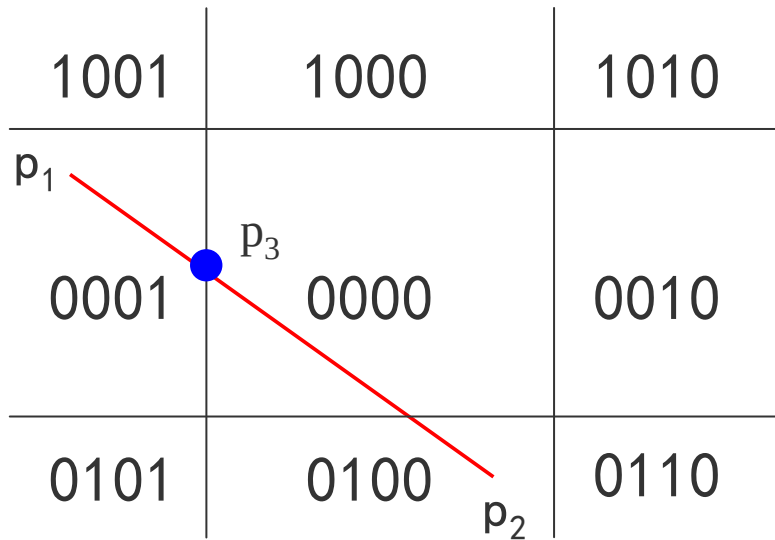


则需求出直线段与窗口边界的交点在交点处把线段一分为二

下面根据该算法步骤来裁剪如图所示的直线段 P_1P_2 ：

首先对 P_1P_2 进行编码

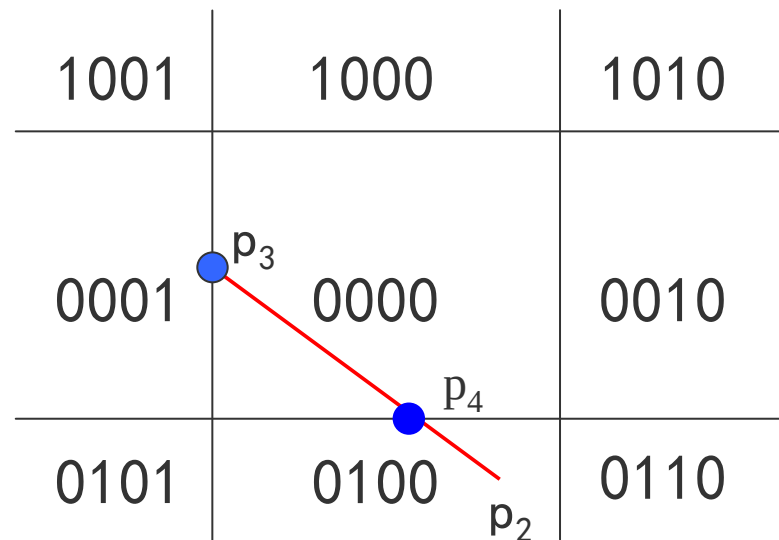
或	0001	与	0001
	0100		0100
	0101		0000



按左、右、下、上的顺序求出直线段与窗口左边界的交点为 P_3 ， P_1P_3 必在窗口外，可简弃

对 P_2P_3 重复上述处理

或	0000		0000
	0100		0100
	0100		0000



剩下的直线段 (P_3P_4) 再进行进一步判断, $code_1 | code_2 = 0$, 全在窗口中, 简取之。

小 结

Cohen-Sutherland算法用编码的方法实现了对直线段的裁剪

编码的思想在图形学中甚至在计算机科学里也是非常重要的，一个很简单的思想可以带来很了不起的作用。

比较适合两种情况：一是大部分线段完全可见；二是大部分线段完全不可见。

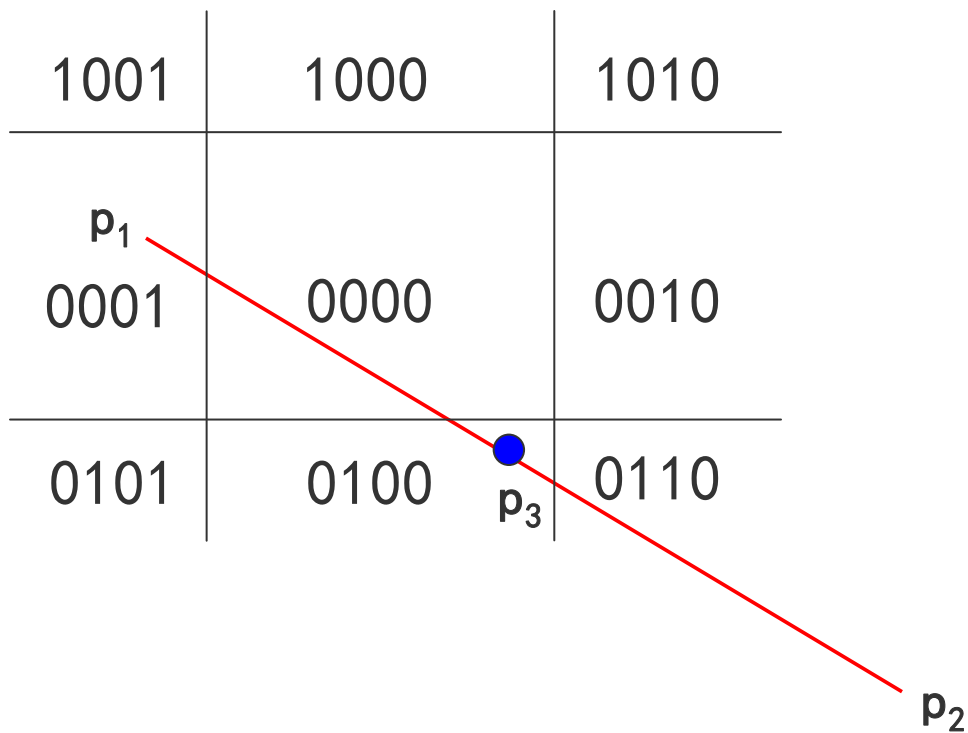
二、中点分割算法

和上面讲到的Cohen-Sutherland算法一样，首先对直线段的端点进行编码。

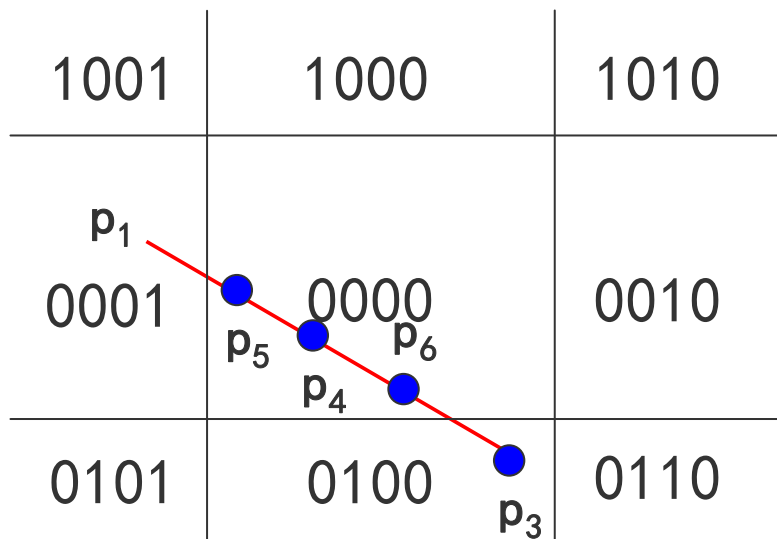
把线段和窗口的关系分成三种情况：

- 1、完全在窗口内
- 2、完全在窗口外
- 3、和窗口有交点

中点分割算法的**核心思想**是通过**二分逼近**来确定直线段与窗口的交点。

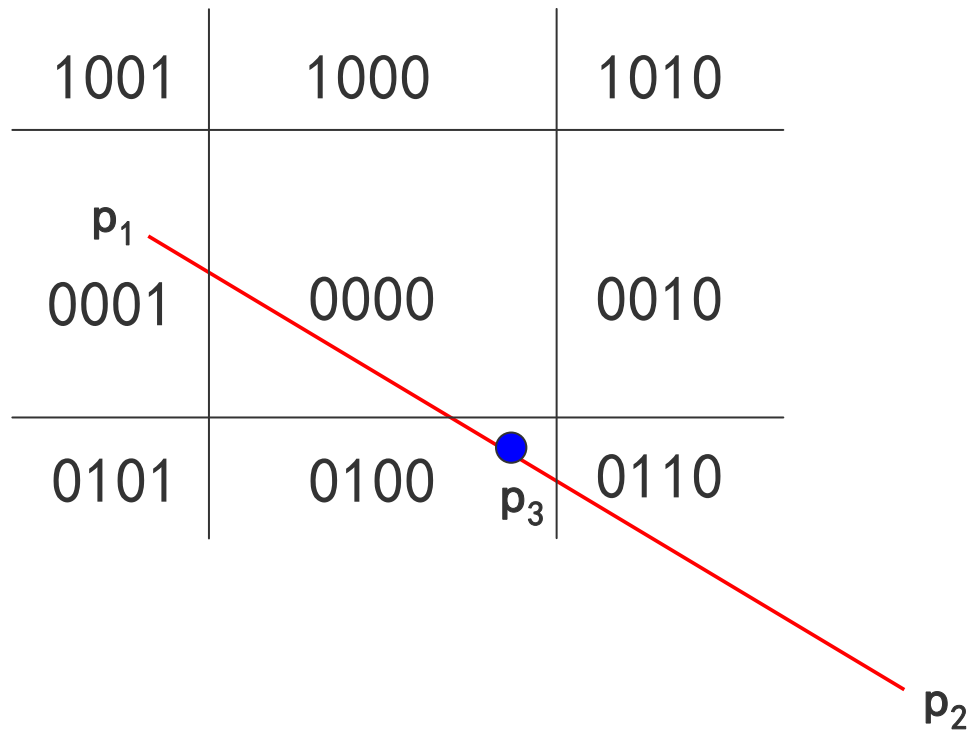


中点分割算法的**核心思想**是通过**二分逼近**来确定直线段与窗口的交点。

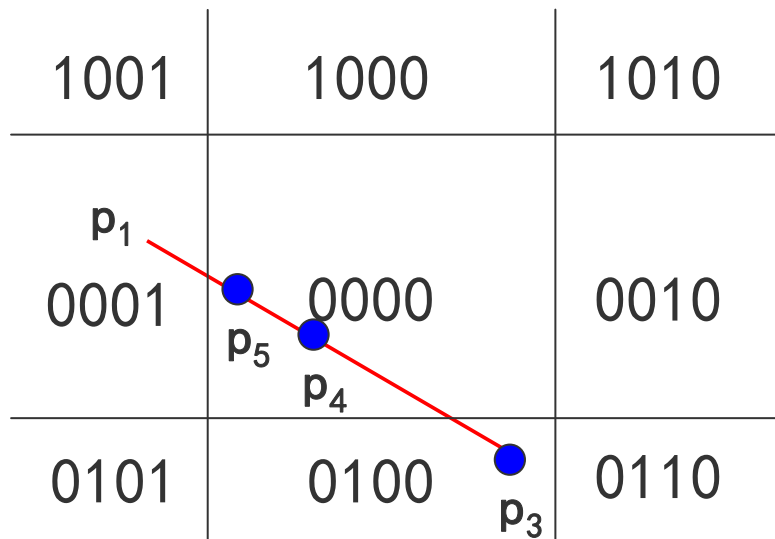


注意:

1、若中点不在窗口内，
则把中点和离窗口边界
最远点构成的线段丢掉，
以线段上的另一点
和该中点再构成线段求
其中点



2、如中点在窗口内，则又以中点和最远点构成线段，并求其中点，直到中点与窗口边界的坐标值在规定的误差范围内相等



问题：

中点分割算法会不会无限循环二分下去？

三、Liang-Barsky算法

在Cohen-Sutherland算法提出后，梁友栋和Barsky又针对标准矩形窗口提出了更快的Liang-Barsky直线段裁剪算法。

上世纪80年代，梁友栋先生提出了著名的Liang-Barsky算法，至今仍是计算机图形学中最经典的算法之一，也是写进国内外主流《计算机图形学》教科书里的唯一一个以中国人命名的算法。

You-Dong Liang; Barsky, B.A. **A new concept and method for line clipping**, ACM Transactions on Graphics, Vol.3 1-22,1984

A New Concept and Method for Line Clipping

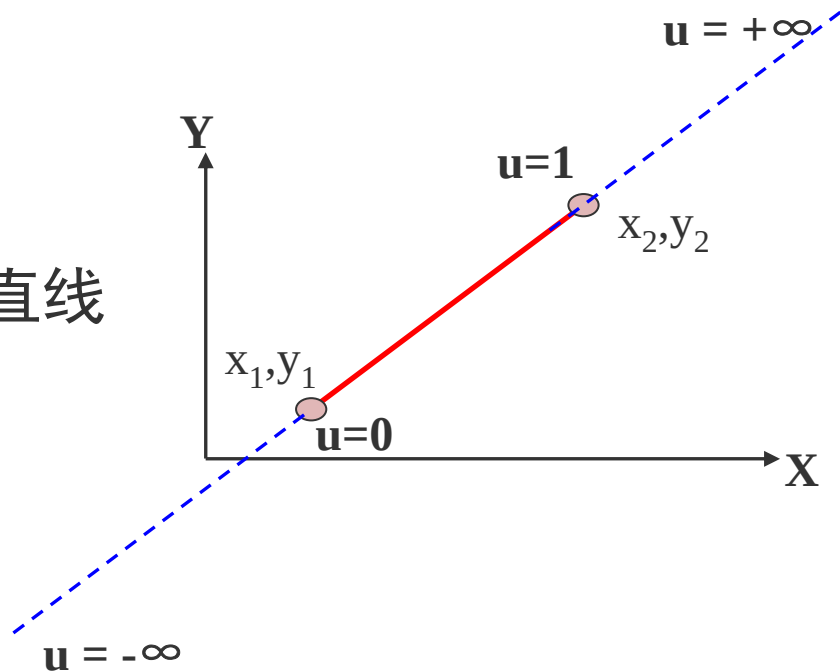
YOU-DONG LIANG and BRIAN A. BARSKY

University of California, Berkeley

A new concept and method for line clipping is developed that describes clipping in an exact and mathematical form. The basic ideas form the foundation for a family of algorithms for two-dimensional, three-dimensional, and four-dimensional (homogeneous coordinates) line clipping. The

梁算法的主要思想:

(1) 用参数方程表示一条直线



$$x = x_1 + u \cdot (x_2 - x_1) = \underline{x_1 + \Delta x \cdot u}$$

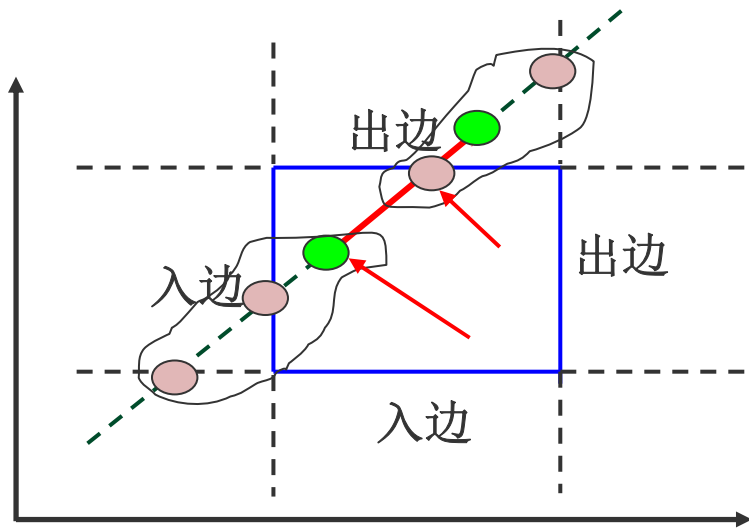
$$y = y_1 + u \cdot (y_2 - y_1) = \underline{y_1 + \Delta y \cdot u}$$

$$0 \leq u \leq 1$$

梁算法的主要思想：

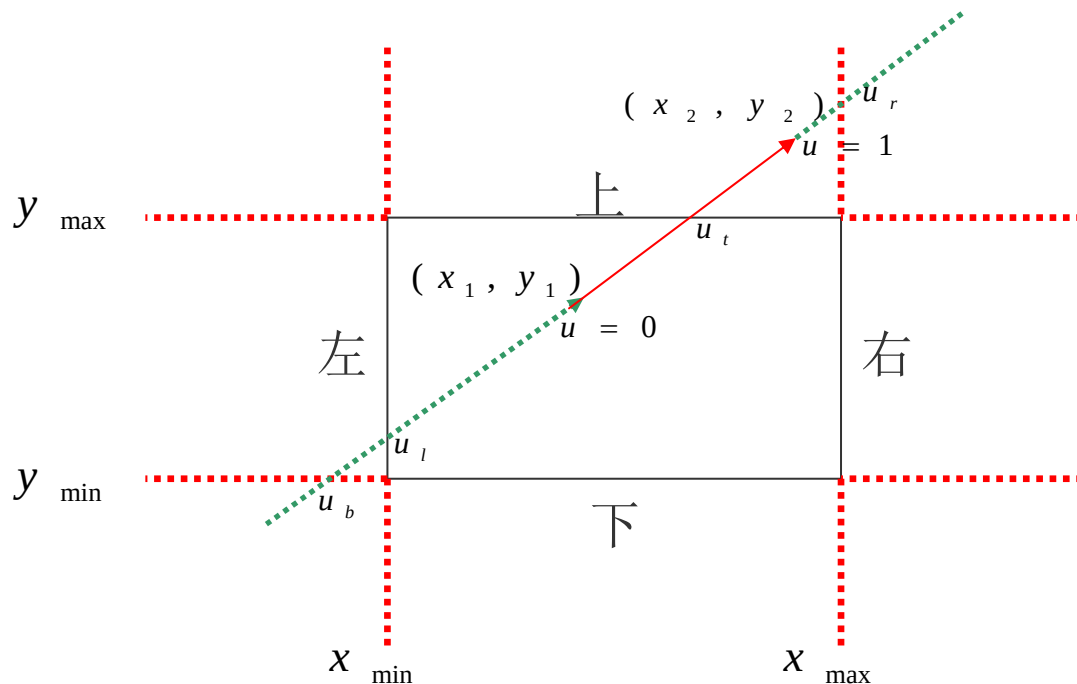
(2) 把被裁剪的红色直线段看成是一条有方向的线段，把窗口的四条边分成两类：

入边和出边



裁剪结果的线段起点是直线和两条入边的交点以及始端点三个点里最前面的一个点，即参数 u 最大的那个点；

裁剪线段的终点是和两条出边的交点以及端点最后面的一个点，取参数 u 最小的那个点。

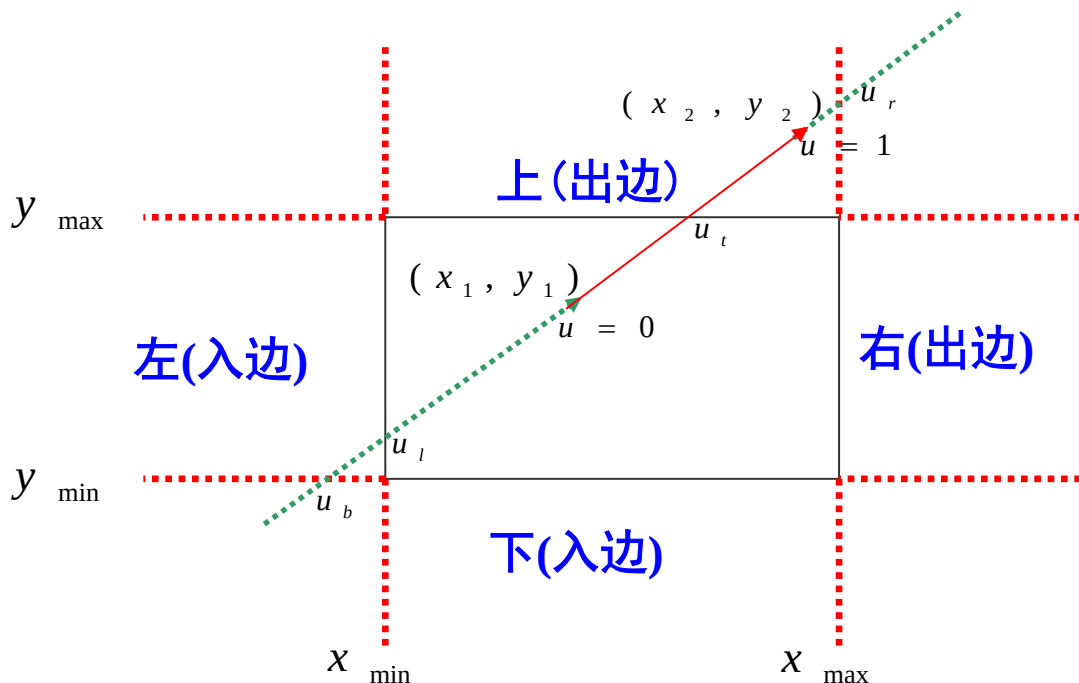


值得注意的是，当 u 从 $-\infty$ 到 $+\infty$ 遍历直线时，首先对裁剪窗口的两条边界直线（下边和左边）从外向里面移动，再对裁剪窗口两条边界直线（上边和右边）从里面向外面移动。

如果用 u_1 , u_2 分别表示
线段 ($u_1 \leq u_2$) 可见部
分的开始和结束

$$\underline{u_1 = \max(0, u_l, u_b)}$$

$$\underline{u_2 = \min(1, u_t, u_r)}$$



这就是梁先生的重大发现！

Liang-Bar sky算法的基本出发点是直线的参数方程

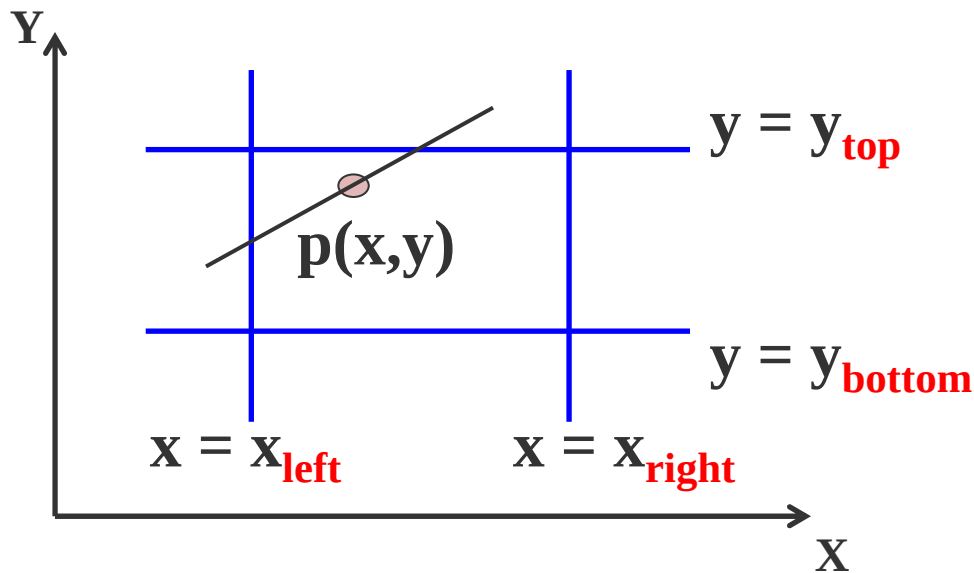
$$x = x_1 + u \cdot (x_2 - x_1)$$

$$y = y_1 + u \cdot (y_2 - y_1)$$

$$0 \leq u \leq 1$$

$$x_{\text{left}} \leq x_1 + u \cdot \Delta x \leq x_{\text{right}}$$

$$y_{\text{bottom}} \leq y_1 + u \cdot \Delta y \leq y_{\text{top}}$$



$$x_{left} \leq x_1 + u \cdot \Delta x \leq x_{right}$$

$$y_{bottom} \leq y_1 + u \cdot \Delta y \leq y_{top}$$

$$u \cdot (-\Delta x) \leq x_1 - x_{left}$$

$$u \cdot \Delta x \leq x_{right} - x_1$$

$$u \cdot (-\Delta y) \leq y_1 - y_{bottom}$$

$$u \cdot \Delta y \leq y_{top} - y_1$$

$$u \cdot (-\Delta x) \leq x_1 - x_{left}$$

$$u \cdot \Delta x \leq x_{right} - x_1$$

$$u \cdot (-\Delta y) \leq y_1 - y_{bottom}$$

$$u \cdot \Delta y \leq y_{top} - y_1$$

令:

$$p_1 = -\Delta x \quad q_1 = x_1 - x_{left}$$

$$p_2 = \Delta x \quad q_2 = x_{right} - x_1$$

$$p_3 = -\Delta y \quad q_3 = y_1 - y_{bottom}$$

$$p_4 = \Delta y \quad q_4 = y_{top} - y_1$$

令：

$$\begin{array}{ll} p_1 = -\Delta x & q_1 = x_1 - x_{left} \\ p_2 = \Delta x & q_2 = x_{right} - x_1 \\ p_3 = -\Delta y & q_3 = y_1 - y_{bottom} \\ p_4 = \Delta y & q_4 = y_{top} - y_1 \end{array}$$

于是有： $u \cdot p_k \leq q_k$ 其中， $k = 1, 2, 3, 4$

$$\begin{array}{llll} p_1 = -\Delta x & q_1 = x_1 - x_{left} & p_2 = \Delta x & q_2 = x_{right} - x_1 \\ p_3 = -\Delta y & q_3 = y_1 - y_{bottom} & p_4 = \Delta y & q_4 = y_{top} - y_1 \end{array}$$

入边： 左边和下边

出边： 右边和上边

Liang-Bar sky算法的基本出发点是直线的参数方程

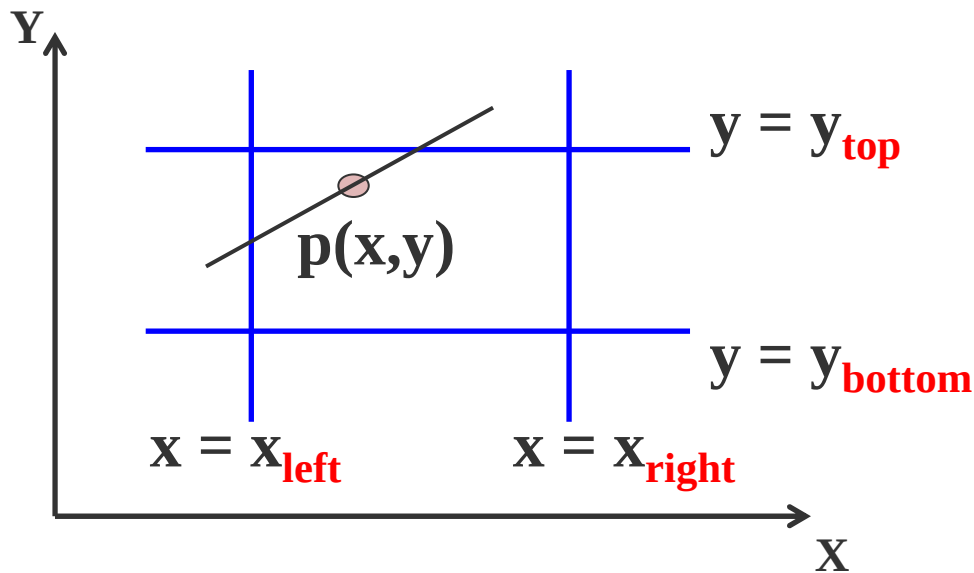
$$x = x_1 + u \cdot (x_2 - x_1)$$

$$y = y_1 + u \cdot (y_2 - y_1)$$

$$0 \leq u \leq 1$$

$$x_{left} \leq x_1 + u \cdot \Delta x \leq x_{right}$$

$$y_{bottom} \leq y_1 + u \cdot \Delta y \leq y_{top}$$



令：

$$\begin{array}{ll} p_1 = -\Delta x & q_1 = x_1 - x_{left} \\ p_2 = \Delta x & q_2 = x_{right} - x_1 \\ p_3 = -\Delta y & q_3 = y_1 - y_{bottom} \\ p_4 = \Delta y & q_4 = y_{top} - y_1 \end{array}$$

于是有： $u \cdot p_k \leq q_k$ 其中， $k = 1, 2, 3, 4$

$$\begin{array}{llll} p_1 = -\Delta x & q_1 = x_1 - x_{left} & p_2 = \Delta x & q_2 = x_{right} - x_1 \\ p_3 = -\Delta y & q_3 = y_1 - y_{bottom} & p_4 = \Delta y & q_4 = y_{top} - y_1 \end{array}$$

入边： 左边和下边

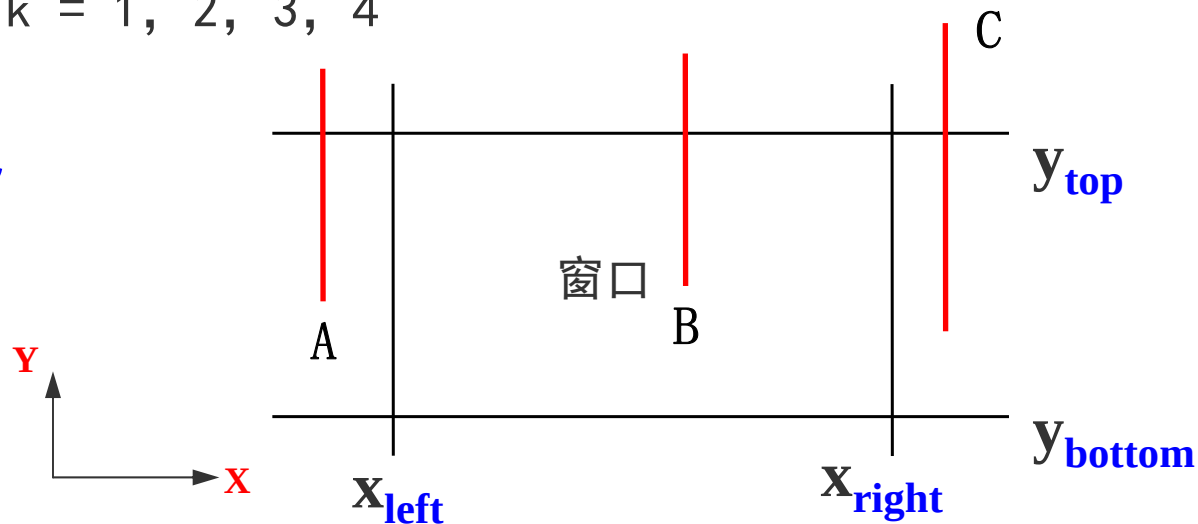
出边： 右边和上边

$$\begin{array}{ll}
 p_1 = -\Delta x & q_1 = x_1 - x_{left} \\
 p_2 = \Delta x & q_2 = x_{right} - x_1 \\
 p_3 = -\Delta y & q_3 = y_1 - y_{bottom} \\
 p_4 = \Delta y & q_4 = y_{top} - y_1
 \end{array}$$

$$u \cdot p_k \leq q_k \quad \text{其中, } k = 1, 2, 3, 4$$

(1) 分析 $P_k=0$ 的情况

$$\text{若 } P_1 = P_2 = 0$$



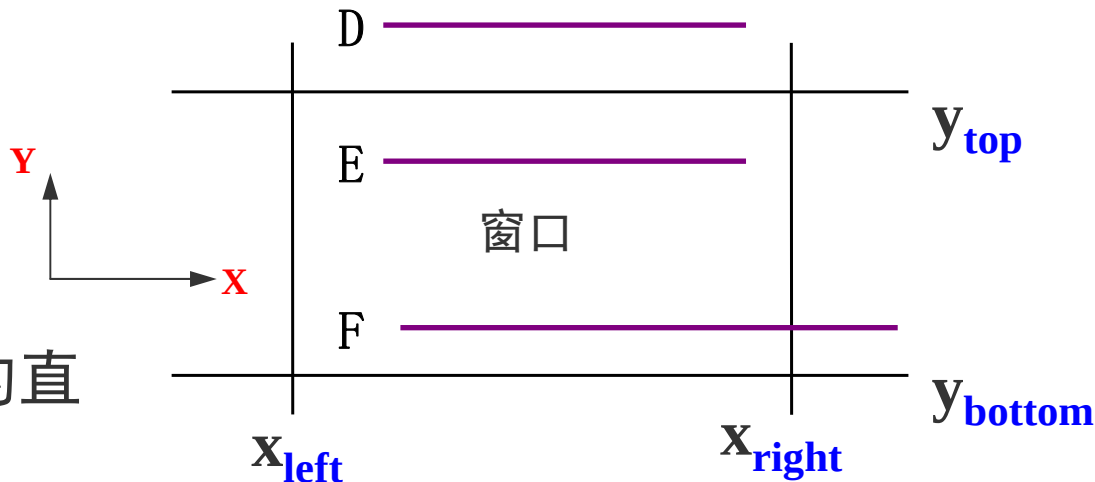
$$\begin{array}{ll}
 p_1 = -\Delta x & q_1 = x_1 - x_{left} \\
 p_2 = \Delta x & q_2 = x_{right} - x_1 \\
 p_3 = -\Delta y & q_3 = y_1 - y_{bottom} \\
 p_4 = \Delta y & q_4 = y_{top} - y_1
 \end{array}$$

$$u \cdot p_k \leq q_k \quad \text{其中, } k = 1, 2, 3, 4$$

(1) 分析 $p_k=0$ 的情况

若 $p_3=p_4=0$

任何平行于窗口某边界的直线，其 $p_k=0$



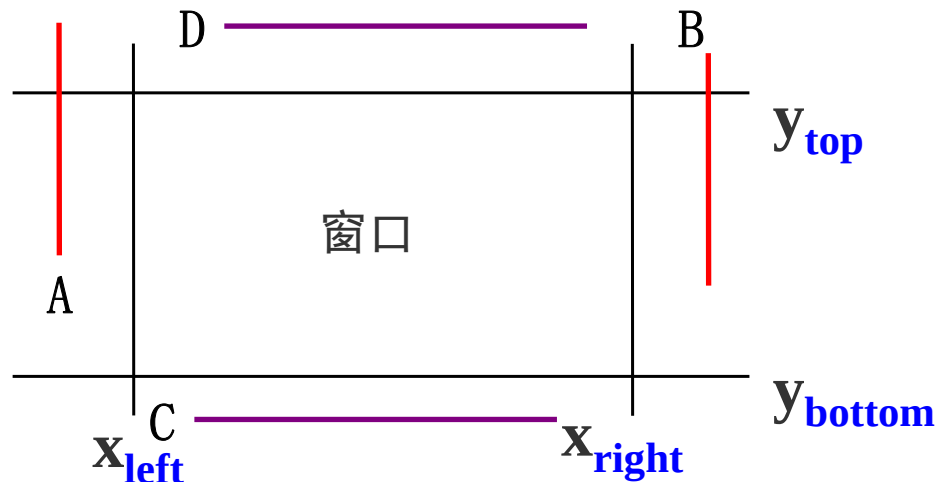
$$\begin{array}{ll}
 p_1 = -\Delta x & q_1 = x_1 - x_{left} \\
 p_2 = \Delta x & q_2 = x_{right} - x_1 \\
 p_3 = -\Delta y & q_3 = y_1 - y_{bottom} \\
 p_4 = \Delta y & q_4 = y_{top} - y_1
 \end{array}$$

$$u \cdot p_k \leq q_k \quad \text{其中, } k = 1, 2, 3, 4$$

(1) 分析 $p_k=0$ 的情况

如果还满足 $q_k < 0$

则线段完全在边界外, 应舍弃该线段



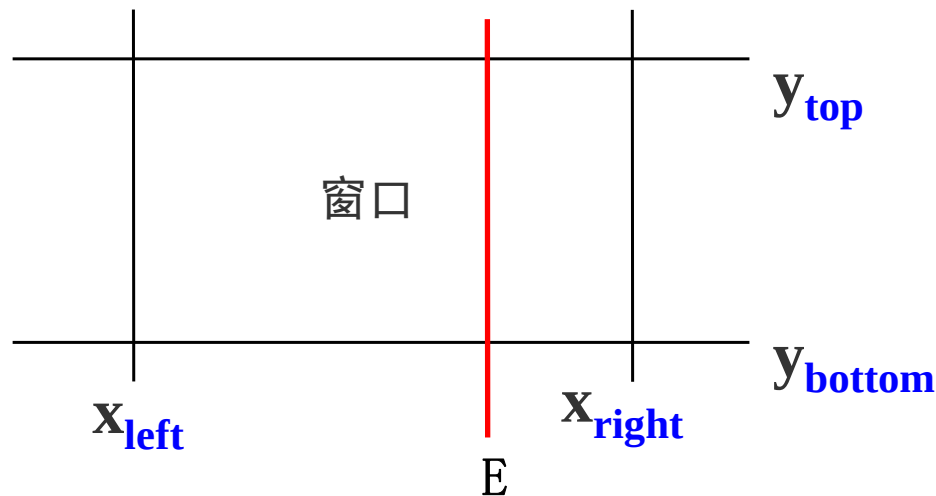
$$\begin{array}{ll}
 p_1 = -\Delta x & q_1 = x_1 - x_{left} \\
 p_2 = \Delta x & q_2 = x_{right} - x_1 \\
 p_3 = -\Delta y & q_3 = y_1 - y_{bottom} \\
 p_4 = \Delta y & q_4 = y_{top} - y_1
 \end{array}$$

$$u \cdot p_k \leq q_k \quad \text{其中, } k = 1, 2, 3, 4$$

(1) 分析 $p_k=0$ 的情况

如果 $q_k \geq 0$

则进一步判断



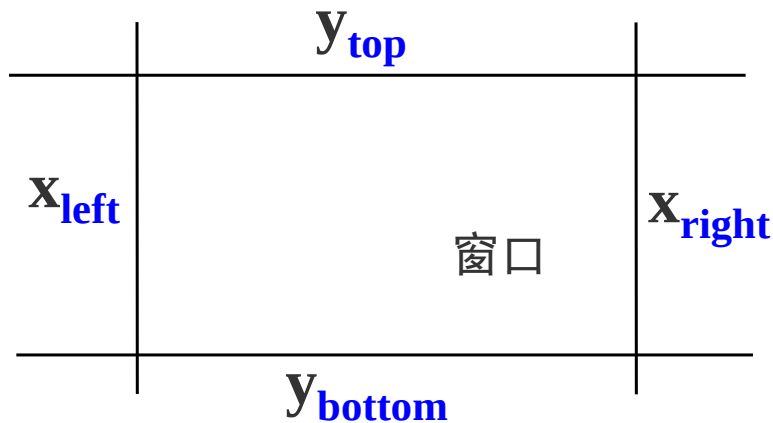
$$\begin{array}{ll}
 p_1 = -\Delta x & q_1 = x_1 - x_{\text{left}} \\
 p_2 = \Delta x & q_2 = x_{\text{right}} - x_1 \\
 p_3 = -\Delta y & q_3 = y_1 - y_{\text{bottom}} \\
 p_4 = \Delta y & q_4 = y_{\text{top}} - y_1
 \end{array}$$

$$u \cdot p_k \leq q_k \quad \text{其中, } k = 1, 2, 3, 4$$

(2) 当 $p_k \neq 0$ 时:

当 $p_k < 0$ 时

线段从裁剪边界延长线的外部延伸到内部，是入边交点



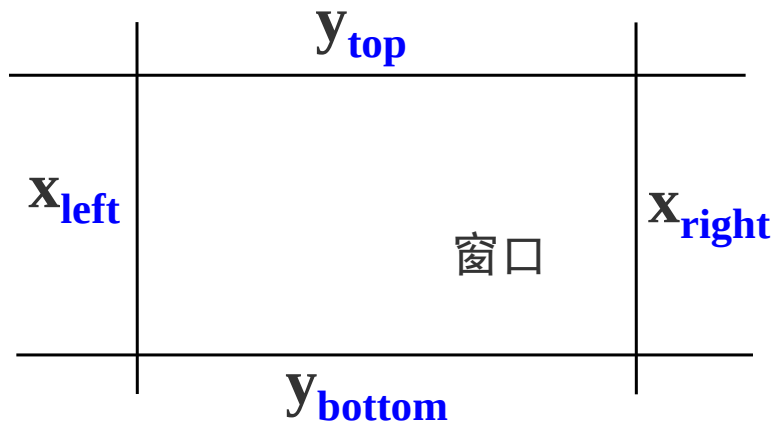
$$\begin{array}{ll}
 p_1 = -\Delta x & q_1 = x_1 - x_{\text{left}} \\
 p_2 = \Delta x & q_2 = x_{\text{right}} - x_1 \\
 p_3 = -\Delta y & q_3 = y_1 - y_{\text{bottom}} \\
 p_4 = \Delta y & q_4 = y_{\text{top}} - y_1
 \end{array}$$

$$u \cdot p_k \leq q_k \quad \text{其中, } k = 1, 2, 3, 4$$

(2) 当 $p_k \neq 0$ 时:

当 $p_k > 0$ 时

线段从裁剪边界延长线的内部
延伸到外部, 是出边交点



线段和窗口边界一共有四个交点，根据 p_k 的符号，就知道哪两个是入交点，哪两个是出交点

当 $p_k < 0$ 时：对应入边交点

当 $p_k > 0$ 时：对应出边交点

一共四个 u 值，再加上 $u=0$ 、 $u=1$ 两个端点值，总共六个值

把 $p_k < 0$ 的两个 u 值和0比较去找最大的，把 $p_k > 0$ 的两个 u 值和1比较去找最小的，这样就得到两个端点的参数值

$$u_k = \frac{q_k}{p_k} \quad (p_k \neq 0, k = 1, 2, 3, 4)$$

u_k 是窗口边界及其延长线的交点的对应参数值

分别计算 $u_{\max}$ 和 $u_{\min}$:

$$u_{\max} = \max (0, u_k|_{p_k < 0}, u_k|_{p_k < 0})$$

$$u_{\min} = \min (u_k|_{p_k > 0}, u_k|_{p_k > 0}, 1)$$

注意: $p_k < 0$, 代表入边; $p_k > 0$ 代表出边

若 $u_{\max} > u_{\min}$ ，则直线段在窗口外，删除该直线

若 $u_{\max} \leq u_{\min}$ ，将 $u_{\max}$ 和 $u_{\min}$ 代回直线参数方程，即求出直线与窗口的两实交点坐标。

$$x = x_1 + u \cdot (x_2 - x_1)$$

$$y = y_1 + u \cdot (y_2 - y_1)$$

注意：因为对于实交点 $0 \leq u \leq 1$ ，因此 $u_{\max}$ 不能小于0， $u_{\min}$ 不能大于1

下面写出Liang-Barsky裁剪算法步骤：

(1) 输入直线段的两端点坐标 (x_1, y_1) 、 (x_2, y_2) ，以及窗口的四条边界坐标： wxl 、 wxr 、 wyb 和 wyt

(2) 若 $\Delta X=0$ ，则 $p_1=p_2=0$ ，此时进一步判断是否满足 $q_1<0$ 或 $q_2<0$ ，若满足，则该直线段不在窗口内，算法转(7)-结束。否则，满足 $q_1\geq 0$ 且 $q_2\geq 0$ ，则进一步计算 $u_{\max}$ 和 $u_{\min}$ ：

$$u_{\max} = \max(0, u_k \mid p_k < 0)$$

$$u_{\min} = \min(u_k \mid p_k > 0, 1)$$

其中, $u_k = \frac{q_k}{p_k} \quad (p_k \neq 0, k = 3, 4)$ 。算法转 (5)

(3) 若 $\Delta y=0$ ，则 $p_3=p_4=0$ ，此时进一步判断是否满足 $q_3<0$ 或 $q_4<0$ ，若满足，则该直线段不在窗口内，算法转（7）。否则，满足 $q_3\geq 0$ 且 $q_4\geq 0$ ，则进一步计算 $u_{\max}$ 和 $u_{\min}$ ：

$$u_{\max} = \max(0, u_k \mid p_k < 0)$$

$$u_{\min} = \min(u_k \mid p_k > 0, 1)$$

其中， $u_k = \frac{q_k}{p_k} \quad (p_k \neq 0, k = 3, 4)$ 。算法转（5）

(4) 若上述两条均不满足, 则有 $p_k \neq 0$ ($k=1, 2, 3, 4$), 此时计

算 $u_{\max}$ 和 $u_{\min}$:

$$u_{\max} = \max (0, u_k |_{p_k < 0}, u_k |_{p_k = 0})$$

$$u_{\min} = \min (u_k |_{p_k > 0}, u_k |_{p_k = 0}, 1)$$

其中, $u_k = \frac{q_k}{p_k} \quad (p_k \neq 0, k = 1, 2, 3, 4)$

(5) 求得 $u_{\max}$ 和 $u_{\min}$ 后, 进行判断: 若 $u_{\max} > u_{\min}$, 则直线段在窗口外, 算法转 (7)。若 $u_{\max} \leq u_{\min}$, 利用直线的参数方程:

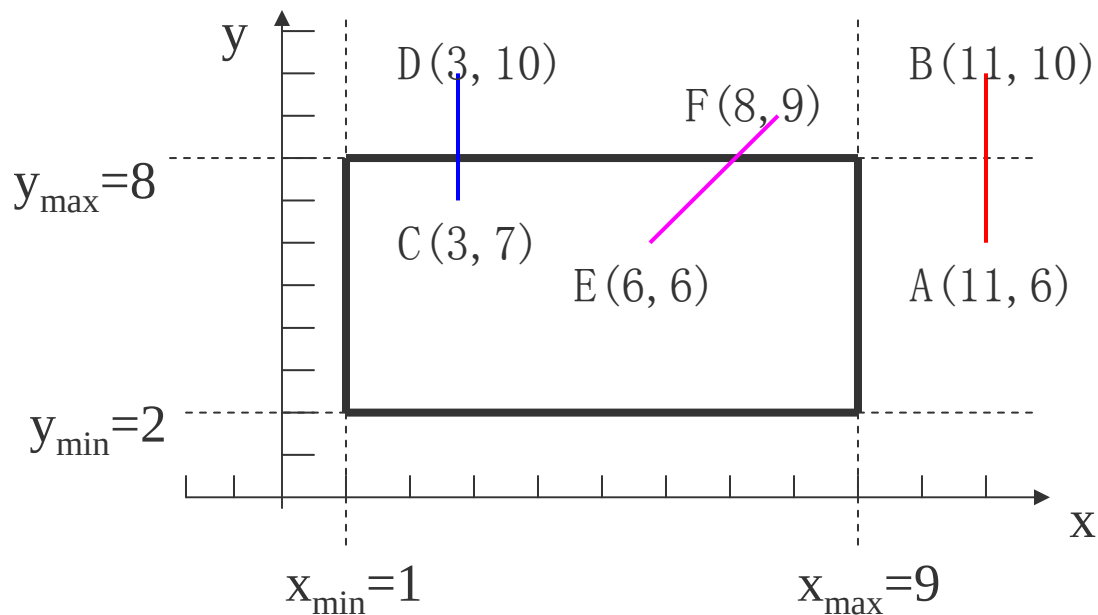
$$x = x_1 + u \cdot (x_2 - x_1)$$

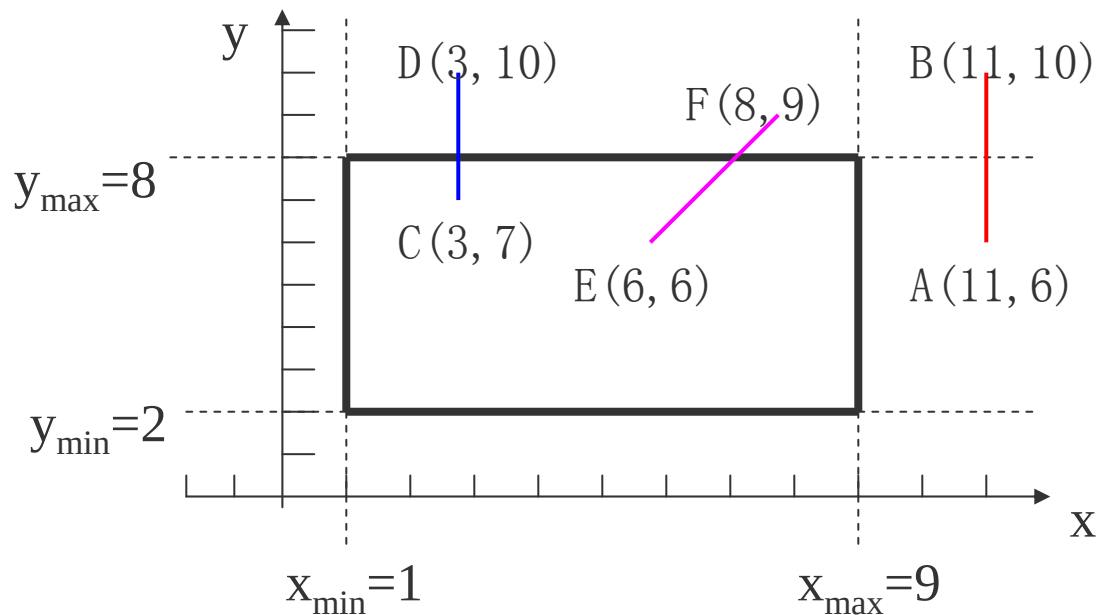
$$y = y_1 + u \cdot (y_2 - y_1)$$

(6) 利用直线的扫描转换算法绘制在窗口内的直线段。

(7) 算法结束。

用Liang-Barsky算法裁减直线段举例





令:

$$\begin{aligned}
 p_1 &= -\Delta x & q_1 &= x_1 - x_{\text{left}} \\
 p_2 &= \Delta x & q_2 &= x_{\text{right}} - x_1 \\
 p_3 &= -\Delta y & q_3 &= y_1 - y_{\text{bottom}} \\
 p_4 &= \Delta y & q_4 &= y_{\text{top}} - y_1
 \end{aligned}$$

对于直线AB, 有:

$$p_1 = 0 \quad q_1 = 10$$

$$p_2 = 0 \quad q_2 = -2$$

$$p_3 = -4 \quad q_3 = 4$$

$$p_4 = 4 \quad q_4 = 2$$

AB完全在右边界之右

对于直线CD, 有:

$$p_1 = 0 \quad q_1 = 2$$

$$p_2 = 0 \quad q_2 = 6$$

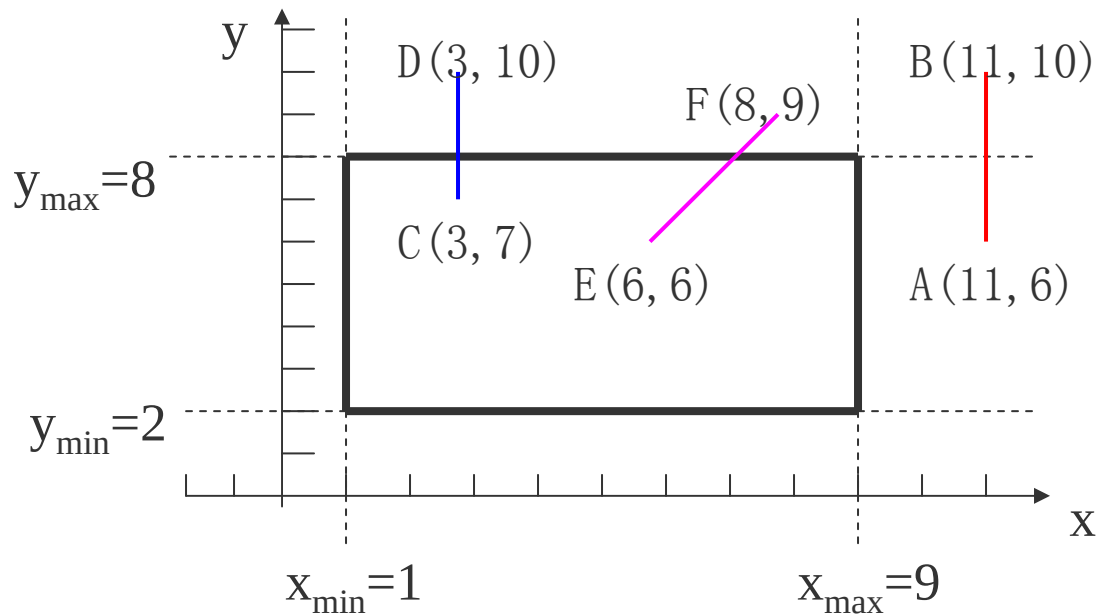
$$p_3 = -3 \quad q_3 = 5$$

$$p_4 = 3 \quad q_4 = 1$$

$$u_3 = -5/3 \quad u_4 = 1/3$$

$$u_{\max} = \max(0, -5/3) = 0$$

$$u_{\min} = \min(1, 1/3) = 1/3$$

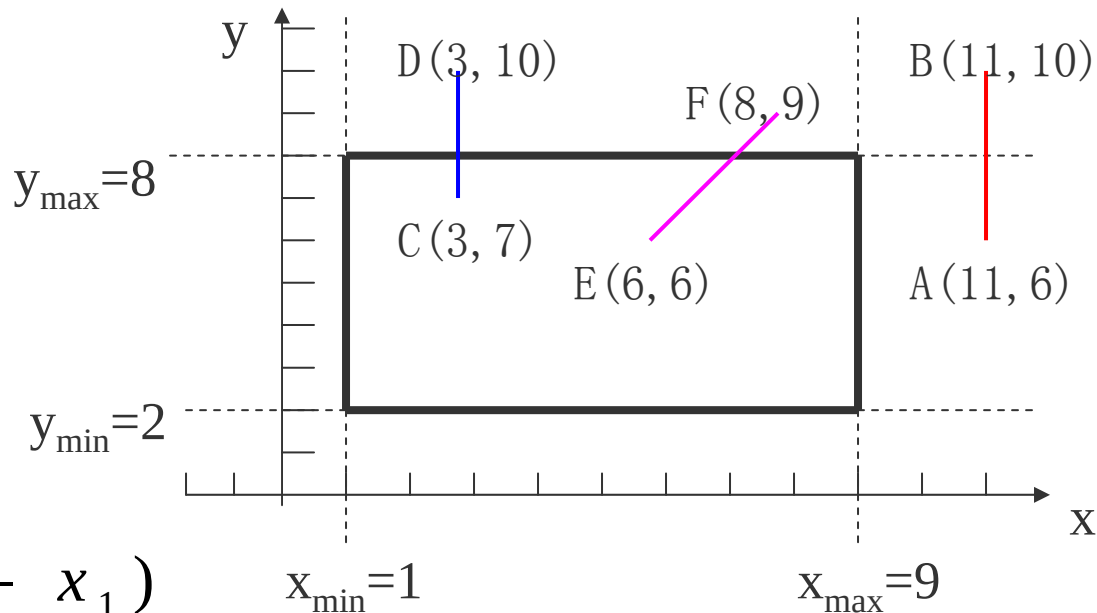


$$u_{\max} = 0$$

$$u_{\min} = 1/3$$

$$x = x_1 + u \cdot (x_2 - x_1)$$

$$y = y_1 + u \cdot (y_2 - y_1)$$



裁减后直线的两个端点是 $(3, 7)$ 和 $(3, 8)$ 。

对于直线EF，有：

$$p_1 = -2 \quad q_1 = 5$$

$$p_2 = 2 \quad q_2 = 3$$

$$p_3 = -3 \quad q_3 = 4$$

$$p_4 = 3\frac{5}{2} \quad q_4 = 3\frac{1}{2}$$

$$u_3 = -\frac{4}{3} \quad u_4 = \frac{2}{3}$$

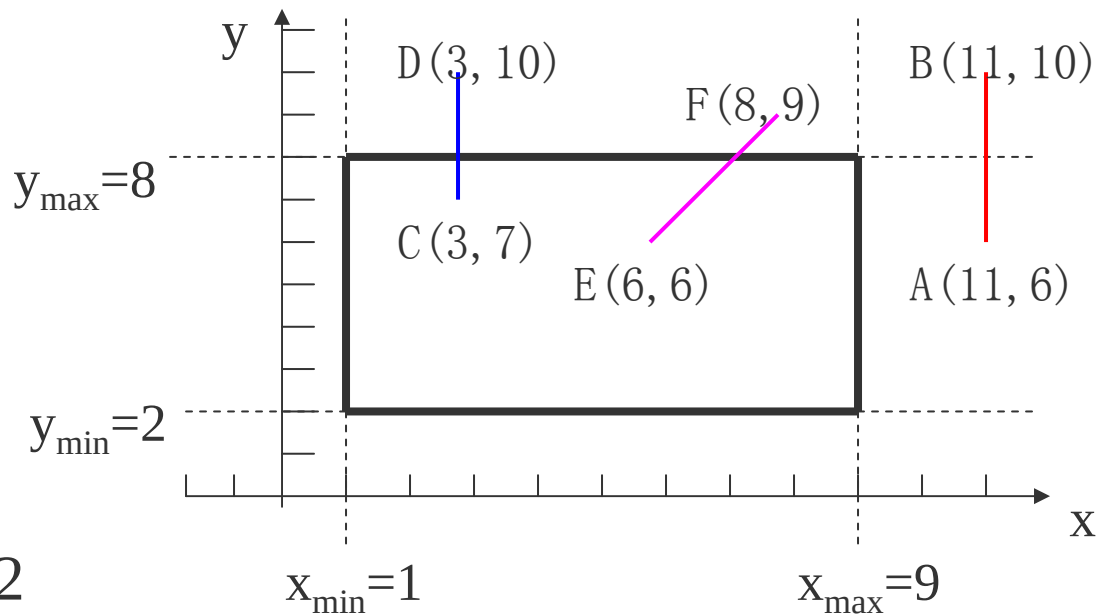
$$u_{\max} = \max(0, -\frac{5}{2}, -\frac{4}{3}) = 0$$

因此：

$$u_{\min} = \min(1, \frac{3}{2}, \frac{2}{3}) = \frac{2}{3}$$

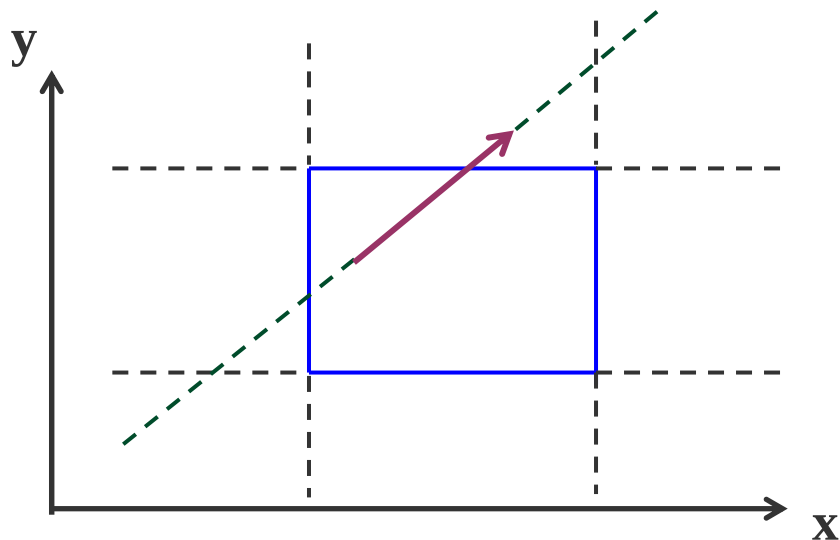
$$u_{\max} < u_{\min}$$

裁剪后的直线的两个端点是 (6, 6) 和 (7.33, 8)



Liang-Bar sky算法小结

1、 直线段看成是有方向的



2、直线参数化

$$x = x_1 + u \cdot (x_2 - x_1)$$

$$y = y_1 + u \cdot (y_2 - y_1)$$

3、判断线段上一点是否在窗口内，需满足下面两个不等式

$$x_{left} \leq x_1 + u \cdot \Delta x \leq x_{right}$$

$$y_{bottom} \leq y_1 + u \cdot \Delta y \leq y_{top}$$

$$u \cdot p_k \leq q_k$$

4、 线段和窗口边界一共有四个交点

$$u_k = \frac{q_k}{p_k} \quad (p_k \neq 0, k = 1, 2, 3, 4)$$

$$u_{\max} = \max (0, u_k|_{pk < 0}, u_k|_{pk < 0})$$

$$u_{\min} = \min (u_k|_{pk > 0}, u_k|_{pk > 0}, 1)$$

$$x = x_1 + u \cdot (x_2 - x_1)$$

$$y = y_1 + u \cdot (y_2 - y_1)$$

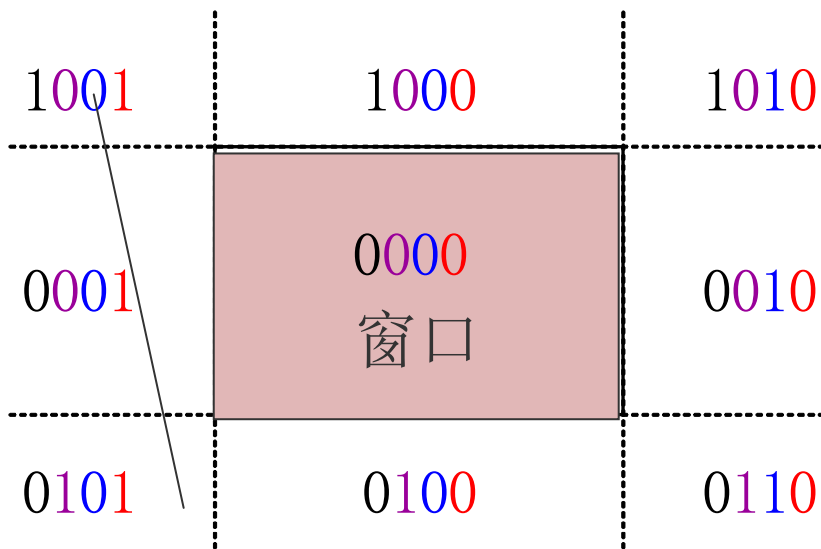
梁算法引进了一种新的思想，把窗口的4条边根据线段的方向走向分成入边和出边，裁剪的算法就变得比较简单。

它首先把线段的裁剪变成了射线的裁剪，也就是把线段赋以方向。发现最终裁剪结果的起始端点肯定是入边的两个交点和线段的起始点里面的一点；而裁剪结果的终点肯定是出边的两个交点和线段的终点里面的一点；前三个点取最大，后三个点取最小。这样就把一个经典的裁剪问题变了解不等式。

这也是中国人的算法第一次出现在了所有图形学教科书都必须提的一个算法。这就叫原始创新！

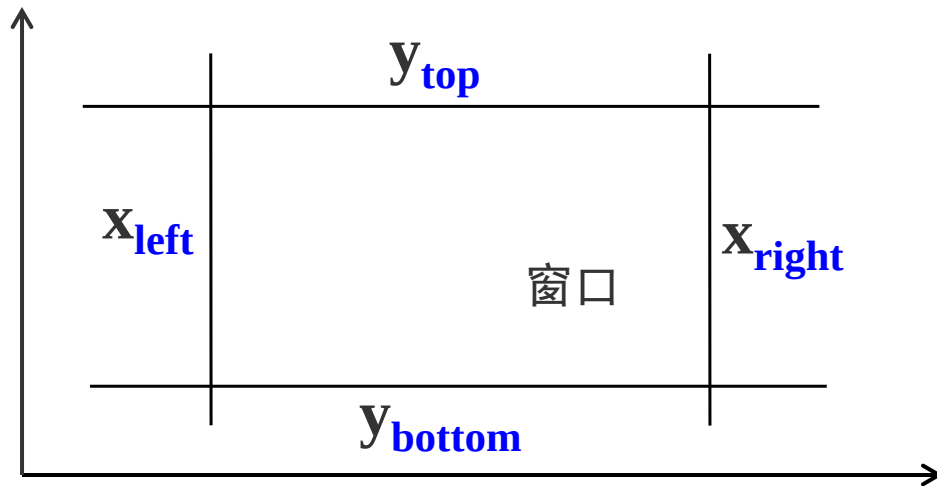
Cohen-Sutherland and Liang-Barsky 裁剪算法比较

1、Cohen-Sutherland 算法的核心思想是编码



$D_3 D_2 D_1 D_0$
窗口及区域编码

2、如果被裁剪的图形大部分线段要么在窗口内或者要么完全在窗口外，很少有贯穿窗口的。Cohen-Sutherland算法效果非常好



3、在一般情况下，Liang-Barsky裁剪算法的效率则优于Cohen-Sutherland算法

4、Cohen-Sutherland和Liang-BarSKy只能应用于矩形窗口

裁剪算法是非常底层的算法，任何一个图形显示的算法和软件都离不开这些底层算法

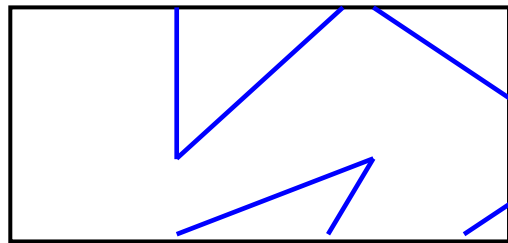
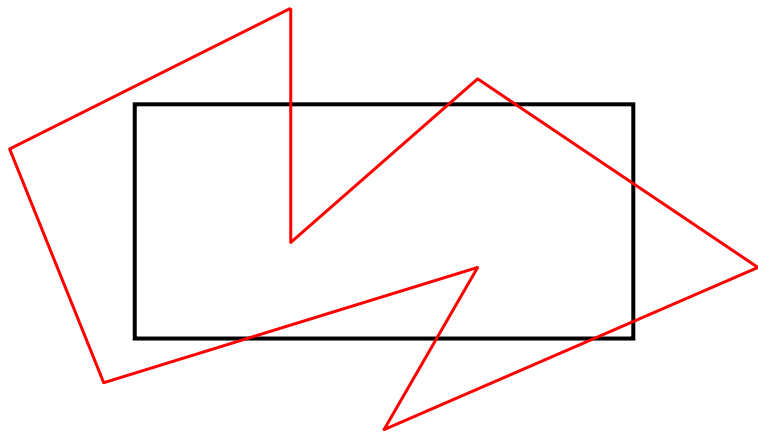
这些底层算法目前都已经固化到计算机硬件里了。现在做一个图形软件，不需要再研究裁剪算法

一是了解图形学底层算法的思想

另一方面，改进底层算法对提高图形显示和处理效率具有至关重要的作用

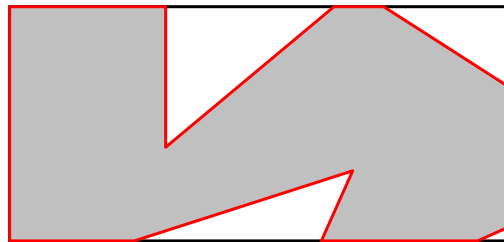
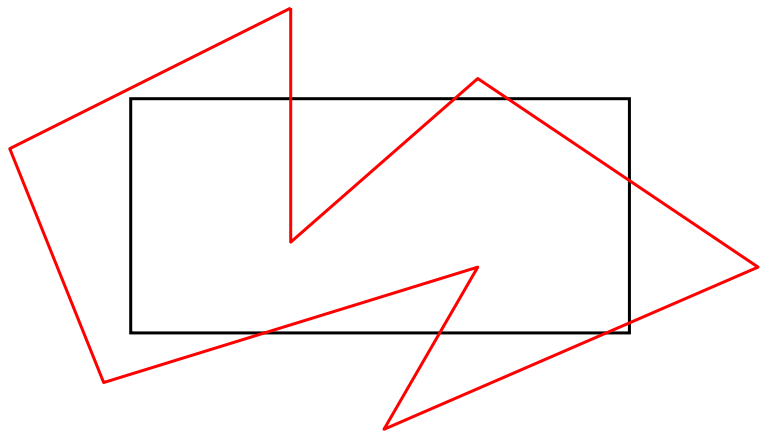
多边形、字符裁剪

一、多边形的裁剪



即得到一系列不连续的直线段！

而实际上，应该得到的是下图所示的有边界的区域：

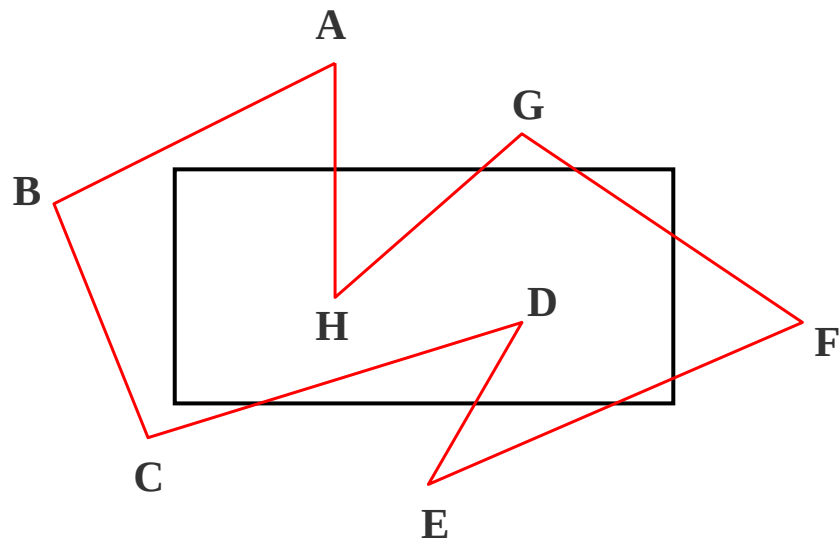


多边形裁剪算法的输出应该是裁剪后的多边形边界的顶点序列！

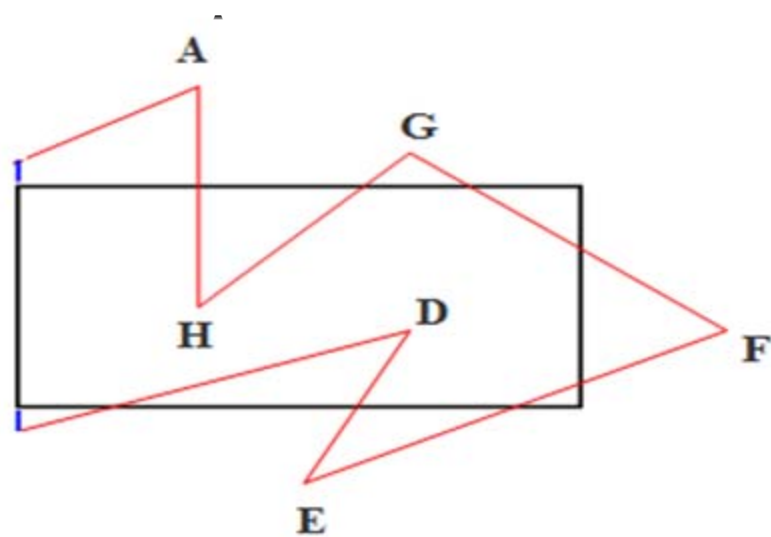
需要构造能产生一个或多个封闭区域的多边形裁剪算法

二、Sutherland-Hodgeman多边形裁剪

该算法的基本思想是将多边形边界作为一个整体，每次用窗口的一条边对要裁剪的多边形和中间结果多边形进行裁剪，体现一种分而治之的思想

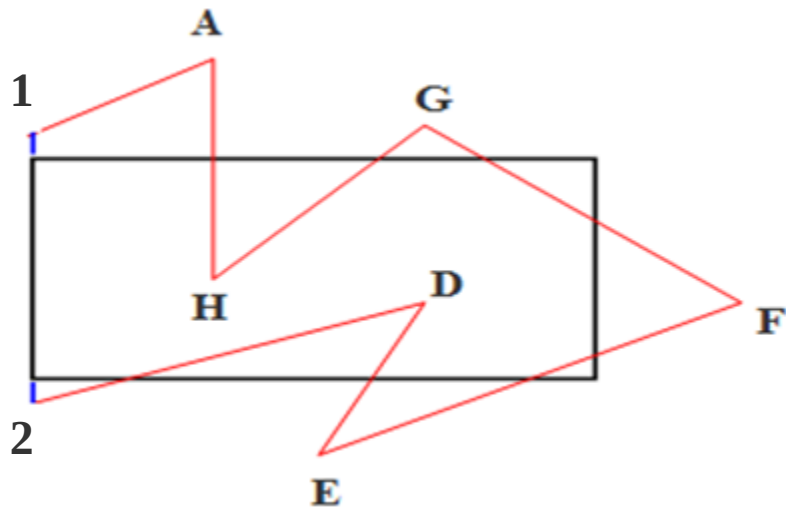


算法的输入: **ABCDEFGH**

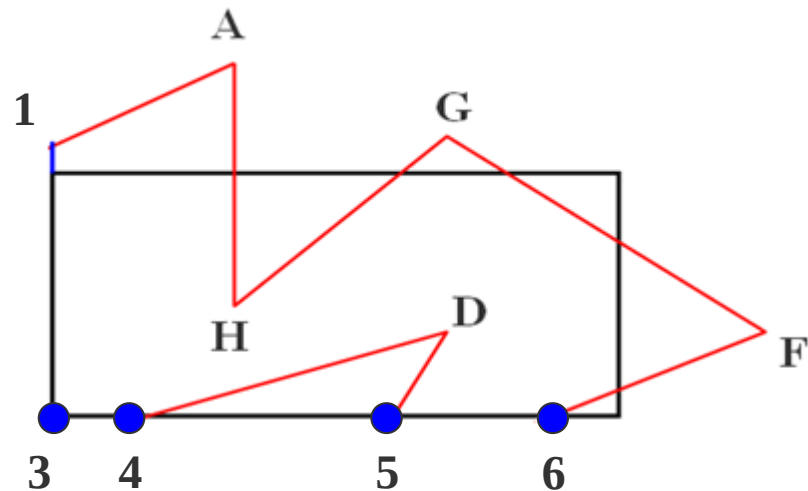


输出: **A12DEFGHA**

把平面分为两个区域: 包含窗口区域的一个域称为可见侧;
不包含窗口区域的域为不可见侧



输入：A12DEFGH



输出：A134D56FGH

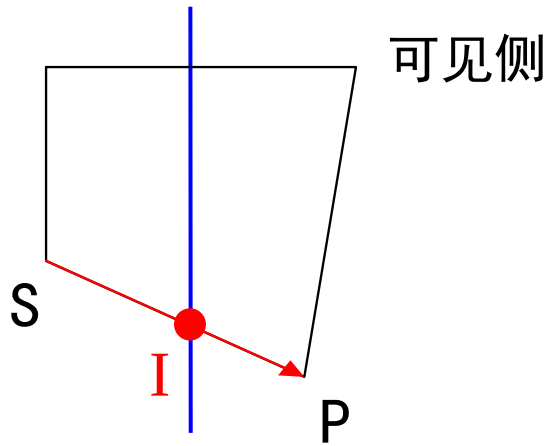
裁剪得到的结果多边形的顶点有两部分组成：

- (1) 落在可见一侧的原多边形顶点
- (2) 多边形的边与裁剪窗口边界的交点

根据多边形每一边与窗口边所形成的位置关系，沿着多边形依次处理顶点会遇到四种情况：

(1) 第一点S在不可见侧面，而第二点P在可见侧

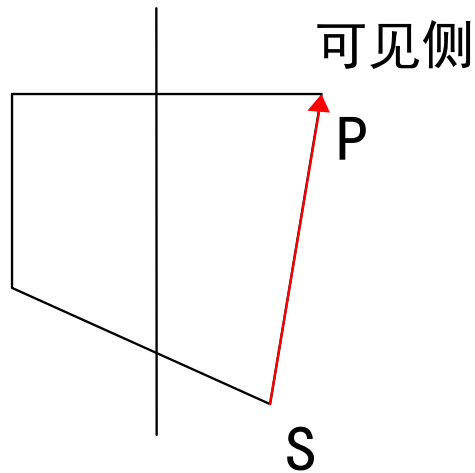
交点I与点P均被加入到输出顶点表中。



(1) 输出I、P

(2) 是S和P都在可见侧

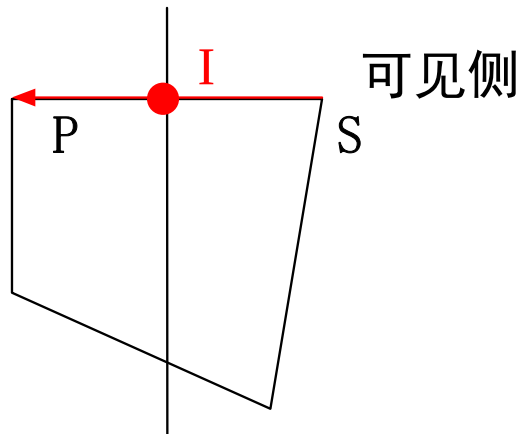
则P被加入到输出顶点表中



(2) 输出P

(3) S在可见侧，而P在不可见侧

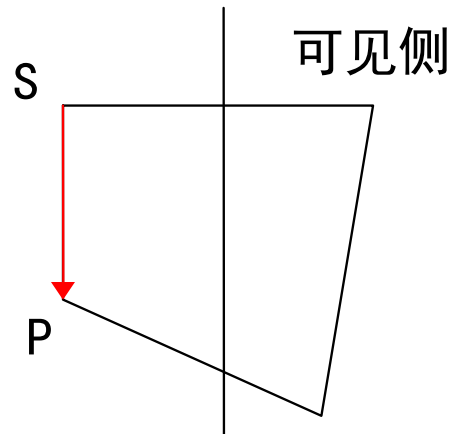
则交点I被加入到输出顶点表中



(3) 输出I

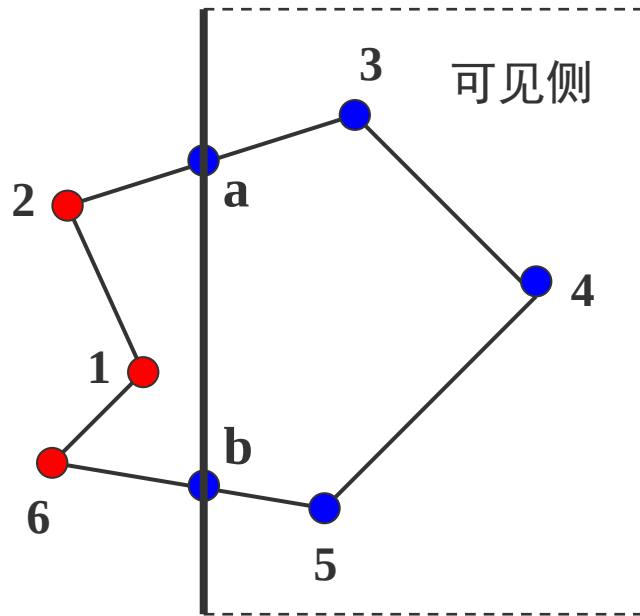
(4) 如果S和P都在不可见侧

输出顶点表中不增加任何顶点



(4) 不输出

在窗口的一条裁剪边界处理完所有顶点后，其输出顶点表将用窗口的下一条边界继续裁剪



输入: 1 2 3 4 5 6

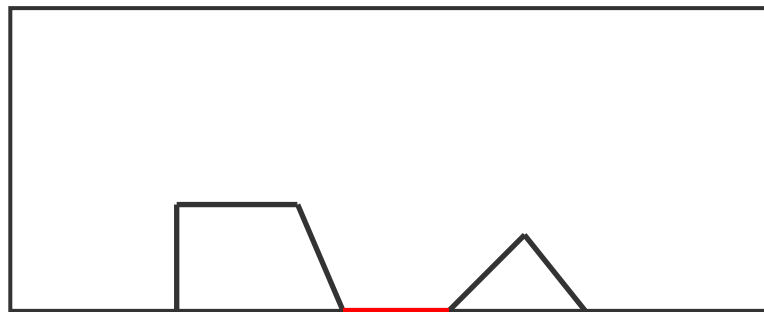
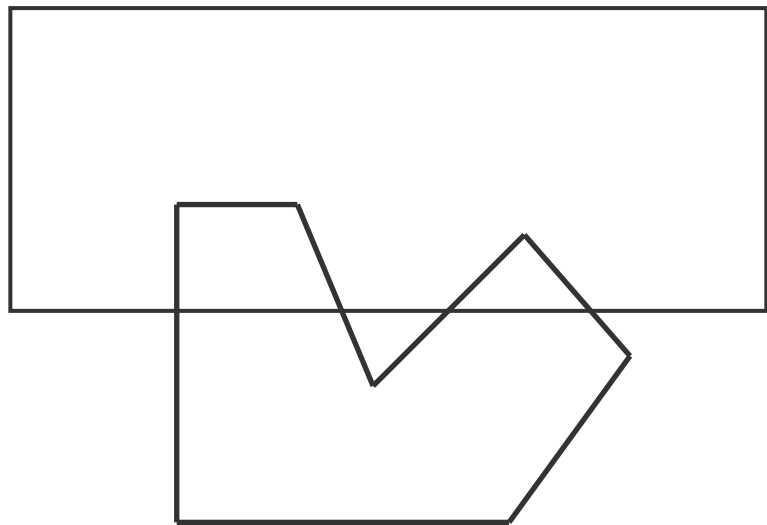
输出:

a 3 4 5 b

```
while 对于每一个窗口边或面 do
  begin
    if  $P_1$  在窗口边的可见一侧 then 输出  $P_1$ 
    for  $i=1$  to  $n$  do
      begin
        if  $P_1$  在窗口边的可见一侧 then
          if  $P_{1+i}$  在窗口边的可见一侧 then 输出  $P_{1+i}$ 
          else 计算交点并输出交点
        else if  $P_{i+1}$  在窗口可见一侧, then 计算交点
          并输出交点, 同时输出  $P_{i+1}$ 
      end
    end
  end
end
```

Sutherland-Hodgeman算法不足之处

利用Sutherland-Hodgeman裁剪算法对凸多边形进行裁剪可以获得正确的裁剪结果，但是。。。



多余线段

三、文字裁剪

屏幕上显示的不仅仅是多边形和直线等，还显示**字符**。字符如何处理？

字符并不是由直线段组成的。文字裁剪包括以下几种：

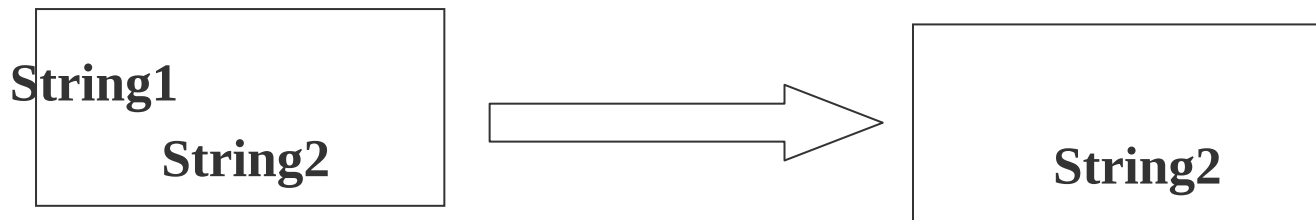
串精度裁剪

字符精度裁剪

笔划/像素精度裁剪

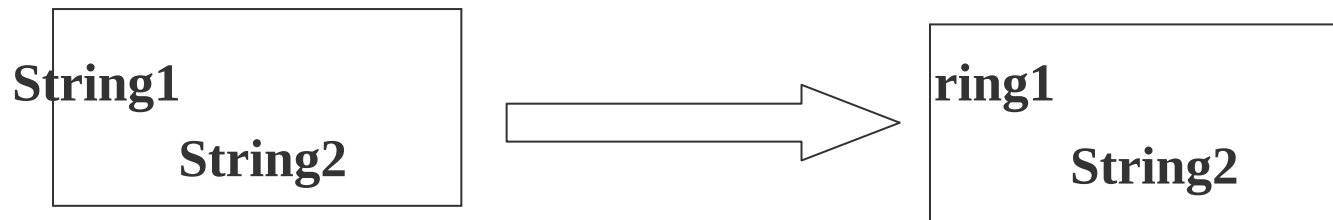
1、串精度裁剪

当字符串中的所有字符都在裁剪窗口内时，就全部保留它，否则舍弃整个字符串



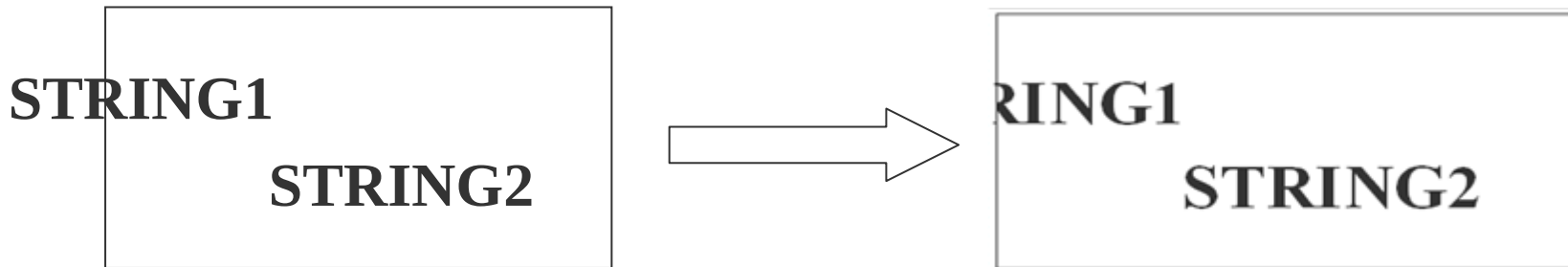
2、字符精度裁剪

在进行裁剪时，任何与窗口有重叠或落在窗口边界以外的字符都被裁剪掉



3、笔画/像素精度裁剪

将笔划分解成直线段对窗口作裁剪。需判断字符串中各字符的哪些像素、笔划的哪一部分在窗口内，保留窗口内部分，裁剪掉窗口外的部分



计算机图形学

第二章：光栅图形学算法

光栅图形学算法的研究内容

- 直线段的扫描转换算法
- 多边形的扫描转换与区域填充算法
- 直线裁剪算法
- 反走样算法
- 消隐算法

主要讲述的内容：

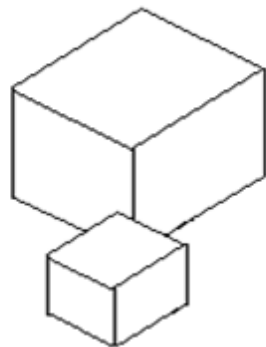
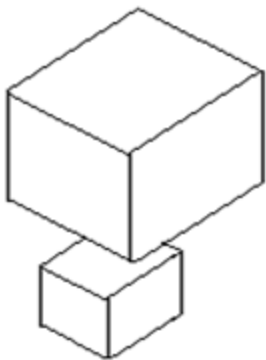
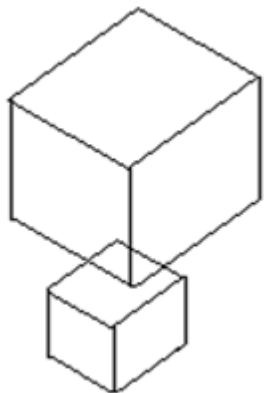
消隐的分类，如何消除隐藏线、隐藏面，主要介绍以下几个算法：

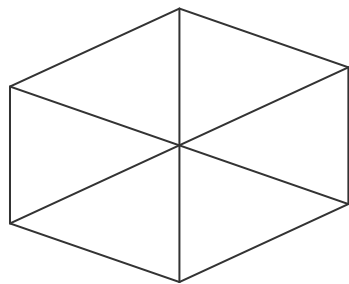
- Z缓冲区(Z-Buffer)算法
- 扫描线Z-buffer算法
- 区域子分割算法

一、消隐

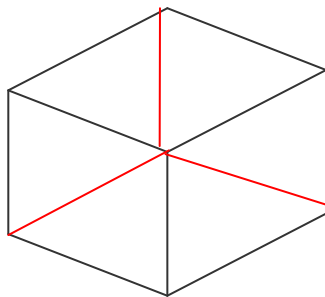
当我们观察空间任何一个不透明的物体时，只能看到该物体朝向我们的那些表面，其余的表面由于被物体所遮挡我们看不到

如果把可见和不可见的线都画出来，对视觉会造成多义性

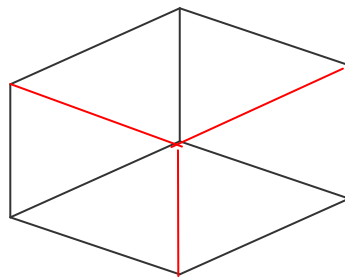




(a)



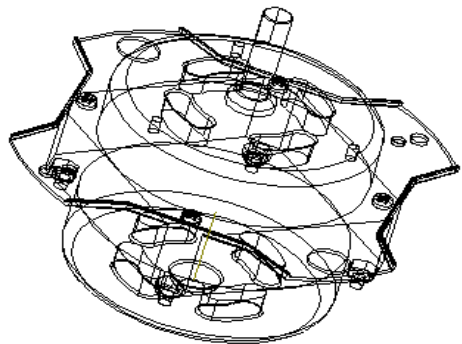
(b)



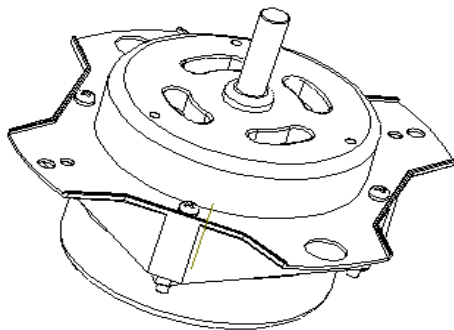
(c)

要消除二义性，就必须在绘制时消除被遮挡的不可见的线或面，习惯上称作消除隐藏线和隐藏面，简称为**消隐**。

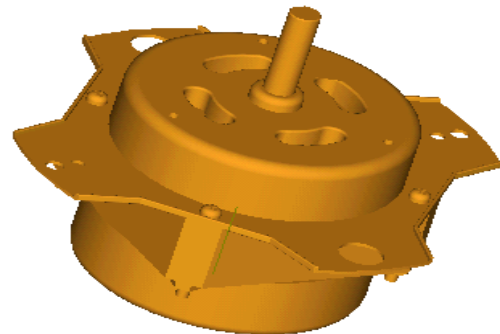
要绘制出意义明确的、富有真实感的立体图形，首先必须消去形体中的不可见部分，而只在图形中表现可见部分



线框图



消隐图

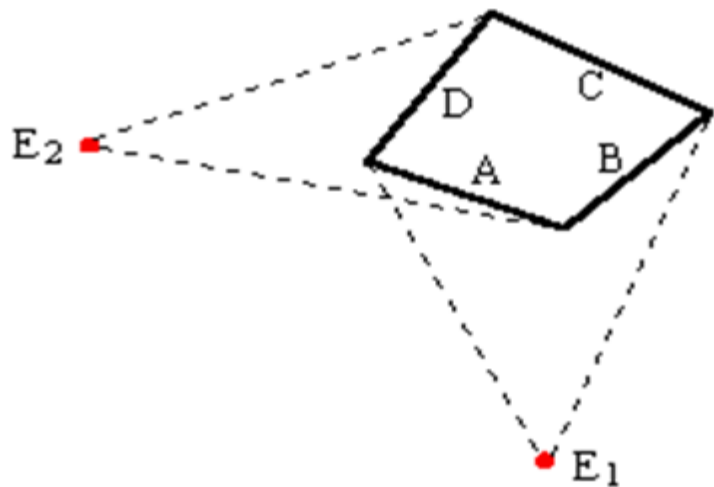


真实感图形

消隐包括消除“隐藏线”和“隐藏面”两个问题

到目前为止，虽然已有数十种算法被提出来了，但是由于物体的形状、大小、相对位置等因素千变万化，因此至今它仍吸引人们作出不懈的努力去探索更好的算法

消隐不仅与消隐对象有关，还与观察者的位置有关



消隐处理从原理上讲并不复杂，但是消隐处理的具体实现并不那么简单，它要求适当的算法及大量的运算。在60年代，消隐问题曾被认为是计算机图形学中的几大难题之一

二、消隐的分类

1、按消隐对象分类

(1) 线消隐

消隐对象是物体上的边，消除的是物体上不可见的边

(2) 面消隐

消隐对象是物体上的面，消除的是物体上不可见的面，通常做真实感图形消隐时用面消隐

2、按消隐空间分类

(1) 物体空间的消隐算法

以场景中的物体为处理单元。假设场景中有 k 个物体，将其中一个物体与其余 $k-1$ 个物体逐一比较，仅显示它可见表面以达到消隐的目的

此类算法通常用于线框图的消隐！

```
for (场景中的每一个物体)  
{ 将该物体与场景中的其  
  它物体进行比较，确定  
  其表面的可见部分；  
  显示该物体表面的可见  
  部分；  
}
```

在物体空间里典型的消隐算法有两个： Roberts算法和光线投射法

Roberts算法数学处理严谨，计算量甚大。算法要求所有被显示的物体都是凸的，对于凹体要先分割成多个凸体的组合

Roberts算法基本步骤:

- 逐个的独立考虑每个物体自身，找出为其自身所遮挡的边和面（自消隐）；
- 将每一物体上留下的边再与其它物体逐个的进行比较，以确定是完全可见还是部分或全部遮挡（两两物体消隐）；
- 确定由于物体之间的相互贯穿等原因，是否要形成新的显示边等，从而使被显示各物体更接近现实

光线投射是求光线与场景的交点，该光线就是所谓的视线（如视点与像素连成的线）

一条视线与场景中的物体可能有许多交点，求出这些交点后需要排序，在前面的才能被看到。人的眼睛可以一目了然，但计算机做需要大量的运算

(2) 图像空间的消隐算法

以屏幕窗口内的每个像素为处理单元。确定在每一个像素处，场景中的 k 个物体哪一个距离观察点最近，从而用它的颜色来显示该像素

```
for (窗口中的每一个像素)  
    {确定距视点最近的物体，  
      以该物体表面的颜色来显示像素;  
    }
```

这类算法是消隐算法的主流！

因为最后看到的图像是在屏幕上的，所以就拿屏幕作为处理对象。针对屏幕上的像素来进行处理，算法的思想是围绕着屏幕的。对屏幕上每个象素进行判断，决定哪个多边形在该象素可见。

三、图像空间的消隐算法

这类算法是消隐算法的主流！

Z-buffer 算法

扫描线算法

Warnock消隐算法

1、Z缓冲区(Z-Buffer)算法

1973年，犹他大学学生艾德·卡姆尔（Edwin Catmull）独立开发出了能跟踪屏幕上每个像素深度的算法 Z-buffer

Z-buffer让计算机生成复杂图形成为可能。Ed Catmull目前担任迪士尼动画和皮克斯动画工作室的总裁

Z缓冲器算法也叫深度缓冲器算法，属于图像空间消隐算法

该算法有帧缓冲器和深度缓冲器。对应两个数组：

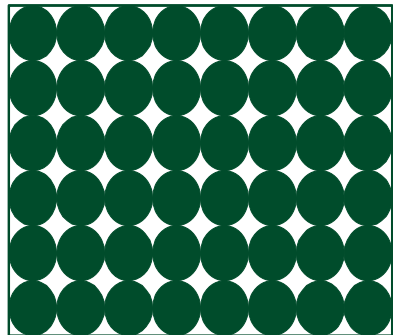
$\text{intensity}(x, y)$ ——属性数组（帧缓冲器）

存储图像空间每个可见像素的光强或颜色

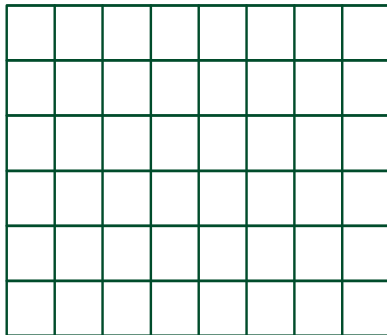
$\text{depth}(x, y)$ ——深度数组（z-buffer）

存放图像空间每个可见像素的z坐标

屏幕

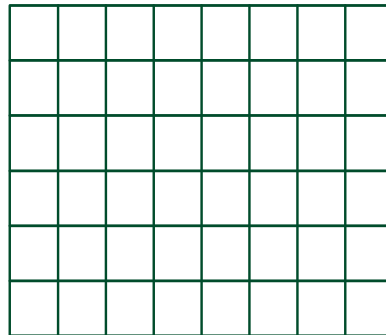


帧缓冲器



每个单元存放对应
像素的颜色值

Z缓冲器

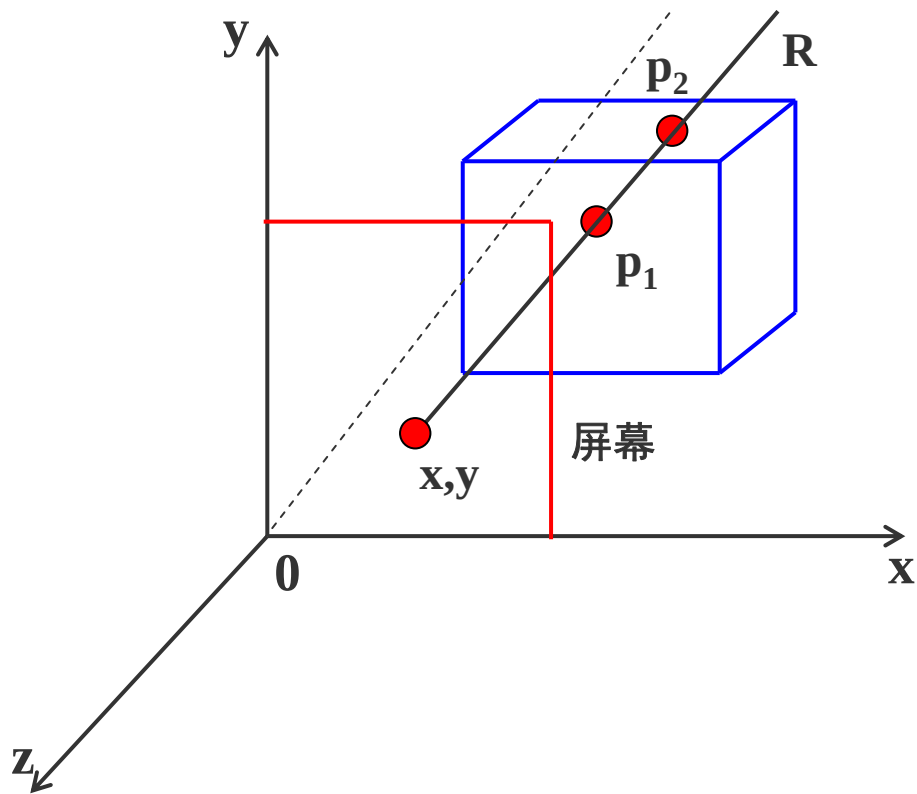


每个单元存放对应
像素的深度值

假定 xoy 面为投影面， z 轴为观察方向

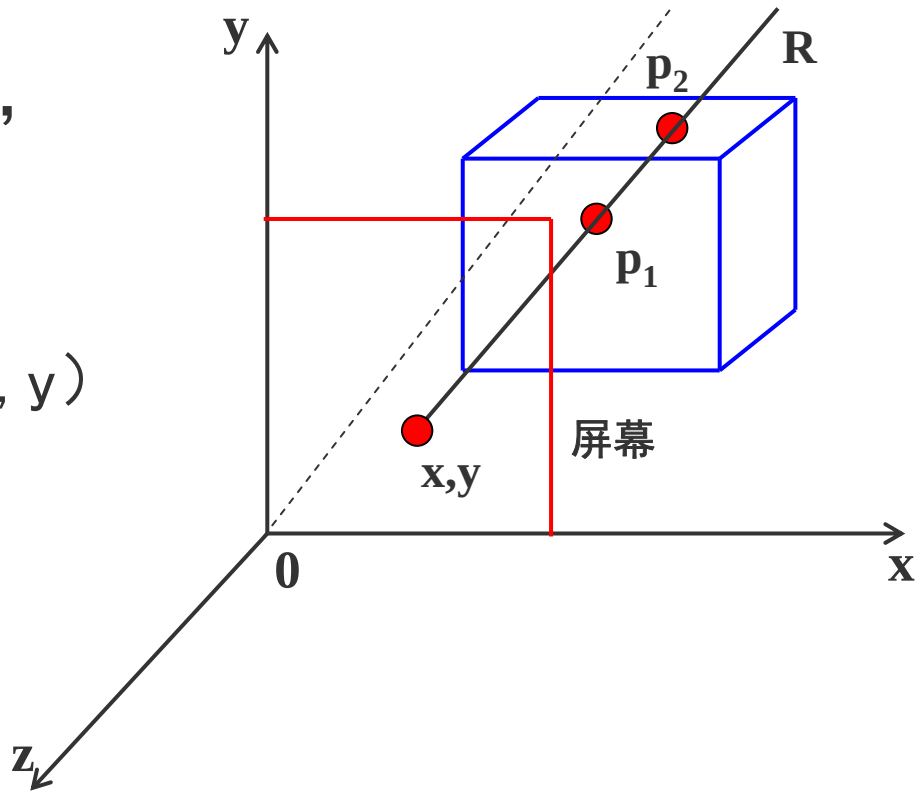
过屏幕上任意像素点 (x, y) 作平行于 z 轴的射线 R ，与物体表面相交于 p_1 和 p_2 点

p_1 和 p_2 点的 z 值称为该点的深度值



z-buffer算法比较 p_1 和 p_2 的z值，
将最大的z值存入z缓冲器中

显然， p_1 在 p_2 前面，屏幕上 (x, y)
这一点将显示 p_1 点的颜色



算法思想：先将 Z 缓冲器中各单元的初始值置为最小值。当要改变某个像素的颜色值时，首先检查当前多边形的深度值是否大于该像素原来的深度值（保存在该像素所对应的 Z 缓冲器的单元中）

如果大于原来的 z 值，说明当前多边形更靠近观察点，用它的颜色替换像素原来的颜色

Z-Buffer算法 ()

{ 帧缓存全置为背景色

深度缓存全置为最小z值

for (每一个多边形)

{ 扫描转换该多边形

for (该多边形所覆盖的每个像素 (x, y))

{ 计算该多边形在该像素的深度值 $Z(x, y)$;

if ($z(x, y)$ 大于 z 缓存在 (x, y) 的值)

{ 把 $z(x, y)$ 存入 z 缓存中 (x, y) 处

把多边形在 (x, y) 处的颜色值存入帧缓存的 (x, y) 处

}

}

}

}

z-Buffer算法的优点：

- (1) Z-Buffer算法比较简单，也很直观
- (2) 在像素级上以近物取代远物。与物体在屏幕上的出现顺序是无关紧要的，有利于硬件实现

z-Buffer算法的缺点：

- (1) 占用空间大
- (2) 没有利用图形的相关性与连续性，这是z-buffer算法的严重缺陷
- (3) 更为严重的是，该算法是在像素级上的消隐算法

2、只用一个深度缓存变量zb的改进算法

一般认为，z-Buffer算法需要开一个与图象大小相等的缓存数组ZB，实际上，可以改进算法，只用一个深度缓存变量zb

z-Buffer算法()

{ 帧缓存全置为背景色

for(屏幕上的每个像素(i, j))

{ 深度缓存变量zb置最小值MinValue

for(多面体上的每个多边形Pk)

{

if(像素点(i, j)在pk的投影多边形之内)

{

计算Pk在(i, j)处的深度值depth;

if(depth大于zb)

{ zb = depth;

indexp = k; (记录多边形的序号)

}

}

}

If(zb != MinValue) 计算多边形 P_{indexp} 在交点 (i, j) 处的光照
颜色并显示

}

}

关键问题：判断像素点 (i, j) 是否在 p_k 的投影多边形之内，不是一件容易的事。节省了空间但牺牲了时间。计算机的很多问题就是在时间和空间上找平衡

另一个问题计算多边形 P_k 在点 (i, j) 处的深度。设多边形 P_k 的平面方程为：

$$ax + by + cz + d = 0 \quad \text{depth} = -\frac{ai + bj + d}{c}$$

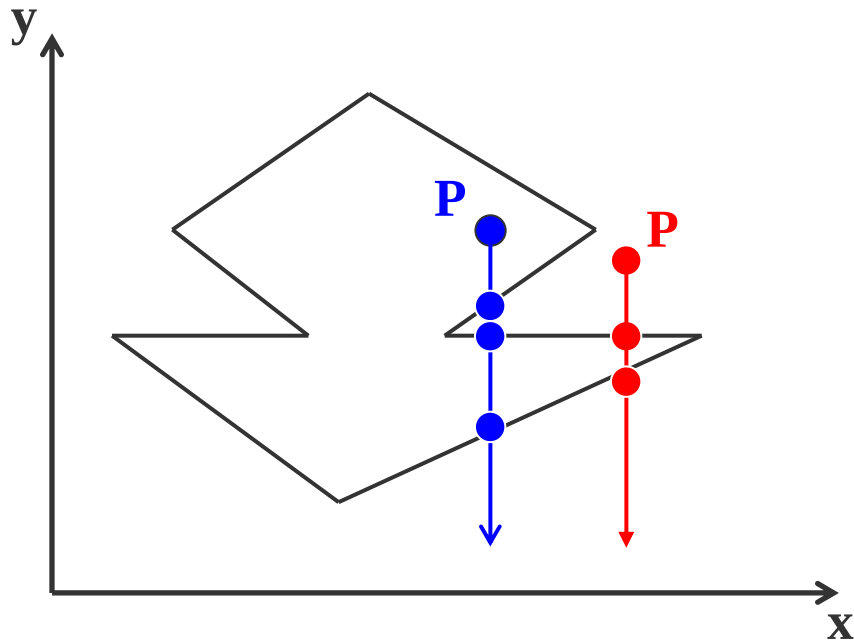
点与多边形的包含性检测：

(1) 射线法

由被测点P处向 $y = -\infty$ 方向作射线

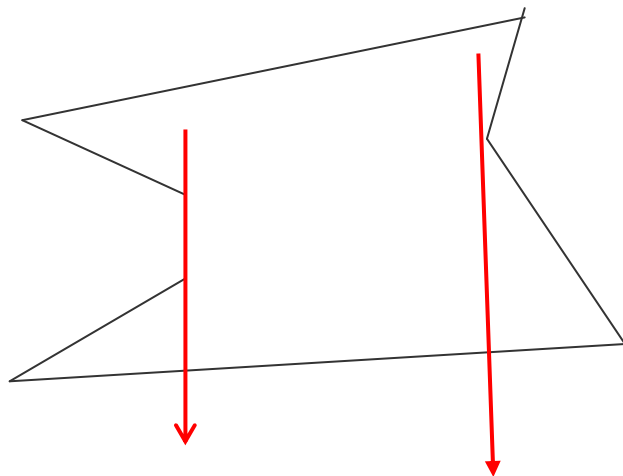
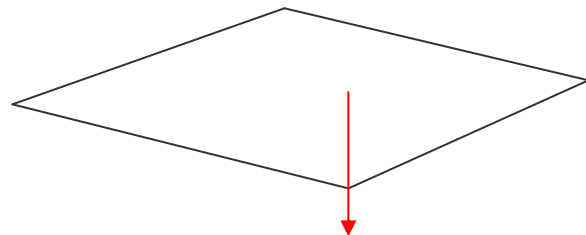
交点个数是奇数，则被测点在多边形内部

交点个数是偶数表示在多边形外部



若射线正好经过多边形的顶点，则采用“左开右闭”的原则来实现

即：当射线与某条边的顶点相交时，若边在射线的左侧，交点有效，计数；若边在射线的右侧，交点无效，不计数

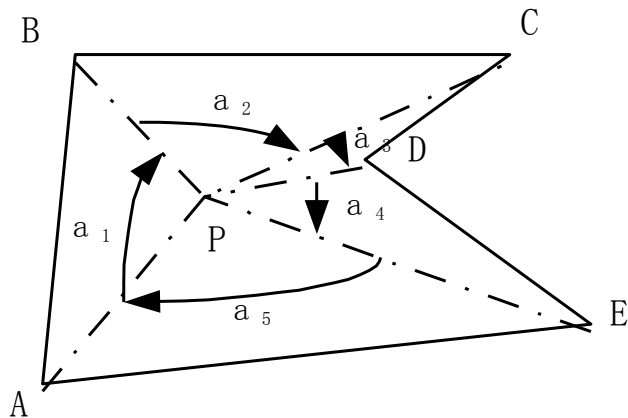
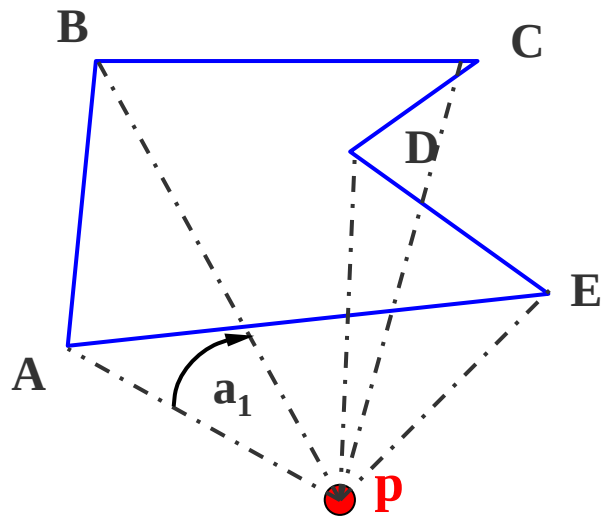


用射线法来判断一个点是否在多边形内的弊端：

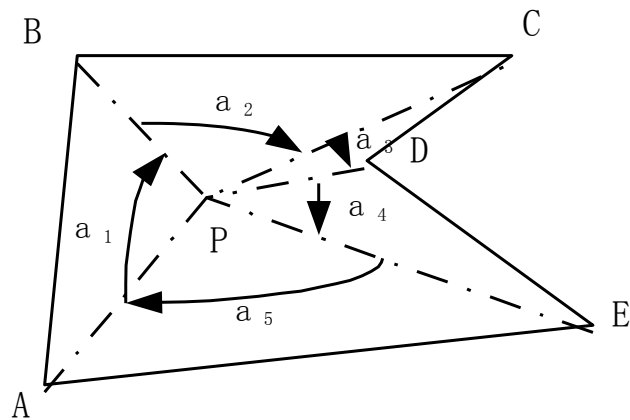
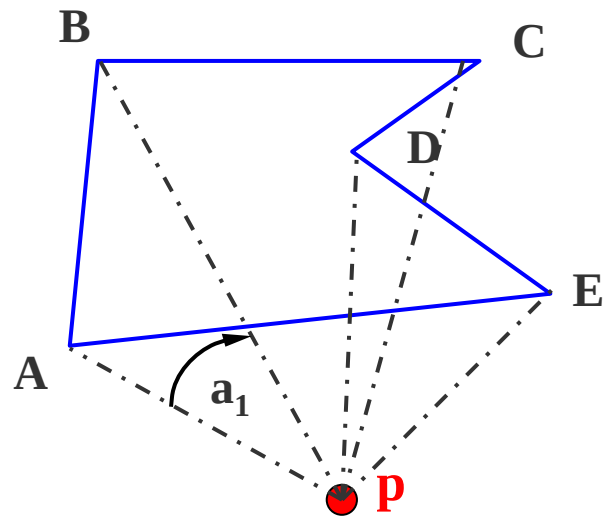
(1) 计算量大

(2) 不稳定

(2) 弧长法



以 p 点为圆心，作单位圆，把边投影到单位圆上，对应一段段弧长，规定逆时针为正，顺时针为负，计算弧长代数和



代数和为0, 点在多边形外部
 代数和为 2π , 点在多边形内部
 代数和为 π , 点在多边形边上

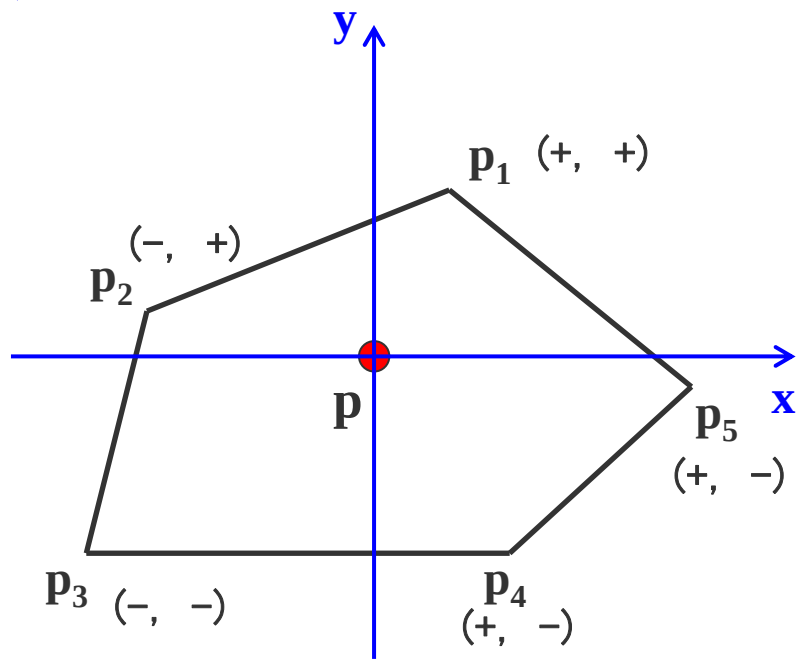
这个算法为什么是稳定的？假如算出来后代数和不是0，而是0.2或0.1，那么基本上可以断定这个点在外部，可以认为是有计算误差引起的，实际上是0。

但这个算法效率也不高，问题是算弧长并不容易，因此又派生出一个新的方法——以顶点符号为基础的弧长累加方法

(3) 以顶点符号为基础的弧长累加方法

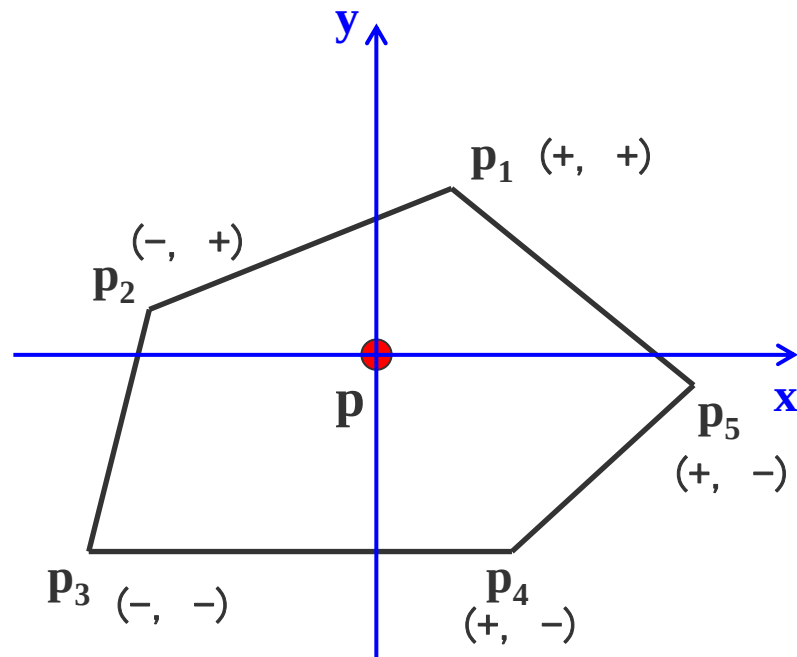
p 是被测点，按照弧长法， p 点的代数和为 2π

不要计算角度，做一个规定来取代原来的弧长计算



弧长变化 象限变化

(+ +)	(+ +)	0	I → I
(+ +)	(- +)	$\pi/2$	I → II
(+ +)	(- -)	$\pm\pi$	I → III
(+ +)	(+ -)	$-\pi/2$	I → IV
...	...	...	...



同一个象限认为是0，跨过一个象限是 $\pi/2$ ，跨过二个象限是 π 。这样当要计算代数和的时候，就不要去投影了，只要根据点所在的象限一下子就判断出多少度，这样几乎没有什么计算量，只有一些简单的判断，效率非常高

z-buffer算法是非常经典和重要的，在图形加速卡和固件里都有。只用一个深度缓存变量zb的改进算法虽然减少了空间，但仍然没考虑相关性和连贯性

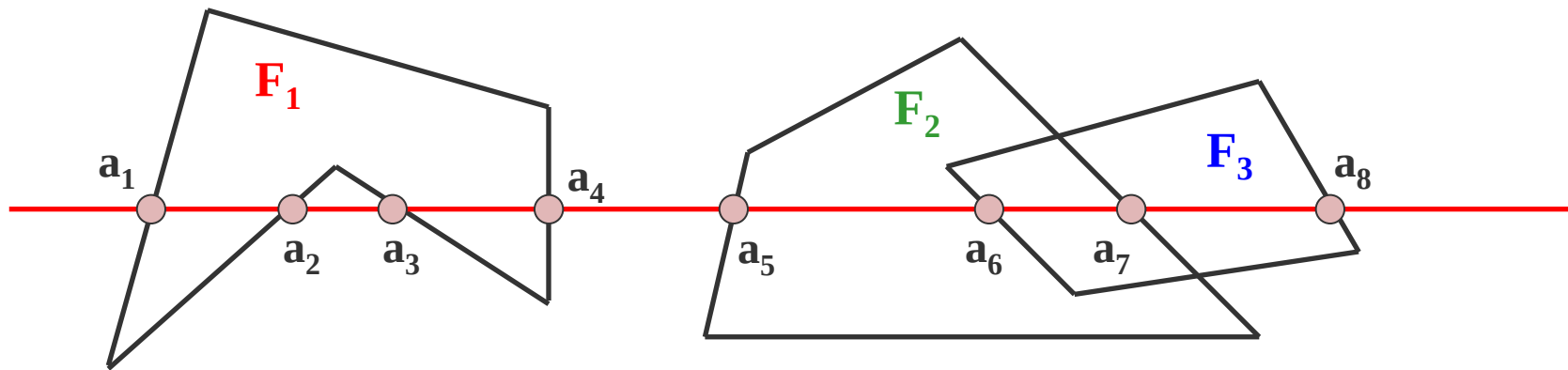
二、区间扫描线算法

前面介绍了经典的z-buffer算法，思想是开一个和帧缓存一样大小的存储空间，利用空间上的牺牲换取算法上的简洁

还介绍了只开一个缓存变量的z-buffer算法，是把问题转化成判别点在多边形内，通过把空间多边形投影到屏幕上，判别该像素是否在多边形内

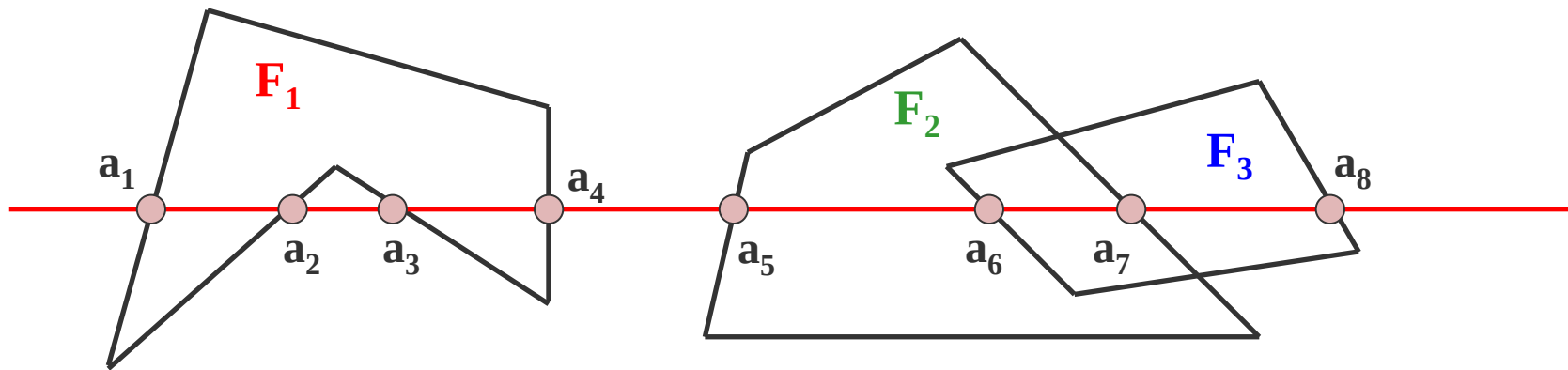
下面介绍区间扫描线算法。该算法放弃了z-buffer的思想，是一个新的算法，这个算法被认为是消隐算法中最快的

因为不管是哪一种z-buffer算法，都是在像素级上处理问题，要进行消隐，每个像素都要进行计算判别，甚至一个像素要进行多次（一个像素可能会被多个多边形覆盖）



扫描线的交点把这条扫描线分成了若干个区间，每个区间上必然是同一种颜色

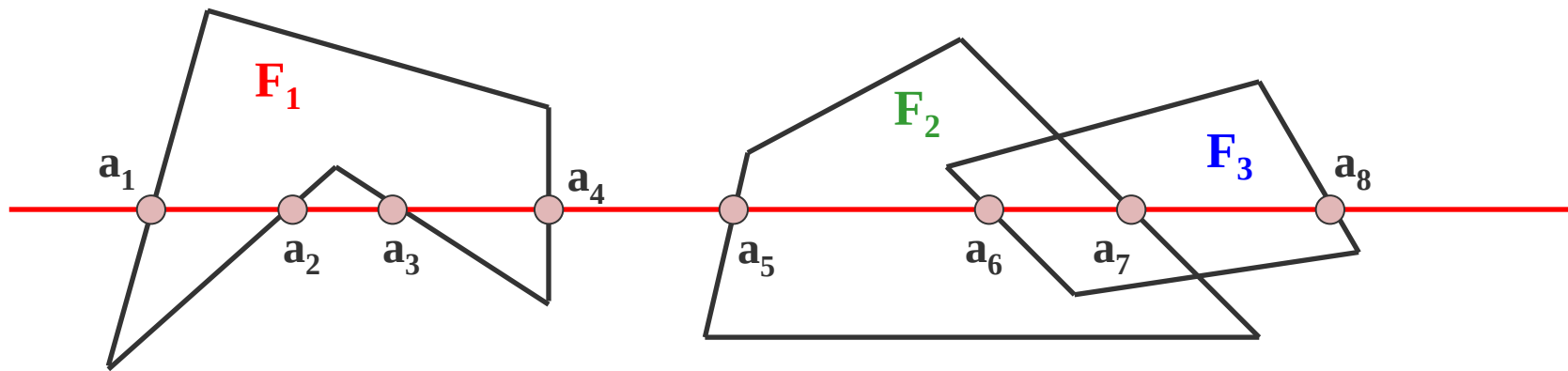
对于有重合的区间，如 a_6a_7 这个区间，要么显示 F_2 的颜色，要么显示 F_3 的颜色，不会出现颜色的跳跃



如果把扫描线和多边形的这些交点都求出来，对每个区间，只要判断一个像素的要画什么颜色，那么整个区间的颜色都解决了，这就是区间扫描线算法的主要思想

算法的优点：将像素计算改为逐段计算，效率大大提高！

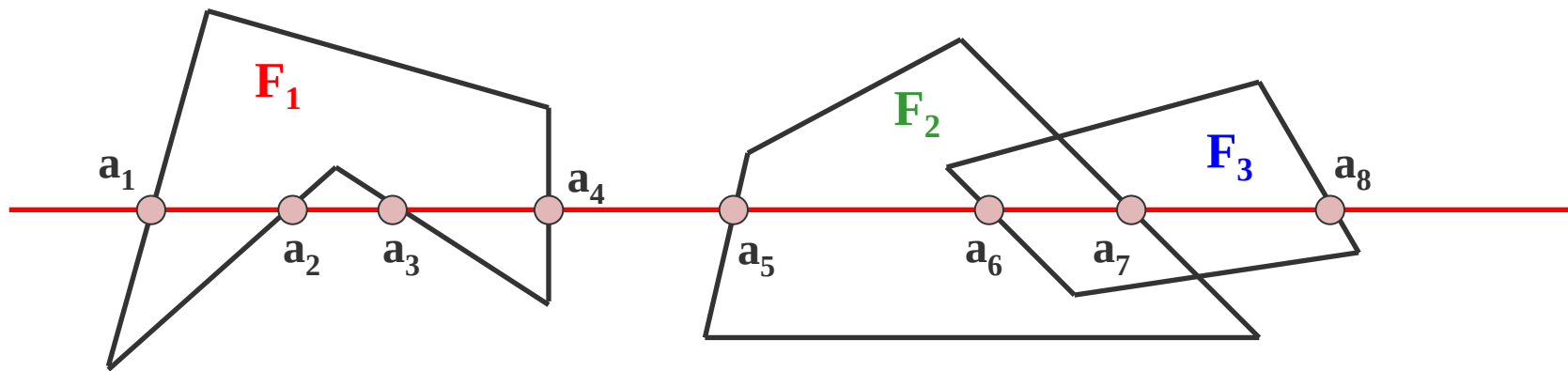
如何实现这个算法？



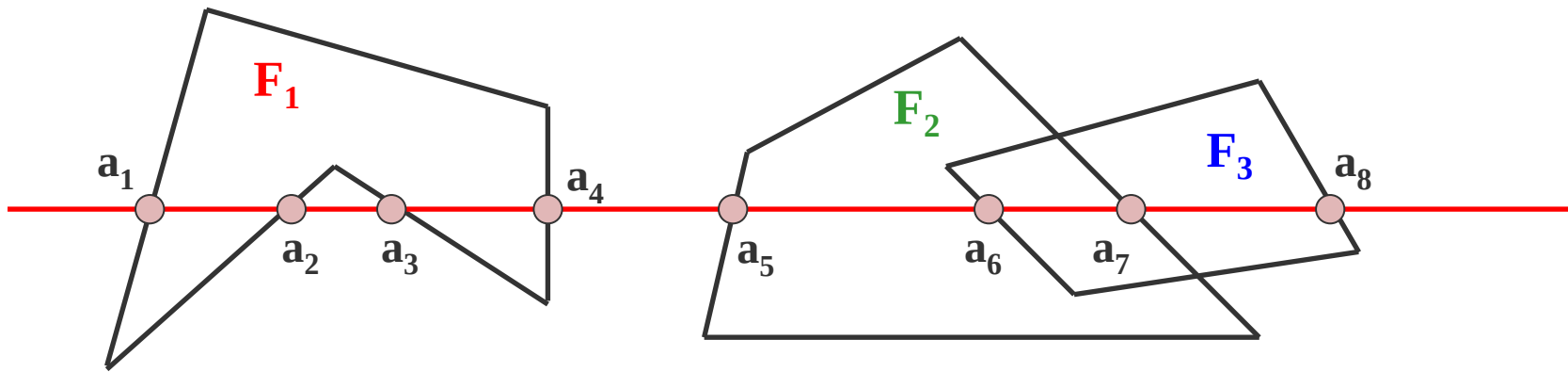
首先要有投影多边形，然后求交点，然后交点进行排序排序

排序的结果就分成了一个一个区间，然后在每个区间找其中的一个像素 (i, j) ，在 (i, j) 处计算每个相关面的 z 值，对相关深度值 z 进行比较，其中最大的一个就表示是可见的。整个这段区间就画这个 z 值最大面的颜色

如何确定小区间的颜色?



(1) 小区间上没有任何多边形, 如 $[a_4, a_5]$, 用背景色显示



(2) 小区间只有一个多边形，如 $[a_1, a_2]$ ，显示该多边形的颜色

(3) 小区间上存在两个或两个以上的多边形，比如 $[a_6, a_7]$ ，必须通过深度测试判断哪个多边形可见

这个算法存在几个问题：

- 1、真的去求交点吗？ **利用增量算法简化求交！**
- 2、每段区间上要求 z 值最大的面，这就存在一个问题。如何知道在这个区间上有哪些多边形是和这个区间相关的？

三、区域子分割算法（ Warnock算法）

John E. Warnock 博士，Adobe创始人之一，曾担任董事会主席

In his 1969 doctoral thesis,
Warnock invented the Warnock
algorithm for hidden surface
determination in computer
graphics



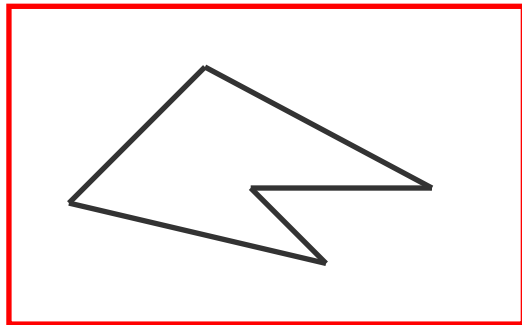
Warnock算法是图像空间中非常经典的一个算法

Warnock算法的重要性不在于它的效率比别的算法高，而在于采用了分而治之的思想，利用了堆栈的数据结构

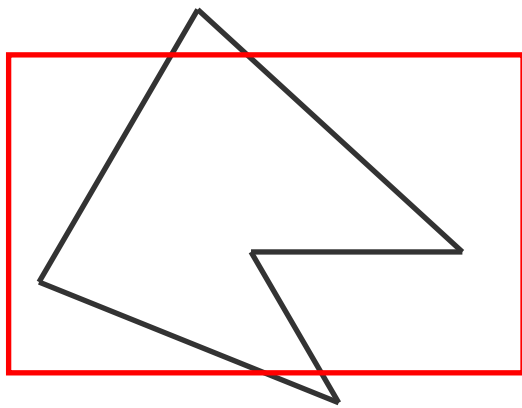
把物体投影到全屏幕窗口上，然后递归分割窗口，直到窗口内目标足够简单，可以显示为止

一、什么样的情况下，画面足够简单可以立即显示？

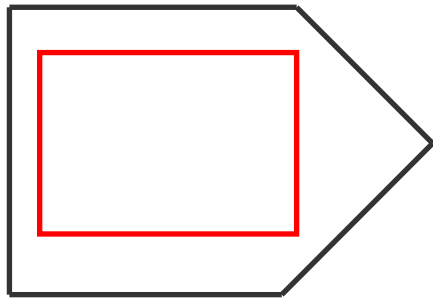
(1) 窗口中仅包含一个多边形



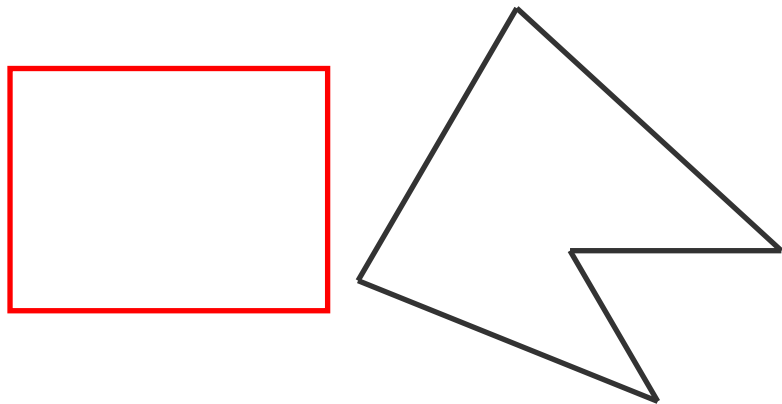
(2) 窗口与一个多边形相交，且
窗口内无其它多边形



(3) 窗口为一个多边形所包围



(4) 窗口与一个多边形相分离

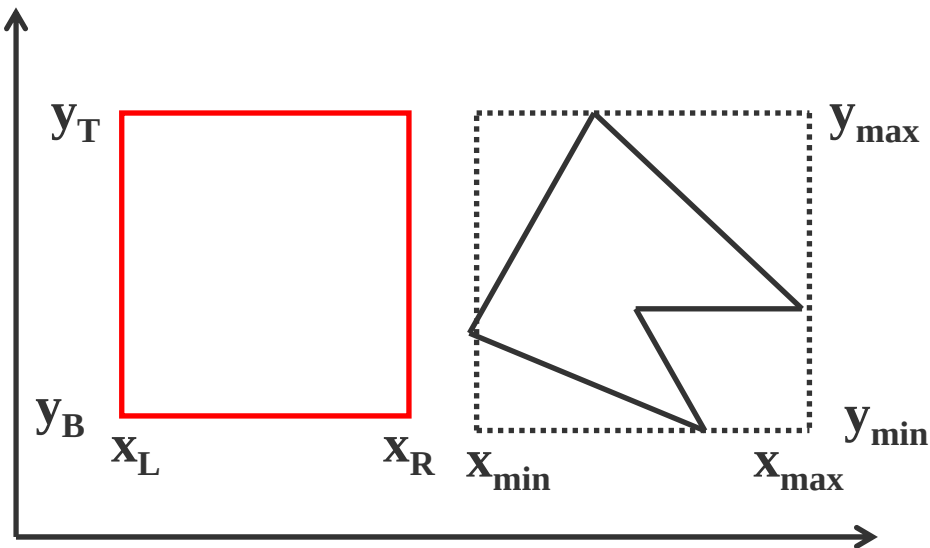


如何判别一个多边形和窗口是分离的？

当满足下列条件时，多边形和窗口分离：

$$x_{\min} > x_R \quad \text{or} \quad x_{\max} < x_L$$

$$y_{\min} > y_T \quad \text{or} \quad y_{\max} < y_B$$

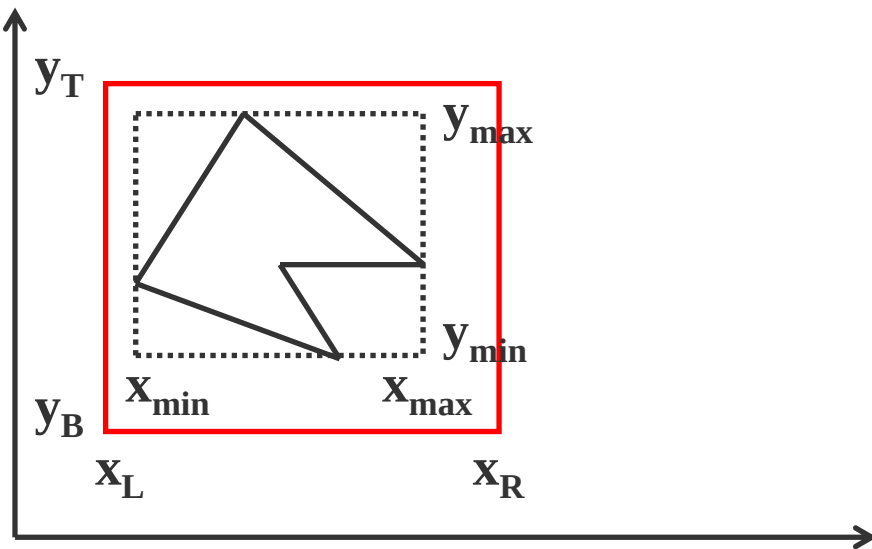


如何判别一个多边形在窗口内？

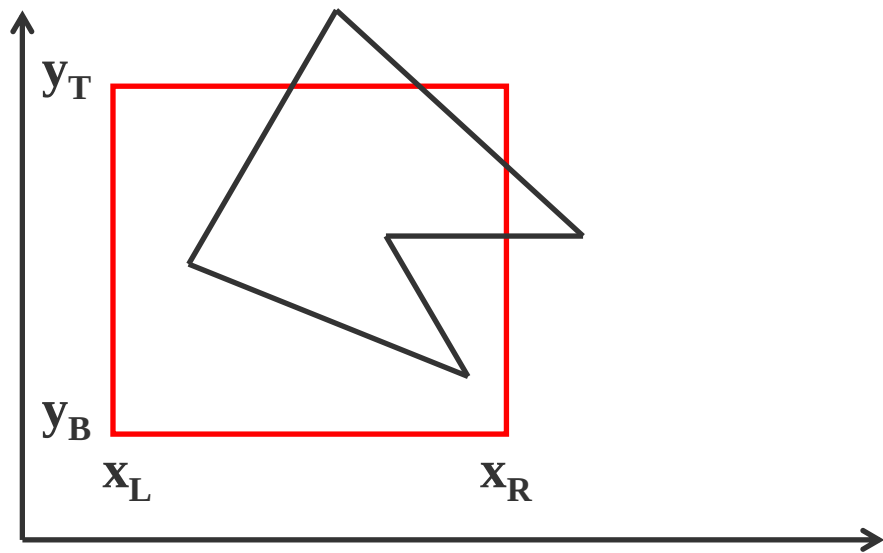
当满足下列条件时，多边形被窗口包含：

$$x_{\min} \geq x_L \quad \& \quad x_{\max} \leq x_R$$

$$y_{\min} \geq y_B \quad \& \quad y_{\max} \leq y_T$$



多边形与窗口相交的判别
，可以采用直线方程作为
判别函数来判定一个多边
形是否与窗口相交



二、窗口有多个多边形投影面，如何显示？

Warnock算法的重要性不在于它的效率比别的算法高，而在于采用了分而治之的思想，利用了堆栈的数据结构

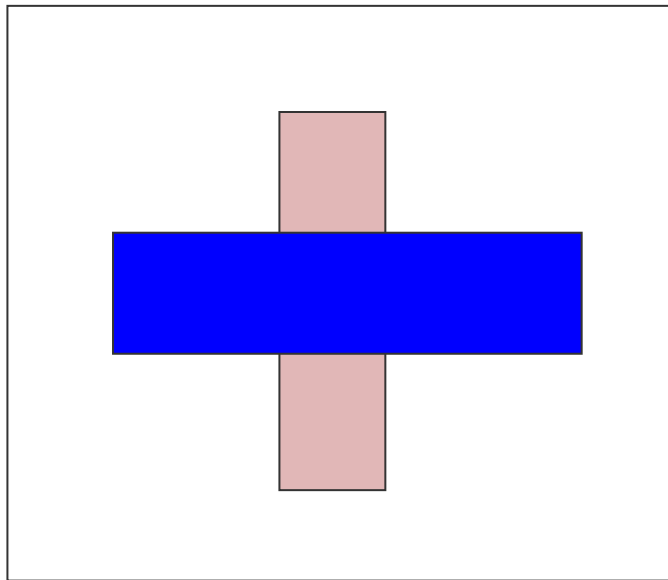
把物体投影到全屏幕窗口上，然后递归分割窗口，直到窗口内目标足够简单，可以显示为止

算法步骤：

(1) 如果窗口内没有物体则按背景色显示

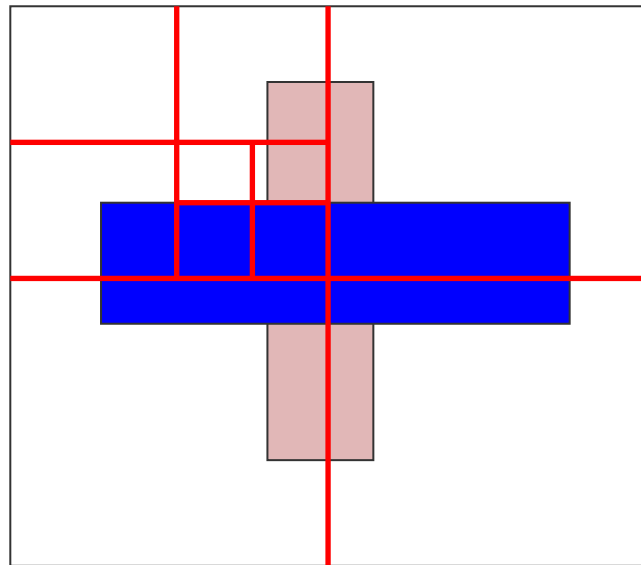
(2) 若窗口内只有一个面，则把该面显示出来

(3) 否则，窗口内含有两个以上的面，则把窗口等分成四个子窗口。对每个小窗口再做上述同样的处理。
这样反复地进行下去



(3) 窗口内含有两个以上的面，
则把窗口等分成四个子窗口。对
每个小窗口再做上述同样的处理
。这样反复地进行下去

把四个子窗口压在一个堆栈里
(后进先出)。



假设显示器分辨率 $(1024*1024)$,
窗口最多分几次?

如果到某个时刻，窗口仅有像素那么大，而窗口内仍有两个以上的面，如何处理？

这时不必再分割，只要取窗口内最近的可见面的颜色或所有可见面的平均颜色作为该像素的值

光栅扫描算法小结

1、直线段的扫描转换算法

这是一维图形的显示基础

- (1) DDA算法主要利用了直线的斜截式方程 ($y=kx+b$) , 在这个算法里引进了增量的思想, 结果把一个乘法和加法变成一个加法

(2) 中点法是采用的直线的一般式方程，也采用了增量的思想，比DDA算法的优点是采用了整数加法

(3) Bresenham算法也采用了增量和整数算法，优点是这个算法还能用于其它二次曲线

2、多边形的扫描转换和区域填充

如何把边界表示的多边形转换成由像素逐点描述的多边形
这是二维图形显示的基础

有四个步骤：求交、排序、配对、填色。这里引进了一个新的思想——图形的连贯性。手段就是利用增量算法和特殊的数据结构（多边形y表、边y表、活化多边形表、活化边表），
2个指针数组和2个指针链表

3、直线和多边形裁剪

关于裁剪算法主要讲了两个经典算法：

cohen-Sutherland算法、梁-barsky算法

cohen-Sutherland算法的核心思想是**编码**。把屏幕（窗口，场景空间）分成9个部分，用4位编码来描述这9个区域，通过4位编码的“**与**”、“**或**”运算来判断直线段是否在窗口内或外

梁算法主要的思想：

(1) 直线方程用参数方程表示

(2) 把被裁剪的直线段看成是一条有方向的线段，把窗口的四条边分成两类：入边和出边

这也是中国人的算法第一次出现在了所有图形学教科书都必须提的一个算法。这就叫原始创新！

4、走样、反走样

因为用离散量表示连续量，有限的表示无限的自然会导致一些失真，这种现象称为走样

反走样主要有三种方法：

提高分辨率

区域采样

加权区域采样。

提高分辨率无非是把分辨率增加，这样可以提高反走样的效果；但这个方法是有物理上的限制的，分辨率不能无限增加

区域采样算法是在关键的直线段、关键的区域上绘制的时候并非非黑即白，可以把关键部位变得模糊一点，有颜色的过渡区域，这样会产生一种好的视觉效果

加权区域是不但要考虑区域采样，而且要考虑不同区域的权重，用积分、滤波等技巧来做

5、消隐

在绘制场景时消除被遮挡的不可见的线或面，称作消除隐藏线和隐藏面，简称为**消隐**

消隐算法按消隐空间分类：

- (1) 物体空间 以场景中的物体为处理单元
- (2) 图像空间 以屏幕窗口内的每个像素为处理单元

首先介绍了经典的z-buffer算法。为了避免一个z-buffer二维数组的开销，引进了单个变量的z-buffer算法，把数组变成了单个变量

单个变量的z-buffer算法的主要问题是要不断地判一个点是否在多边形内。为了解决这个问题，又讲了3个算法：即如何判点在多边形内部，有射线法、代数弧长累加法、以顶点符号为基础的弧长累加方法。

区间扫描线算法：发现扫描线和多边形的交点把扫描线分成若干区间，每个区间只有一个多边形可以显示。利用这个特点可以把逐点处理变成逐段处理，提高了算法效率

Warnock消隐算法：采用了分而治之的思想，利用了堆栈的数据结构

把物体投影到全屏幕窗口上，然后递归分割窗口，直到窗口内目标足够简单，可以显示为止

核心思想

- (1) 增量思想：通过增量算法可以减少计算量
- (2) 编码思想
- (3) 符号判别 $\rightarrow$ 整数算法 尽可能的提高底层算法的效率，底层上提高效率才是真正解决问题

核心思想

- (4) 图形连贯性：利用连贯性可以大大减少计算量
- (5) 分而治之：把一个复杂对象进行分块，分到足够简单再进行处理

计算机图形学

第三章： 二维图形观察及变换

主要内容

本章主要讲述向量、世界坐标系、用户坐标系、窗口与视区、齐次坐标、二维变换等。

掌握要点

向量、矩阵以及它们的运算
坐标系的概念和坐标系之间的变换
齐次坐标的概念
二维图形的各种变换
窗口与视区的变换

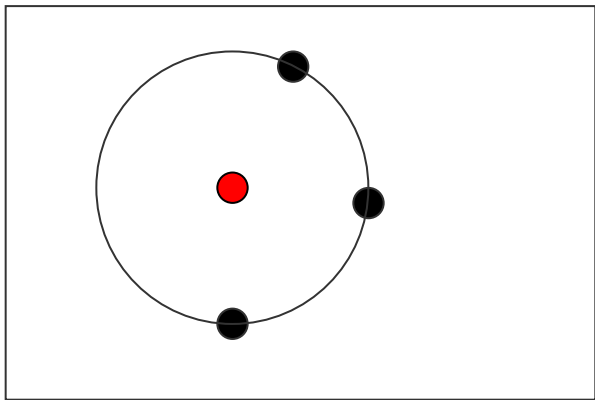
一、向量

1、向量为什么如此重要

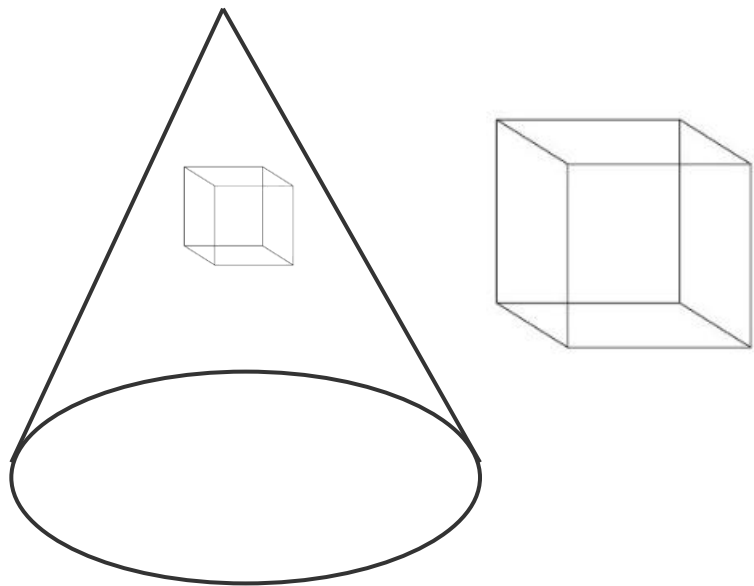
在计算机图形学中，主要处理三维世界中的物体对象。所有需要绘制的对象，都拥有形状、位置和方向等属性

需要编写合适的计算机程序来大致描述这些对象，并描述出围绕在物体周围的光线强度

计算出最终在显示器上的每一个像素的值



在给定坐标系下，所求圆的圆心在哪里呢？如果不使用向量，这个问题会很棘手



在给定圆锥体、立方体和摄像机位置的前提下，怎样才能获得反射影像的准确位置，它的颜色和形状又如何？

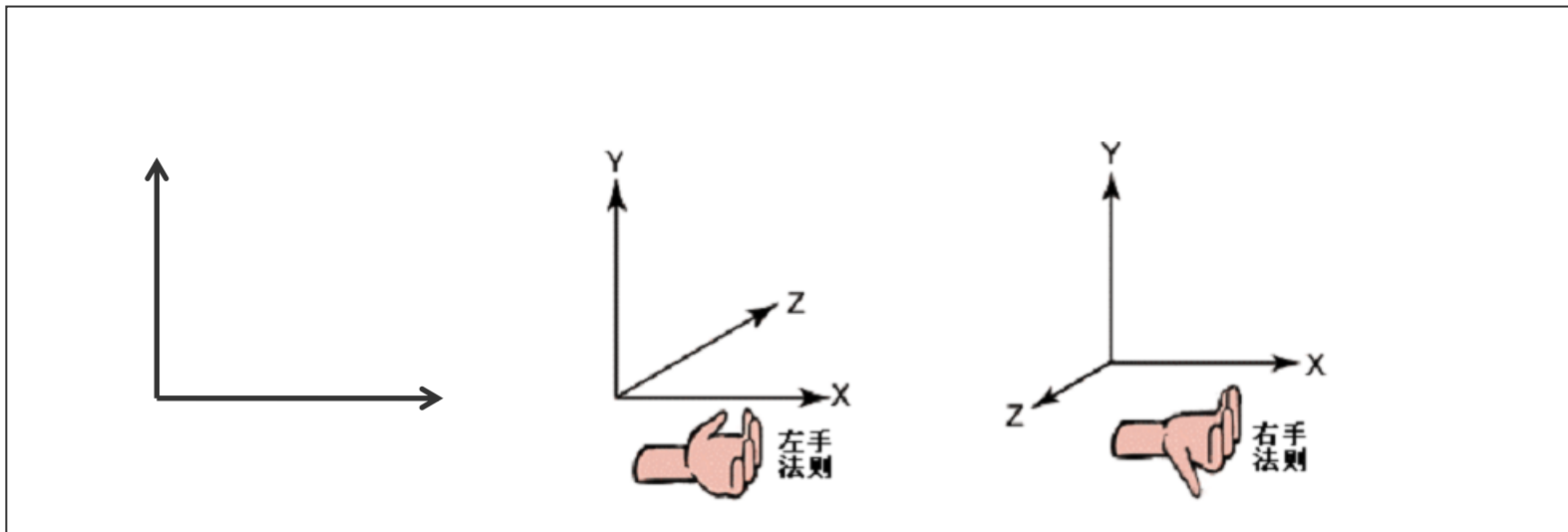
在图形学中，有两大基本工具能帮助我们实现上述要求：

向量分析 图形变换

通过学习这些工具，可以设计出一些方法来描述我们所遇到的各种几何对象，并学会如何把这些几何方法转换成数字

2、向量一些基础知识

我们所使用的所有点和向量都是基于某一坐标系定义的

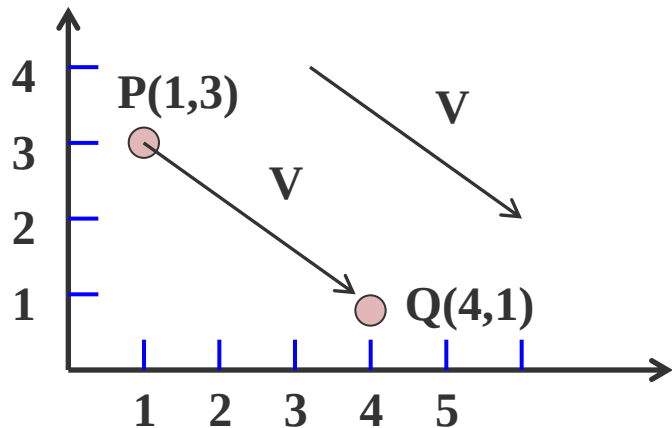


从几何的角度看，向量是具有长度和方向的实体，但是没有位置

而点是只有位置，没有长度和方向

在几何中把向量看成从一个点到另一个点的位移。向量算法提供了一种统一的方法对几何思想进行代数的表示

(1) 向量的表示



从P点到Q点的位移用向量

$v = (3, -2)$ 表示

v 是从点P到点Q的向量。两个点的差是一个向量：

$$v = Q - P$$

$$\boldsymbol{v} = \boldsymbol{Q} - \boldsymbol{P}$$

换个角度，可有说点Q是由点P平移向量 $\boldsymbol{v}$ 得到的；或者说 $\boldsymbol{v}$ 偏移（offset）P得到Q

$$\boldsymbol{Q} = \boldsymbol{P} + \boldsymbol{v}$$

可以把向量表示成它所有分量的列表，一个n维向量就是一个n元组：

$$\boldsymbol{w} = (w_1, w_2, \dots, w_n) \quad \boldsymbol{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \quad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

(2) 向量基本运算

向量允许两个基本操作：

向量相加 标量（实数）的数乘

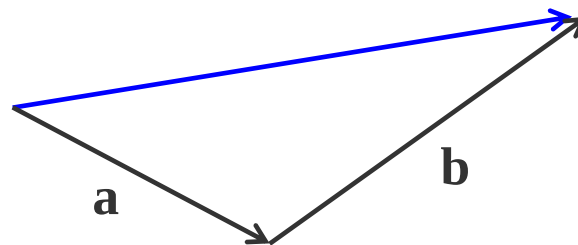
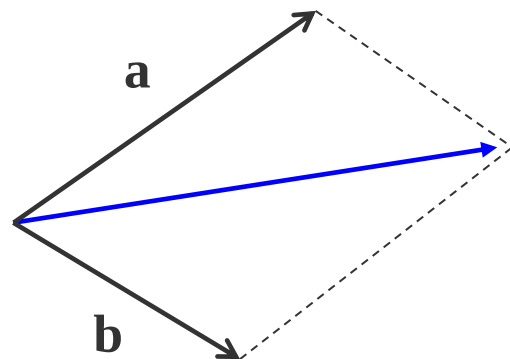
如果 a 和 b 是两个向量， s 是一个标量，

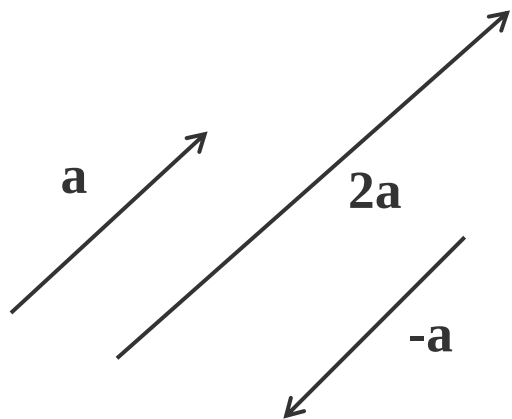
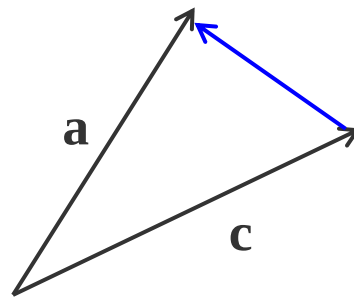
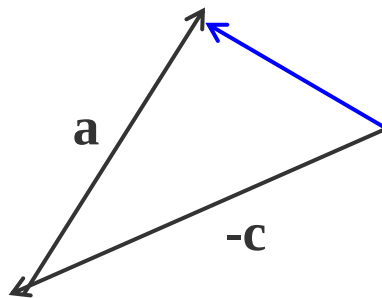
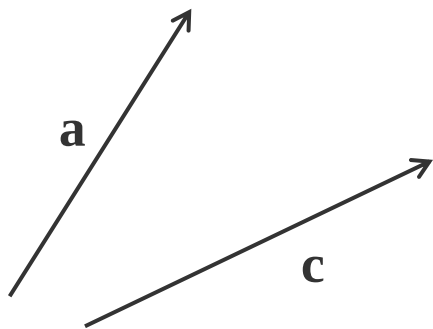
$a+b$ 和 sa 都是有意义的

$$a = (2, 6) \qquad b = (3, 1)$$

$$a + b = (5, 7) \qquad 2a = (4, 12)$$

向量的加（减）法可以采用“平行四边形法则”





(3) 向量线性组合

掌握了向量的加法和数乘，就可以定义任意多个向量的线性组合

m个向量 $v_1, v_2, \dots, v_m$ 的线性组合具有如下形式的向量：

$$w = a_1 v_1 + a_2 v_2 + \dots + a_n v_n$$

$$2(3, -1) + 6(-1, 2)$$

$$(0, 10)$$

有两种特殊的线性组合在计算机图形学中很重要：

仿射组合 凸组合

① 仿射组合

如果线性组合的系数 $a_1, a_2, \dots, a_m$ 的和等于1，那么它就是仿射组合

$$a_1 + a_2 + \dots + a_m = 1$$

② 向量的凸组合

凸组合在数学中具有重要的位置，在图形学中也有很多应用。凸组合是对仿射组合加以更多的限制得来的

$$a_1 + a_2 + \dots + a_m = 1$$

$$i = 1, 2, \dots, m \quad a_i \geq 0$$

3、向量的点积和叉积

有两个功能强大的工具一直推动着向量的应用：

点积 叉积

点积得到一个标量，叉积产生一个新的向量

(1) 向量的点积

$$a = (a_1, a_2) \quad b = (b_1, b_2)$$

$$a \bullet b = a_1 b_1 + a_2 b_2$$

也就是说，计算点积时，只需将两个向量相应的分量相乘，然后将结果相加即可

$$d = v \bullet w = \sum_{i=1}^n v_i w_i$$

点积最重要的应用就是计算两个向量的夹角，或者两条直线的夹角

$$b \bullet c = |b||c| \cos \theta$$

其中 θ 是 b 和 c 之间的夹角

$$\cos \theta = \frac{b \bullet c}{|b||c|} = \hat{b} \bullet \hat{c}$$

由于两个向量的点积和它们之间夹角的余弦成正比，可以得出以下关于两个非零向量夹角与点积的关系：

$$b \bullet c > 0 \quad \theta < 90^{\circ}$$

$$b \bullet c = 0 \quad \theta = 90^{\circ}$$

$$b \bullet c < 0 \quad \theta > 90^{\circ}$$

余弦定理和新闻的分类

谷歌、百度的新闻是自动分类和整理的。所谓新闻的分类无非是要把相似的新闻放到一类中

如何设计一个算法来算出任意两篇新闻的相似性？

用一个向量来描述一篇新闻

当夹角的余弦接近于1时，两条新闻相似，从而可以归成一类；夹角的余弦越小，两条新闻越不相关

(2) 向量的叉积

两个向量的叉积是另一个三维向量。叉积只对三维向量有意义。它有许多有用的属性，但最常用的一个是它与原来的两个向量都正交。

$$\mathbf{a} = (a_x, a_y, a_z) \quad \mathbf{b} = (b_x, b_y, b_z)$$

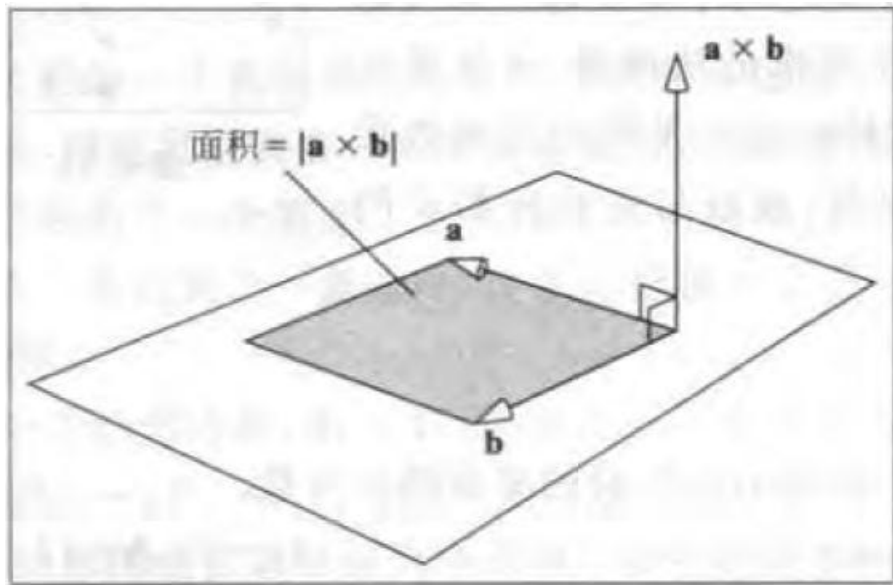
$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$$

两个向量的叉积 $a \times b$ 是另一个向量，但是这个向量与原来的两个向量在几何上有什么关系，为什么对它感兴趣呢？

① $a \times b$ 和 a 、 b 两个向量都正交

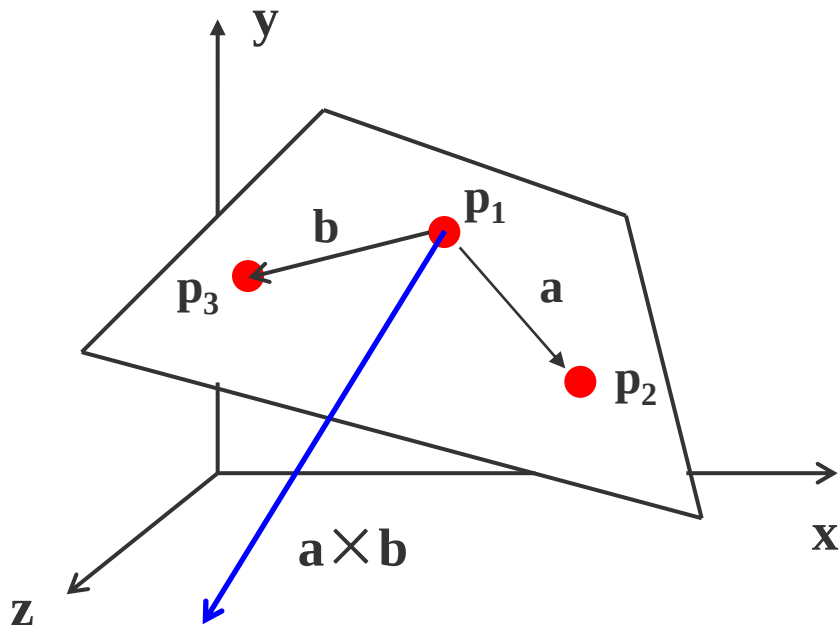
② $a \times b$ 的长度等于由 a 和 b 决定的平行四边形面积

$$a \times b = |a||b|\sin(\theta)$$



利用叉积求平面的法向量

法向量是空间解析几何的一个概念，垂直于平面的直线所表示的向量为该平面的法向量



将向量分析应用到几何场景处理中是值得研究的内容

把一个场景中众多向量的各种属性搜集起来，并将它们与在图形学中遇到的实际几何问题相结合，寻找出一个解决方案，将是非常有用的

图形坐标系

在图形学中，有两大基本工具：

向量分析

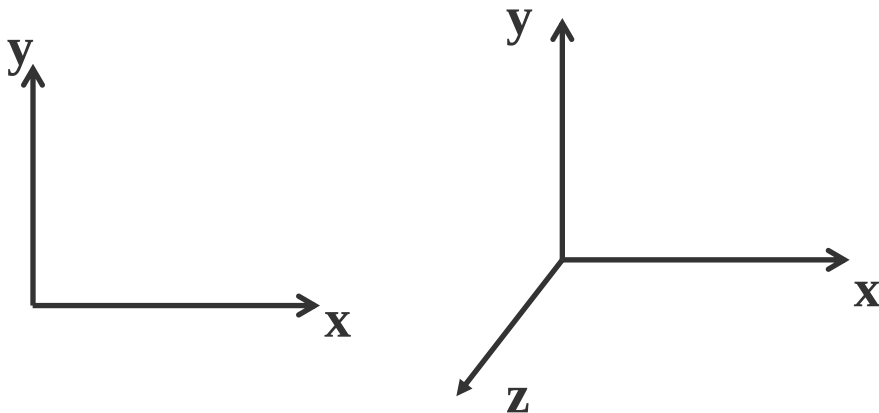
图形变换

一、坐标系的基本概念

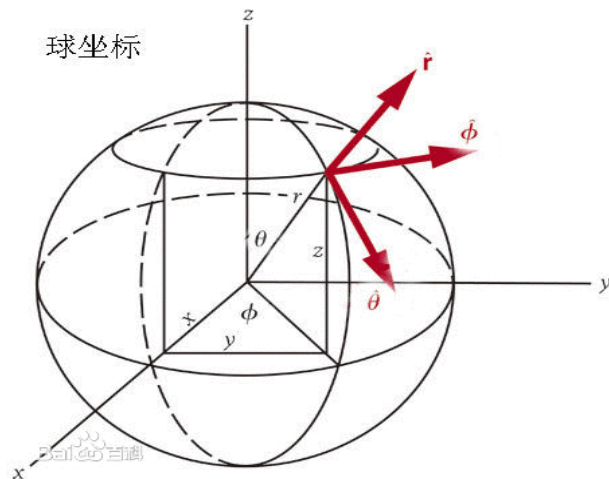
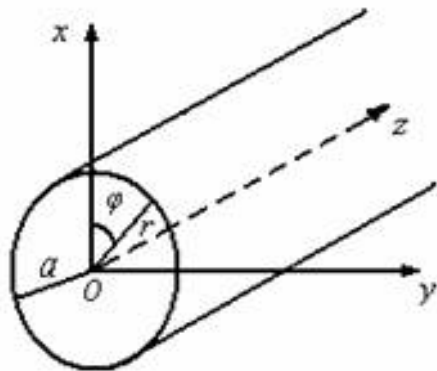
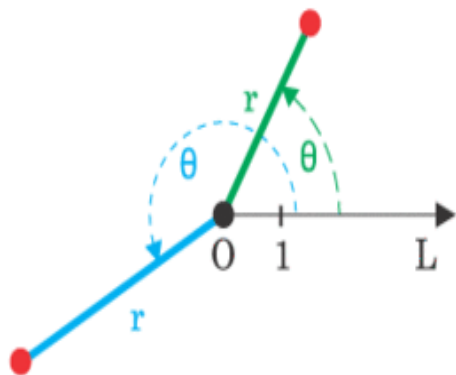
坐标系是建立图形与数之间对应联系的参考系

1、坐标系的分类

从维度上看，可分为一维、二维、三维坐标系

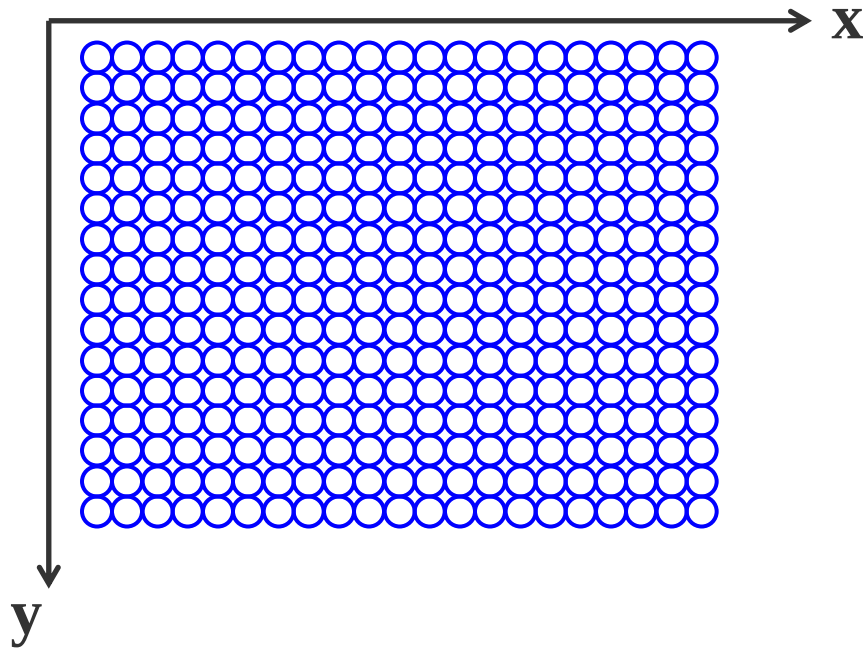


从坐标轴之间的空间关系来看，可分为直角坐标系、极坐标系、圆柱坐标系、球坐标系等



在计算机图形学中，从物体（场景）的建模，到在不同显示设备上显示、处理图形时同样使用一系列的坐标系

对于一个给定的问题，并不总是按像素坐标来考虑



显然，希望将程序中用于描述对象几何信息的数值，和那些用于表示对象中大小和位置的数值区分开来

前者通常被看作一个建模（modeling）的任务，后者是一个观察（viewing）的任务

图形显示的过程就是几何（对象）模型在不同坐标系之间的映射变换

2、计算机图形学中坐标系的分类

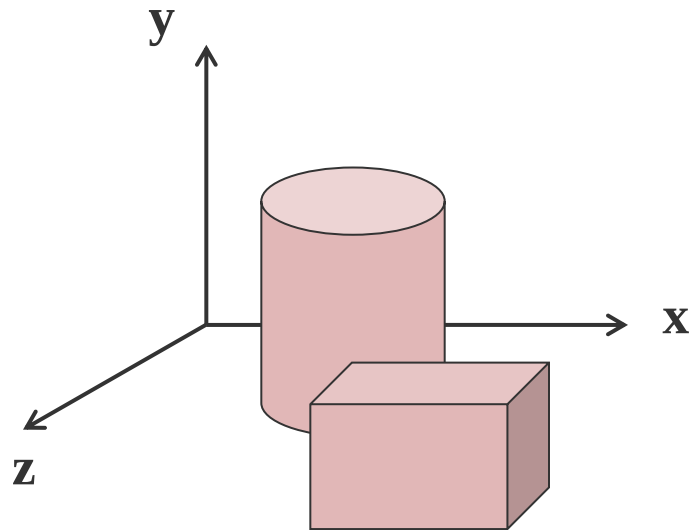
(1) 世界坐标系

程序员可以用最适合他们手中问题的坐标系来描述对象，并且可以自动的缩放和平移图形，使得其能正确地在屏幕窗口中显示

这个描述对象的空间被称为世界坐标系，即场景中物体在实际世界中的坐标

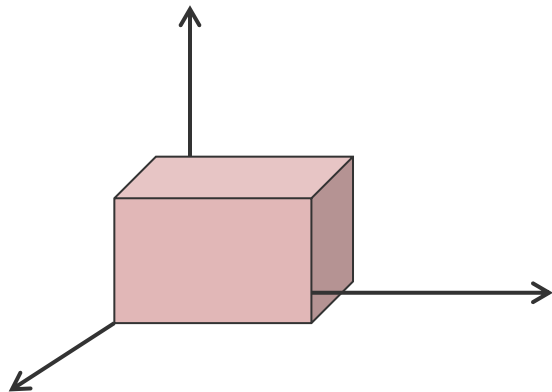
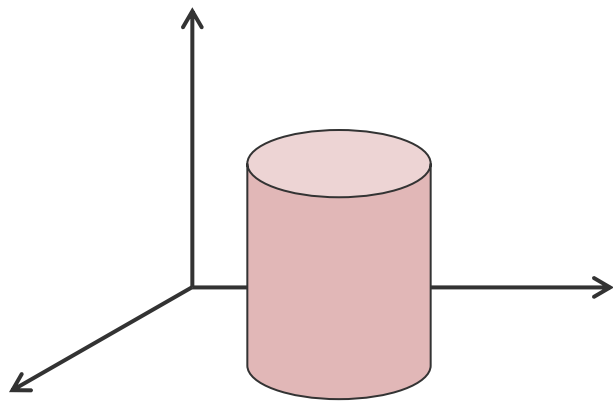
世界坐标系是一个公共坐标系，是现实中物体或场景的统一参照系

计算机图形系统中涉及的其它坐标系都是参照它进行定义的



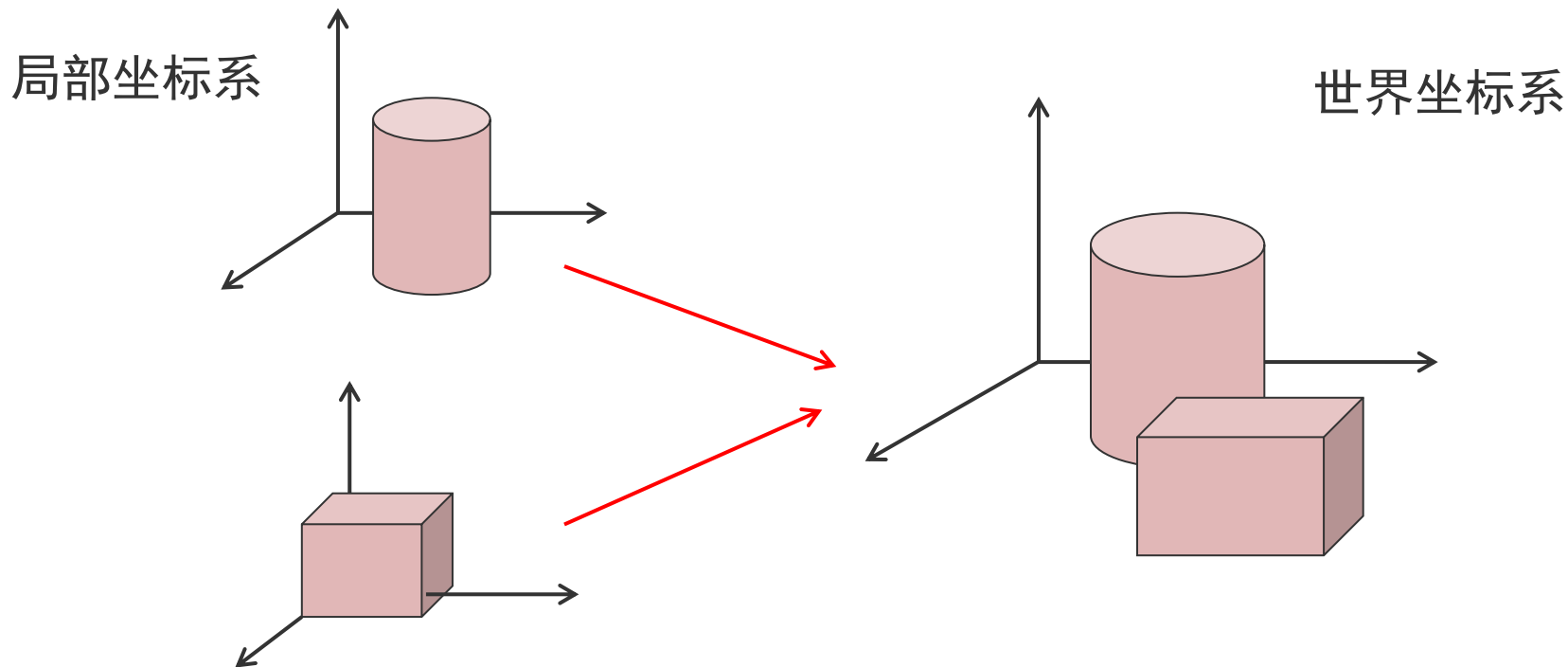
(2) 建模坐标系

又称为局部坐标系。每个物体（对象）有它自己的局部中心和坐标系



建模坐标系独立于世界坐标系来定义物体的几何特性

一旦定义了“局部”物体，就可以很容易地将“局部”物体放入世界坐标系内，使它由局部上升为全局

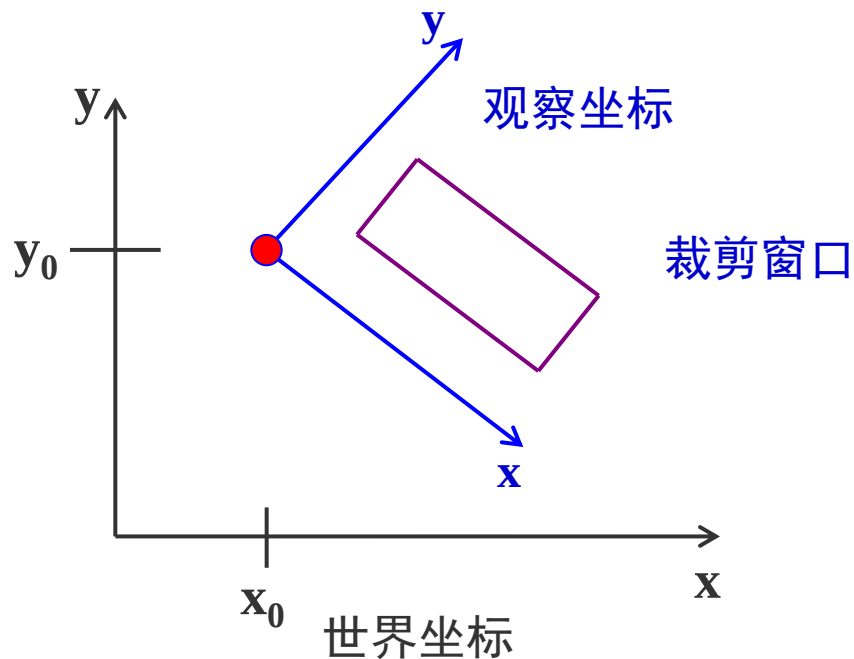


(3) 观察坐标系

观察坐标系主要用于从观察者的角度对整个世界坐标系内的对象进行重新定位和描述

依据观察窗口的方向和形状在世界坐标系中定义的坐标系称为观察坐标系。观察坐标系用于指定图形的输出范围

二维观察变换的一般方法是在世界坐标系中指定一个观察坐标系统，以该系统为参考通过选定方向和位置来制定矩形剪裁窗口



(4) 设备坐标系

适合特定输出设备输出对象的坐标系。比如屏幕坐标系

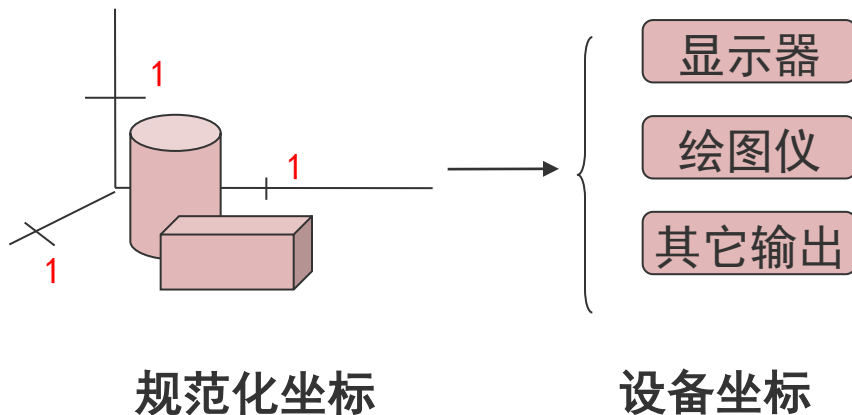
在多数情况下，对于每一个具体的显示设备，都有一个单独的坐标系统

注意：设备坐标是**整数**

(5) 规范化坐标系

规范化坐标系独立于设备，能容易地转变为设备坐标系，是一个中间坐标系。

为使图形软件能在不同的设备之间移植，采用规范化坐标，坐标轴取值范围是0-1



要想建立观察坐标系，需要已知三个要素：

观察点的位置

观察的方向

世界坐标系的上向量

观察坐标系通常以视点的位置为原点，由视点的位置和观察的方向即可确定 Z 轴

确定与 X 轴垂直的平面，世界坐标系的上向量在该平面上的投影即 Y 轴；由 Z 轴和 Y 轴，通过左手定则即可确定 X 轴

二维图形变换

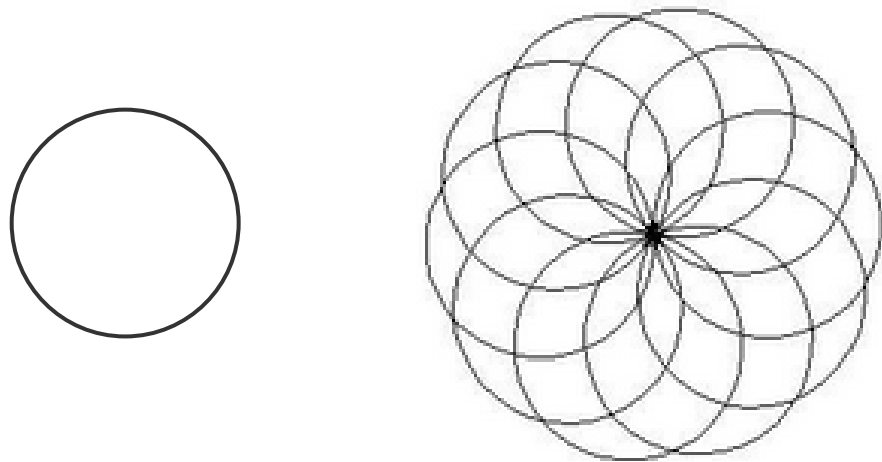
一、图形变换

1、图形变换的用途

图形变换和观察是计算机图形学的基础内容之一，也是图形显示过程中不可缺少的一个环节

一个简单的图形，通过各种变换（如：比例、旋转、镜象、错切、平移等）可以形成一个丰富多彩的图形或图案

(1) 由一个基本的图案，经过变换组合成另外一个复杂图形



(2) 用很少的物体组成一个场景

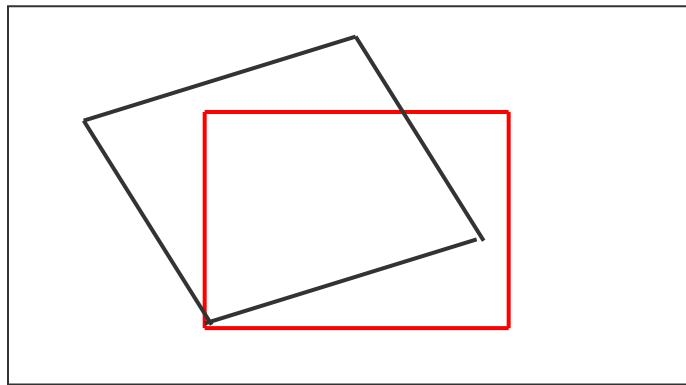
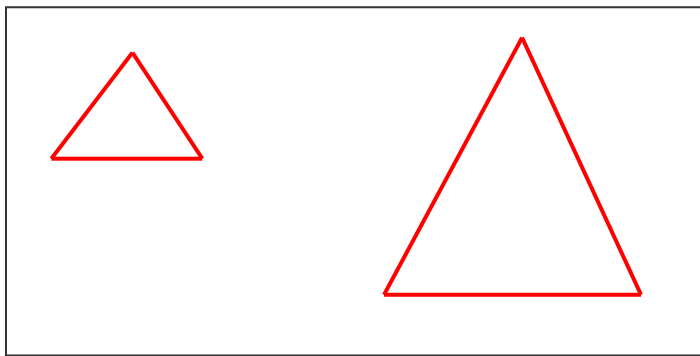


(3) 可以通过图形变换组合得到动画效果

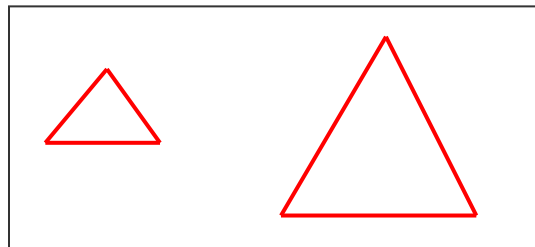
在计算机动画中，经常有几个物体之间的相对运动，可以通过平移和旋转这些物体的局部坐标系得到这种动画效果

2、图形变换的基本原理

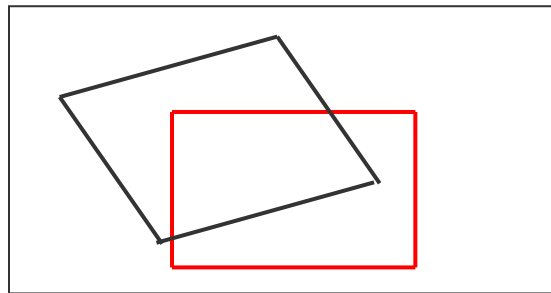
图形是根据什么样的原理来实现各种变换的呢？



(1) 图形变化了，但原图形的
连边规则没有改变



(2) 图形的变化，是因为顶点
位置的改变决定的



变换图形就是要变换图形的几何关系，即改变顶点的坐标；同时，保持图形的原拓扑关系不变

仿射变换（Affine Transformation或 Affine Map）是一种二维坐标到二维坐标之间的线性变换

（1）“平直性”。即：直线经过变换之后依然是直线

（2）“平行性”。即：平行线依然是平行线，且直线上点的位置顺序不变）

$$\begin{cases} x' = ax + by + m \\ y' = cx + dy + n \end{cases}$$

称为**二维仿射变换**（ affine transformation ），其中坐标 x' 和 y' 都是原始坐标 x 和 y 的线性函数

参数 a , b , c , d , m 和 n 是函数的系数

在几何中把向量看成从一个点到另一个点的位移。向量算法提供了一种统一的方法来对几何思想进行代数的表示

二、齐次坐标

在二维平面内，我们是用一对坐标值 (x, y) 来表示一个点在平面内的确切位置，或着说是用一个向量 (x, y) 来标定一个点的位置

假如变换前的点坐标为 (x, y) ，变换后的点坐标为 (x^*, y^*) ，这个变换过程可以写成如下矩阵形式：

$$[x^*, y^*] = [x, y] \bullet M$$

$$\begin{cases} x^* = a_1 x + b_1 y + c_1 \\ y^* = a_2 x + b_2 y + c_2 \end{cases}$$

$$[x^*, y^*] = [x \quad y \quad 1] \begin{bmatrix} a_1 & a_2 \\ b_1 & b_2 \\ c_1 & c_2 \end{bmatrix}$$

上两式是完全等价的。对于向量 $(x, y, 1)$ ，可以在几何意义上理解为是在第三维为常数的平面上的一个二维向量。

这种用三维向量表示二维向量，或者一般而言，用一个 $n+1$ 维的向量表示一个 n 维向量的方法称为齐次坐标表示法

n 维向量的变换是在 $n+1$ 维的空间进行的，变换后的 n 维结果是被反投回到感兴趣的特定的维空间内而得到的。

如 n 维向量 $(p_1, p_2, \dots, p_n)$ 表示为 $(hp_1, hp_2, \dots, hp_n, h)$,
其中 h 称为哑坐标。

普通坐标与齐次坐标的关系为“一对多”：

普通坐标 $\times h \rightarrow$ 齐次坐标

齐次坐标 $\div h \rightarrow$ 普通坐标

当 $h = 1$ 时产生的齐次坐标称为“规格化坐标”，因为前 n 个坐标就是普通坐标系下的 n 维坐标

为什么要采用齐次坐标？

在笛卡儿坐标系内，向量 (x, y) 是位于 $z=0$ 的平面上的点；而向量 $(x, y, 1)$ 是位于 $z=1$ 的等高平面上的点

对于图形来说，没有实质性的差别，但是却给后面矩阵运算提供了可行性和方便性

采用了齐次坐标表示法，就可以统一地把二维线形变换表示如下式所示的规格化形式：

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} a_1 & a_2 & 0 \\ b_1 & b_2 & 0 \\ c_1 & c_2 & 1 \end{bmatrix}$$

对于一个图形，可以用顶点表来描述图形的几何关系，用连边表来描述图形的拓扑关系。所以对图形的变换，最后是只要变换图形的顶点表

三、基本几何变换

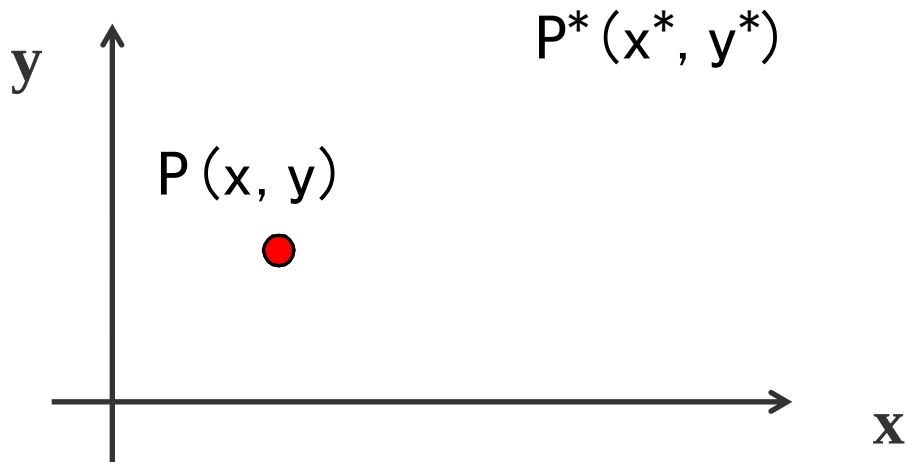
图形的**几何变换**是指对图形的几何信息经过平移、比例、旋转等变换后产生新的图形

$p(x, y)$ $\longrightarrow$ 表示平面上一个未被变换的点

$p^*(x^*, y^*)$ $\longrightarrow$ 变换后的新点坐标

1、平移变换

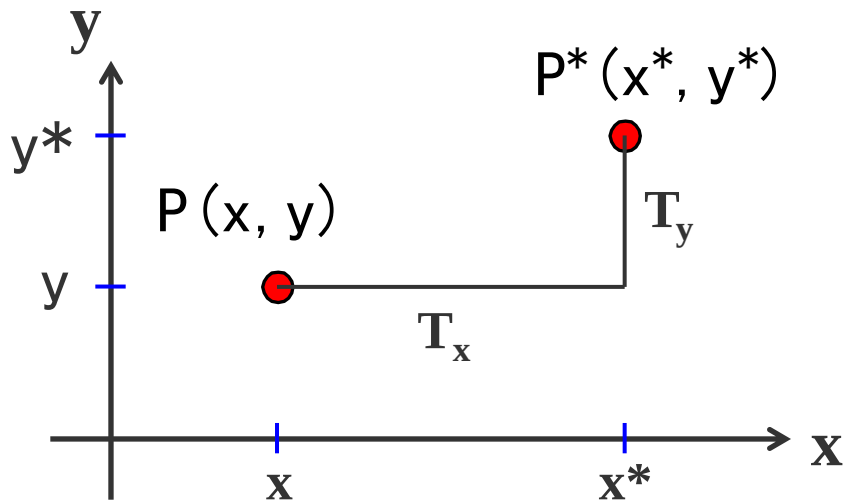
平移是指将p点沿直线路径从一个坐标位置移到另一个坐标位置的重定位过程



即新的坐标分别在x方向和y方向增加了一个增量和，使得：

$$\begin{cases} x^* = x + T_x \\ y^* = y + T_y \end{cases}$$

T_x , T_y 称为**平移矢量**



$$\begin{cases} x^* = x + Tx \\ y^* = y + Ty \end{cases}$$

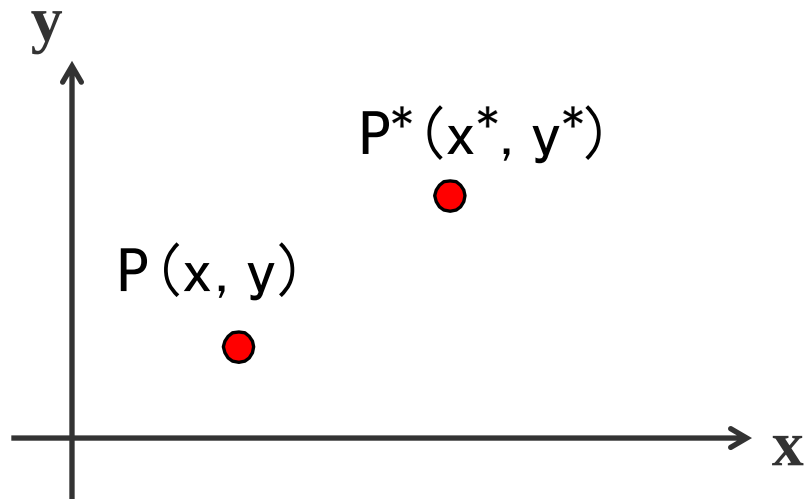
$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ Tx & Ty & 1 \end{bmatrix} = \begin{bmatrix} x + Tx & y + Ty & 1 \end{bmatrix}$$

平移是一种不产生变形而移动物体的**刚体变换**，即物体上的每个点移动相同数量的坐标

2、比例变换

比例变换是指对p点相对于坐标原点沿x方向放缩 S_x 倍，沿y方向放缩 S_y 倍。其中 S_x 和 S_y 称为比例系数

$$\begin{cases} x^* = x \bullet S_x \\ y^* = y \bullet S_y \end{cases}$$

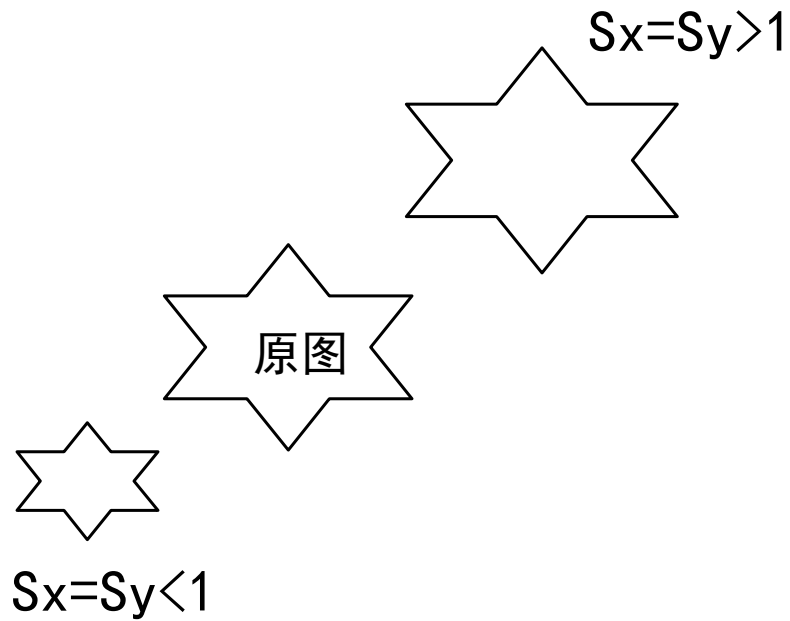


比例变换的齐次坐标计算形式如下：

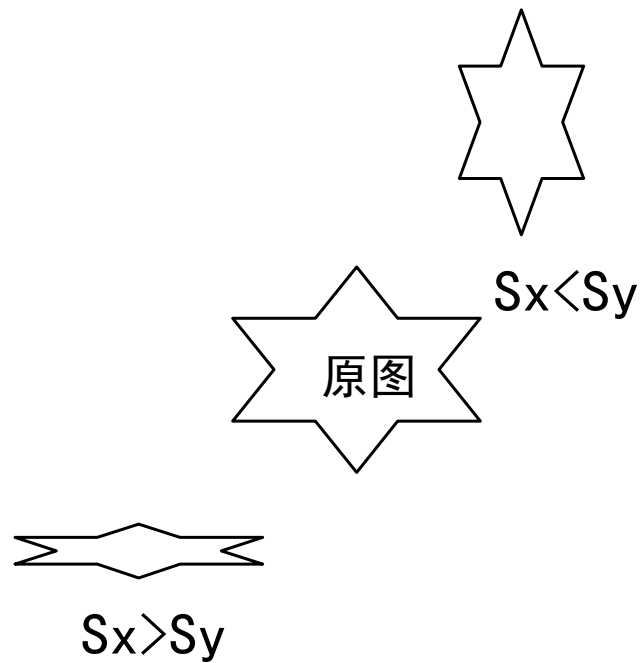
$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} S_x \bullet x & S_y \bullet y & 1 \end{bmatrix}$$

缩放系数 S_x 和 S_y 可赋予任何正整数。值小于1缩小物体的尺寸，值大于1则放大物体，都指定为1，物体尺寸就不会改变

(a) $S_x=S_y$ 比例



(b) $S_x\neq S_y$ 比例



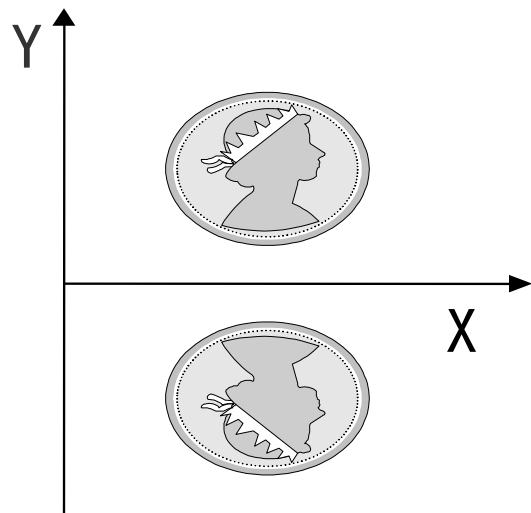
当 $S_x=S_y$ 时，变换成为整体比例变换，用以下矩阵进行计算：

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & S \end{bmatrix} = \begin{bmatrix} x & y & S \end{bmatrix} = \begin{bmatrix} \frac{x}{S} & \frac{y}{S} & 1 \end{bmatrix}$$

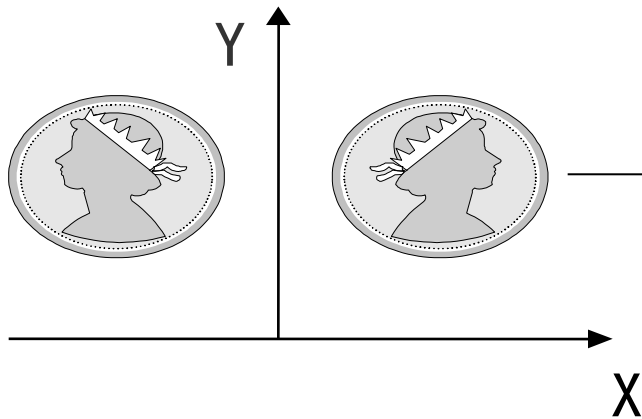
整体比例变换时，若 $S>1$ ，图形整体缩小；若 $0<S<1$ ，图形整体放大；若 $S<0$ ，发生关于原点的对称等比变换

3、对称变换

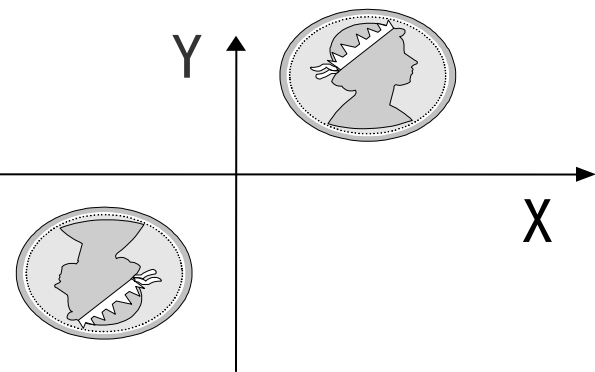
对称变换也称为反射变换或镜像变换，变换后的图形是原图形关于某一轴线或原点的镜像。



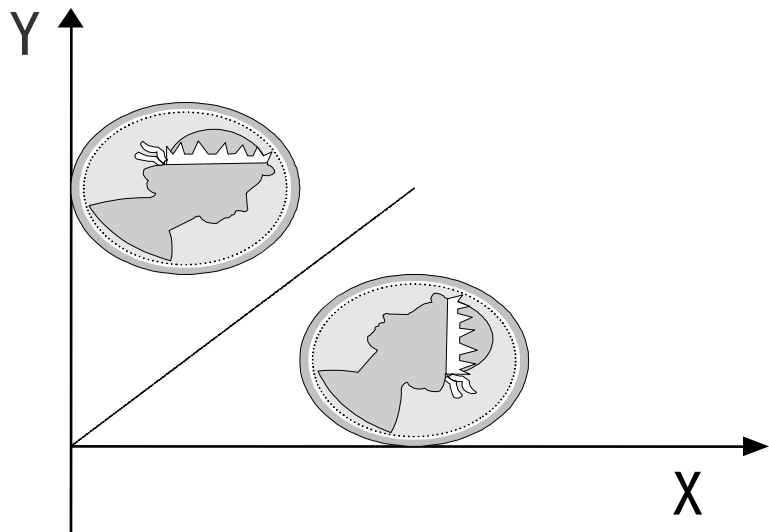
关于x轴对称



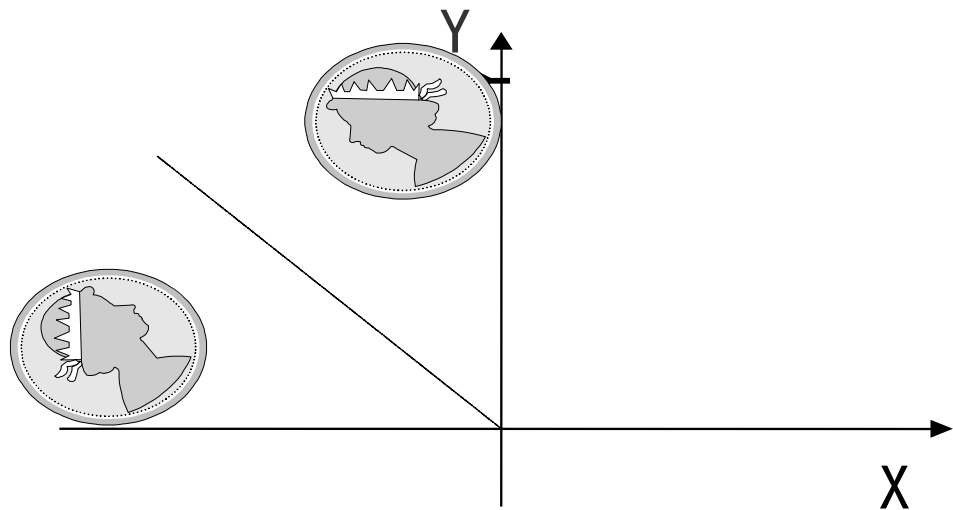
关于y轴对称



关于原点对称



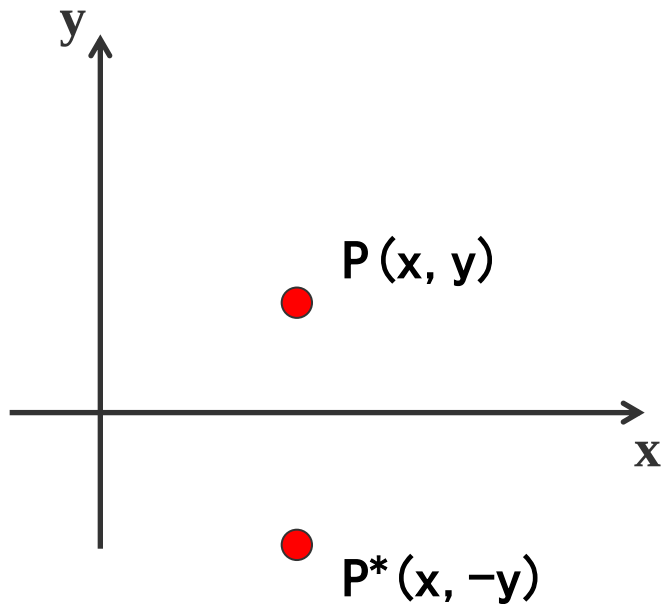
关于 $x=y$ 对称



关于 $x=-y$ 对称

(a) 关于X轴对称

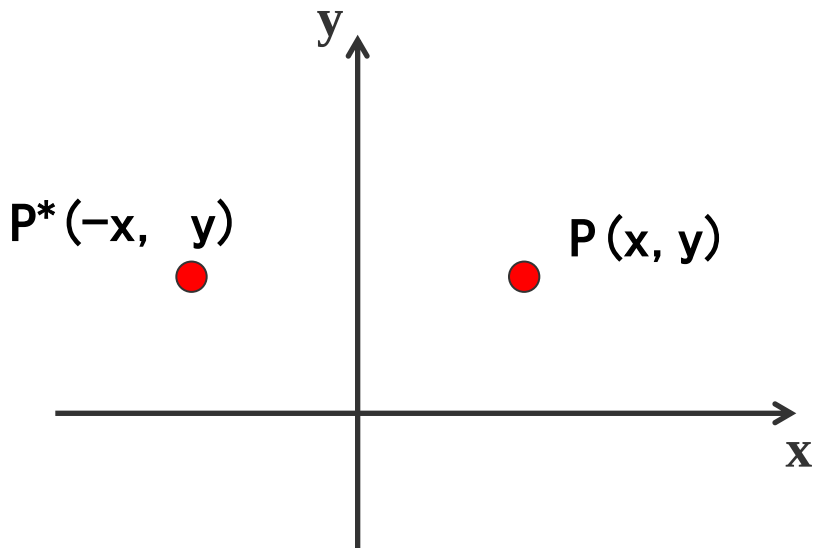
点P经过关于X轴的对称变换后形成点P^{*}，则 $x^*=x$ 且 $y^*=-y$ ，写成齐次坐标的计算形式为：



$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x & -y & 1 \end{bmatrix}$$

(b) 关于y轴对称

点P经过关于X轴的对称变换后形成点P^{*}，则 $x^*=-x$ 且 $y^*=y$ ，写成齐次坐标的计算形式为：

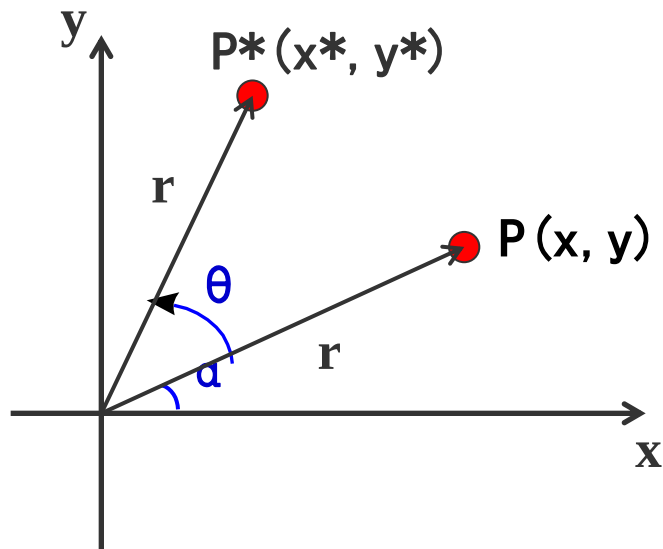


$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -x & y & 1 \end{bmatrix}$$

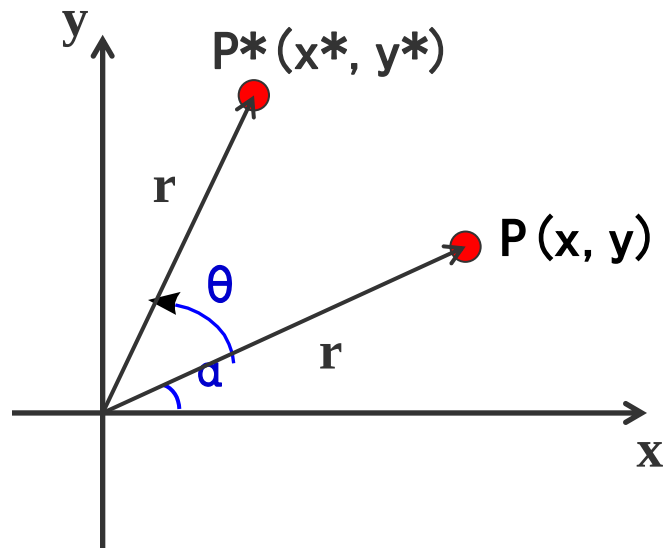
4、旋转变换

二维旋转是指将P点绕坐标原点转动某个角度 θ （逆时针为正，顺时针为负）得到新的点 P^* 的重定位过程

首先确定当基准点为坐标原点时，点位置P旋转的变换方程



应用标准三角特性，利用角度 α 和 θ 将转换后的坐标表示为：



$$\begin{cases} x^* = r \cos(\alpha + \theta) = r \cos \alpha \cos \theta - r \sin \alpha \sin \theta \\ y^* = r \sin(\alpha + \theta) = r \cos \alpha \sin \theta + r \sin \alpha \cos \theta \end{cases}$$

在极坐标系中点的原始坐标为：

$$\begin{cases} x = r \cos \alpha \\ y = r \sin \alpha \end{cases}$$

将 x 、 y 代入上式，就得到相对于原点将在位置 (x, y) 处的点旋转 θ 角的变换方程：

$$\begin{cases} x^* = x \cos \theta - y \sin \theta \\ y^* = x \sin \theta + y \cos \theta \end{cases}$$

二维图形绕原点逆时针旋转 θ 角的齐次坐标计算形式可写为：

$$\begin{aligned} \begin{bmatrix} x^* & y^* & 1 \end{bmatrix} &= \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} x \bullet \cos \theta - y \bullet \sin \theta & x \bullet \sin \theta + y \bullet \cos \theta & 1 \end{bmatrix} \end{aligned}$$

绕原点顺时针旋转 θ 角的齐次坐标计算形式可写为：

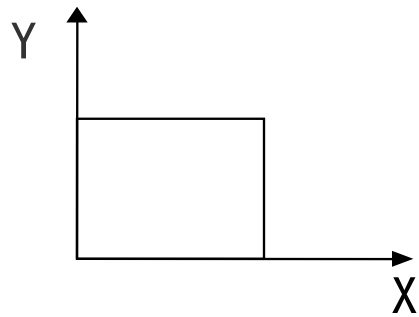
$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} \cos(-\theta) & \sin(-\theta) & 0 \\ -\sin(-\theta) & \cos(-\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5、错切变换

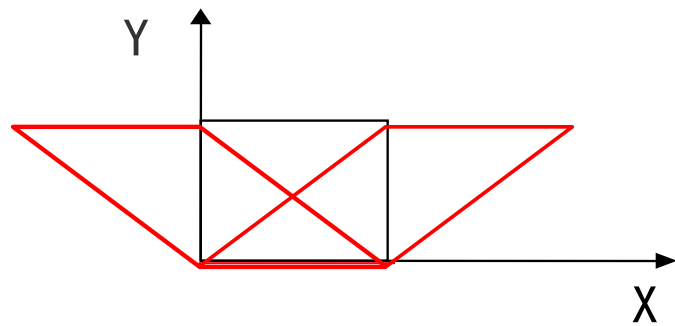
在图形学的应用中，有时需要产生弹性物体的变形处理，这就要用到错切变换。

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ T_x & T_y & 1 \end{bmatrix} \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

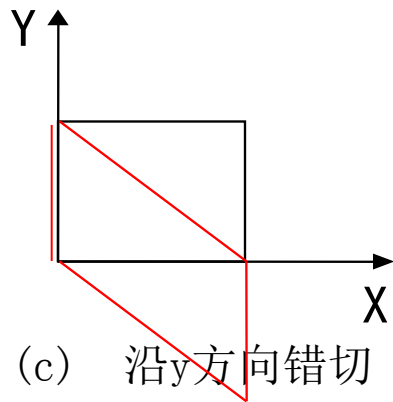
变换矩阵中的非对角线元素大都为零，若变换矩阵中的非对角元素不为0，则意味着x, y同时对图形的变换起作用。也就是说，变换矩阵中非对角线元素起着把图形沿x方向或y方向错切的作用。



(a) 原图



(b) 沿x方向错切



(c) 沿y方向错切

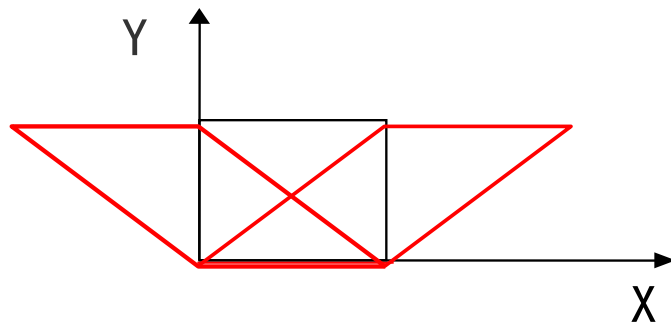
x值或y值越小，错切量越小； x值或y值越大，错切量越大。
其变换矩阵为：

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & b & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x + cy & bx + y & 1 \end{bmatrix}$$

(1) 沿x方向错切

当 $b = 0$ 时，有：

$$\begin{cases} x^* = x + cy \\ y^* = y \end{cases}$$



为什么要采用齐次坐标？

假如变换前的点坐标为 (x, y) ，变换后的点坐标为 (x^*, y^*) ，这个变换过程可以写成如下矩阵形式：

$$\begin{bmatrix} x^* & y^* \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \bullet T_{2 \times 2}$$

(1) 比例变换

$$\begin{cases} x^* = x \bullet S_x \\ y^* = y \bullet S_y \end{cases}$$

$$\begin{bmatrix} x^* & y^* \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \bullet \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix}$$

(2) 关于X轴对称

$$\begin{bmatrix} x^* & y^* \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{cases} x \\ -y \end{cases}$$

(3) 关于y=x对称

$$\begin{bmatrix} x^* & y^* \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \bullet \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{cases} y \\ x \end{cases}$$

(4) 旋转变换

$$\begin{cases} x^* = x \cos \theta - y \sin \theta \\ y^* = x \sin \theta + y \cos \theta \end{cases}$$

$$\begin{bmatrix} x^* & y^* \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \bullet \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$$

(5) 平移变换

$$\begin{cases} \mathbf{x}^* = \mathbf{x} + \mathbf{l} \\ \mathbf{y}^* = \mathbf{y} + \mathbf{m} \end{cases} \quad (1)$$

$$[x^* \quad y^*] = [x \quad y] \bullet \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad (2)$$

无论矩阵中的几何元素如何变化，都不能获得式（1）的形式。
也就是说，不能用式（2）表示平移变换

将 $T_{2 \times 2}$ 矩阵扩展成 $T_{3 \times 3}$ 矩阵，写成如下的形式：

$$T = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ l & m & 1 \end{bmatrix}$$

根据矩阵乘法定义，只有当第一个矩阵的列数与第二个矩阵的行数相等时才能相乘。将二维点用三维向量来表示：

$$\begin{bmatrix} x & y & 1 \end{bmatrix}$$

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ l & m & 1 \end{bmatrix} = \begin{bmatrix} x+l & y+m & 1 \end{bmatrix}$$

四、复合变换

复合变换是指图形作一次以上的几何变换，变换结果是每次的变换矩阵相乘

从另一方面看，任何一个复杂的几何变换都可以看作基本几何变换的组合形式。

$$\begin{aligned} P' &= P \cdot T = P \cdot (T_1 \cdot T_2 \cdot T_3 \cdots T_n) \\ &= P \cdot T_1 \cdot T_2 \cdot T_3 \cdots T_n \quad (n > 1) \end{aligned}$$

(1) 二维复合平移

p点经过两次连续平移后，其变换矩阵可写为：

$$\begin{aligned} T_t = T_{t1} \cdot T_{t2} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ T_{x1} & T_{y1} & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ T_{x2} & T_{y2} & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ T_{x1} + T_{x2} & T_{y1} + T_{y2} & 1 \end{bmatrix} \end{aligned}$$

(2) 二维复合比例平移

p点经过两个连续比例变换后，其变换矩阵可写为：

$$\begin{aligned} T_s = T_{s1} \cdot T_{s2} &= \begin{bmatrix} S_{x1} & 0 & 0 \\ 0 & S_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} S_{x2} & 0 & 0 \\ 0 & S_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} S_{x1} \cdot S_{x2} & 0 & 0 \\ 0 & S_{y1} \cdot S_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

结果矩阵表明连续比例变换是相乘的

(3) 二维复合旋转

p点经过两个连续旋转变换后，其变换矩阵可写为：

$$\begin{aligned} T_r = T_{r1} \cdot T_{r2} &= \begin{bmatrix} \cos \theta_1 & \sin \theta_1 & 0 \\ -\sin \theta_1 & \cos \theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta_2 & \sin \theta_2 & 0 \\ -\sin \theta_2 & \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos(\theta_1 + \theta_2) & \sin(\theta_1 + \theta_2) & 0 \\ -\sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

结果矩阵表明两个连续旋转是相加的

在进行复合变换时，需要注意的是矩阵相乘的顺序

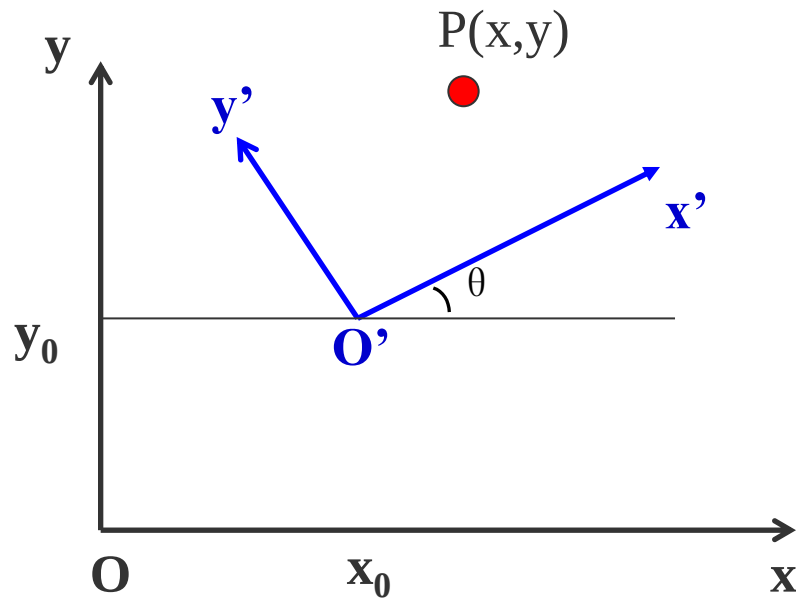
由于矩阵乘法**不满足交换率**，因此通常 $T_1 * T_2 \neq T_2 * T_1$ ，即

矩阵相乘的顺序不可交换

(4) 坐标系之间的变换

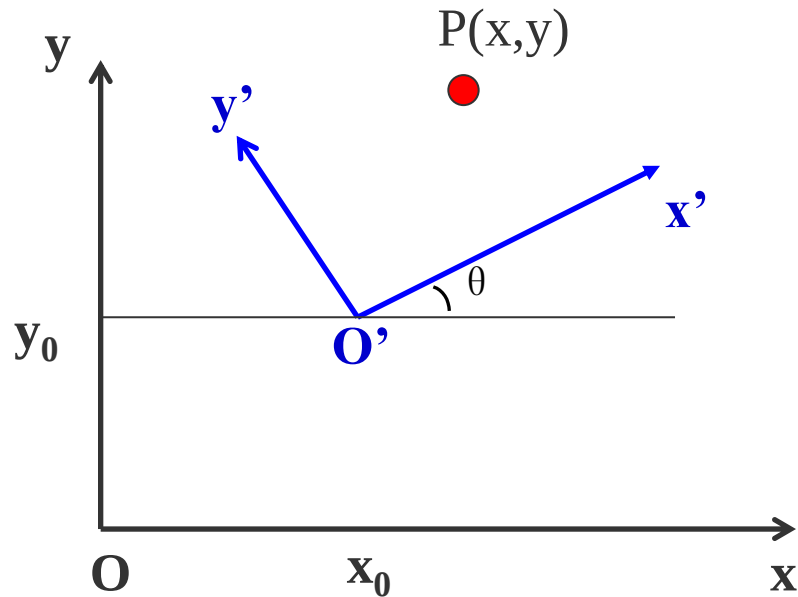
图形变换经常需要从一个坐标系变换到另一个坐标系

例：右图显示了两个笛卡儿坐标系 x_0y_0 和 $x' y'$ ，而 O' 点在 x_0y_0 坐标系的 (x_0, y_0) 处。



为了将 $p(x_p, y_p)$ 点从 x_0y_0 坐标系变换到 $x' \ O' \ y'$ 坐标系, 如何进行计算?

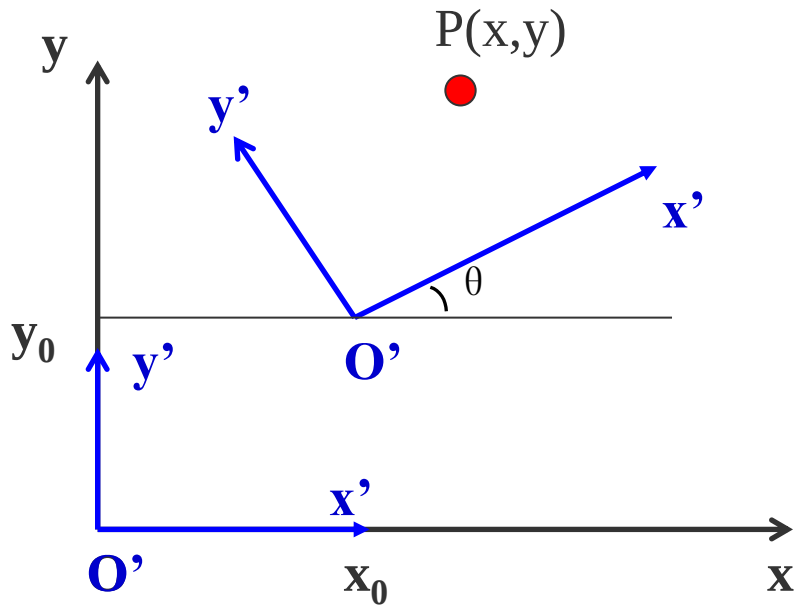
需建立变换使 $x' \ O' \ y'$ 坐标系与 x_0y_0 坐标系重合



可以分两步来进行：

① 将 x' $0'$ y' 坐标系的原点平移至 x_0y 坐标系的原点——**平移变换**

② 将 x' 轴旋转到 x 轴上——**旋转变换**



上述变换步骤可用变换矩阵表示：

$$T = T_t \cdot T_r = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_0 & -y_0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos(-\theta) & \sin(-\theta) & 0 \\ -\sin(-\theta) & \cos(-\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_0 & -y_0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(5) 相对任意参考点的二维几何变换

比例、旋转变换等均与参考点相关。如要对某个参考点 (x_f, y_f) 作二维几何变换，其变换过程如下：

- a、将固定点移至坐标原点，此时进行平移变换
- b、针对原点进行二维几何变换
- c、进行反平移，将固定点又移回到原来的位置

五、二维变换矩阵

二维空间中某点的变化可以表示成点的齐次坐标与3阶的二维变换矩阵 T_{2d} 相乘，即：

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot T_{2D}$$
$$= \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} a & b & p \\ c & d & q \\ l & m & s \end{bmatrix}$$

$T_1 = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 对图形进行比例、旋转、对称、错切等变换；

$T_2 = \begin{bmatrix} l & m \end{bmatrix}$ 对图形进行平移变换

$T_3 = \begin{bmatrix} p \\ q \end{bmatrix}$ 是对图形作投影变换

$T_4 = [s]$ 是对图形作整体比例变换

$$\left[\begin{array}{cc|c} a & b & p \\ c & d & q \\ \hline l & m & s \end{array} \right]$$

六、二维图形几何变换的计算

几何变换均可表示成： $P^*=P \cdot T$ 的形式，其中， P 为变换前二维图形的规范化齐次坐标， P^* 为变换后的规范化齐次坐标， T 为变换矩阵。

1、点的变换

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet T$$

2、直线的变换

直线的变换可以通过对直线两端点进行变换，从而改变直线的位置和方向

$$\begin{bmatrix} x_1^* & y_1^* & 1 \\ x_2^* & y_2^* & 1 \end{bmatrix} = \begin{bmatrix} x_1 & y_1 & 1 \\ y_1 & y_2 & 1 \end{bmatrix} \bullet T$$

3、多边形的变换

多边形变换是将变换矩阵作用到每个顶点的坐标位置，并按新的顶点坐标值和当前属性设置来生成新的多边形

$$P = \begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \\ \dots & \dots & \dots \\ x_n & y_n & 1 \end{bmatrix}$$

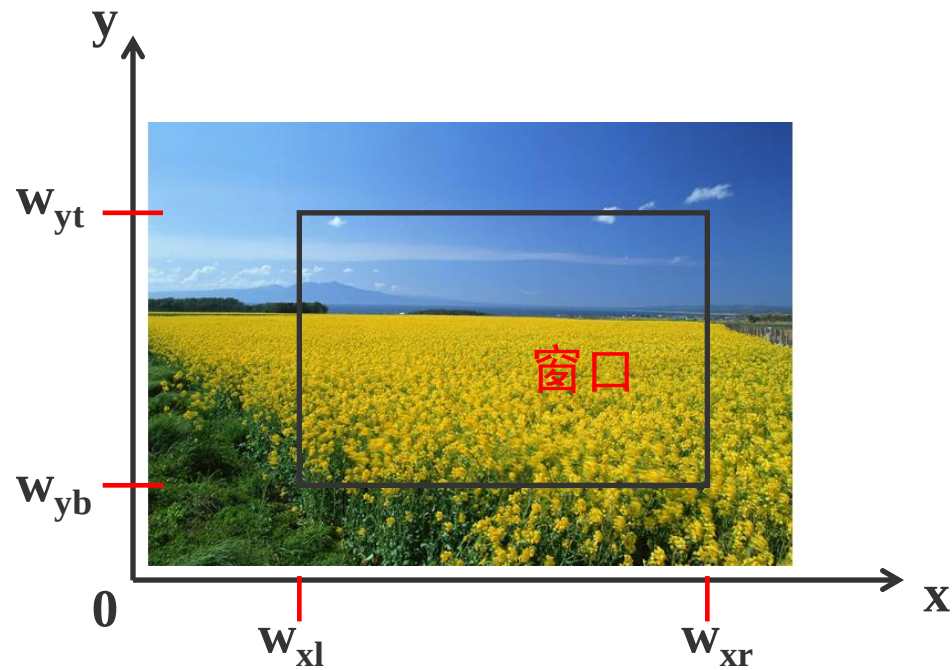
窗口、视区及变换

一、窗口和视区

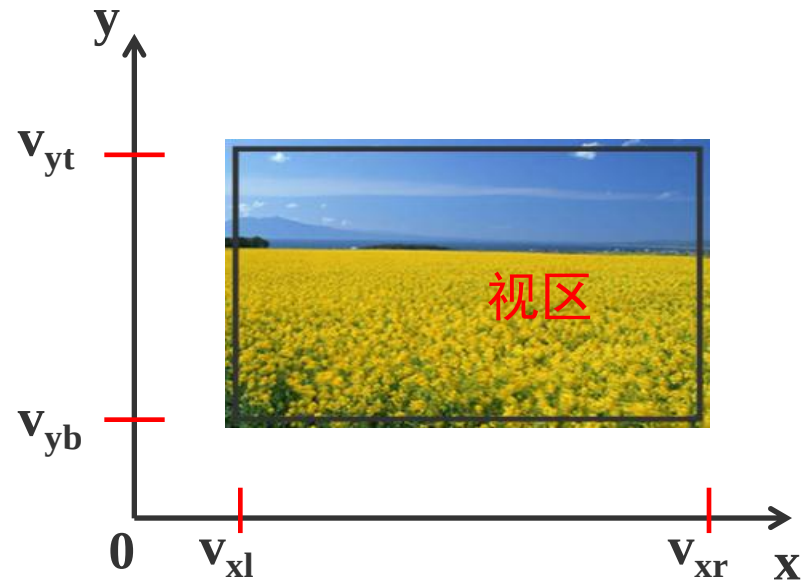
世界坐标系中要显示的区域（通常在观察坐标系内定义）称为窗口

窗口映射到显示器(设备)上的区域称为视区

窗口定义显示什么；视区定义在何处显示

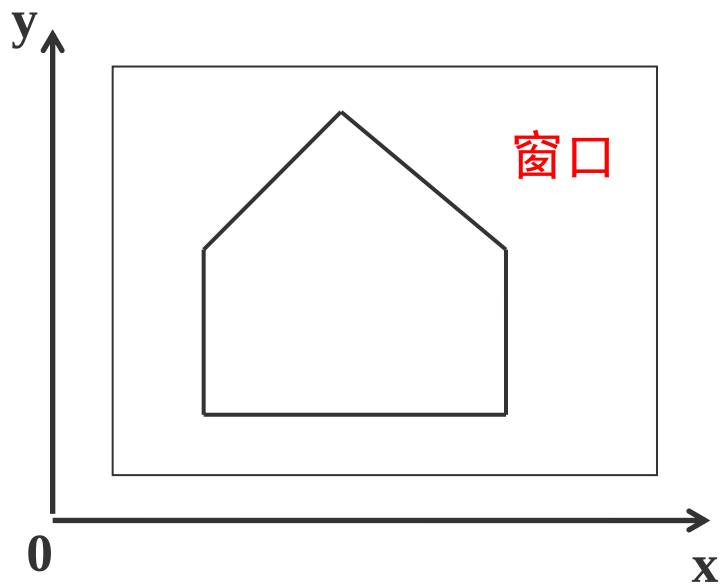


世界坐标系

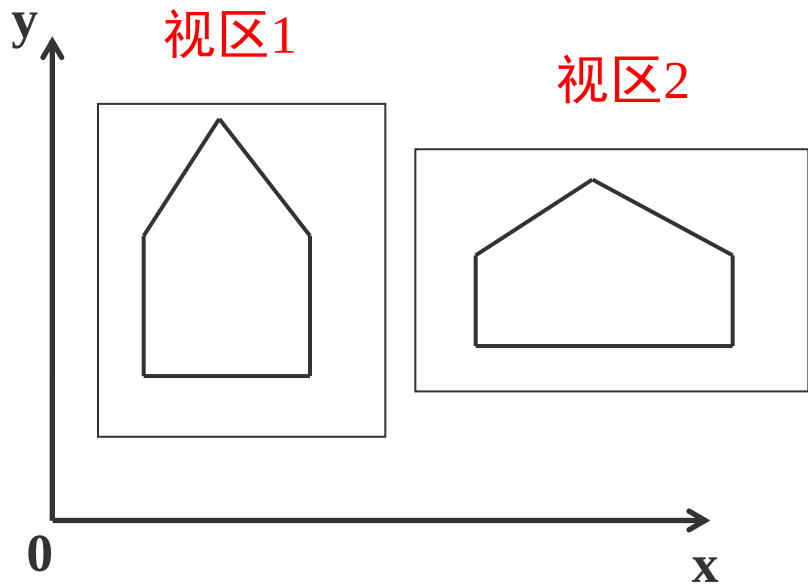


设备坐标系

世界坐标系中的一个窗口可以对应于多个视区



世界坐标系



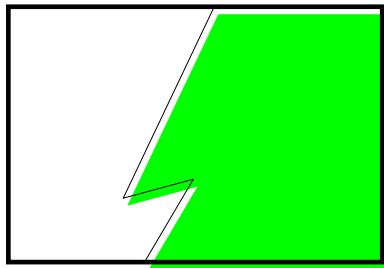
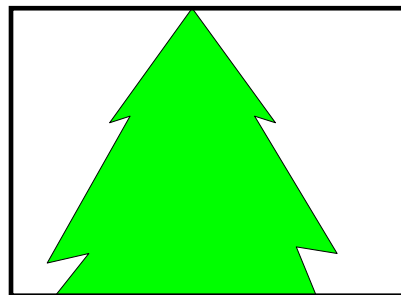
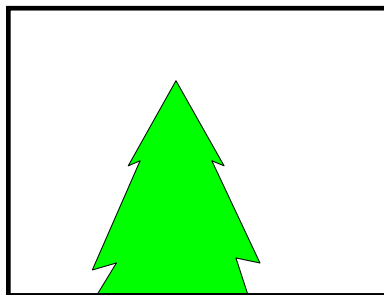
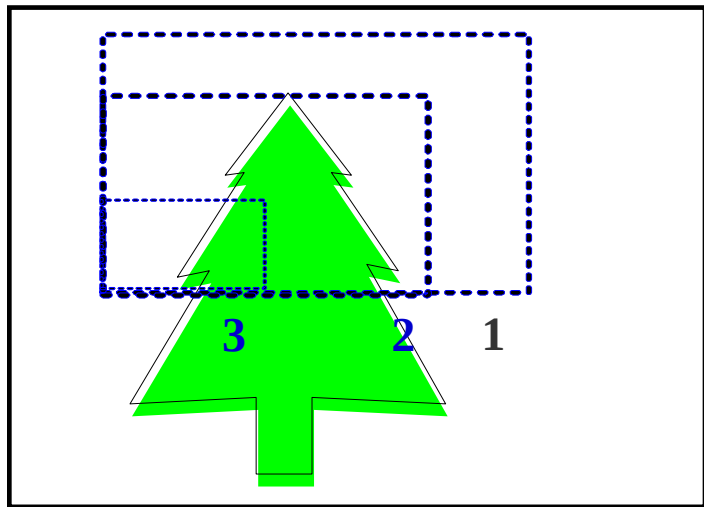
屏幕坐标系

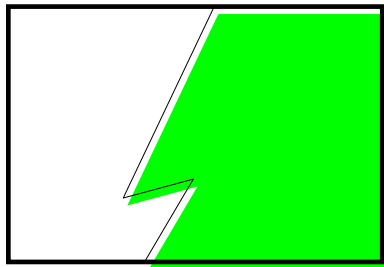
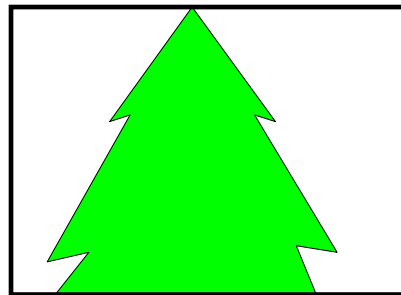
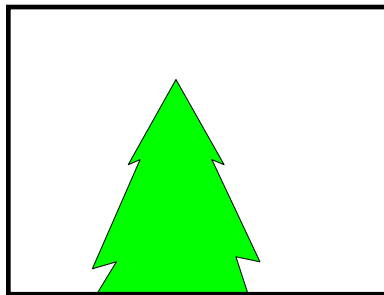
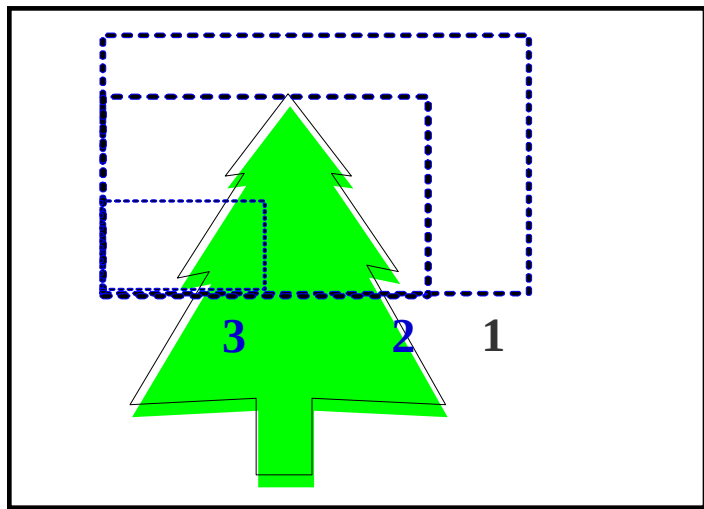
如何将窗口内的图形在视区中显示出来呢？

必须经过将窗口到视区的变换处理，这种变换就是观察变换（Viewing Transformation）

二、观察变换

1、变焦距效果





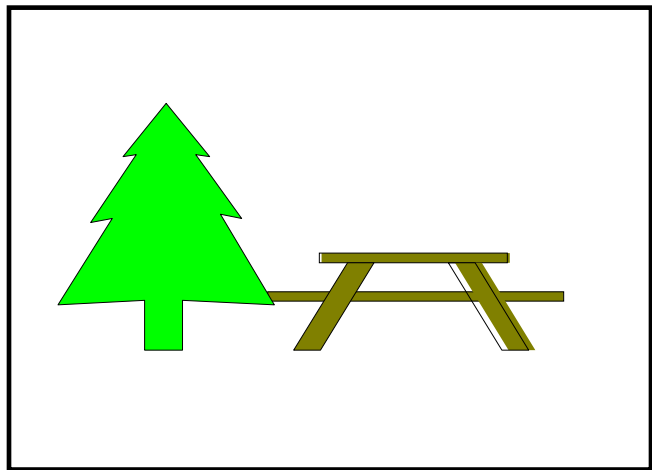
当窗口变小时，由于视区大小不变，就可以放大图形对象的某一部分，从而观察到在较大的窗口时未显示出的细节

而当窗口变大，视区不变时，会出现什么情况呢？

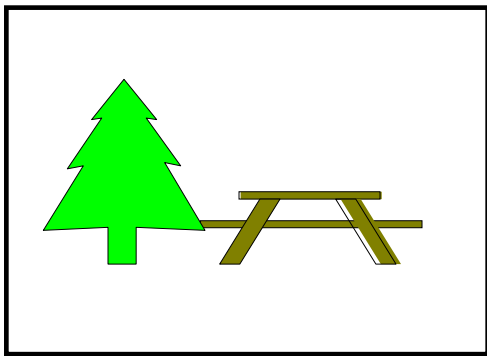
这类似于照相机的变焦处理

2、整体缩放效果

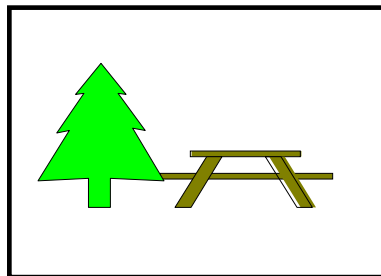
当窗口大小不变而视区大小发生变化时，得到整体放缩效果。这种放缩不改变观察对象的内容



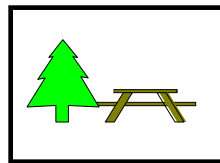
原图及窗口



视区1



视区2



视区3

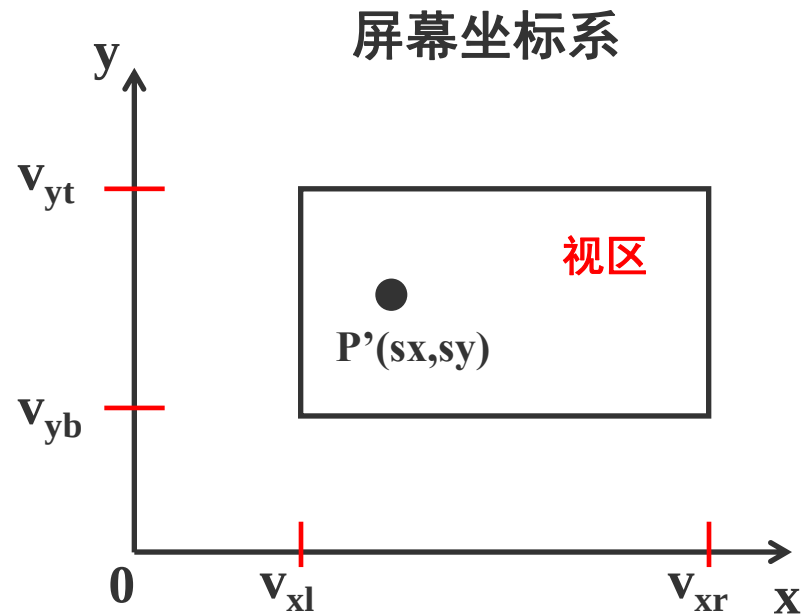
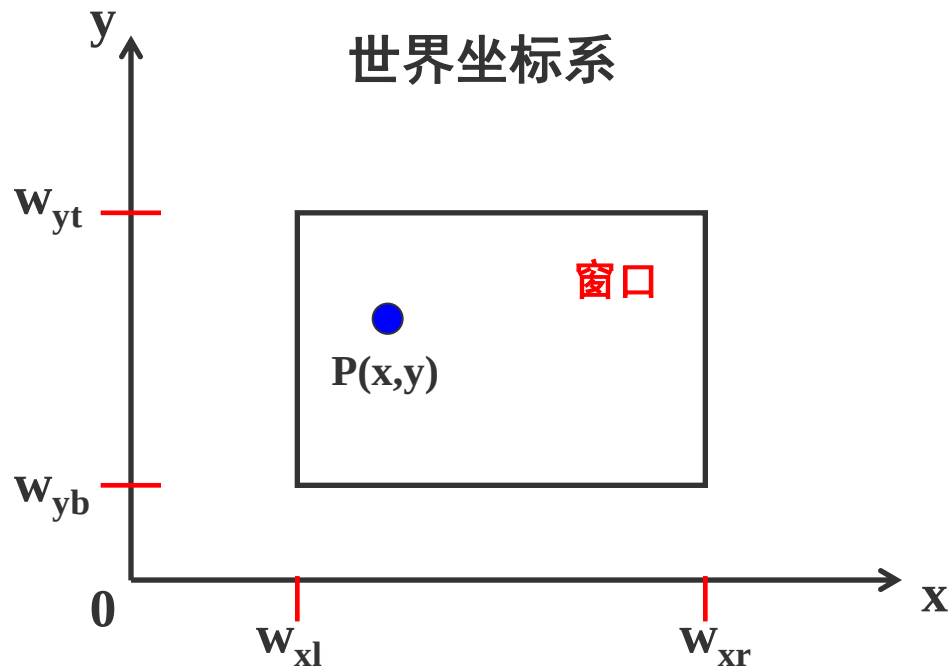
如果把一个固定大小的窗口在一幅大图形上移动，视区不变，会产生什么效果？

漫游效果！

三、窗口到视区的变换

为了全部、如实地在视区中显示出窗口内的图形对象，就必须求出图形在窗口和视区间的映射关系

需要根据用户所定义参数，找到窗口和视区之间的坐标对应关系



窗口到视区的映射是基于一个等式，即对每一个在世界坐标下的点 (x, y) ，产生屏幕坐标系中的一个点 (sx, sy)

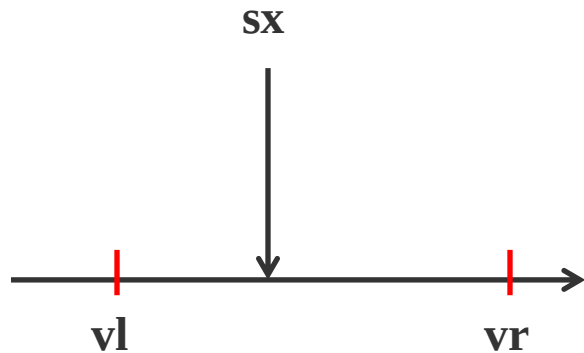
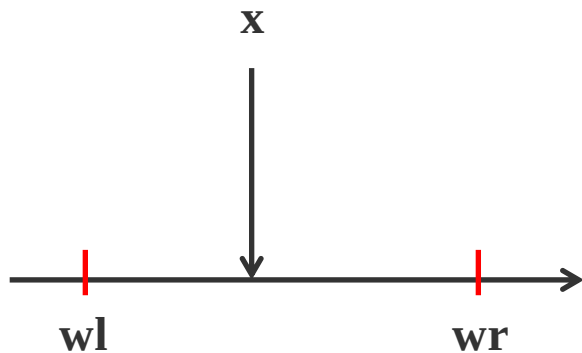
这个映射是“保持比例”的映射

保持比例的性质使得这个映射有线性形式：

$$sx = A * x + C$$

$$sy = B * y + D$$

其中A、B、C、D是常数



首先考虑 x 的映射。保持比例的性质说明：

$$sx - vl \quad vr - vl \quad \longrightarrow \quad \frac{sx - vl}{vr - vl}$$

$$x - wl \quad wr - wl \quad \longrightarrow \quad \frac{x - wl}{wr - wl}$$

$$\frac{sx - vl}{vr - vl} = \frac{x - wl}{wr - wl}$$

$$\begin{aligned} sx &= A * x + C \\ sy &= B * y + D \end{aligned}$$

$$sx = \frac{x - wl}{wr - wl} (vr - vl) + vl$$

$$sx = \frac{vr - vl}{wr - wl} * x + \left(vl - \frac{vr - vl}{wr - wl} * wl \right)$$

A看做放大x的部分，而C看做常数

$$A = \frac{vr - vl}{wr - wl} \quad C = vl - A * wl$$

同理，y方向上保持比例性质满足：

$$\frac{sy - vb}{vt - vb} = \frac{y - wb}{wt - wb} \quad \begin{aligned} sx &= A * x + C \\ sy &= B * y + D \end{aligned}$$

$$B = \frac{vt - vb}{wt - wb} \quad D = vb - B * wb$$

这个映射可用于任意点（x, y），不管它是否在窗口之中。在窗口中的点映射到视口中的点，在窗口外的点映射到视口外的点

二维几何变换小结

主要讲述向量的基本知识、坐标系的分类、齐次坐标、二维变换等、窗口与视区

一、向量基本知识

为了处理二维、三维图形，向量是很重要的一个分析计算工具

讲述了向量的定义、基本运算、线性组合、叉积、点积等

二、坐标系的分类

坐标系是建立图形与数之间对应联系的参考系

在计算机图形学中，从物体（场景）的建模，到在不同显示设备上显示，需要使用一系列不同的坐标系

1、世界坐标系：是一个公共坐标系，是现实中物体或场景的统一参照系

2、局部坐标系：又称为局部坐标系。每个物体（对象）
有它自己的局部中心和坐标系

观察坐标系、设备坐标系、规格化坐标系

三、二维图形几何变换

图形变换和观察是计算机图形学的基础内容之一，也是图形显示过程中不可缺少的一个环节

1、齐次坐标

用一个 $n+1$ 维的向量表示一个 n 维向量的方法称为齐次坐标表示法。对于图形来说，没有实质性的差别，但是却给后面矩阵运算提供了可行性和方便性

2、二维几何变换

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} a & b & p \\ c & d & q \\ l & m & s \end{bmatrix}$$

$$T_1 = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad \text{比例、旋转、对称、错切等变换}$$

$$T_4 = [s]$$

整体比例变换

$$T_2 = \begin{bmatrix} l & m \end{bmatrix} \quad \text{平移变换}$$

$$T_3 = \begin{bmatrix} p \\ q \end{bmatrix} \quad \text{投影变换}$$

3、物体变换和坐标变换

有两种方式去看待一个变换：一种是物体变换，另一种是坐标变换。物体变换使用同一个规则改变物体上所有的点，但是保证底层坐标系不变

坐标变换按照原坐标系定义了一个全新的坐标系，然后在新坐标系下表示物体上所有的点

两种变换紧密联系，各有各的长处

4、复合坐标

也称组合变换。实际应用中很少只需要单一的基本变换，通常要构造一个几种基本变换的组合变换

组合变换的变换矩阵是几个单独变换矩阵的乘积

由于矩阵乘法不满足交换率，因此在进行复合变换时，需要注意的是矩阵相乘的顺序

四、窗口视区及变换

1、窗口

世界坐标系中要显示的区域称为窗口

2、视区

窗口映射到显示器(设备)上的区域称为视区

3、窗口到视区的变换

为了全部、如实地在视区中显示出窗口内的图形对象，就必须求出图形在窗口和视区间的映射关系

计算机图形学

第三章： 三维图形几何变换

主要解决以下几个问题：

如何对三维图形进行方向、尺寸和形状方面的变换？

三维物体如何在二维输出设备上输出？

通过三维图形变换，可由简单图形得到复杂图形，三维图形变换则分为三维几何变换和投影变换

一、三维物体基本几何变换

三维物体的几何变换是在二维方法基础上增加了对 z 坐标的考虑而得到的

有关二维图形几何变换的讨论，基本上都适合于三维空间。从应用角度看，三维空间几何变换直接与显示、造型有关，因此更为重要

同二维变换一样，三维基本几何变换都是相对于坐标原点和坐标轴进行的几何变换：有平移、比例、旋转、对称和错切等

与二维变换类似，引入齐次坐标表示，即：三维空间中某点的变换可以表示成点的齐次坐标与四阶的三维变换矩阵相乘

$$p' = \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = p \cdot T_{3D}$$

$$= \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} a & b & c & p \\ d & e & f & q \\ g & h & i & r \\ l & m & n & s \end{bmatrix}$$

根据 T_{3D} 在变换中所起的具体作用，进一步可将 T_{3D} 分成四个矩阵。即：

$$T_{3D} = \left[\begin{array}{ccc|c} a & b & c & p \\ d & e & f & q \\ g & h & i & r \\ \hline l & m & n & s \end{array} \right]$$

$$T_1 = \left[\begin{array}{ccc} a & b & c \\ d & e & f \\ g & h & i \end{array} \right] \quad \text{对点进行比例、对称、旋转、错切变换}$$

$$T_2 = [l \quad m \quad n] \quad \text{对点进行平移变换}$$

$$T_3 = \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

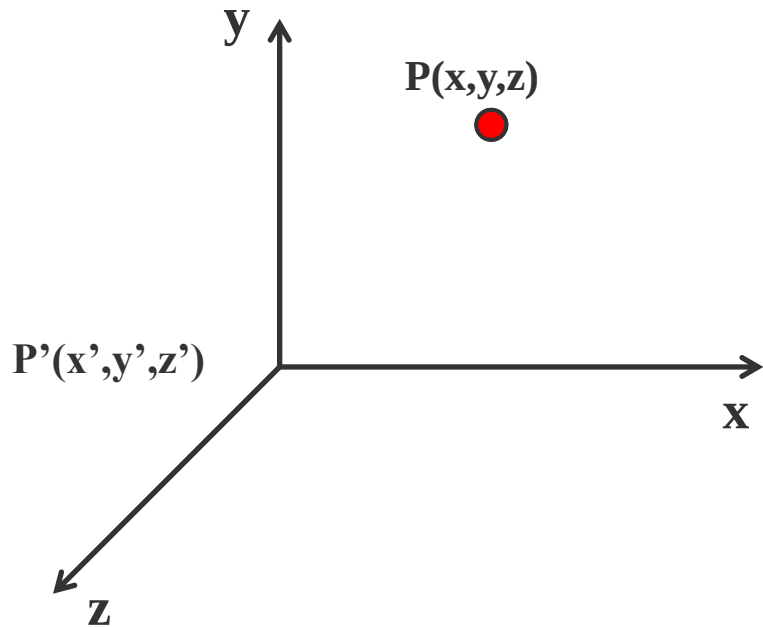
作用是进行透视投影变换

$$T_4 = [s]$$

作用是产生整体比例变换

1、平移变换

若三维物体沿 x , y , z 方向上移动一个位置，而物体的大小与形状均不变，则称为平移变换



点P的平移变换矩阵表示如下：

$$[x' \quad y' \quad z' \quad 1] = [x \quad y \quad z \quad 1] \cdot T_t$$

$$= [x \quad y \quad z \quad 1] \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ T_x & T_y & T_z & 1 \end{bmatrix}$$

$$= [x + T_x \quad y + T_y \quad z + T_z \quad 1]$$

2、比例变换

比例变换分局部比例变换和整体比例变换

(1) 局部比例变换

局部比例变换由 T_{2D} 中主对角线元素决定，其它元素均为零。
当对 x, y, z 方向分别进行比例变换时，其变换的矩阵表示为：

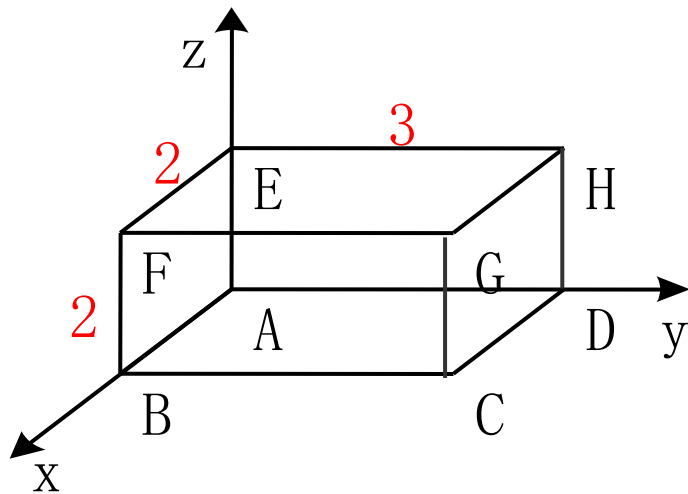
$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_s$$

$$= \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & e & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} ax & ey & iz & 1 \end{bmatrix}$$

对下图所示的长方形体进行比例变换，其中：

$a=1/2$ ， $e=1/3$ ， $i=1/2$ ，求变换后的长方形体各点坐标

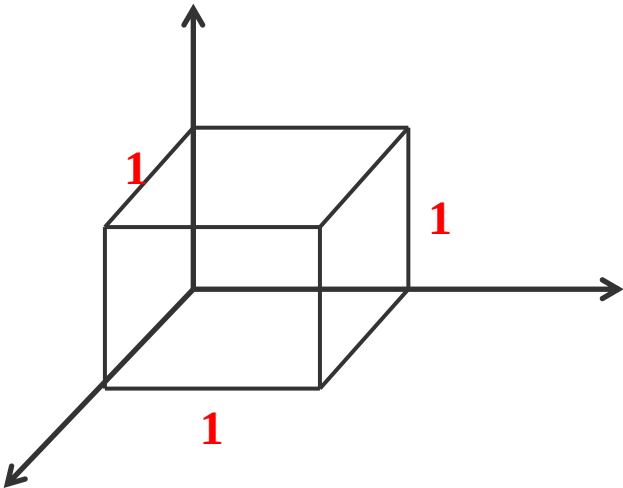


将其写为矩阵计算形式如下：

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 2 & 0 & 0 & 1 \\ 2 & 3 & 0 & 1 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 1 \\ 2 & 0 & 2 & 1 \\ 2 & 3 & 2 & 1 \\ 0 & 3 & 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$=$

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}$$



(2) 整体比例变换

整体比例变换，可用以下矩阵表示：

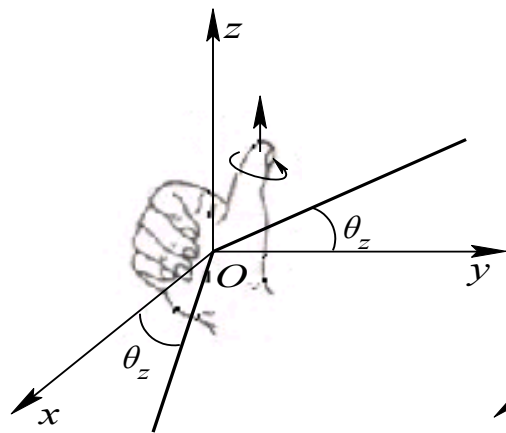
$$\begin{aligned} [x' \quad y' \quad z' \quad 1] &= [x \quad y \quad z \quad 1] \cdot T_s \\ &= [x \quad y \quad z \quad 1] \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & s \end{bmatrix} \\ &= [x \quad y \quad z \quad s] = \begin{bmatrix} \frac{x}{s} & \frac{y}{s} & \frac{z}{s} & 1 \end{bmatrix} \end{aligned}$$

3、旋转变换

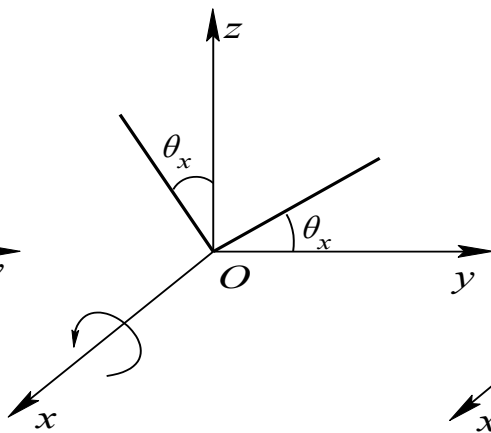
三维立体的旋转变换是指给定的三维立体绕三维空间某个指定的坐标轴旋转 θ 角度

旋转后，立体的空间位置将发生变化，但形状不变

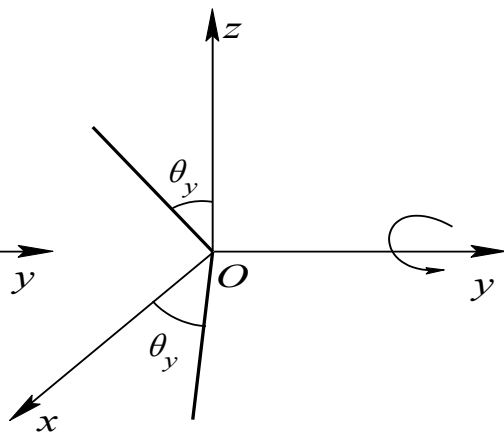
θ 角的正负按右手规则确定，右手大姆指指向旋转轴的正向，其余四个手指指向旋转角的正向



(a)



(b)



(c)

(a) 绕 z 轴正向旋转； (b) 绕 x 轴正旋转； (c) 绕 y 轴正向旋转

(1) 绕z轴旋转 θ

三维空间立体绕z轴正向旋转时，立体上各顶点的x，y坐标改变，而z坐标不变。而x，y坐标可由二维点绕原点旋转公式得到，因此可得：

$$x^* = x \cos \theta - y \sin \theta$$

$$y^* = x \sin \theta + y \cos \theta$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_{Rz}$$

$$= \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} x \cdot \cos \theta - y \cdot \sin \theta & x \cdot \sin \theta + y \cdot \cos \theta & z & 1 \end{bmatrix}$$

(2) 绕x轴旋转

同理，三维点p绕x轴正向旋转 θ 角的矩阵计算形式为：

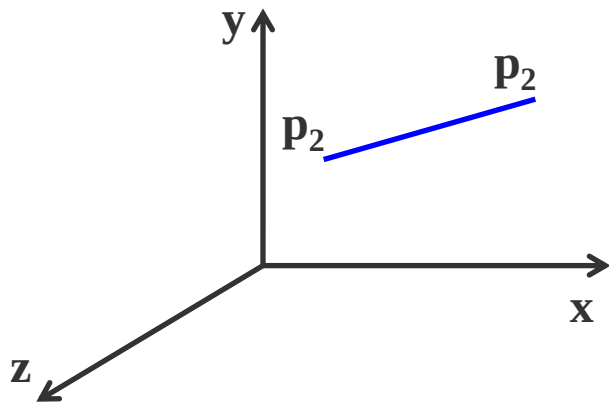
$$\begin{aligned} \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} &= \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_{Rx} \\ &= \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} x & y \cdot \cos \theta - z \cdot \sin \theta & y \cdot \sin \theta + z \cdot \cos \theta & 1 \end{bmatrix} \end{aligned}$$

(3) 绕y轴旋转

三维点p绕y轴正向旋转 θ 角的矩阵计算形式为：

$$\begin{aligned} \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} &= \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_{Ry} \\ &= \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} x \cdot \sin \theta + x \cdot \cos \theta & y & z \cdot \cos \theta - x \cdot \sin \theta & 1 \end{bmatrix} \end{aligned}$$

(3) 绕任意轴旋转



求绕任意直线旋转矩阵的原则：

- ① 任意变换的问题——基本几何变换的问题
- ② 绕任意直线旋转的问题——绕坐标轴旋转的问题

4、对称变换

对称变换有关于坐标平面、坐标轴等的对称变换。

(1) 关于坐标平面的对称

关于xoy平面进行对称变换

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & -z & 1 \end{bmatrix}$$

$$T_{Fxy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} x & y & -z & 1 \end{bmatrix}$$

关于yoz平面进行对称变换的矩阵计算形式为：

$$T_{Fyz} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_{Fyz} = \begin{bmatrix} -x & y & z & 1 \end{bmatrix}$$

关于zox平面进行对称变换的矩阵计算形式为：

$$T_{Fzx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_{Fzx} = \begin{bmatrix} x & -y & z & 1 \end{bmatrix}$$

(2) 关于坐标轴对称

关于x轴进行对称变换的矩阵计算形式为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & -y & -z & 1 \end{bmatrix}$$

$$T_{Fx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

关于y轴进行对称变换的矩阵计算形式为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} -x & y & -z & 1 \end{bmatrix}$$

$$T_{Fy} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

关于z轴进行对称变换的矩阵计算形式为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} -x & -y & z & 1 \end{bmatrix}$$

$$T_{Fz} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

投影变换

如何在二维平面上显示三维物体？

显示器屏幕、绘图纸等是二维的

显示对象是三维的

解决方法——**投影变换**

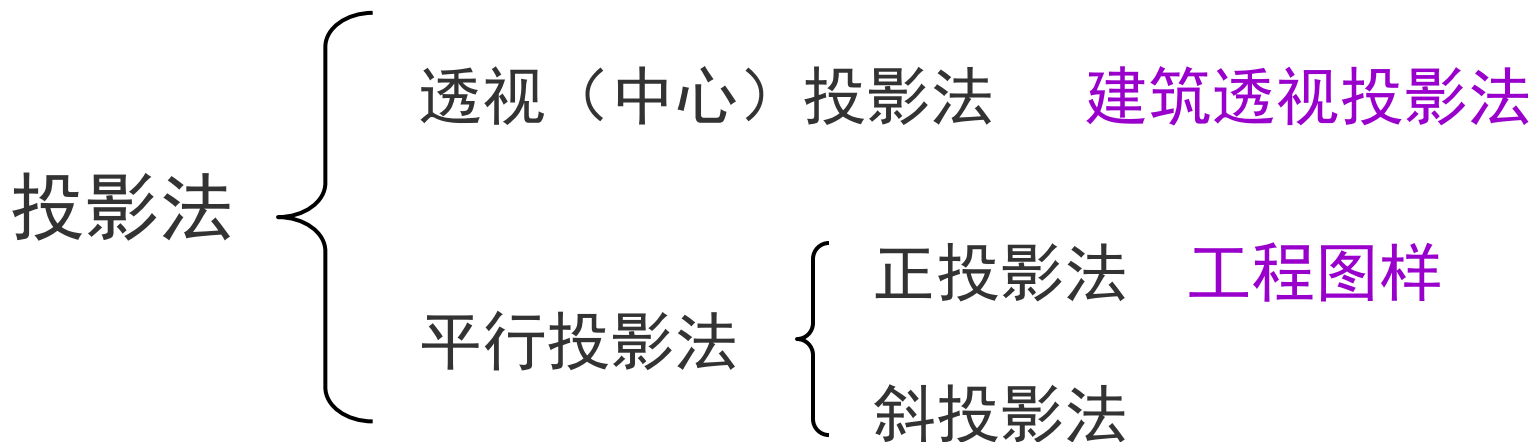
一、平面几何投影

投影变换就是把三维物体投射到投影面上得到二维平面图形

需要记住的一点是，计算机绘图是产生三维物体的二维图象。但在屏幕上绘制图形的时候，必须在三维坐标系下来考虑画法

在创建一个三维图形时，考虑二维平面图象是怎样的

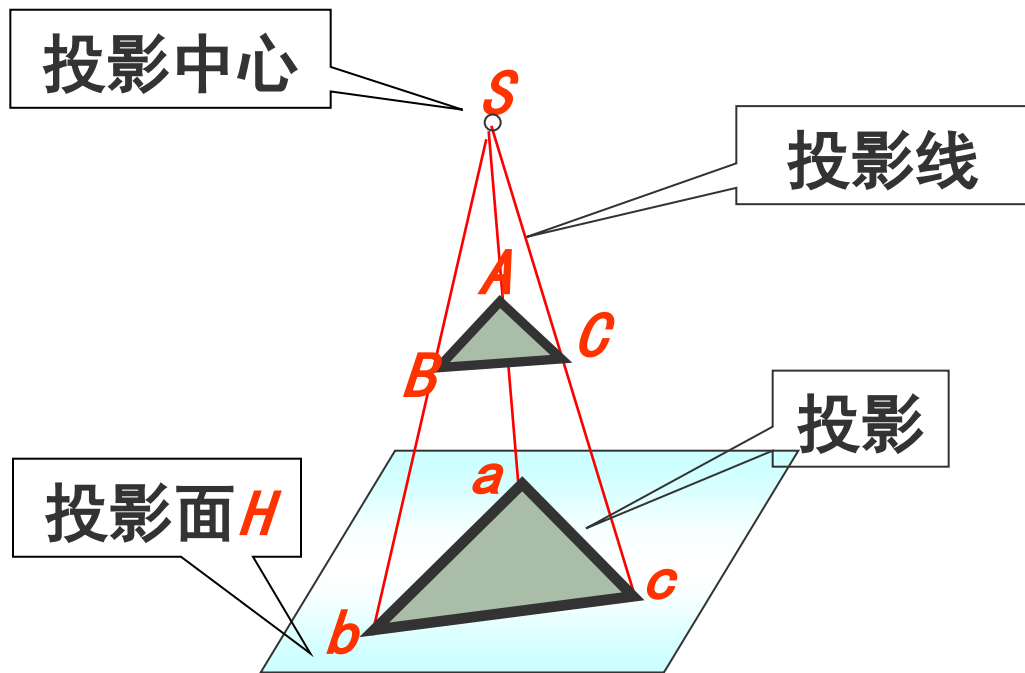
常用的投影法有两大类



两种投影法的本质区别在于透视投影的投影中心到投影面之间的距离是有限的；而另一个的距离是无限的

1、中心（透视）投影

投影线均通过投影中心

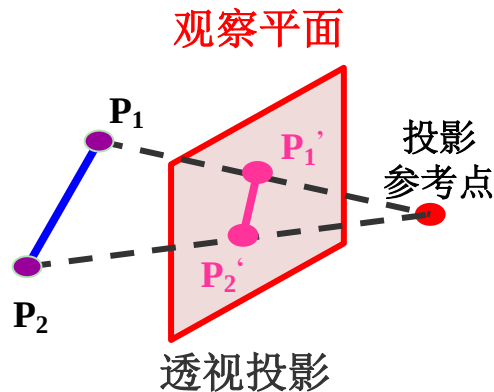


在投影中心相对投影面确定的情况下，空间的一个点在投影面上只存在唯一一个投影。

透视投影特点：

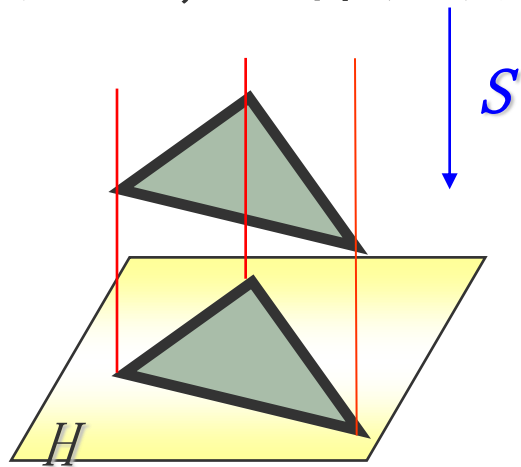
物体的投影视图由计算投影线
与观察平面之交点而得

透视投影生成真实感视图但不保
持相关比例



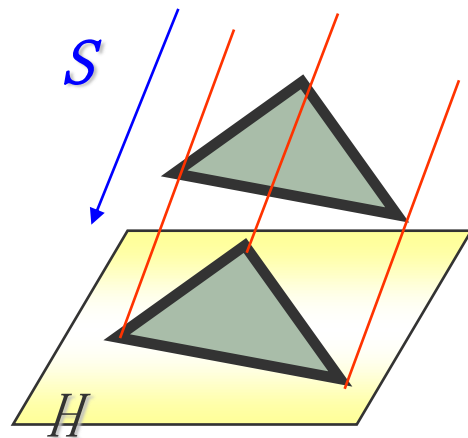
2、平行投影

如果把透视投影的中心移至无穷远处，则各投影线成为相互平行的直线，这种投影法称为平行投影法。



正投影法

投影方向S垂直于投影面H



斜投影法

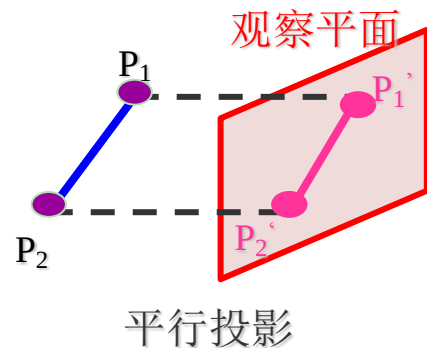
投影方向S倾斜于投影面H

平行投影特点：

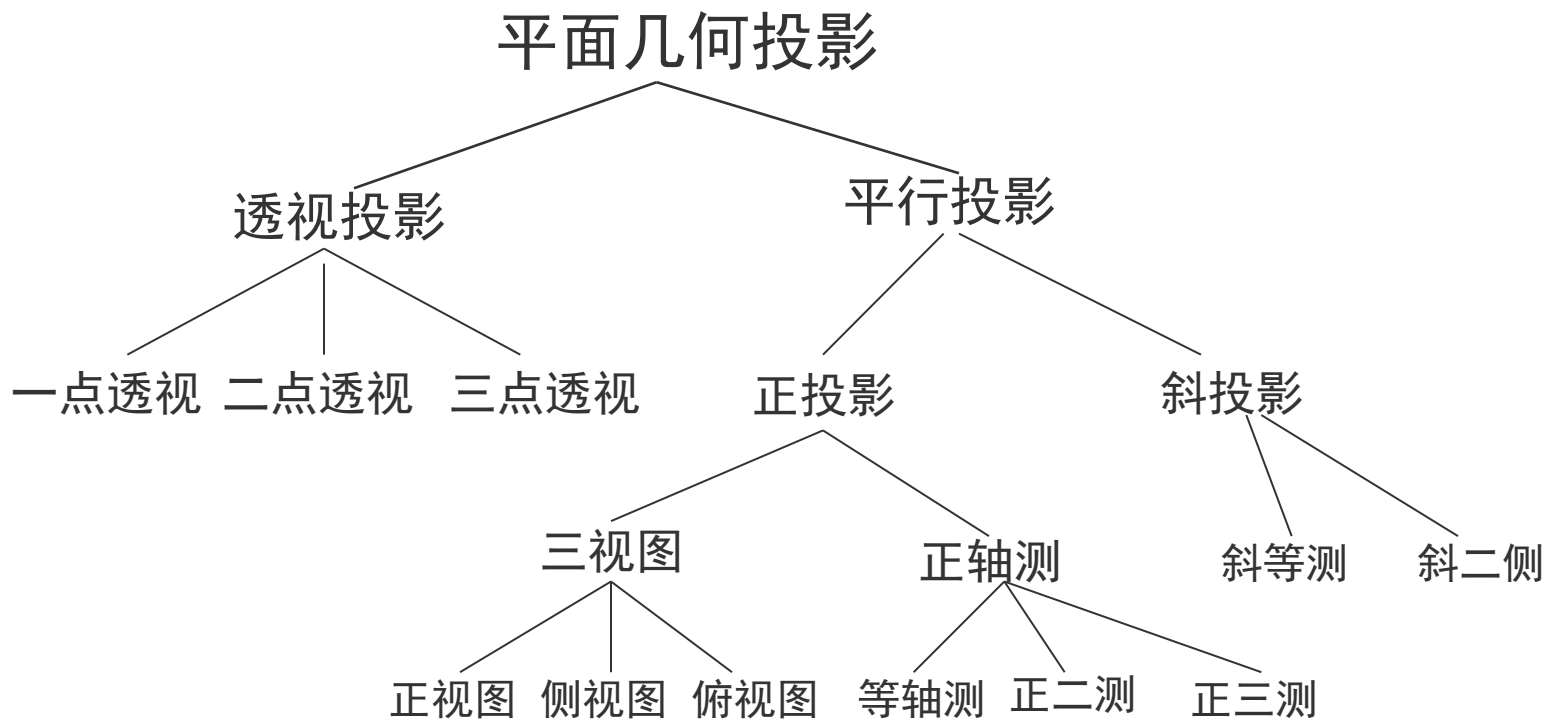
平行投影保持物体的有关比例不变

物体的各个面的精确视图由平行投影而得

没有给出三维物体外表的真实性表示

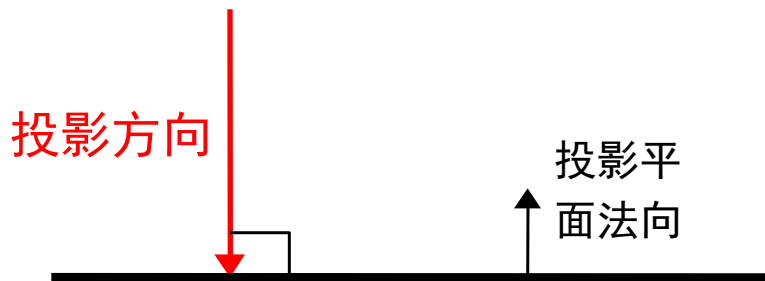


下面给出平面几何投影的分类：

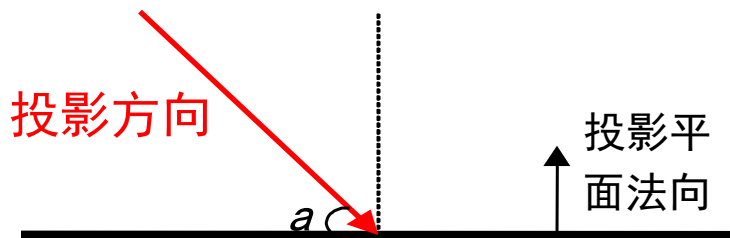


二、平行投影

平行投影可根据投影方向与投影面的夹角分成两类：**正投影**和**斜投影**。



投影平面
(a) 正投影

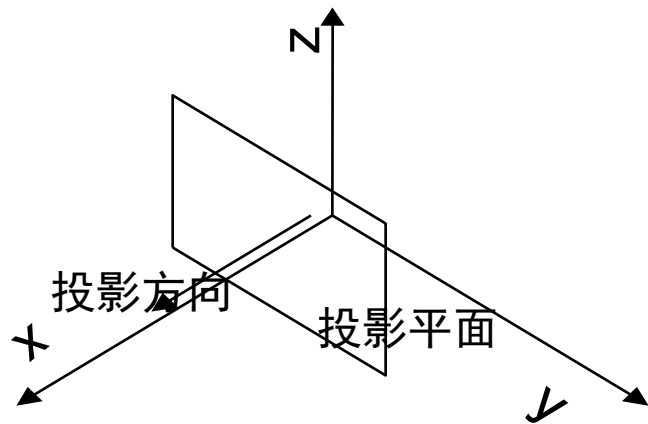


投影平面
(b) 斜投影

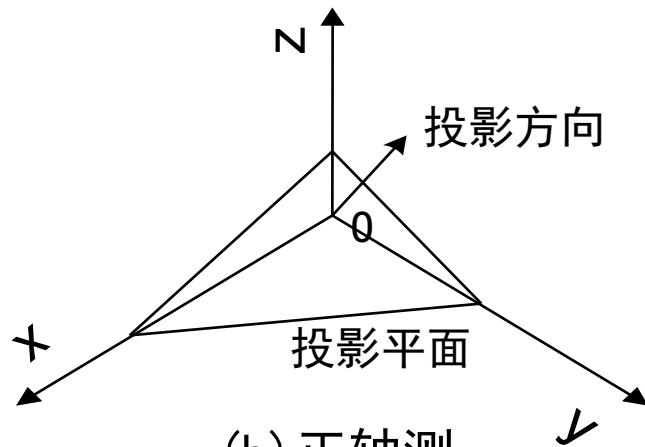
1、正投影

正投影根据投影面与坐标轴的**夹角**又可分为两类：**三视图**和**正轴侧图**

当投影面与某一坐标轴垂直时，得到的投影为三视图，这时投影方向与这个坐标轴的方向一致；否则，得到的投影为正轴侧图



(a) 三视图

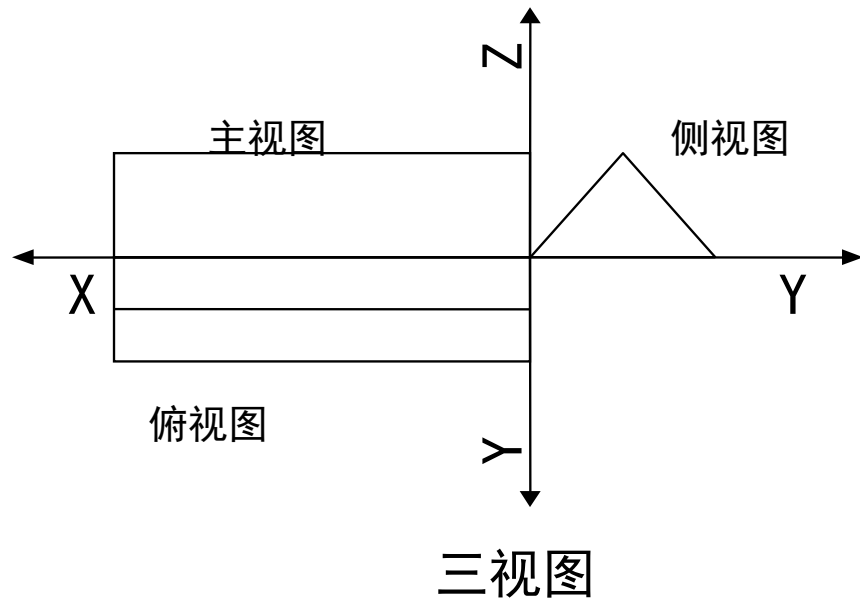
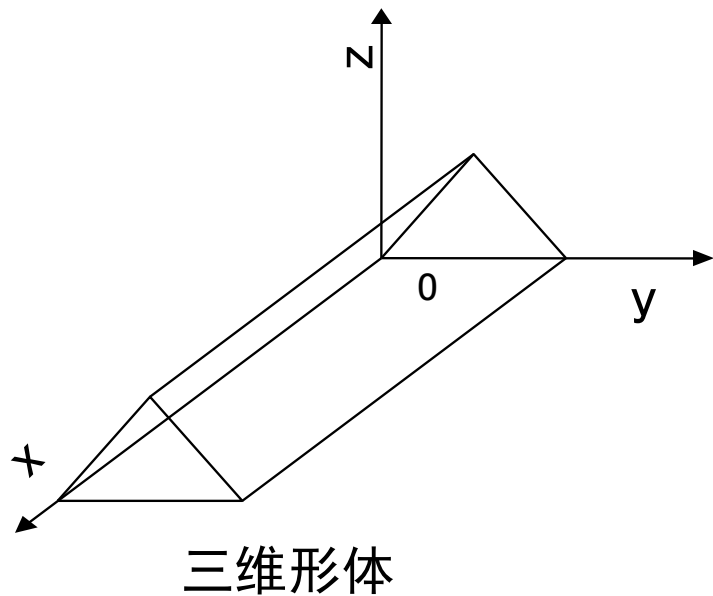


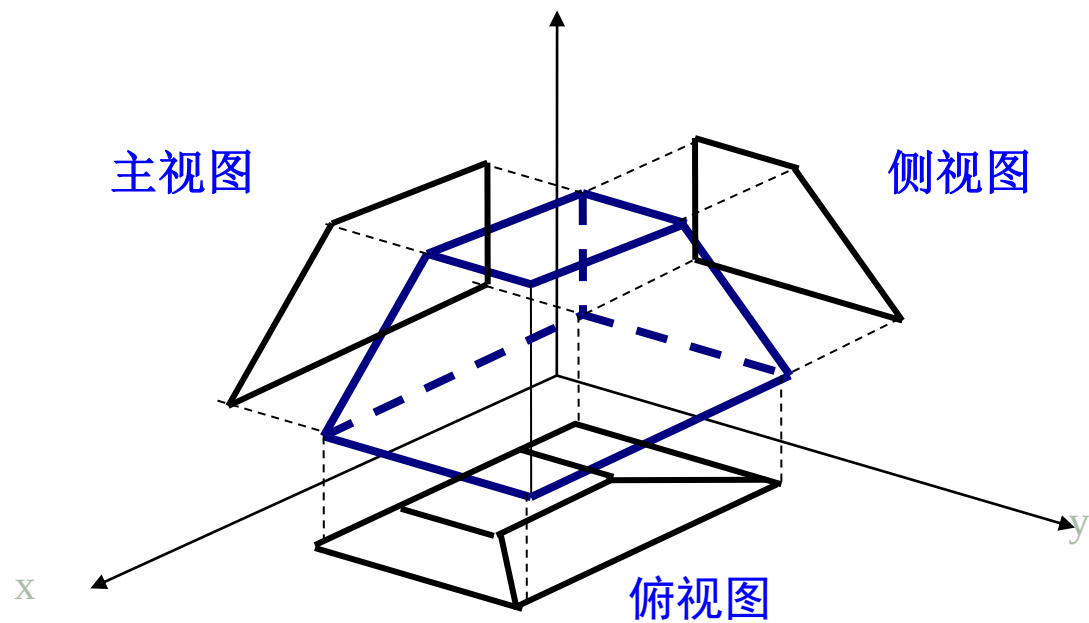
(b) 正轴测

正投影

2、三视图

通常所说的三视图包括主视图、侧视图和俯视图三种，投影面分别与x轴、y轴和z轴垂直





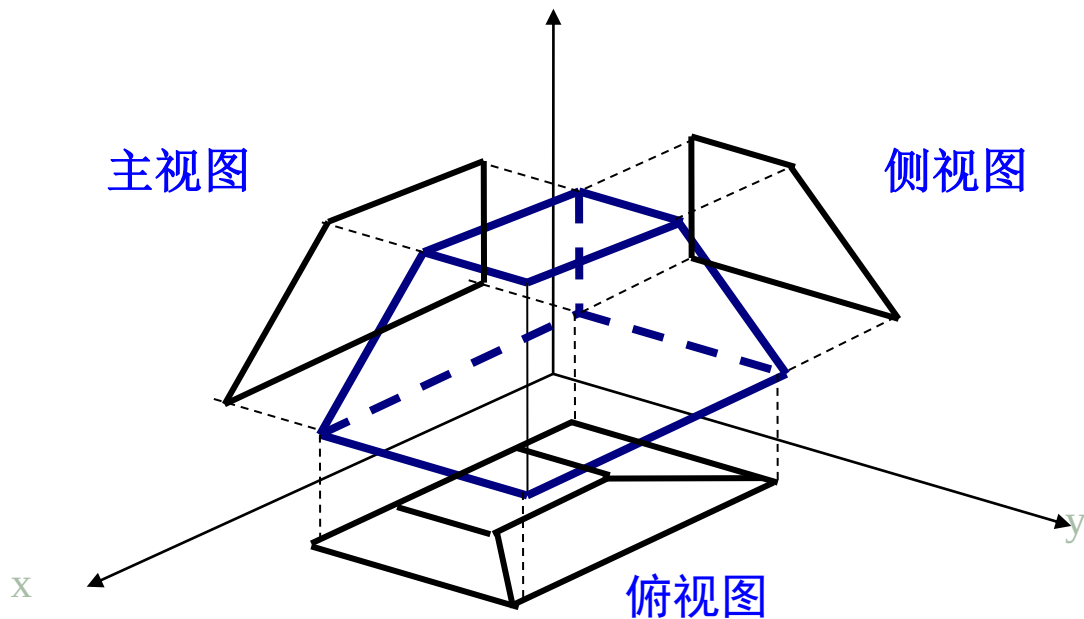
一个直角棱台的三视图

三视图的特点是物体的一个坐标面平行于投影面，其投影能反映形体的实际尺寸。工程制图中常用三视图来测量形体间的距离、角度以及相互位置关系

不足之处是一种三视图上只有物体一个面的投影，所以三视图难以形象地表示出形体的三维性质，只有将主、侧、俯三个视图放在一起，才能综合出物体的空间形状

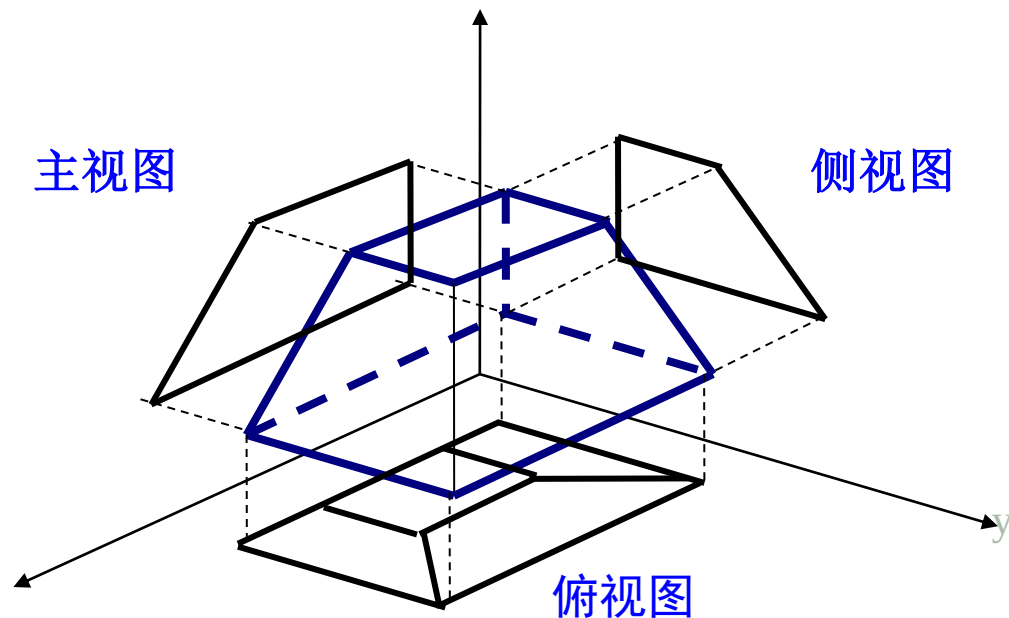
(1) 三视图的计算

主视图、俯视图和侧视图是分别将三维物体对正面、水平面和侧面作正平行投影而得到的三个基本视图



一个直角棱台的三视图

显然，只要求得这种正平行投影的变换矩阵，就可以得到三维物体上任意点经变换后的相应点，有这些变换后的点^x即可绘出三维物体投影后的三视图



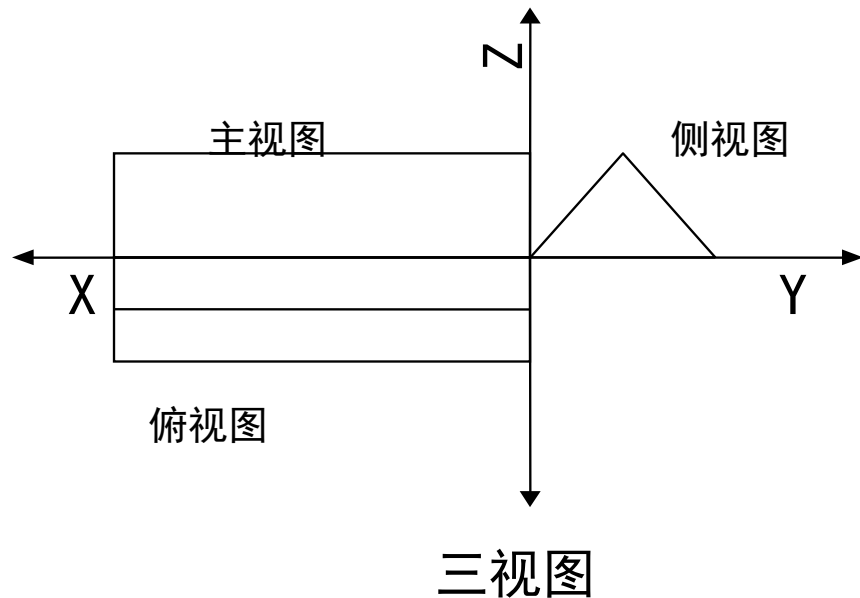
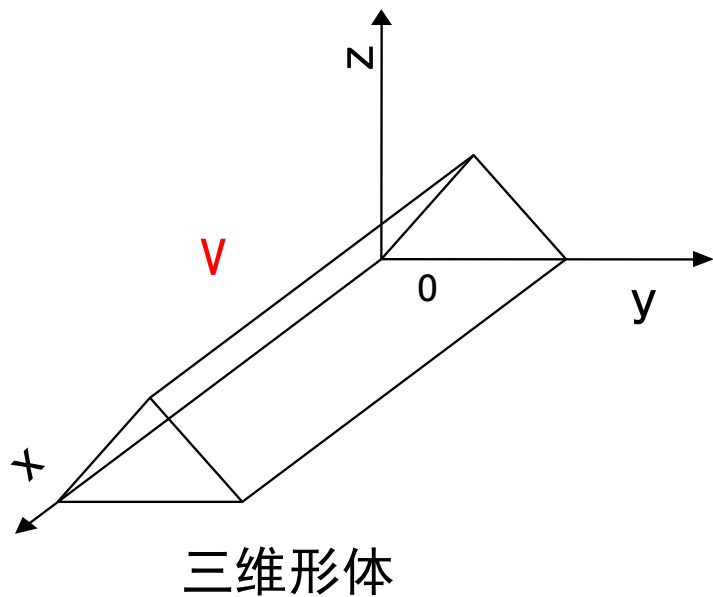
一个直角棱台的三视图

具体计算步骤如下：

- a、确定三维物体上各点的位置坐标；
- b、引入齐次坐标，求出所作变换相应的变换矩阵；
- c、将所作变换用矩阵表示，通过运算求得三维物体上各点经变换后的点坐标值；
- d、由变换后得到的二维点绘出三维物体投影后的三视图

(2) 主视图

将三维物体 xOz 面（又称V面）作垂直投影，得到主视图



由投影变换前后三维物体上点到主视图上点的关系，此投影变换的变换矩阵应为：

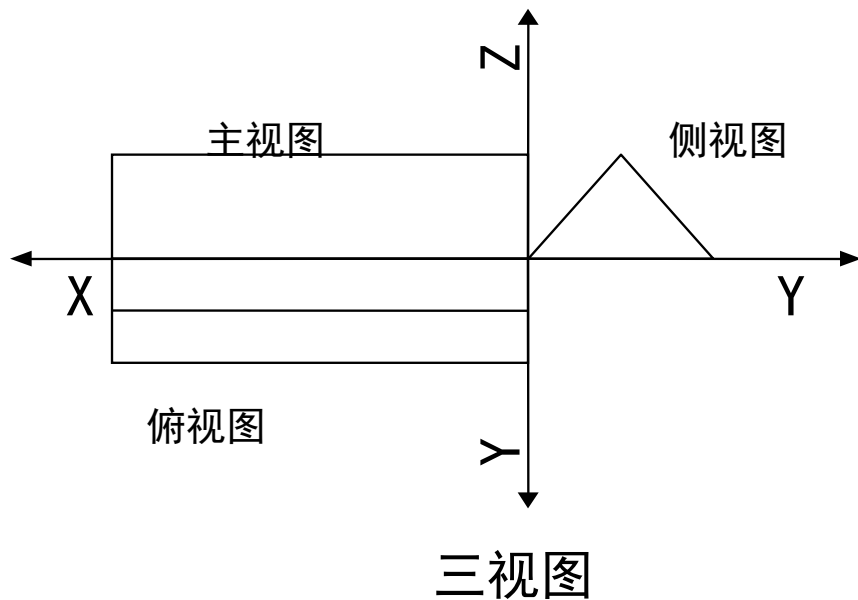
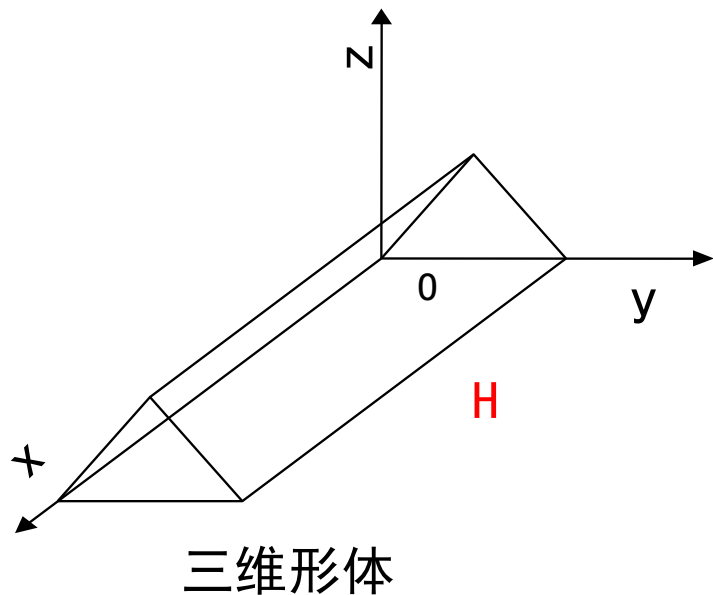
$$T_v = T_{xOz} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

通常称 T_v 为主视图的投影变换矩阵。于是，由三维物体到主视图的投影变换矩阵表示为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_v = \begin{bmatrix} x & 0 & z & 1 \end{bmatrix}$$

(3) 俯视图

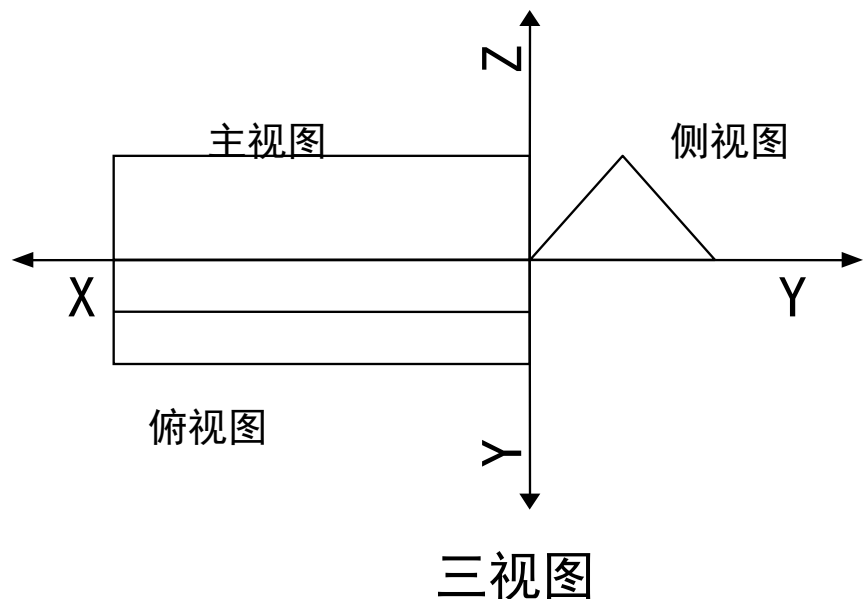
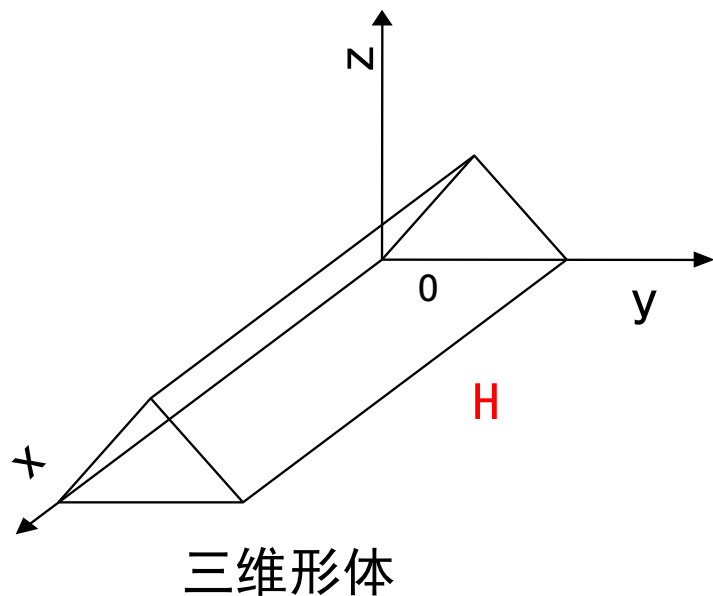
将三维物体xOy面（又称H面）作垂直投影得到俯视图



其投影变换矩阵应为：

$$T_H = T_{xOy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[x' \quad y' \quad z' \quad 1] = [x \quad y \quad z \quad 1] \cdot T_H = [x \quad y \quad 0 \quad 1]$$



为了使俯视图与主视图都画在一个平面内，就要使H面绕x轴顺时针转 90° ，即应有一个旋转变换，其变换矩阵为：

$$T_{Rx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(-90^\circ) & \sin(-90^\circ) & 0 \\ 0 & -\sin(-90^\circ) & \cos(-90^\circ) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

为了使主视图和俯视图有一定的间距，还要使H面沿z方向平移一段距离 $-z_0$ ，其变换矩阵为：

$$T_{tz} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -z_0 & 1 \end{bmatrix}$$

于是，俯视图的投影变换矩阵：

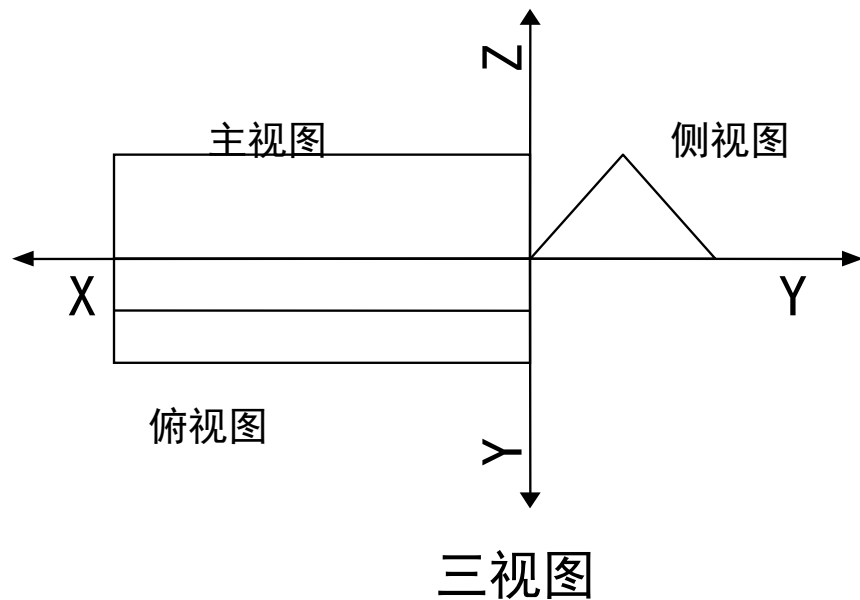
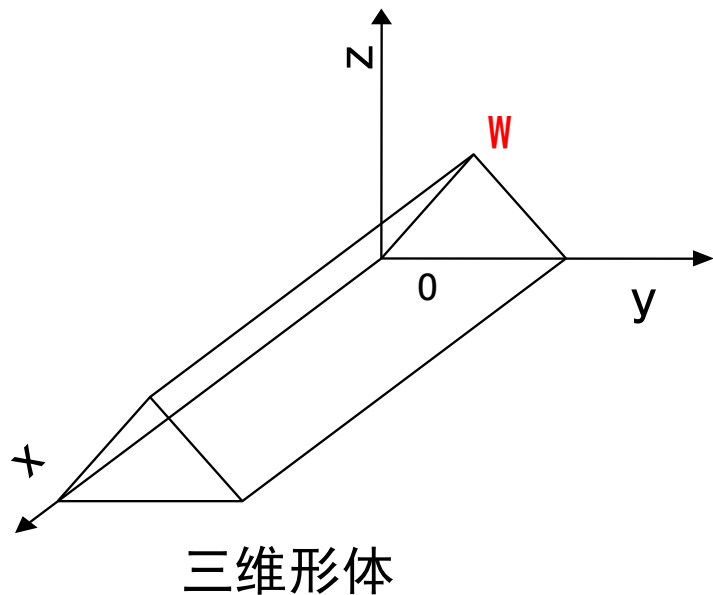
$$T_H = T_{xOy} \cdot T_{Rx} \cdot T_{tz} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -z_0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -z_0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -z_0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_H = \begin{bmatrix} x & 0 & -(y + z_0) & 1 \end{bmatrix}$$

(4) 侧视图

将三维物体 yOz 面（又称 W 面）作垂直投影得到侧视图



其投影变换矩阵应为：

$$T_W = T_{yOz} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

为了使侧视图与主视图也在一个平面内，就要使W面绕z轴正转 90° ，其旋转变换矩阵为：

$$T_{Rz} = \begin{bmatrix} \cos 90^0 & \sin 90^0 & 0 & 0 \\ -\sin 90^0 & \cos 90^0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

为使主视图和侧视图有一定的间距，还要使W面沿负x方向平移一段距离 $-x_0$ ，该平移变换矩阵为：

$$T_{tx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -x_0 & 0 & 0 & 1 \end{bmatrix}$$

于是，侧视图的投影变换矩阵为：

$$\begin{aligned} T_W &= T_{yOz} \cdot T_{Rz} \cdot T_{tx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -x_0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -x_0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot T_W = \begin{bmatrix} -(y+x_0) & 0 & z & 1 \end{bmatrix}$$

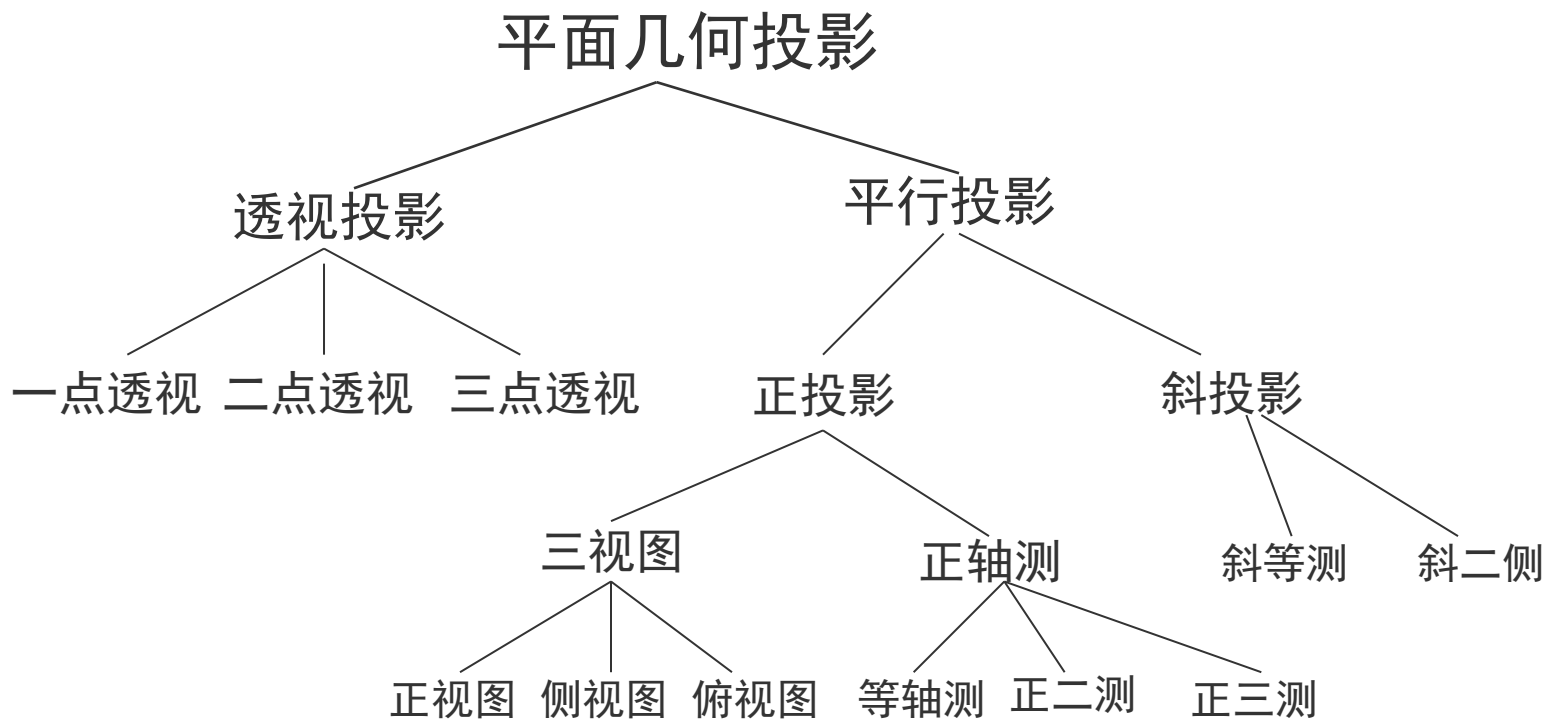
主视图: $[x' \ y' \ z' \ 1] = [x \ 0 \ z \ 1]$

俯视图: $[x' \ y' \ z' \ 1] = [x \ 0 \ -(y + z_0) \ 1]$

侧视图: $[x' \ y' \ z' \ 1] = [-(y + x_0) \ 0 \ z \ 1]$

三个视图中的 y' 均为0, 表明三个视图均落在 $x0z$ 面上

3、正轴侧图投影变换矩阵

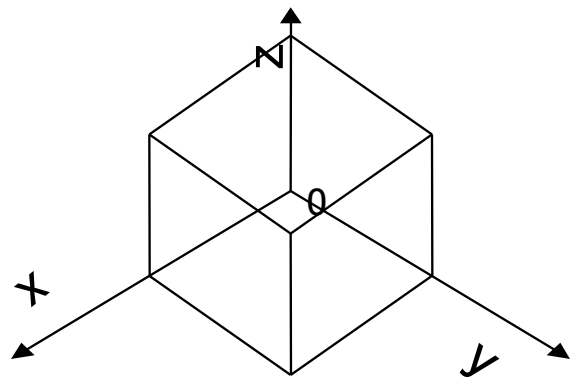
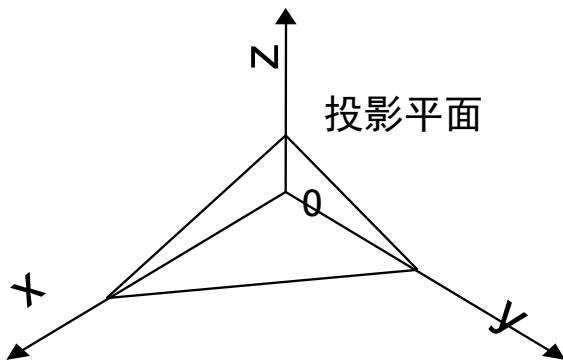
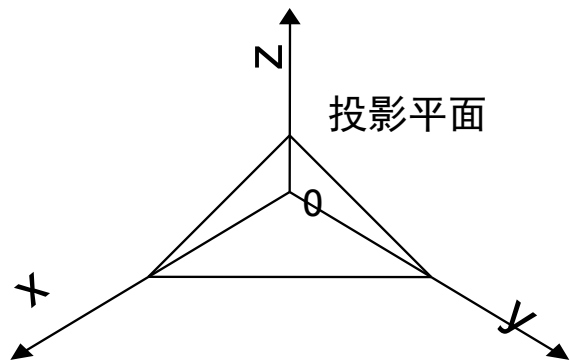
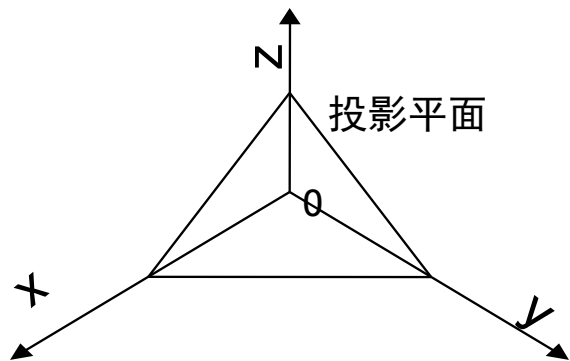


正轴测有等轴测、正二测和正三测三种：

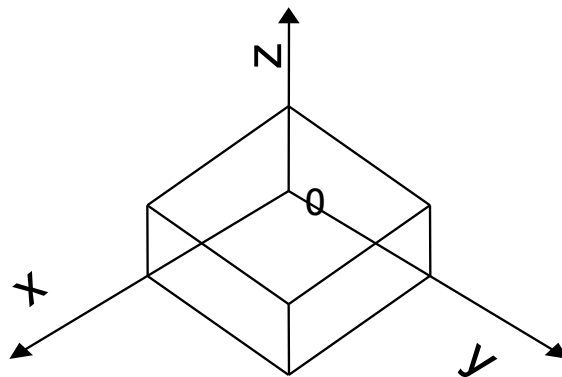
当投影面与三个坐标轴之间的夹角都相等时为**等轴测**

当投影面与两个坐标轴之间的夹角相等时为**正二测**

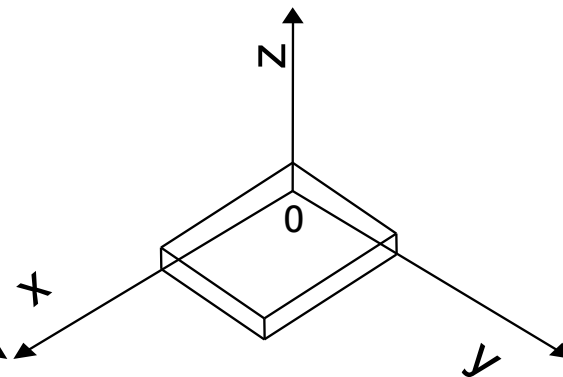
当投影面与三个坐标轴之间的夹角都不相等时为**正三测**



(a) 等轴测

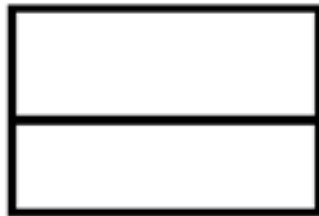


(b) 正二测

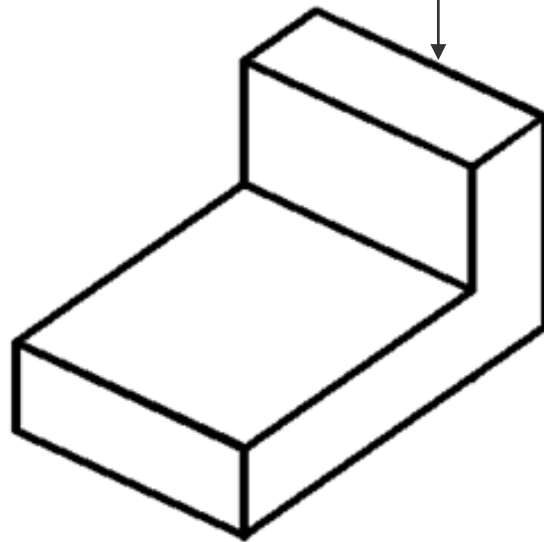


(c) 正三测

正轴测投影面及一个立方体的正轴测投影图



比较



正投影图

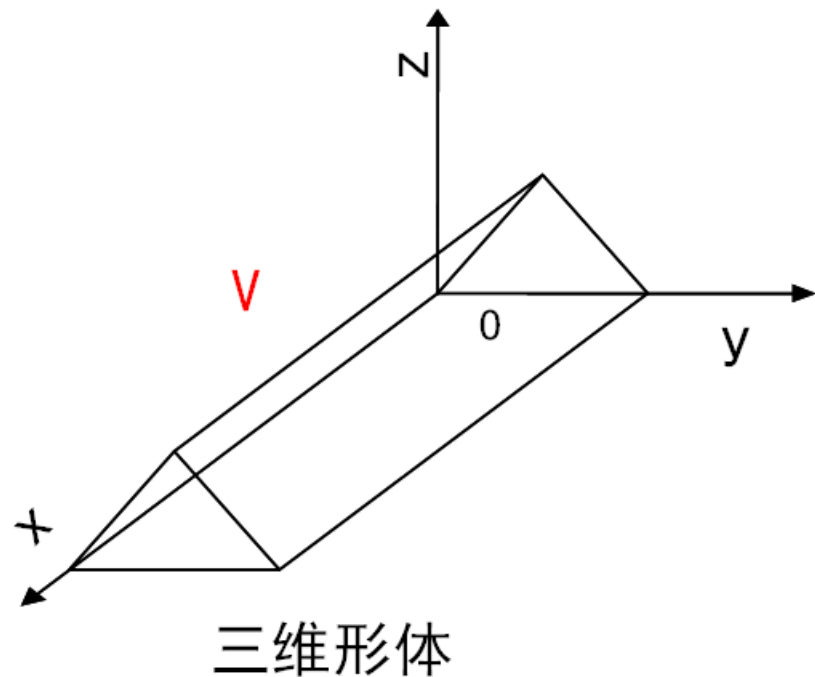
轴测图

表现力和度量性好，但直观性差

直观性好，但度量性差，辅助图样

空间物体的正轴测图是以V面为轴测投影面，先将物体绕Z轴转 γ 角，接着绕X轴转 $-\alpha$ 角，最后向V面投影。其变换矩阵为：

$$\mathbf{T}_{\text{正}} = \mathbf{T}_Z \cdot \mathbf{T}_X \cdot \mathbf{T}_V$$



$$T_{\text{IE}} = T_Z \cdot T_X \cdot T_V$$

$$= \begin{bmatrix} \cos \gamma & \sin \gamma & 0 & 0 \\ -\sin \gamma & \cos \gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \gamma & 0 & -\sin \gamma & \sin \alpha & 0 \\ -\sin \gamma & 0 & -\cos \gamma & \sin \alpha & 0 \\ 0 & 0 & \cos \alpha & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

即：

$$\begin{cases} x^* = x \cos \gamma - y \sin \gamma \\ y^* = 0 \\ z^* = -x \sin \gamma \sin \alpha - y \cos \gamma \sin \alpha + z \cos \alpha \end{cases}$$

(1) 正等轴测图的变换矩阵

根据画法几何学，作正等轴测投影时， $\gamma = 45^\circ$ ， $\alpha = -35.26^\circ$ ，将 γ 、 α 代入上式，其变换矩阵为：

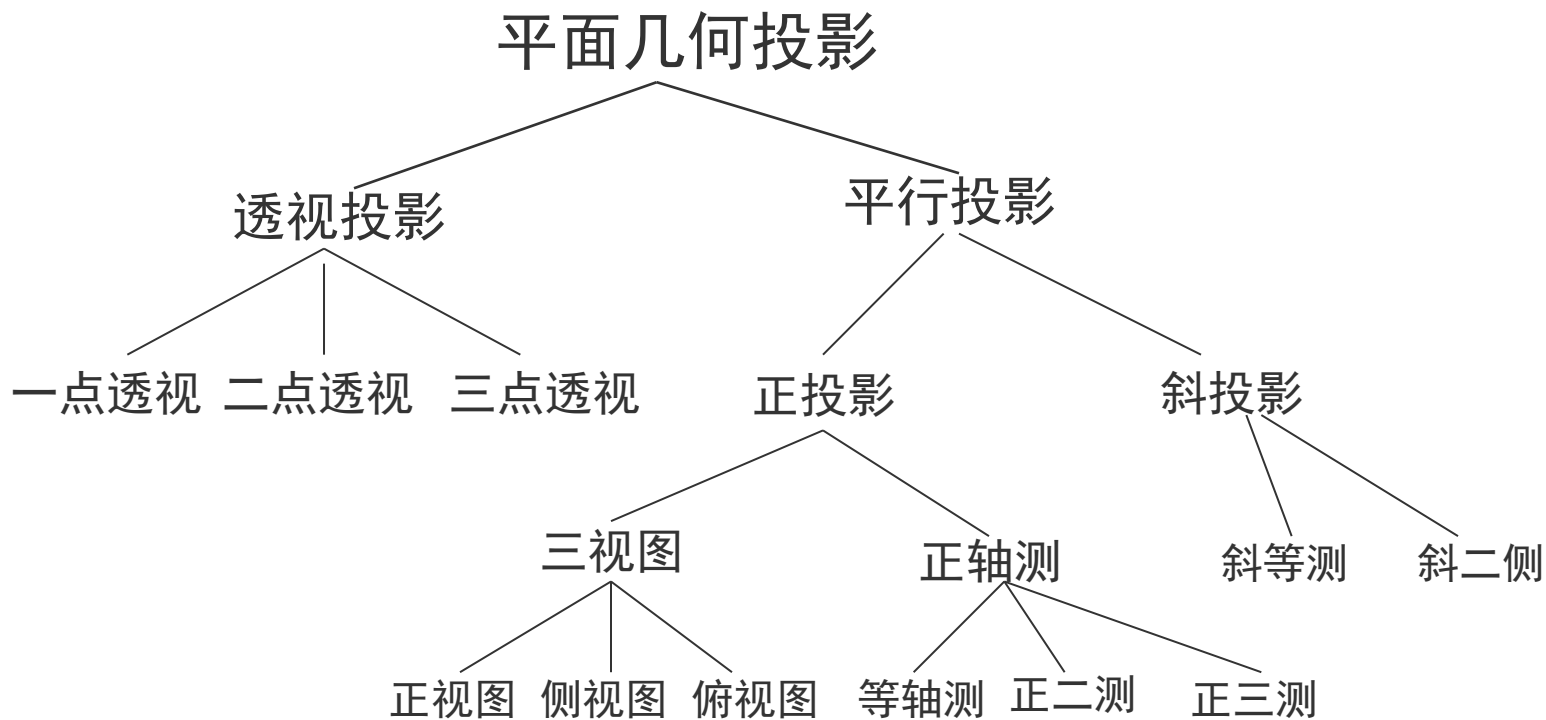
$$T_{\text{正等轴侧}} = \begin{bmatrix} 0.7071 & 0 & -0.4082 & 0 \\ -0.7071 & 0 & -0.4082 & 0 \\ 0 & 0 & 0.8165 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(2) 正二测图的变换矩阵

做正二测图时， $\gamma = 20.7^\circ$ ， $\alpha = 19.47^\circ$ ， 其变换矩阵为：

$$T_{\text{正二等}} = \begin{bmatrix} 0.9354 & 0 & -0.1178 & 0 \\ -0.7071 & 0 & -0.3118 & 0 \\ 0 & 0 & 0.9428 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

三、透视投影



有两种基本的投影—**平行投影**和**透视投影**。它们分别用于解决基本的、彼此独立的图形表示问题

平行投影表示真实大小和形状的物体

透视投影表示真实看到的物体

透视投影比轴测图更富有立体感和真实感

透视投影（Perspective Projection）是为了获得接近真实三维物体的视觉效果而在二维的纸或者画布平面上绘图或者渲染的一种方法，能逼真地反映形体的空间形象，也称为透视图

透视投影是3D渲染的基本概念，也是3D程序设计的基础

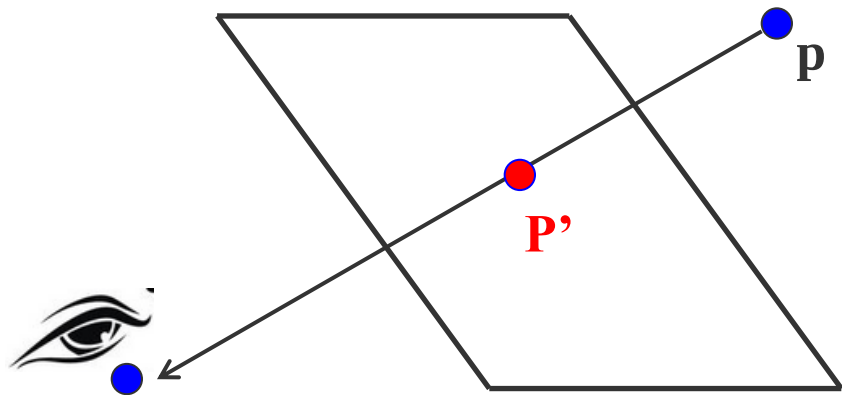
轴测投影图是用平行投影法形成的，视点在无穷远处；而透视投影图是用中心投影法形成的，视点在有限远处

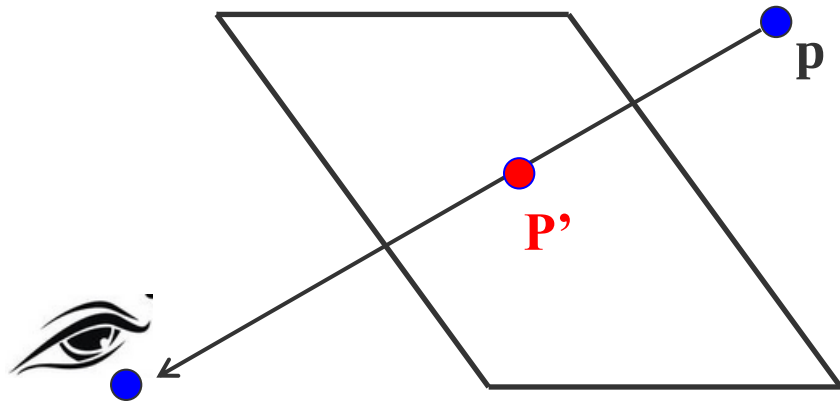
其中的 $[p, q, r]$ 能产生透视变换的效果

$$T_{3D} = \left[\begin{array}{ccc|c} a & b & c & p \\ d & e & f & q \\ g & h & i & r \\ \hline l & m & n & s \end{array} \right]$$

1、透视基本原理

众所周知，位于空间的任何一个点，它之所以能被人们的眼睛所看见，是因为从该点出发射出来的一条光线能够到达人们的眼睛





该平面为透视投影面，穿点 P' 为P的透视投影

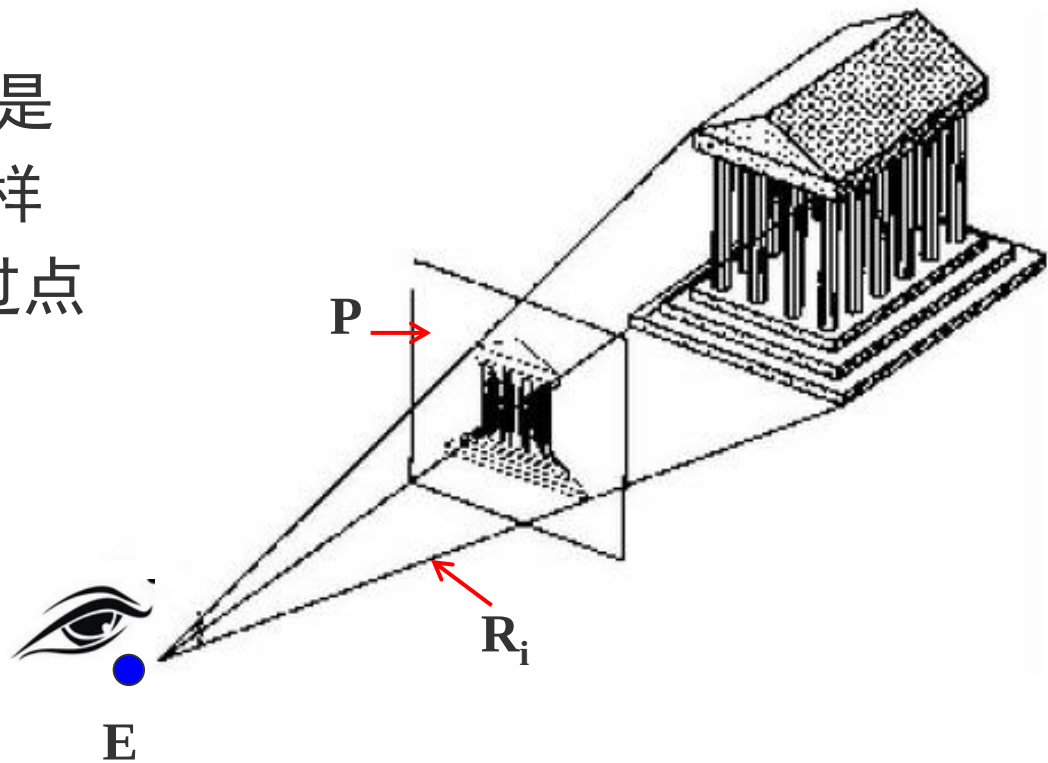
假如求空间点的透视投影问题得到了解决，那么空间任何线段、多边形或立体的透视投影也就可以方便地求得

因为一条直线段是由两点确定，多边形平面由围成该多边形的各顶点和边框线段确定，而任何立体也可以看成是由它的顶点和各棱边所构成的一个框体

这就是说，可以通过求出这些顶点的透视投影而获得空间任意立体的透视投影

三维世界的物体可以看作是由点集合 $\{X_i\}$ 构成的，这样依次构造起点为E，并经过点 X_i 的射线 R_i

这些射线与投影面P的交点集合便是三维世界在当前视点的透视图



2、一点透视

先假设 $q \neq 0$ ， $p=r=0$ 。然后对点 (x, y, z) 进行变换：

$$T_{3D} = \left[\begin{array}{ccc|c} a & b & c & p \\ d & e & f & q \\ g & h & i & r \\ \hline l & m & n & s \end{array} \right]$$

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & q \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & qy + 1 \end{bmatrix}$$

对其结果进行齐次化处理得：

$$[x \quad y \quad z \quad qy + 1]$$

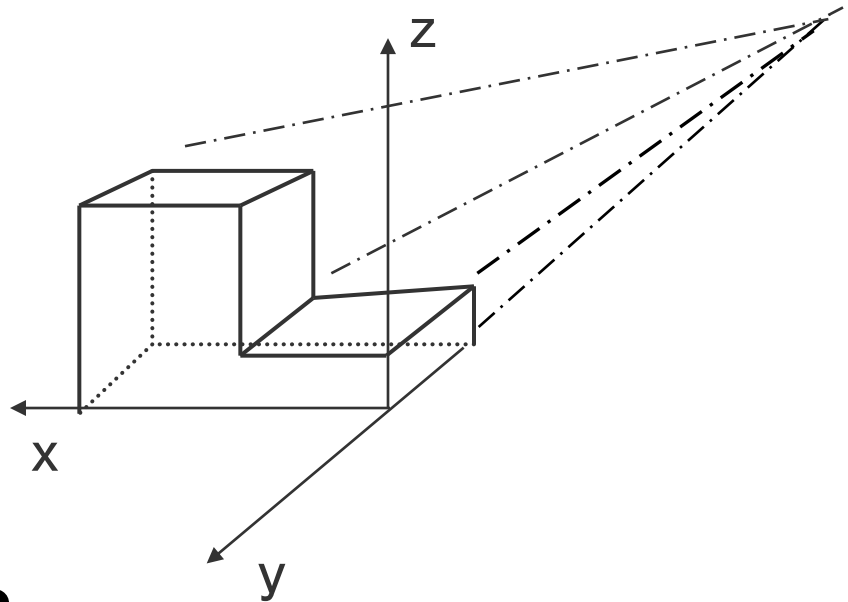
$$\left[\frac{x}{qy + 1} \quad \frac{y}{qy + 1} \quad \frac{z}{qy + 1} \quad 1 \right] = [x' \quad y' \quad z' \quad 1]$$

$$\begin{bmatrix} \frac{x}{qy+1} & \frac{y}{qy+1} & \frac{z}{qy+1} & 1 \end{bmatrix} = \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix}$$

A、当 $y=0$ 时，得：

$$\begin{cases} x' = x \\ y' = 0 \\ z' = z \end{cases}$$

说明处于 $y=0$ 平面内的点，经过变换以后没有发生变化

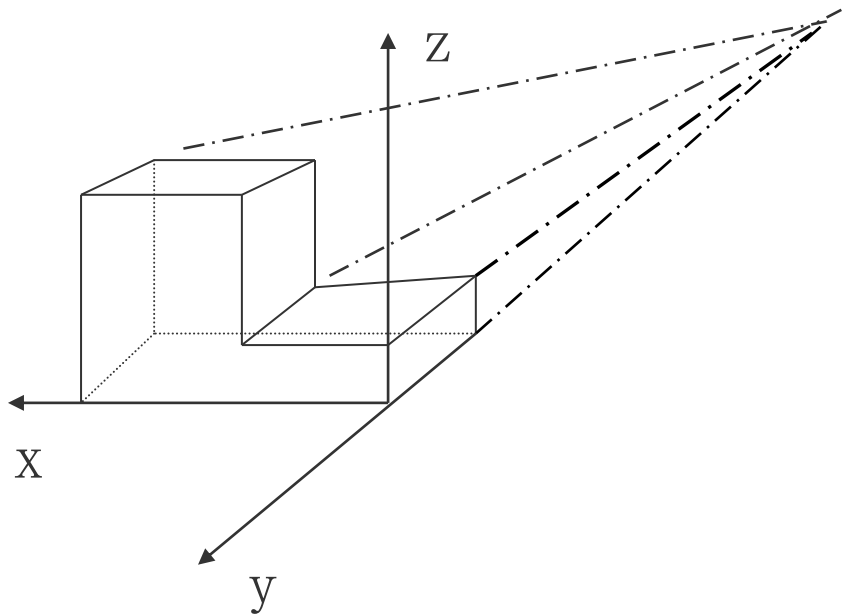


B、当 $y \rightarrow \infty$ 时，得：

$$\begin{cases} x' = 0 \\ y' = \frac{1}{q} \\ z' = 0 \end{cases}$$

这说明，当 $y \rightarrow \infty$ 时，所有点的变换结果都集中到了y轴上的 $1/q$ 处

即所有平行于y轴的直线将延伸相交于此点 $(0, 1/q, 0)$ 。
该点称为**灭点**，而像这样形成一个灭点的透视变换称为一点透视



根据同样的道理，当 $p \neq 0$ ， $q=r=0$ 时，则将在 x 轴上的 $1/p$ 处产生一个灭点，其坐标值为 $(1/p, 0, 0)$ 。在这种情况下，所有平行于 x 轴的直线将延伸交于该点

$$[x \quad y \quad z \quad 1] \cdot \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = [x \quad y \quad z \quad px + 1]$$

$$[x' \quad y' \quad z' \quad 1] = \begin{bmatrix} \frac{x}{px + 1} & \frac{y}{px + 1} & \frac{z}{px + 1} & 1 \end{bmatrix}$$

当 $r \neq 0$, $q=p=0$ 时, 则将在 z 轴上的 $1/r$ 处产生一个灭点, 其坐标值为 $(0, 0, 1/r)$ 。在这种情况下, 所有平行于 z 轴的直线将延伸交于该点。

3、多点透视

根据一点透视的原理予以推广，如果 p, q, r 三个元素中有两个为非零元素时，将会生成两个灭点，因此得到两点透视。如当 $p \neq 0, r \neq 0$ 时，结果为：

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & px + rz + 1 \end{bmatrix}$$

经过齐次化处理后结果为：

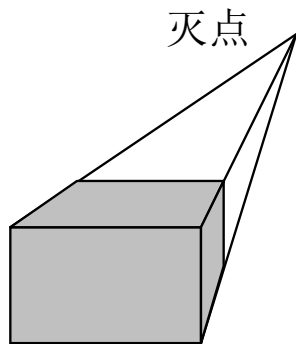
$$\left\{ \begin{array}{l} x' = \frac{x}{(px + rz + 1)} \\ y' = \frac{y}{(px + rz + 1)} \\ z' = \frac{z}{(px + rz + 1)} \end{array} \right.$$

从以上结果可以看到：

当 $x \rightarrow \infty$ 时，一个灭点在x轴上的 $1/p$ 处

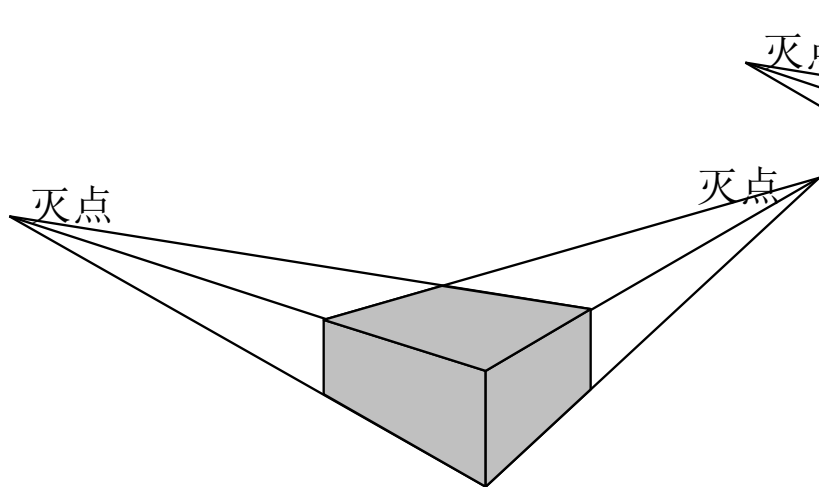
当 $z \rightarrow \infty$ 时，一个灭点在z轴上的 $1/r$ 处

同理，当 p, q, r 三个元素全为非零时，结果将会产生三个灭点，从而形成三点透视。产生的三个灭点分别在x轴上的 $1/p$ 处、y轴上的 $1/q$ 处和z轴上的 $1/r$ 处



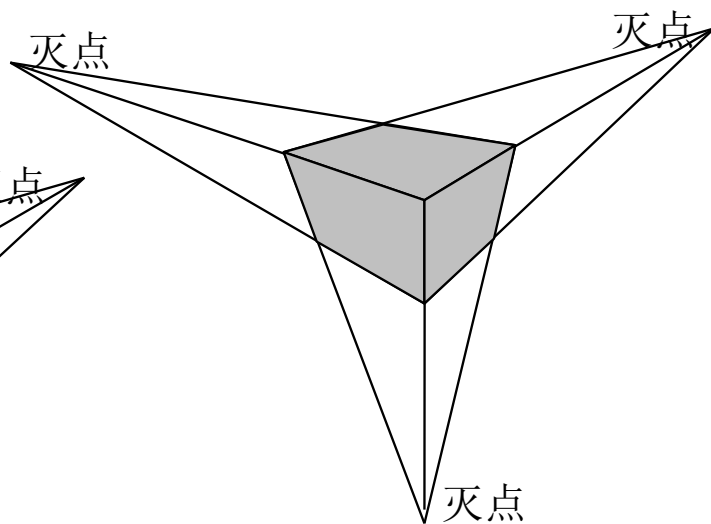
(a) 一点透视

一点透视有一个主灭点，即投影面与一个坐标轴正交，与另外两个坐标轴平行



(b) 二点透视

两点透视有两个主灭点，即投影面与两个坐标轴相交，与另一个坐标轴平行

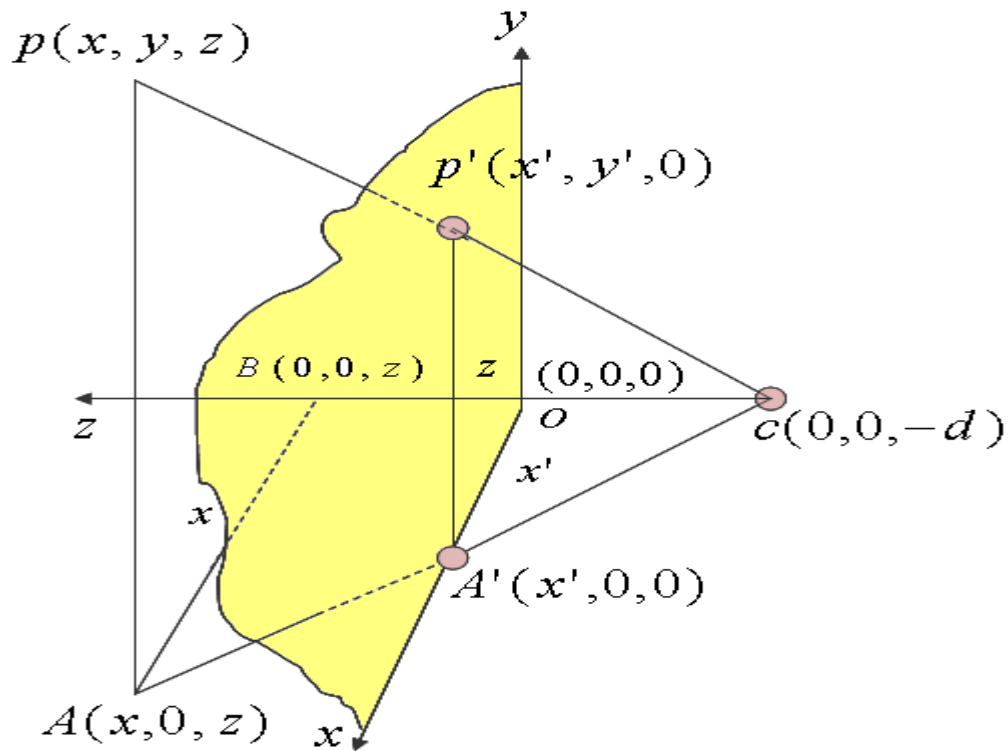


(c) 三点透视

三点透视有三个主灭点，即投影面与三个坐标轴都相交

4、生成透视投影图的方法

如右图所示，假定投影中心在 z 轴上（ $z = -d$ 处），投影面在面 xOy 上，与 z 轴垂直，现在求空间一点 $p(x, y, z)$ 的透视投影 $p'(x', y', 0)$ （ x', y', z' ）点的坐标。



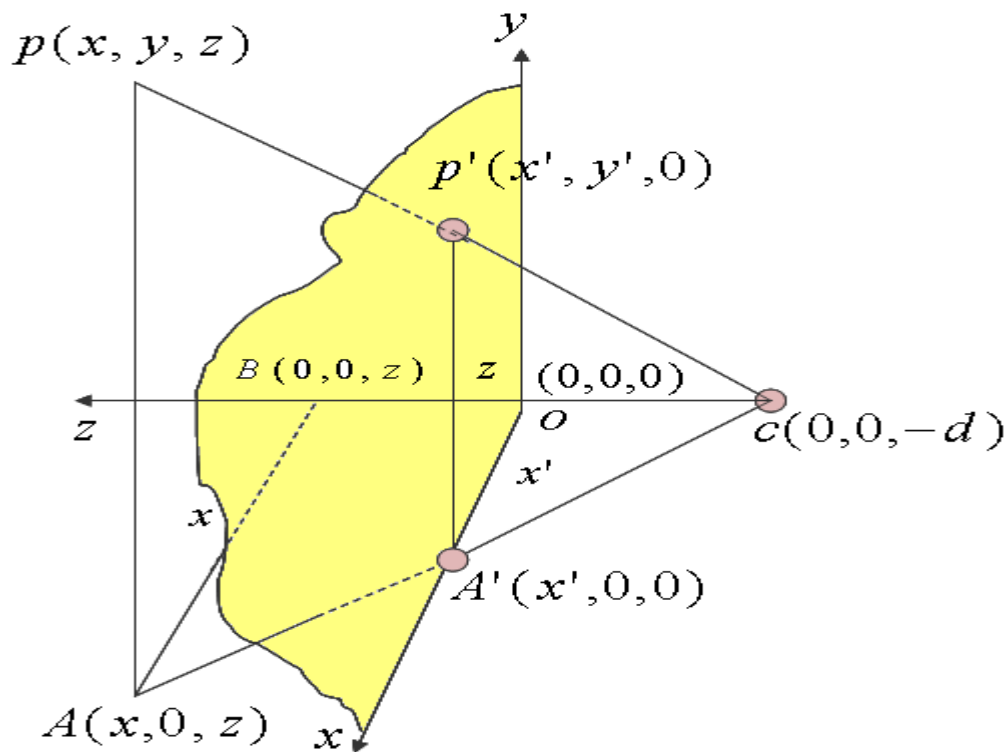
z轴负方向的点C(0, 0, -d)
是投影中心。

三角形ABC和A' OC是相似
三角形，因此：

$$\frac{x'}{x} = \frac{y'}{y} = \frac{d}{d+z}$$

$$x' = \frac{x}{1 + \frac{z}{d}} \quad y' = \frac{y}{1 + \frac{z}{d}}$$

$$z' = 0$$



$$x' = \frac{x}{1 + \frac{z}{d}} \quad y' = \frac{y}{1 + \frac{z}{d}} \quad z' = 0$$

- (1) 透视坐标与z值成反比。即z值越大，透视坐标值越小
- (2) d的取值不同，可以对形成的透视图有放大和缩小的功能。当值较大时，形成的透视图变大；反之缩小。

该过程写为变换矩阵形式为：

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{d} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} x & y & z & \frac{z}{d} + 1 \end{bmatrix}$$

然后再乘以向投影面投影的变换矩阵，就得到了点在画面上的投影。

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{d} \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

若投影中心在无穷远处，则 $1/d \rightarrow 0$ ，上式变为平行投影

$$[x' \quad y' \quad z' \quad 1] = [x \quad y \quad z \quad 1] * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{d} \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

由该矩阵还可以看出透视投影的特性：透视缩小效应：

三维物体透视投影的**大小**与物体到投影中心的**距离**成**反比**，即透视缩小效应。这种效应所产生的视觉效果十分类似于照相系统和人的视觉系统

四、透视投影实例

1、一点透视

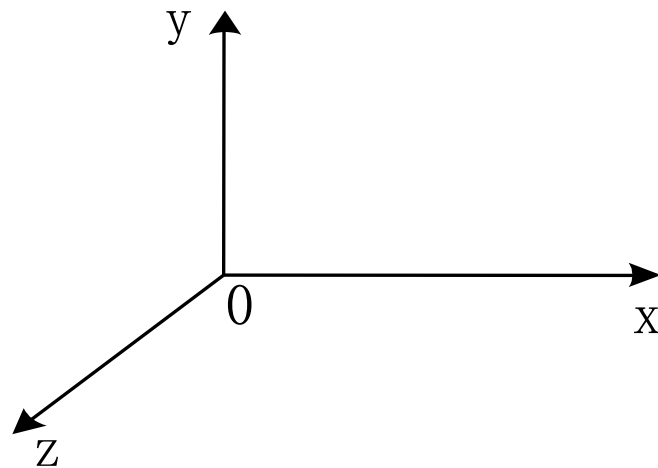
一点透视只有一个灭点。进行透视投影，要很好地考虑图面布局，以避免三维物体的平面或直线积聚成直线或点而影响直观性。具体地说，就是要考虑下列几点：

(1) 三维形体与画面（投影面）的相对位置；

(2) 视距，即视点（投影中心）与画面的距离

(3) 视点的高度

假定视点（投影中心）在 z 轴上（ $z = -d$ 处），投影面在 xOy 面上，则一点透视的步骤如下：



- (1) 将三维物体平移到适当位置 l 、 m 、 n
- (2) 进行透视变换
- (3) 最后，为了绘制的方便，向 xoy 平面作正投影变换，将结果变换到 xoy 平面上。

$$T_{p1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ l & m & n & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{d} \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

给出三维物体中任一点

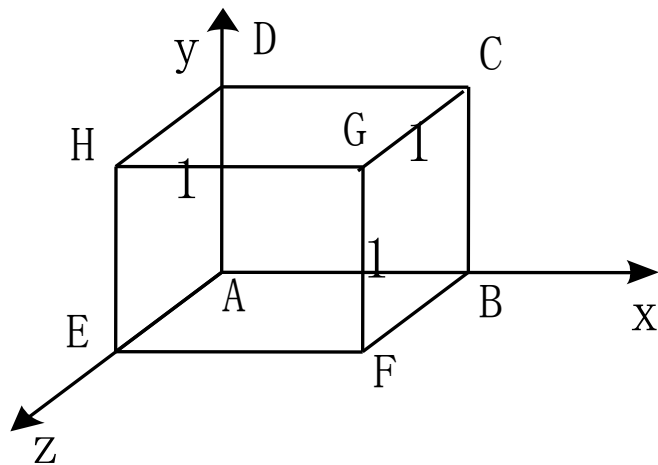
(x, y, z) 一点透视变换
矩阵形式:

$$T_{p1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{d} \\ l & m & 0 & \frac{n}{d} + 1 \end{bmatrix}$$

$$[x' \quad y' \quad z' \quad 1] = [x \quad y \quad z \quad 1] \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{d} \\ l & m & 0 & \frac{n}{d} + 1 \end{bmatrix}$$

$$= \begin{bmatrix} \frac{x+l}{\frac{(n+z)}{d} + 1} & \frac{y+m}{\frac{(n+z)}{d} + 1} & 0 & 1 \end{bmatrix}$$

例：试绘制如下图所示的单位立方体的一点透视图。



设： $l = -0.8$, $m = -1.6$, $n = -2$, 视距 $d = -2.5$,

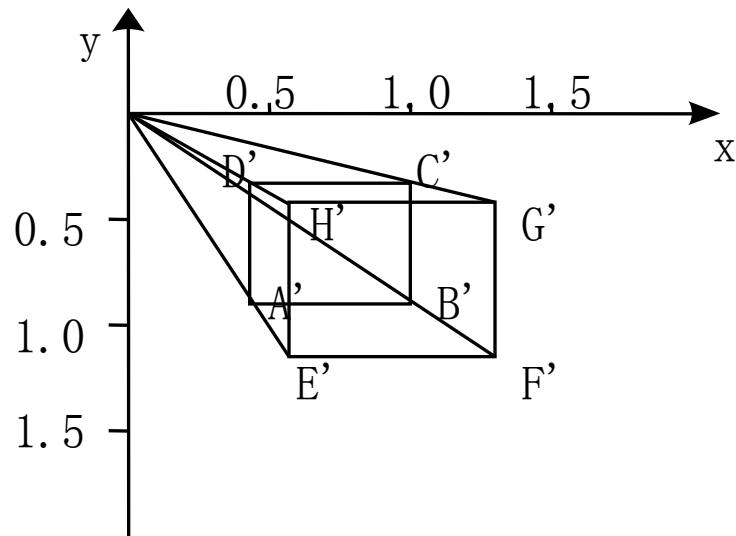
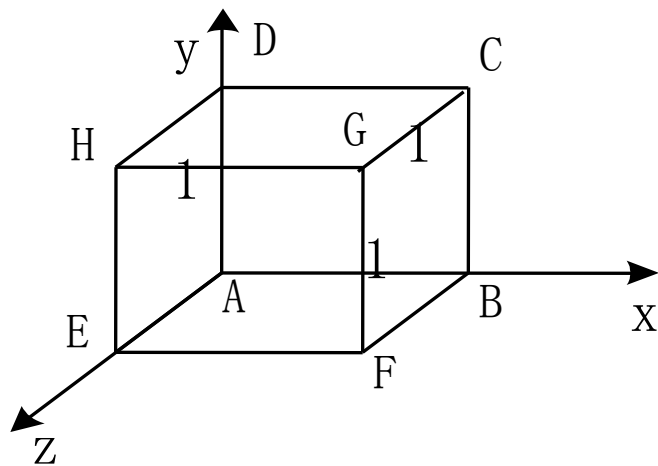
将上述值代入一点透视变换矩阵，得到：

$$\begin{matrix} A \\ B \\ C \\ D \\ E \\ F \\ G \\ H \end{matrix} \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -0.4 \\ -0.8 & -1.6 & 0 & 1.8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{d} \\ l & m & 0 & \frac{n}{d} + 1 \end{bmatrix}$$

$$\begin{aligned} l &= -0.8, \quad m = -1.6, \\ n &= -2, \quad d = -2.5 \end{aligned}$$

$$= \begin{bmatrix} 0.8 & -1.6 & 0 & 1.8 \\ 1.8 & -1.6 & 0 & 1.8 \\ 1.8 & -0.6 & 0 & 1.8 \\ 0.8 & -0.6 & 0 & 1.8 \\ 0.8 & -1.6 & 0 & 1.4 \\ 1.8 & -1.6 & 0 & 1.4 \\ 1.8 & -0.6 & 0 & 1.4 \\ 0.8 & -0.6 & 0 & 1.4 \end{bmatrix} = \begin{bmatrix} 0.44 & -0.89 & 0 & 1 \\ 1 & -0.89 & 0 & 1 \\ 1 & -0.33 & 0 & 1 \\ 0.44 & -0.33 & 0 & 1 \\ 0.57 & -1.14 & 0 & 1 \\ 1.29 & -1.14 & 0 & 1 \\ 1.29 & -0.43 & 0 & 1 \\ 0.57 & -0.43 & 0 & 1 \end{bmatrix} \begin{matrix} A' \\ B' \\ C' \\ D' \\ E' \\ F' \\ G' \\ H' \end{matrix}$$

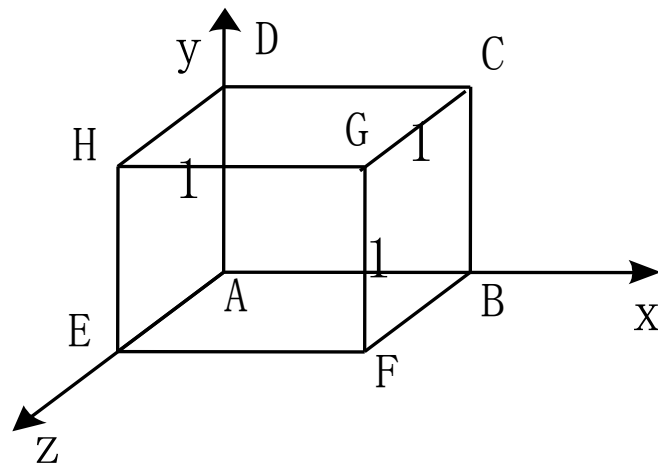


单位立方体的一点透视

2、二点透视投影图的生成

做两点透视时，通常要将物体绕y轴旋转 θ 角，以使物体的主要平面不平行于投影面

经透视变换后使物体产生变形，然后再向投影面做正投影



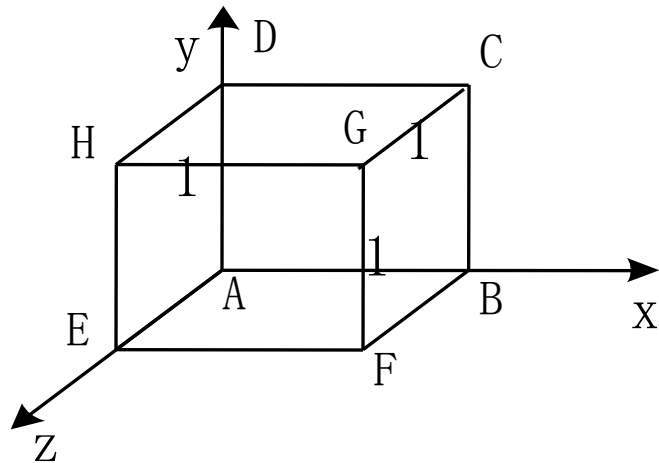
构造二点透视的一般步骤如下：

(1) 将物体平移到适当位置 l 、 m 、 n

(2) 将物体绕 y 轴旋转 θ 角

(3) 进行透视变换

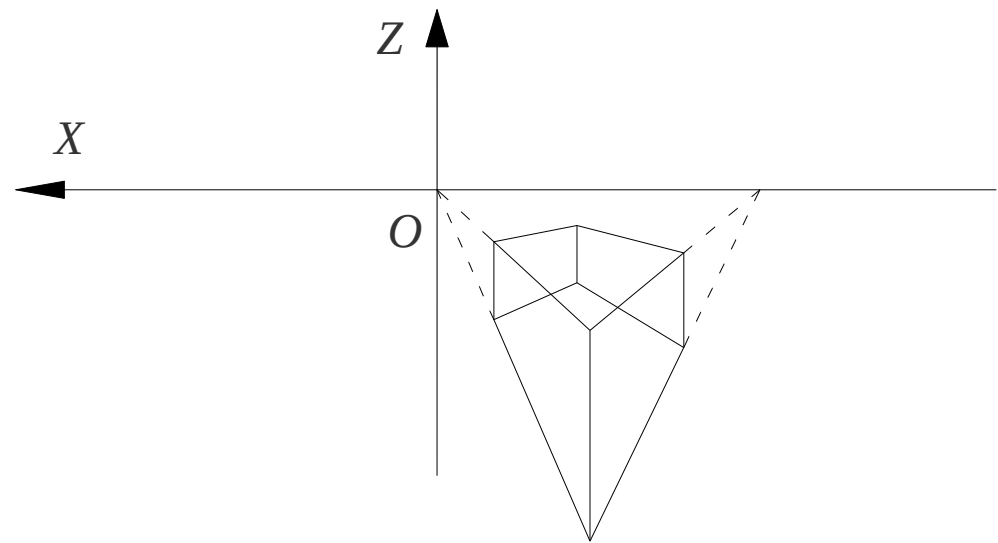
(4) 最后向 xoy 面做正投影，即得二点透视图



$$T_{p2} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ l & m & n & 1 \end{bmatrix} \bullet \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta & 0 & 0 & p \cdot \cos \theta - r \sin \theta \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & 0 & p \cdot \sin \theta + r \cdot \cos \theta \\ l \cdot \cos \theta + n \cdot \sin \theta & m & 0 & p(l \cdot \cos \theta + n \cdot \sin \theta) + r(n \cdot \cos \theta - l \cdot \sin \theta) + 1 \end{bmatrix}$$

变换结果如下图所示：



3、三点透视投影图的生成

构造三点透视的一般步骤如下：

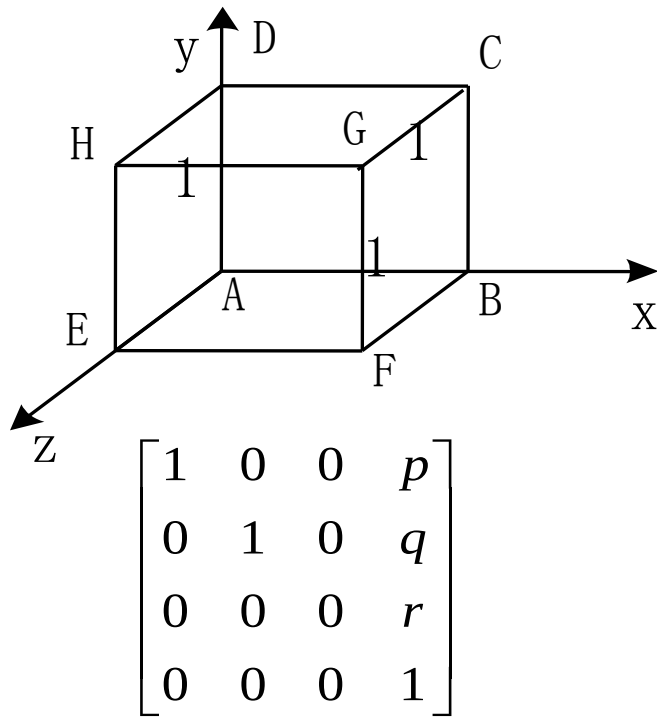
(1) 将物体平移到适当位置

(2) 将物体绕y轴旋转 θ 角

(3) 再绕x轴旋转 α 角

(4) 进行透视变换

(5) 最后向xoy面做正投影，即得三点透视图



三维图形变换小结

三维物体基本几何变换

三维物体的投影变换

一、三维物体基本几何变换

三维物体的几何变换是在二维方法基础上增加了对 z 坐标的考虑而得到的

有关二维图形几何变换的讨论，基本上都适合于三维空间

根据 T_{3D} 在变换中所起的具体作用，进一步可将 T_{3D} 分成四个矩阵。即：

$$T_{3D} = \left[\begin{array}{ccc|c} a & b & c & p \\ d & e & f & q \\ g & h & i & r \\ \hline l & m & n & s \end{array} \right]$$

$$T_1 = \left[\begin{array}{ccc} a & b & c \\ d & e & f \\ g & h & i \end{array} \right] \quad \text{对点进行比例、对称、旋转、错切变换}$$

$$T_2 = \begin{bmatrix} l & m & n \end{bmatrix} \quad \text{对点进行平移变换}$$

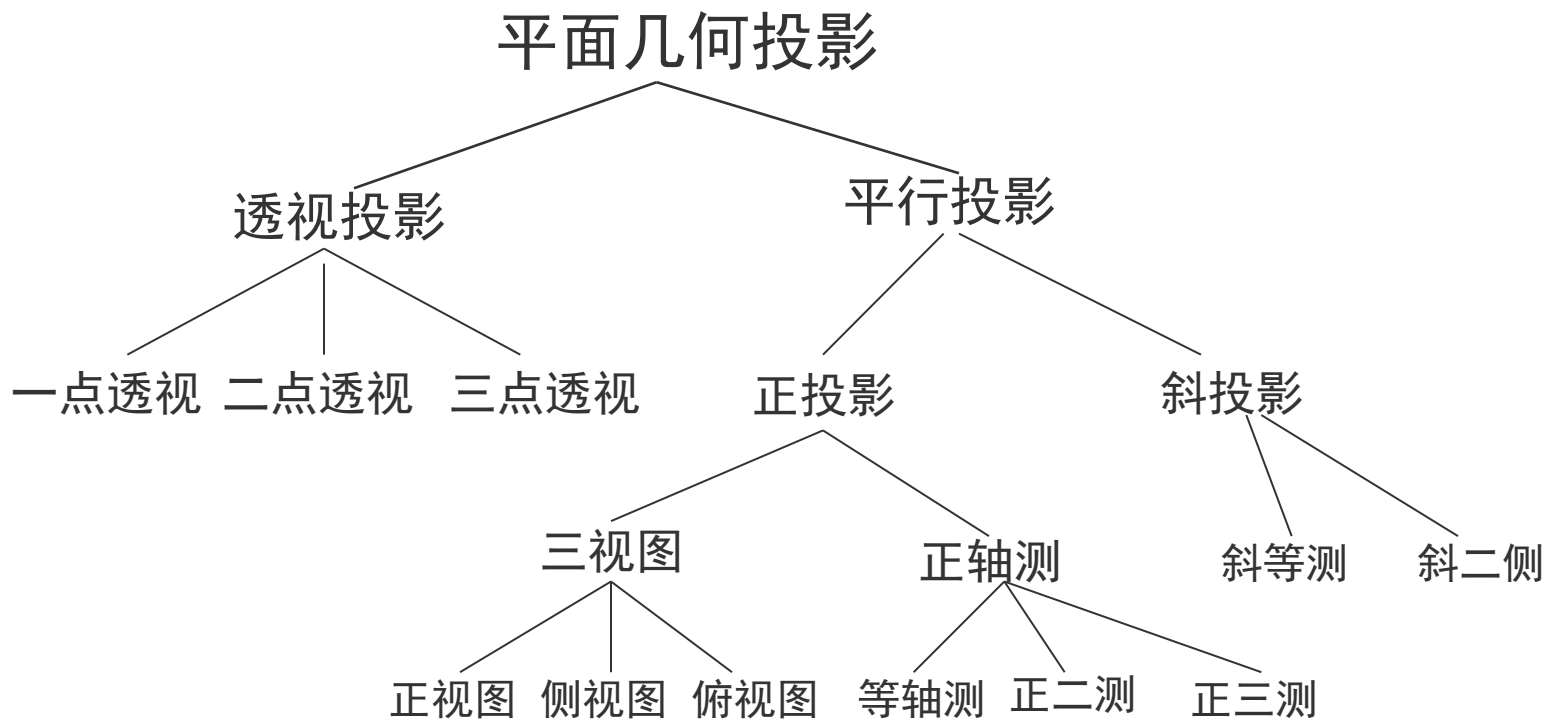
$$T_3 = \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

作用是进行透视投影变换

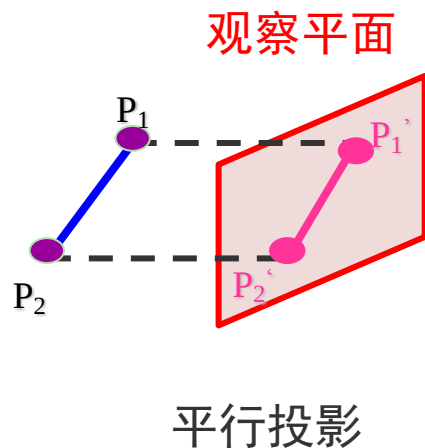
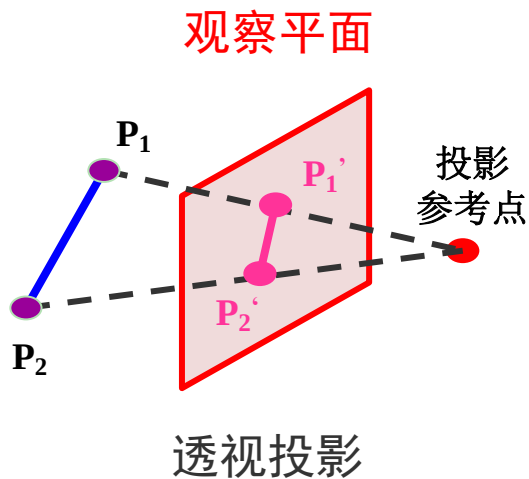
$$T_4 = [s]$$

作用是产生整体比例变换

二、三维物体的投影变换



两种投影法的本质区别在于：透视投影的投影中心到投影面之间的距离是有限的；而另一个的距离是无限的

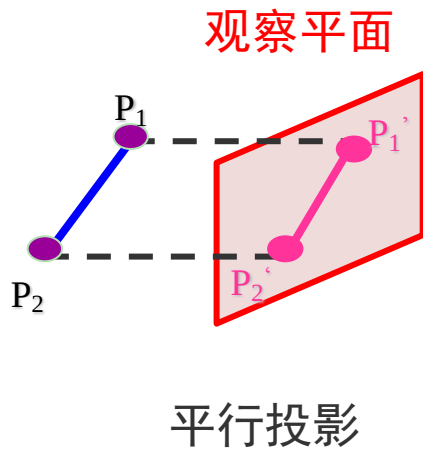


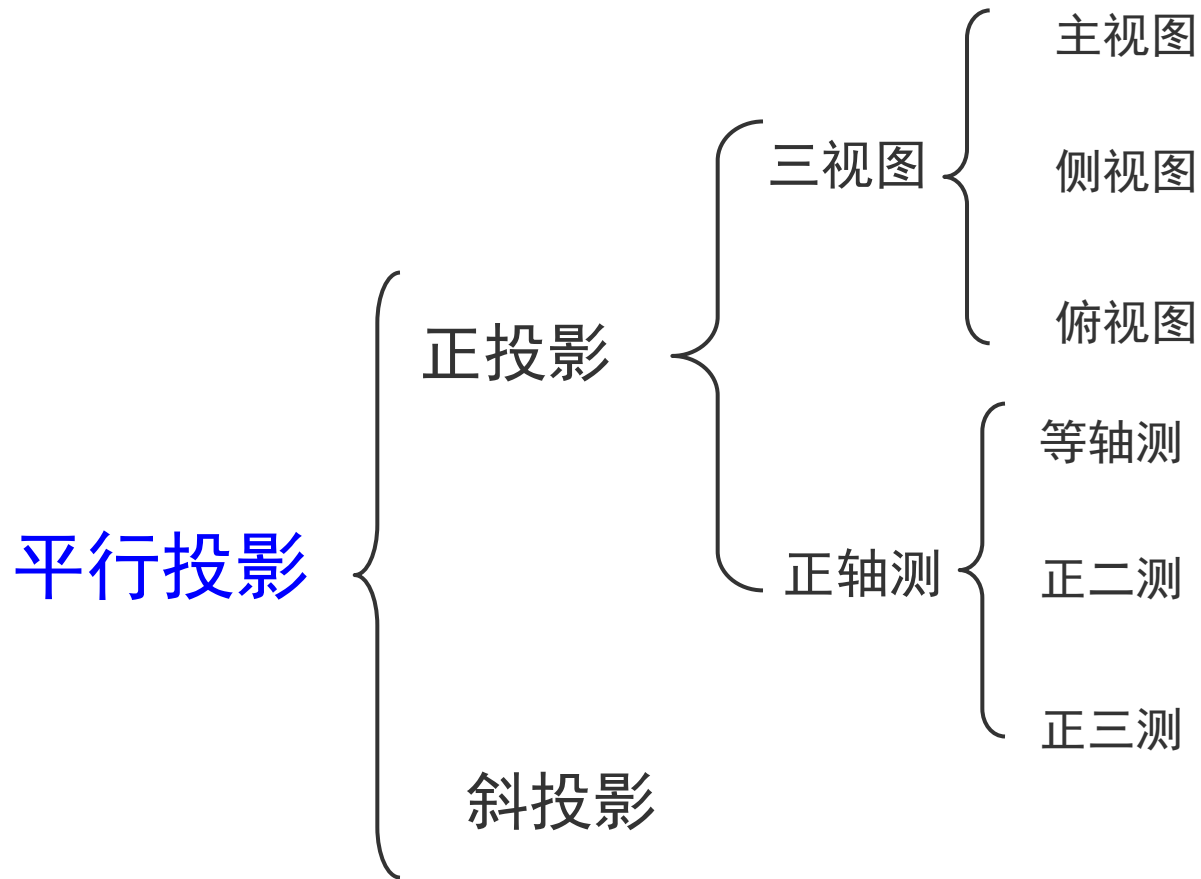
平行投影特点：

平行投影保持物体的有关比例不变

物体的各个面的精确视图由平行投影而得

没有给出三维物体外表的真实性表示

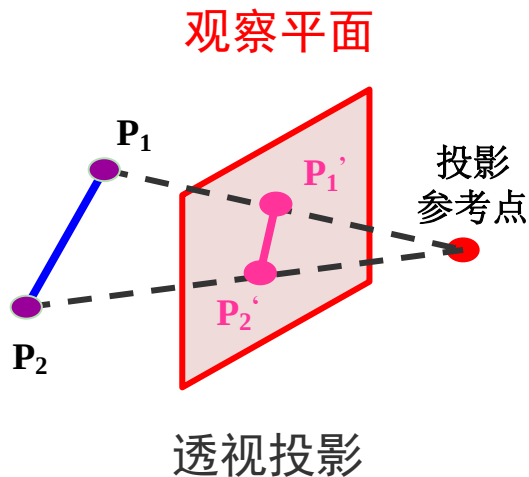




透视投影特点：

物体的投影视图由计算投影线
与观察平面之交点而得

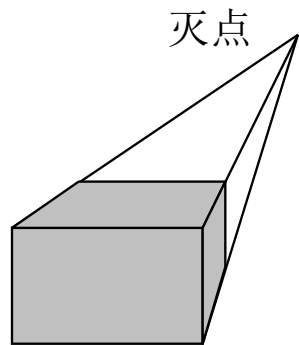
透视投影生成真实感视图但不保
持相关比例



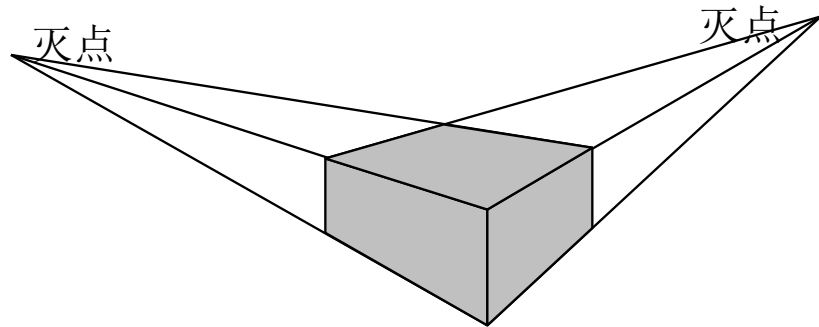
透视投影比轴测图更富有立体感和真实感

其中的[p, q, r]能产生透
视变换的效果

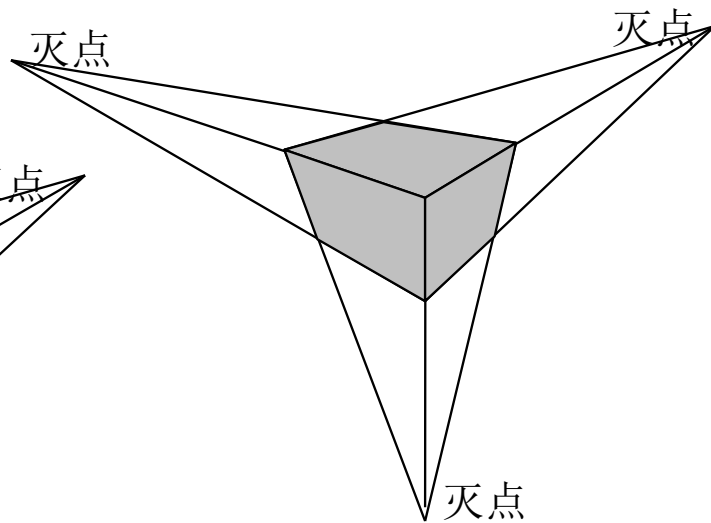
$$T_{3D} = \left[\begin{array}{ccc|c} a & b & c & p \\ d & e & f & q \\ g & h & i & r \\ \hline l & m & n & s \end{array} \right]$$



(a) 一点透视



(b) 二点透视



(c) 三点透视

曲线曲面基本理论

计算机图形学三大块内容：**光栅图形显示**、**几何造型技术**、**真实感图形显示**。光栅图形学是图形学的基础，有大量的思想和算法

几何造型技术是一项研究在计算机中，如何表达物体模型形状的技术

描述物体的三维模型有三种：

线框模型、曲面模型和实体模型

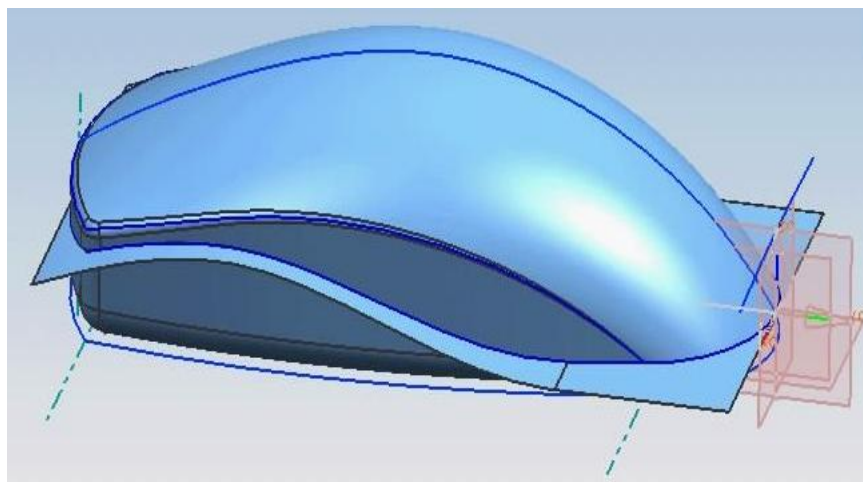
线框模型用顶点和棱边来表示物体

曲面模型只描述物体的表面和表面的连接关系，不描述物体内部的点的属性

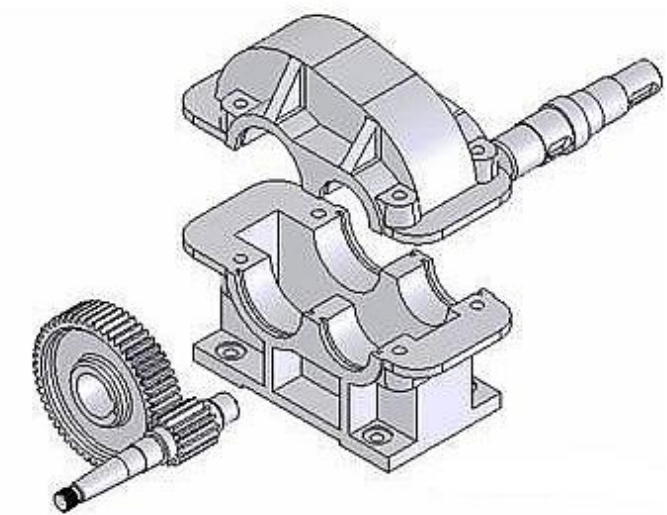
实体模型不但有物体的外观而且也有物体内点的描述

一、几何造型的历史

曲面造型：60年代，法国雷诺汽车公司的Pierre Bézier研发了汽车外形设计的UNISURF系统



实体造型：1973英国剑桥大学CAD小组的Build系统、美国罗彻斯特大学的PADL-1系统等



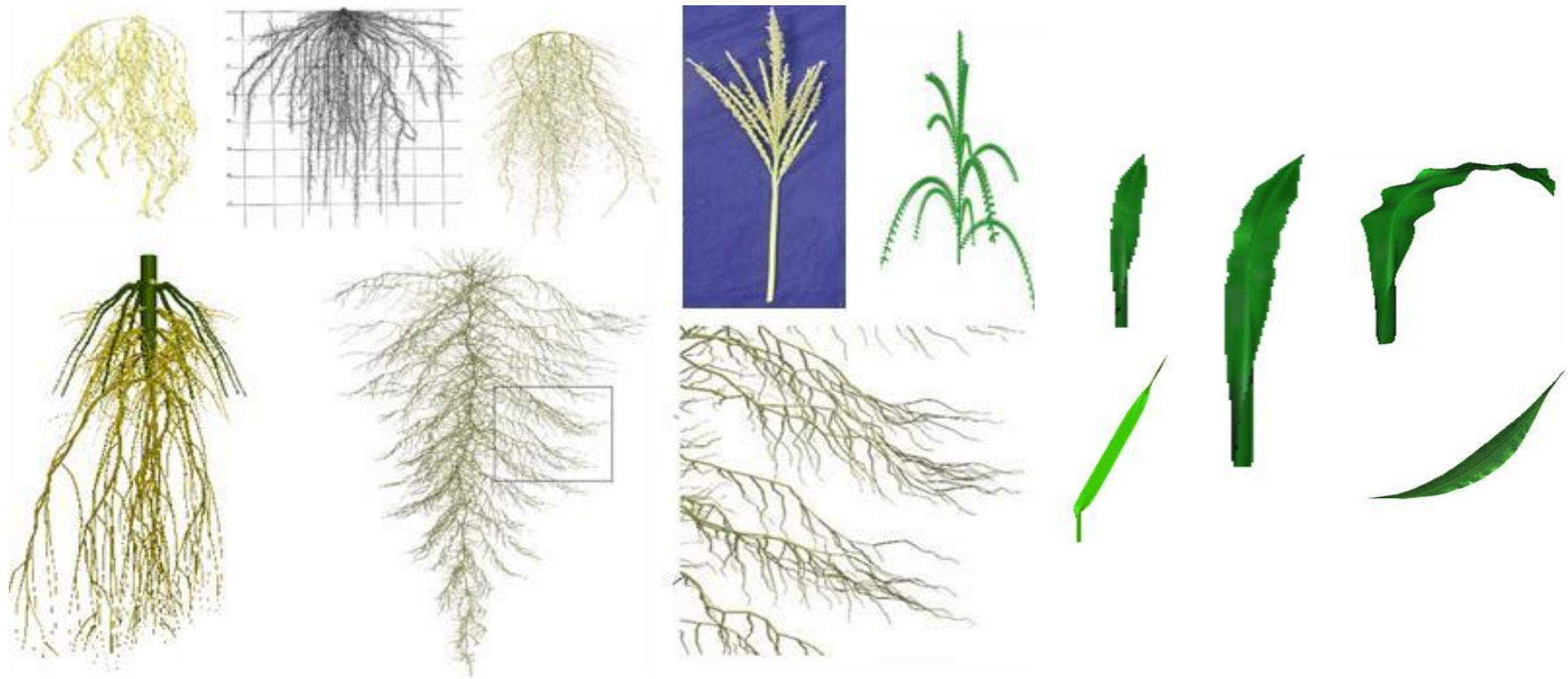
- In 1962, **Pierre Bézier**, an engineer of French Renault Car company, propose a new kind of curve representation, and finally developed a system **UNISURF** for car surface design in 1972.



- Solid Modeling（实体造型）：
 - In 1973, Ian Braid of Cambridge University developed a solid modeling system for designing engineering parts.
 - Ian Braid presented in his dissertation “Designing with volumes”, this work being demonstrated with the BUILD-1 system.







二、曲线曲面基础

1、显示、隐式和参数表示

曲线和曲面的表示方程有参数表示和非参数表示之分，非参数表示又分为显式表示和隐式表示。

对于一个平面曲线，显式表示一般形式是： $y = f(x)$

$$y = f(x)$$

在此方程中，一个x值与一个y值对应，所以显式方程不能表示封闭或多值曲线

如果一个平面曲线方程，表示成 $f(x, y) = 0$ 的形式，称之为隐式表示。隐式表示的优点是易于判断一个点是否在曲线上

2、显式或隐式表示存在的问题

- (1) 与坐标轴相关
- (2) 用隐函数表示不直观，作图不方便
- (3) 用显函数表示存在多值性
- (4) 会出现斜率为无穷大的情形

3、参数方程

为了克服以上问题，曲线曲面方程通常表示成参数的形式，假定用 t 表示参数，平面曲线上任一点 P 可表示为：

$$p(t) = [x(t), y(t)]$$

空间曲线上任一三维点 P 可表示为：

$$p(t) = [x(t), y(t), z(t)]$$

它等价于笛卡儿分量表示：

$$p(t) = x(t)i + y(t)j + z(t)k$$

这样，给定一个t值，就得到曲线上一点的坐标

假设曲线段对应的参数区间为 $[a, b]$ ，即 $a \leq t \leq b$ 。为方便期间，可以将区间 $[a, b]$ 规范化成 $[0, 1]$ ，参数变换为：

$$t' = \frac{t - a}{b - a}$$

参数曲线一般可写成：

$$p = p(t) \quad t \in [0, 1]$$

$$p = p(t) \quad t \in [0, 1]$$

该形式把曲线上表示一个点的位置矢量的各个分量合写在一起当成一个整体，考虑的是曲线上点之间的相对位置关系而不是它们与所取坐标系之间的相对位置关系

类似地，可把曲面表示成为双参数 u 和 v 的矢量函数

$$p(u, v) = p(x(u, v), y(u, v), z(u, v)) \quad (u, v) \in [0, 1] \times [0, 1]$$

最简单的参数曲线是直线段，端点为 P_1 、 P_2 的直线段参数方程可表示为：

$$p(t) = p_1 + (p_2 - p_1)t \quad t \in [0,1]$$

4、参数方程的优势

在曲线、曲面的表示上，参数方程比显式、隐式方程有更多的优越性，主要表现在：

(1) 可以满足几何不变性的要求

即指形状的数学表示及其所表达的形状不随所取坐标系而改变的性质

(2) 有更大的自由度来控制曲线、曲面的形状

$$y = ax^3 + bx^2 + cx + d$$

只有四个系数控制曲线的形状。而二维三次曲线的参数表达式为：

$$p(t) = \begin{bmatrix} a_1 t^3 + a_2 t^2 + a_3 t + a_4 \\ b_1 t^3 + b_2 t^2 + b_3 t + b_4 \end{bmatrix} \quad t \in (0,1)$$

有8个系数可用来控制此曲线的形状

(3) 直接对参数方程进行几何变换

对非参数方程表示的曲线、曲面进行变换，必须对曲线、曲面上的每个型值点进行几何变换；而对参数表示的曲线、曲面可对其参数方程直接进行几何变换

(4) 便于处理斜率为无穷大的情形，不会因此而中断计算

(5) 界定曲线、曲面的范围十分简单

具有规格化的参数变量 $t \in [0, 1]$

(6) 易于用向量（矢量）和矩阵运算，简化计算

5、参数曲线的基本概念

这部分内容完全来自于微分几何。微分几何是用微分的方法来研究曲线的局部性质，如曲线的弯曲程度等

一条用参数表示的三维曲线是一个有界点集，可写成一个带参数的、连续的、单值的数学函数，其形式：

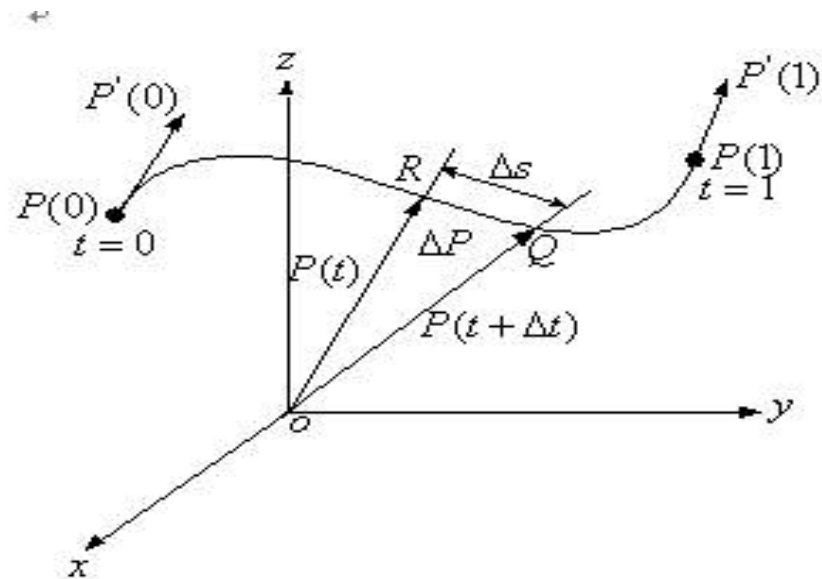
$$\begin{cases} x = x(t) \\ y = y(t) \\ z = z(t) \end{cases} \quad 0 \leq t \leq 1$$

$$p'(t) = \frac{dP}{dt} \quad p''(t) = \frac{d^2 P}{dt^2}$$

(1) 位置矢量

曲线上任一点的位置矢量
可表示为：

$$p(t) = [x(t), y(t), z(t)]$$



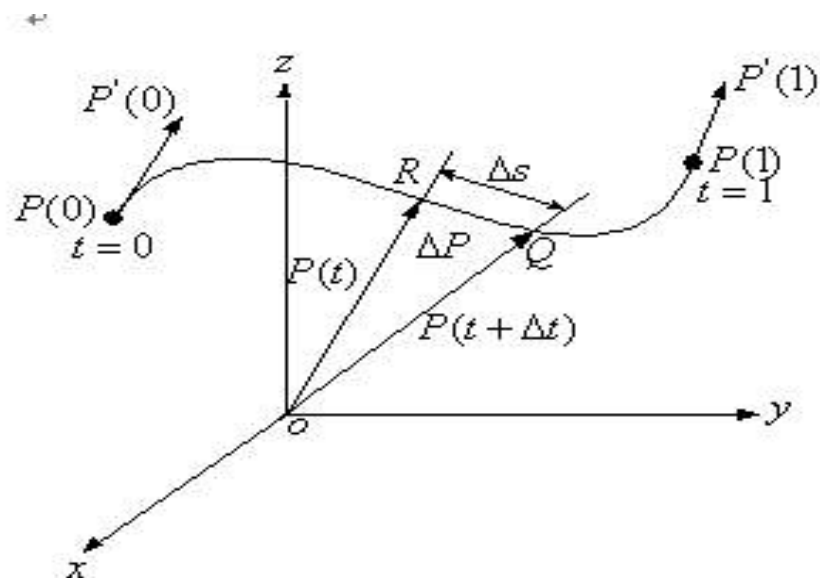
(2) 切矢量

选择弧长 s 作为参数，当
 $\Delta t \rightarrow 0$ 时，弦长 $\Delta s \rightarrow 0$ ，但
方向不能趋向于0

$$T = \frac{dP}{ds} = \lim_{\Delta s \rightarrow 0} \frac{\Delta P}{\Delta s} \quad \text{单位矢量}$$

根据弧长微分公式有：

$$(ds)^2 = (dx)^2 + (dy)^2 + (dz)^2$$



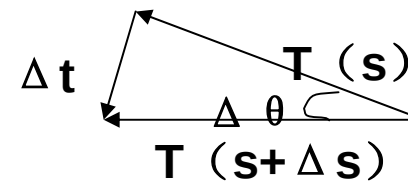
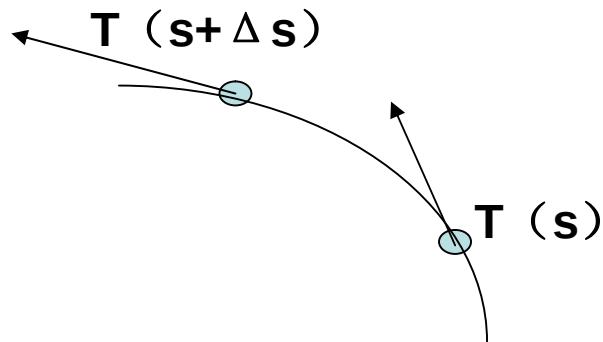
引入参数t，可改写为：

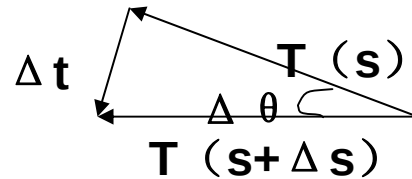
$$(ds / dt)^2 = (dx / dt)^2 + (dy / dt)^2 + (dz / dt)^2 = |P'(t)|^2$$

$$T = \frac{dP}{ds} = \frac{dP}{dt} \cdot \frac{dt}{ds} = \frac{P'(t)}{|P'(t)|} \quad \text{即T是单位切矢量}$$

(3) 曲率

切向量求导，求导以后还是一个向量，称为**曲率**，其几何意义是曲线的单位切向量对弧长的转动率，即刻画这一点的曲线的弯曲程度。





$$k = |T'| = \lim_{\Delta s \rightarrow 0} \left| \frac{\Delta T}{\Delta s} \right| = \lim_{\Delta s \rightarrow 0} \left| \frac{T(s + \Delta s) - T(s)}{\Delta s} \right| = \lim_{\Delta s \rightarrow 0} \left| \frac{\Delta q}{\Delta s} \right|$$

曲率越大，表示曲线的弯曲程度越大

曲率 k 的倒数 $r = \frac{1}{k}$ 称为曲率半径

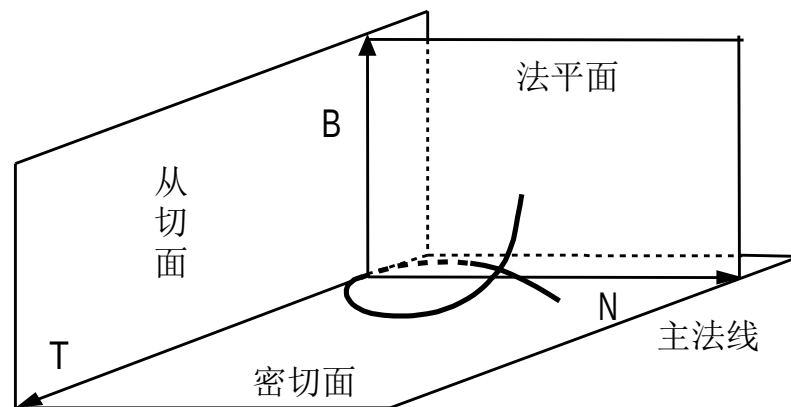
曲率半径越大，圆弧越平缓

曲率半径越小，圆弧越陡

(4) 法矢量

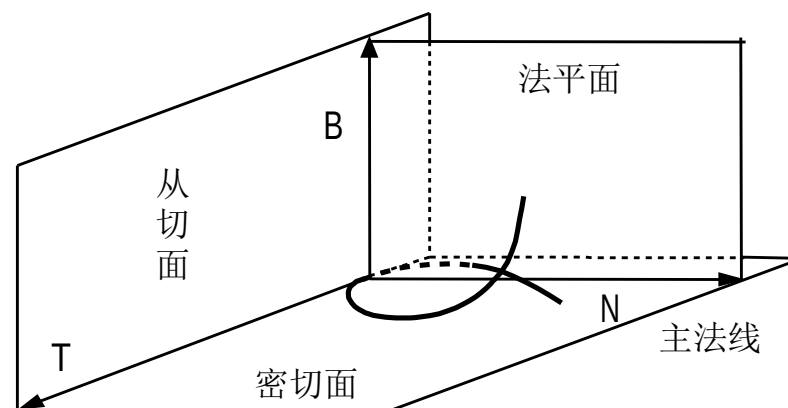
法矢量是与切矢量垂直的向量

N、B 构成的平面称为法平面，N、T 构成的平面称为密切平面，B、T 构成的平面称为从切平面



T(切矢)、N(主法矢)和B(副法矢)构成了曲线上的活动坐标架

$$B = T \times N$$



(5) 挠率

空间曲线不但要弯曲，而且还要扭曲，即要离开它的密切平面。为了能刻画这一扭曲程度，等价于去研究密切平面的法矢量（即曲线的副法矢量）关于弧长的变化率。

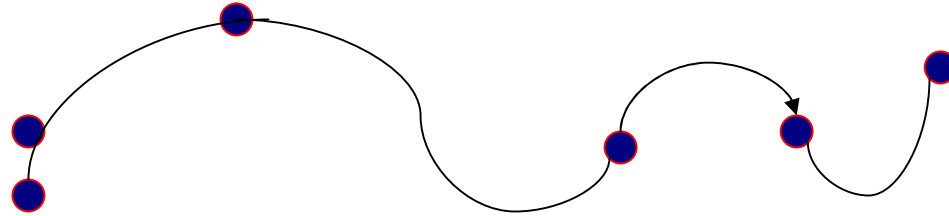
挠率 t 的绝对值等于副法线方向(或密切平面)对于弧长的转动率：

$$|t| = \lim_{\Delta s} \left| \frac{\Delta q}{\Delta s} \right|$$

6、插值

自由曲线和自由曲面一般通过少数分散的点生成，这些点叫做“型值点”、“样本点”或“控制点”

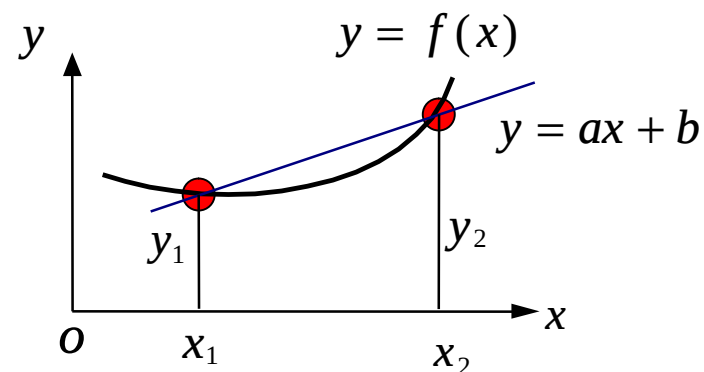
给定一组有序的数据点 $P_i (i=0, 1, 2, \dots, n)$ ，要求构造一条曲线顺序通过这些数据点，称为对这些数据点进行插值（interpolation），所构造的曲线称为插值曲线



把曲线插值推广到曲面，类似地就有插值曲面

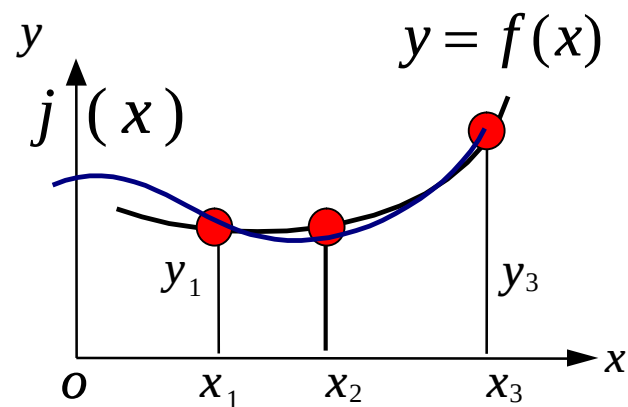
构造插值曲线曲面所采用的数学方法称为曲线曲面插值法

(1) 线性插值：假设给定函数 $f(x)$
在两个不同点 x_1 和 x_2 的值，用
一个线形函数： $y=ax+b$ ，近似
代替，称为的线性插值函数



(2) 抛物线插值：已知三个点的坐标，要求构造一个抛物线函数

$$j(x) = ax^2 + bx + c$$



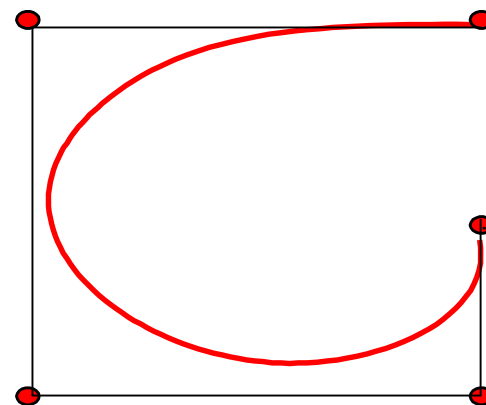
7、拟合

构造一条曲线使之在某种意义下最接近给定的数据点(但未必通过这些点)，所构造的曲线为拟合曲线

在计算数学中，逼近通常指用一些性质较好的函数近似表示一些性质不好的函数。在计算机图形学中，逼近继承了这方面的含义，因此插值和拟合都可以视为逼近

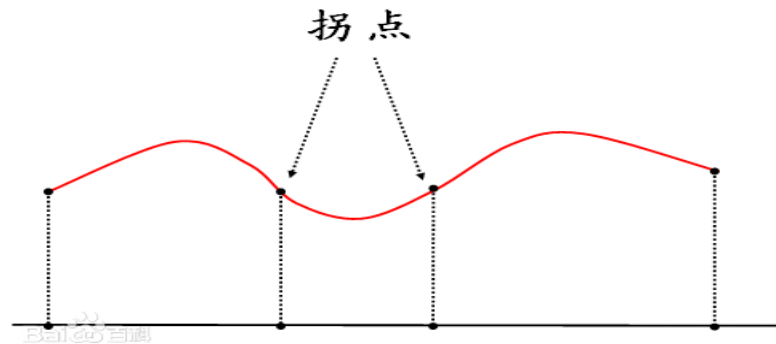
对于逼近样条，连接控制点序列的折线通常被显示出来，以提醒设计者控制点的次序

一般将连接有一定次序控制点的直线序列称为**控制多边形**或**特征多边形**



7、光顺

指曲线的拐点不能太多（有一、二阶导数等）



在数学领域是指：凸曲线与凹曲线的连接点

对平面曲线而言，相对光顺的条件是：

- a. 具有二阶几何连续性(G^2)
- b. 不存在多余拐点和奇异点
- c. 曲率变化较小

8、连续性

当许多参数曲线段首尾相连构成一条曲线时，如何保证各曲线段在连接处具有合乎要求的连续性是一个重要问题。假定参数曲线段 p_i 以参数形式进行描述：

$$p_i = p_i(t) \quad t \in [t_{i0}, t_{i1}]$$

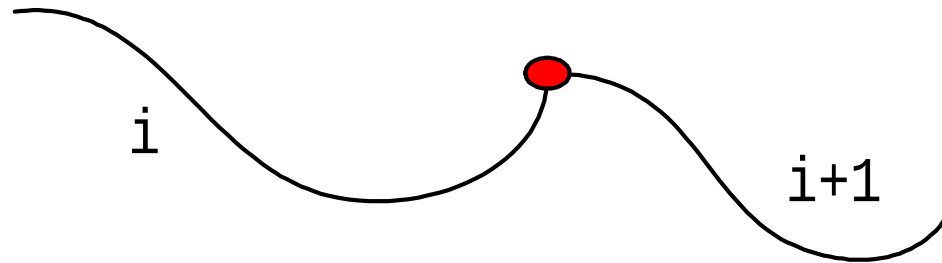
这里讨论参数曲线两种意义上的连续性：即参数连续性和几何连续性

(1) 参数连续性

0阶参数连续性:

记作 C^0 连续性，是指曲线的几何位置连接，即第一个曲线段在 t_{i1} 处的 x, y, z 值与第二个曲线段在 $t_{(i+1)0}$ 处的 x, y, z 值相等：

$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0})$$



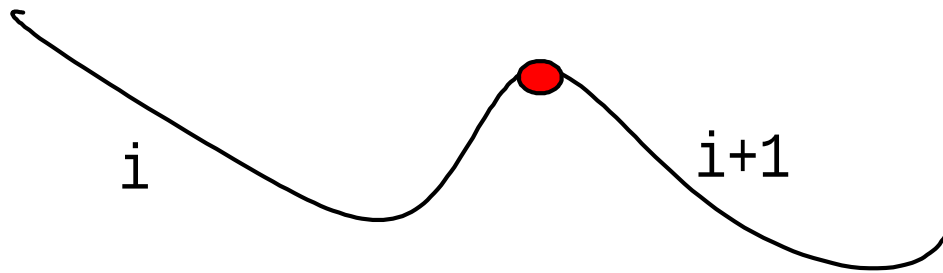
1阶参数连续性:

记作 C^1 连续性, 指代表两个相邻曲线段的方程在相交点处有相同的一阶导数 (切线) :

$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0})$$

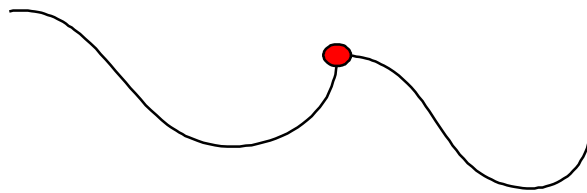
且 $p'_i(t_{i1}) = p'_{(i+1)}(t_{(i+1)0})$

一阶连续性对数字化
绘画及一些设计应用
已经足够

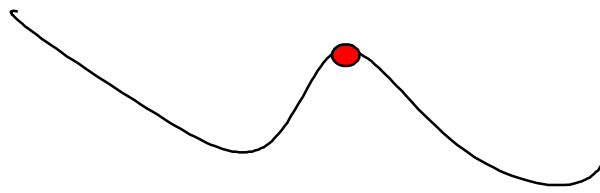


2阶参数连续性:

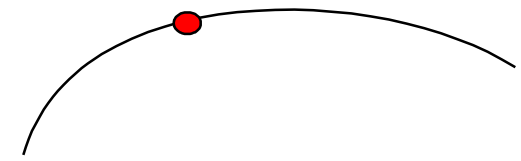
记作 C^2 连续性, 指两个相邻曲线段的方程在相交点处具有相同的一阶和二阶导数。类似地, 还可定义高阶参数连续性



(a) 0阶连续性



(b) 1阶连续性



(c) 2阶连续性

对于 C^2 连续性，交点处的切向量变化率相等，即切线从一个曲线段平滑地变化到另一个曲线段

二阶连续性对电影中的动画途径和很多精密CAD需求有用

经典的参数连续性在图形学里是不适合的，因为太苛刻，所以引进了几何连续性的概念

汽车曲面的设计美观要求很高，但有时候车身的一条曲线并不是参数连连续的，但人眼看上去已经是很光滑的了，因此需要一种更弱的连续性

$$\Phi(t) = \begin{cases} V_0 + \frac{V_1 - V_0}{3}t, & 0 \leq t \leq 1 \\ V_0 + \frac{V_1 - V_0}{3} + (t - 1)\frac{2(V_1 - V_0)}{3}, & 1 \leq t \leq 2 \end{cases}$$

$$\Phi'(1^-) = \frac{1}{3}(V_1 - V_0) \quad \Phi'(1^+) = \frac{2}{3}(V_1 - V_0)$$

$$\Phi'(1^-) = \frac{1}{3}(V_1 - V_0) \quad \Phi'(1^+) = \frac{2}{3}(V_1 - V_0)$$

上面的函数实际上是直线方程。但可以发现，在 $t=1$ 这一点，直线的左右导数不相等

在微积分里，如果一个函数的在一点处它的左导数和右导数都存在并且相等，就说明在这一点是连续的

(2) 几何连续性

曲线段相连的另一个连续性条件是几何连续性。与参数连续性不同的是，它只需曲线段在相交处的参数导数成比例即可

0阶几何连续性：记作 G^0 连续性。与0阶参数连续性的定义相同，满足：

$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0})$$

1阶几何连续性，记作 G^1 连续性。若要求在结合处达到 G^1 连续，就是说两条曲线在结合处在满足 G^0 连续的前提下，并有公共的切矢

$$Q'(0) = a P'(1) \quad (a > 0)$$

2阶几何连续性，记作 G^2 连续性。就是说两条曲线在结合处在满足 G^1 连续的条件下，并有公共的曲率

一阶导数相等和有公共切向量这两个概念差别是什么？导数相等是大小方向都相等，而公共切矢意味着方向相同但大小不等

所谓参数连续意味着导数相等，导数相等意味着两个切向量不但方向相等而且长度也相等

如果是几何连续的话，只是要求切向量一样，方向一样，长度可以不同。条件减弱了

9、参数化

过三点 P_0 、 P_1 和 P_2 构造参数表示的插值多项式是唯一的还是有多个呢？（设置一个课间提问）

插值多项式可以有无数条，这是因为对应地参数 t 在 $[0, 1]$ 中可以有无数种取法

$$t_0 = 0, t_1 = \frac{1}{2}, t_2 = 1 \quad t_0 = 0, t_1 = \frac{1}{3}, t_2 = 1$$

参数方程:

$$\begin{aligned} x(t) &= a_1 t^2 + a_2 t + a_3 \\ y(t) &= b_1 t^2 + b_2 t + b_3 \end{aligned}$$

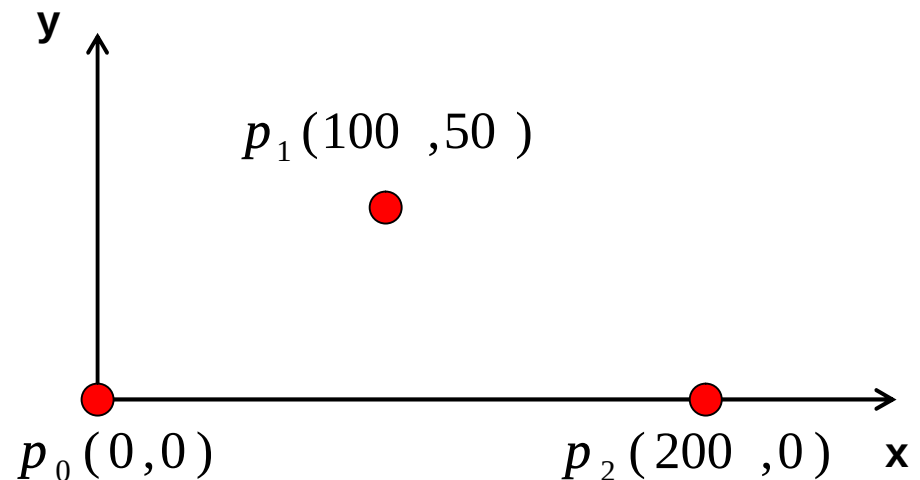
插值问题实际上就是解方程组的问题。但如果参数取的不一样的话，结果是不一样的

每个参数值称为节点（knot）。对于一条插值曲线， p_0 、 p_1 、 p_2 这些点称为型值点

对于一条插值曲线，型值点 $p_0, p_1, \dots, p_n$ 与其参数域 $t \in [t_0, t_1]$ 内的节点之间有一种对应关系。对于一组有序型值点，所确定一种参数分割，称之这组型值点的参数化

现在给定3个点 p_0 、 p_1 、 p_2 ，坐标是 $(0, 0)$ 、 $(100, 50)$ 、 $(200, 0)$ ，求一条2次的多项式曲线来插值这三个点

假设第一个点参数 $t = 0$ ，
第二个点参数取 $t = 1/2$ ，
第三个点的参数取 $t = 1$ 。
如何列这个方程？



$$t_1 = 0 \quad t_2 = \frac{1}{2} \quad t_3 = 1$$

$$x(t) = a_1 t^2 + a_2 t + a_3$$

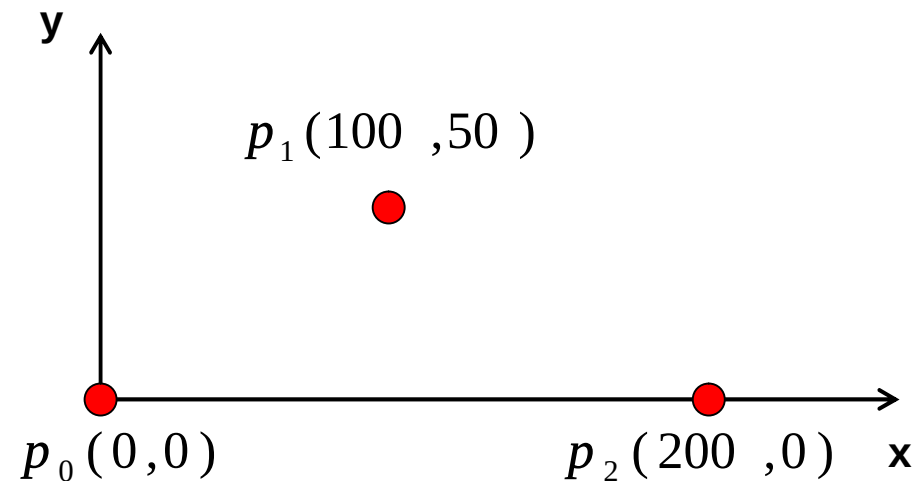
$$y(t) = b_1 t^2 + b_2 t + b_3$$

$$0 = x(0) = a_3$$

$$0 = y(0) = b_3$$

$$100 = x\left(\frac{1}{2}\right) = \frac{a_1}{4} + \frac{a_2}{2} + a_3$$

$$50 = y\left(\frac{1}{2}\right) = \frac{b_1}{4} + \frac{b_2}{2} + b_3$$



$$200 = x(1) = a_1 + a_2 + a_3$$

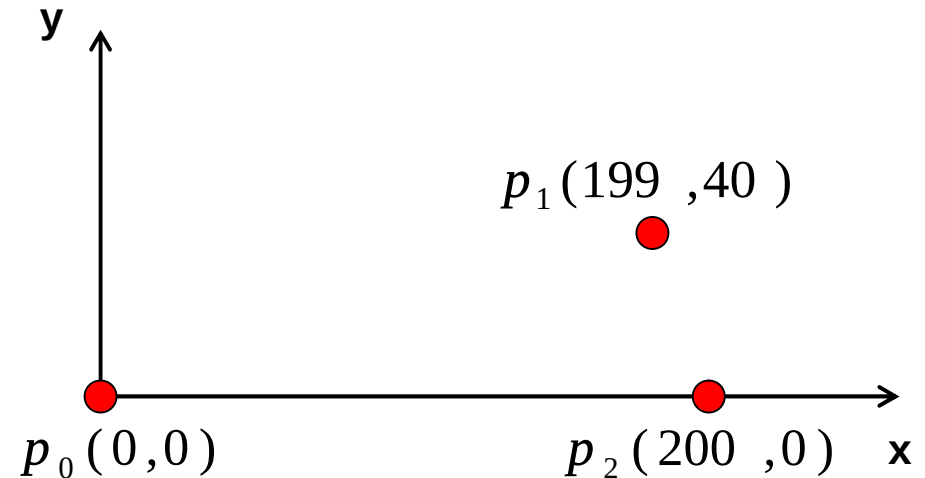
$$0 = y(1) = b_1 + b_2 + b_3$$

6个方程6个未知数，插值问题的本质是方程的个数和未知数的个数是一致的

现在的问题是凭什么取： $t=0$ ， $t=1/2$ ， $t=1$ ？

为什么 t 不可以取别的值，如 $t=0$ ， $t=1/3$ ， $t=1$ ？哪种取法更科学？

这样一条曲线参数 t 应该如何取比较好？如果再取成 $t=0$ ， $t=1/2$ ， $t=1$ 好不好？



参数化的本质就是找一组恰当的参数 t 来匹配这一组不同的型值点。给定一组不同的型值点，就要给出不同的参数化即不同的 t 值，这样才使得这条曲线美观、合理

参数化常用方法：

(1) 均匀参数化

节点在参数轴上呈等距分布。如0、 $1/10$ 、 $2/10$ 。。。

(2) 累加弦长参数化（根据长度的比例关系来确定t）

$$\begin{cases} t_0 = 0 \\ t_i = t_{i-1} + |\Delta P_{i-1}|, i = 1, 2, \dots, n \end{cases} \quad \Delta P_i = P_{i+1} - P_i$$

这种参数法如实反映了型值点按弦长的分布情况，能够克服型值点按弦长分布不均匀的情况下采用均匀参数化所出现的问题

3、向心参数化法

$$t_0 = 0$$

$$t_i = t_{i-1} + \left| \Delta P_{i-1} \right|^{1/2}, i = 1, 2, \dots, n$$

向心参数化法假设在一段曲线弧上的向心力与曲线切矢从该弧段始端至末端的转角成正比，加上一些简化假设，得到向心参数化法。此法尤其适用于非均匀型值点分布。

10、参数曲线的代数和几何形式

以三次参数曲线为例，讨论参数曲线的代数和几何形式

(1)代数形式：

$$\begin{cases} x(t) = a_{3x}t^3 + a_{2x}t^2 + a_{1x}t + a_{0x} \\ y(t) = a_{3y}t^3 + a_{2y}t^2 + a_{1y}t + a_{0y} \\ z(t) = a_{3z}t^3 + a_{2z}t^2 + a_{1z}t + a_{0z} \end{cases} \quad t \in [0,1]$$

上述代数式写成矢量式是：

$$P(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0 \quad t \in [0,1]$$

注意： a_3 ， a_2 ， a_1 ， a_0 是参数曲线的系数，但记住不是常数而是向量。 a_3 对应刚才的 a_{3x} 、 a_{3y} 、 a_{3z} 。改变系数曲线如何变化是不清楚的，这是代数形式的缺点

几何形式：

几何形式是利用一条曲线端点的几何性质来刻画一条曲线。所谓端点的几何性质，就是指曲线的端点位置、切向量、各阶导数等端点的信息。

对三次参数曲线，若用其端点位矢 $P(0)$ 、 $P(1)$ 和切矢 $P'(0)$ 、 $P'(1)$ 描述。需要这四个量来刻画三次参数曲线

将 $P(0)$ 、 $P(1)$ 、 $P'(0)$ 和 $P'(1)$ 简记为：

$$P_0、P_1、P'_0、P'_1$$

$$P(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0 \quad t \in [0,1]$$

$$P_0 = a_0$$

$$P_1 = a_3 + a_2 + a_1 + a_0 \quad \longrightarrow$$

$$P'_0 = a_1$$

$$P'_1 = 3a_3 + 2a_2 + a_1$$

$$\begin{cases} a_0 = P_0 \\ a_1 = P'_0 \\ a_2 = -3P_0 + 3P_1 - 2P'_0 - P'_1 \\ a_3 = 2P_0 - 2P_1 + P'_0 + P'_1 \end{cases}$$

$$\begin{cases} a_0 = P_0 \\ a_1 = P_0' \\ a_2 = -3P_0 + 3P_1 - 2P_0' - P_1' \\ a_3 = 2P_0 - 2P_1 + P_0' + P_1' \end{cases}$$

$$P(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0$$

$$P(t) = (2t^3 - 3t^2 + 1)p_0 + (-2t^3 + 3t^2)p_1 + (t^3 - 2t^2 - t)p_0' + (t^3 - t^2)p_1'$$

得到：

$$P(t) = (2t^3 - 3t^2 + 1)p_0 + (-2t^3 + 3t^2)p_1 + (t^3 - 2t^2 - t)p_0' + (t^3 - t^2)p_1'$$

$$\begin{aligned} \text{令: } F_0(t) &= 2t^3 - 3t^2 + 1 & F_1(t) &= -2t^3 + 3t^2 \\ G_0(t) &= t^3 - 2t^2 + t & G_1(t) &= t^3 - t^2 \end{aligned}$$

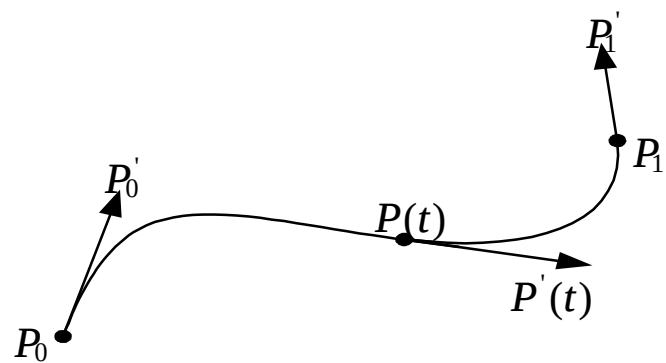
$$P(t) = F_0 P_0 + F_1 P_1 + G_0 P_0' + G_1 P_1' \quad t \in [0,1]$$

$$P(t) = F_0 P_0 + F_1 P_1 + G_0 P_0' + G_1 P_1' \quad t \in [0,1]$$

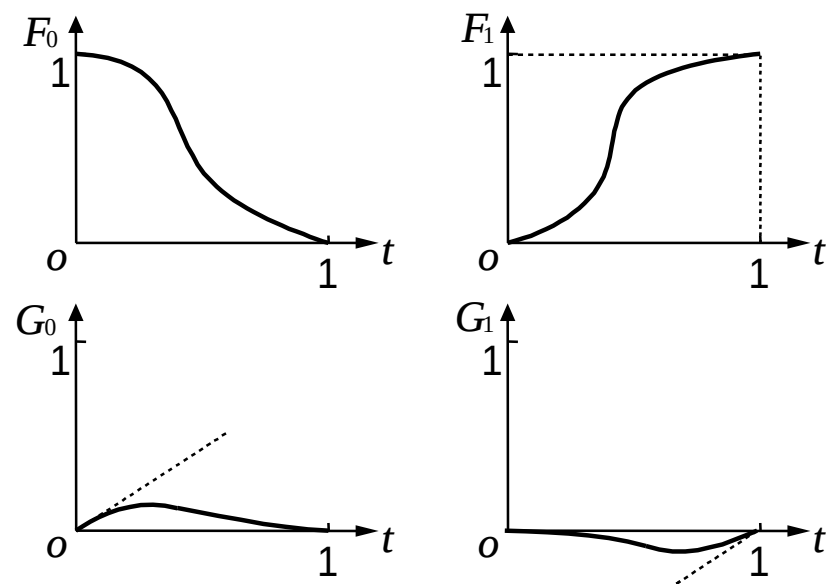
上式是三次Hermite曲线（三次哈密特曲线）的几何形式，几何系数是：

$$P_0、P_1、P_0'、P_1'$$

$F_0、F_1、G_0、G_1$ 称为调和函数（或混合函数）



曲线端点位矢和切矢



三次调和函数

Bezier曲线与曲面

一、Bezier曲线的背景和定义

1、Bezier曲线的背景

给定 $n+1$ 个数据点， $p_0(x_0, y_0), \dots, p_n(x_n, y_n)$ ，生成一条曲线，使得该曲线与这些点所描述的形状相符

如果要求曲线通过所有的数据点，则属于插值问题；如果只要求曲线逼近这些数据点，则属于逼近问题

逼近在计算机图形学中主要用来设计美观的或符合某些美学标准的曲线。为了解决这个问题，有必要找到一种用小的部分即曲线段构建曲线的方法，来满足设计标准

当用曲线段拟合曲线 $f(x)$ 时，可以把曲线表示为许多小线段 $\phi_i(x)$ 之和，其中 $\phi_i(x)$ 称为基（混合）函数。

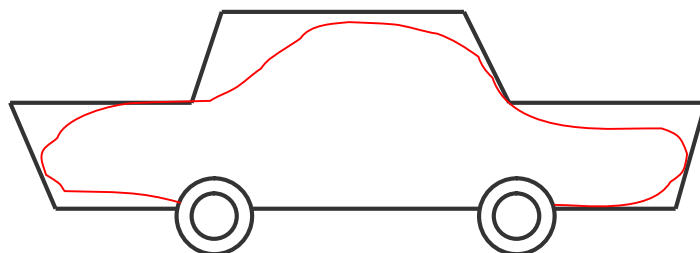
$$f(x) = \sum_{i=0}^n a_i \phi_i(x)$$

这些基（混合）函数是要用于计算和显示的。因此，经常选择多项式作为基（混合）函数

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

1962年，贝塞尔（P.E.Bezier）构造了一种以逼近为基础的参数曲线和曲面的设计方法，并用这种方法完成了一种称为UNISURF 的曲线和曲面设计系统。

想法基点是在进行汽车外形设计时，先用折线段勾画出汽车的外形大致轮廓，然后用光滑的参数曲线去逼近这个折线多边形



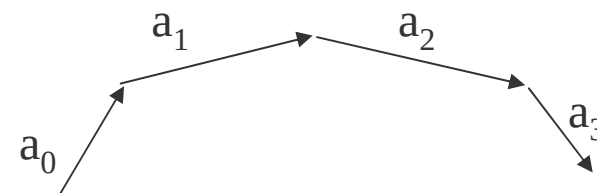
这个折线多边形被称为特征多边形。逼近该特征多边形的曲线被称为Bezier曲线

Bezier方法将函数逼近同几何表示结合起来，使得设计师在计算机上就象使用作图工具一样得心应手

贝塞尔曲线广泛地应用于很多图形图像软件中，例如Flash、Illustrator、CorelDRAW 和 Photoshop 等等

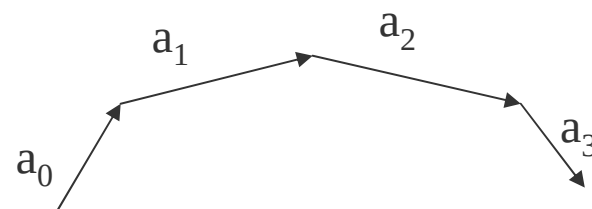
贝塞尔把参数n次曲线表示为：

$$p(t) = \sum_{i=0}^n a_i f_{i,n}(t) \quad 0 \leq t \leq 1$$



其中系数矢量 a_i ($i=0, 1, \dots, n$) 顺序首尾相接

从 a_0 的末端到 a_n 的末端所形成的折线称为控制多边形或贝塞尔多边形



$$p(t) = \sum_{i=0}^n a_i f_{i,n}(t) \quad 0 \leq t \leq 1$$

$$f_{i,n}(t) = \begin{cases} 1, & i = 0, \\ \frac{(-t)^i}{(i-1)!} \frac{d^{i-1}}{dt^{i-1}} \left(\frac{(1-t)^{n-1} - 1}{t} \right) \end{cases}$$

称为贝塞尔基函数

航空学报
1980年第一期

Bézier 基函数的导出

北京航空学院 施法中 韩道康

摘 要

Bézier 教授在《数值控制》一书中,对 Bézier 曲线曲面作了优美而详尽的说明。Bézier 基函数的表达式表于本文 (2) 和 (3)。

Bézier 曲线有着很多重要的和有趣的几何性质。这些性质是 Bézier 基函数性质的直接推论。到目前为止还没有见到关于导出这些基函数的文献。

本文证明了从 Bézier 曲线的三条简单的、合理的几何要求出发,用两种不同的计算方法可以完全确定并导出这些基函数。

1972年，剑桥大学的博士生Forrest 在《Computer Aided Design》发表了他一生中最著名的论文。Forest证明了Bezier曲线的基函数可以简化成伯恩斯坦基函数！

一个连续函数 $y=f(x)$ ，任给一个 $\xi > 0$ ，总能找到一个多项式和这个函数足够逼近。伯恩斯坦有一套逼近的理论，逼近的形式是：

$$B_{i,n}(t) = C_n^i t^i (1-t)^{n-i} = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} \quad (i = 0, 1, \dots, n)$$

C_n^i 从n个不同元素中，任取i($i \leq n$)个元素并成一组，叫做从n个不同元素中取出i个元素的一个组合

Forest证明了Bezier曲线的基函数可以简化成伯恩斯坦基函数

2、Bezier曲线的定义

针对Bezier曲线，给定空间 $n+1$ 个点的位置矢量 P_i ($i=0, 1, 2, \dots, n$)，则Bezier曲线段的参数方程表示如下：

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

其中 $p_i (x_i, y_i, z_i)$, $i=0,1,2\cdots n$ 是控制多边形的 $n+1$ 个顶点，即构成该曲线的特征多边形； $B_{i,n}(t)$ 是Bernstein基函数，有如下形式：

$$B_{i,n}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = C_n^i t^i (1-t)^{n-i} \quad (i = 0,1,\dots, n)$$

二项式定理，又称牛顿二项式定理。该定理给出两个数之和的整数次幂的恒等式

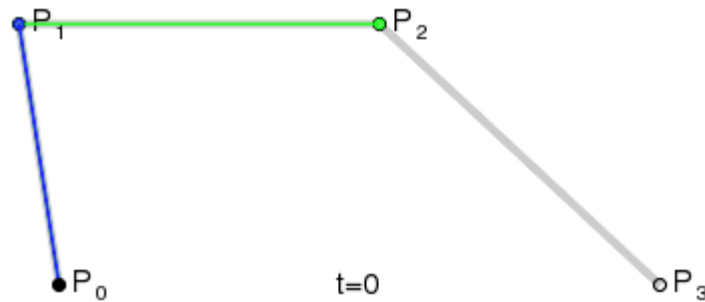
$$B_{i,n}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = C_n^i t^i (1-t)^{n-i} \quad (i = 0, 1, \dots, n)$$

$\sum_{i=0}^n B_{i,n}(t)$ 恰好是二项式 $[t + (1-t)]^n$ 的展开式!

注意：当 $i=0$, $t=0$ 时, $t^i=1$, $i!=1$ 。即： $0^0=1$, $0!=1$

P_i 代表空间的很多点， t 在0到1之间，把 t 代进去可以算出一个数--即平面或空间一个点

随着 t 值的变化，点也在变化。当 t 从0变到1时，就得到空间的一个图形，这个图形就是bezier曲线



(1) 一次Bezier曲线

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

当 $n=1$ 时，有两个控制点 p_0 和 p_1 ，Bezier多项式是一次多项式：

$$p(t) = \sum_{i=0}^1 P_i B_{i,1}(t) = P_0 B_{0,1}(t) + P_1 B_{1,1}(t)$$

$$B_{i,n}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i}$$

$$B_{i,n}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i}$$

$$B_{0,1}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{1!}{0!(1-0)!} t^0 (1-t)^{1-0}$$

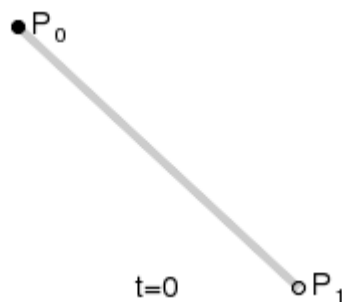
$$= (1-t)$$

$$B_{1,1}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{1!}{1!(1-1)!} t^1 (1-1)^{1-1}$$

$$= t$$

$$\begin{aligned}
 p(t) &= \sum_{i=0}^1 P_i B_{i,1}(t) = P_0 B_{0,1}(t) + P_1 B_{1,1}(t) \\
 &= (1-t)P_0 + tP_1
 \end{aligned}$$

这恰好是连接起点 p_0 和终点 p_1 的直线段！



(2) 二次Bezier曲线

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

当 $n=2$ 时，有3个控制点 p_0 、 p_1 和 p_2 ，Bezier多项式是二次多项式：

$$p(t) = \sum_{i=0}^2 P_i B_{i,2}(t) = P_0 B_{0,2}(t) + P_1 B_{1,2}(t) + P_2 B_{2,2}(t)$$

$$\begin{aligned} B_{0,2}(t) &= \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{2!}{0!(2-0)!} t^0 (1-t)^{2-0} \\ &= (1-t)^2 \end{aligned}$$

$$B_{0,2}(t) = (1 - t)^2$$

$$\begin{aligned} B_{1,2}(t) &= \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{2!}{1!(2-1)!} t^1 (1-t)^{2-1} \\ &= 2t(1-t) \end{aligned}$$

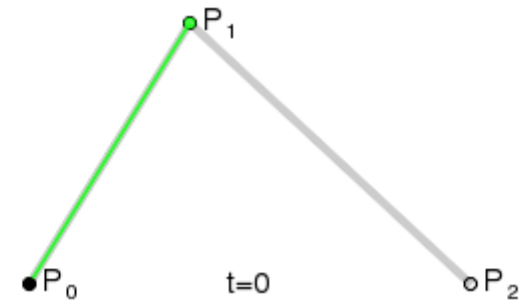
$$\begin{aligned} B_{2,2}(t) &= \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{2!}{2!(2-2)!} t^2 (1-t)^{2-2} \\ &= t^2 \end{aligned}$$

$$\begin{aligned} p(t) &= \sum_{i=0}^2 P_i B_{i,2}(t) = P_0 B_{0,2}(t) + P_1 B_{1,2}(t) + P_2 B_{2,2}(t) \\ &= (1-t)^2 P_0 + 2t(1-t) P_1 + t^2 P_2 \end{aligned}$$

$$\begin{aligned}
 p(t) &= (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2 \\
 &= (P_2 - 2P_1 + P_0)t^2 + 2(P_1 - P_0)t + P_0
 \end{aligned}$$

二次Bezier曲线为抛物线，其矩阵形式为：

$$p(t) = \begin{bmatrix} t^2 & t & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & -2 & 1 \\ -2 & 2 & 0 \\ 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} P_0 \\ P_1 \\ P_2 \end{bmatrix}$$



(3) 三次Bezier曲线

三次Bezier曲线由4个控制点生成，这时 $n=3$ ，有4个控制点 p_0 、 p_1 、 p_2 和 p_3 ，Bezier多项式是三次多项式：

$$p(t) = \sum_{i=0}^3 P_i B_{i,3}(t) = P_0 B_{0,3}(t) + P_1 B_{1,3}(t) + P_2 B_{2,3}(t) + P_3 B_{3,3}(t)$$

$$B_{0,3}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{3!}{0!(3-0)!} t^0 (1-t)^{3-0} = (1-t)^3$$

$$B_{1,3}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{3!}{1!(3-1)!} t^1 (1-t)^{3-1} = 3t(1-t)^2$$

$$B_{2,3}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{3!}{2!(3-2)!} t^2 (1-t)^{3-2} = 3t^2(1-t)$$

$$B_{3,3}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = \frac{3!}{3!(3-3)!} t^3 (1-t)^{3-3} = t^3$$

$$p(t) = \sum_{i=0}^3 P_i B_{i,3}(t) = (1-t^3)P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3$$

其中

$$B_{0,3}(t) = (1 - t)^3$$

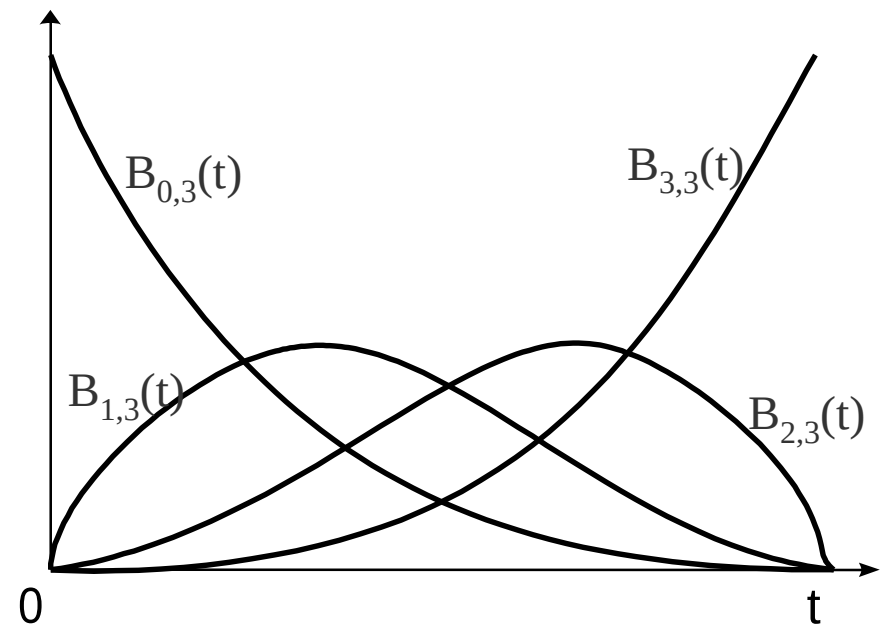
$$B_{1,3}(t) = 3t(1 - t)^2$$

$$B_{2,3}(t) = 3t^2(1 - t)$$

$$B_{3,3}(t) = t^3$$

为三次Bezier曲线的基函数。

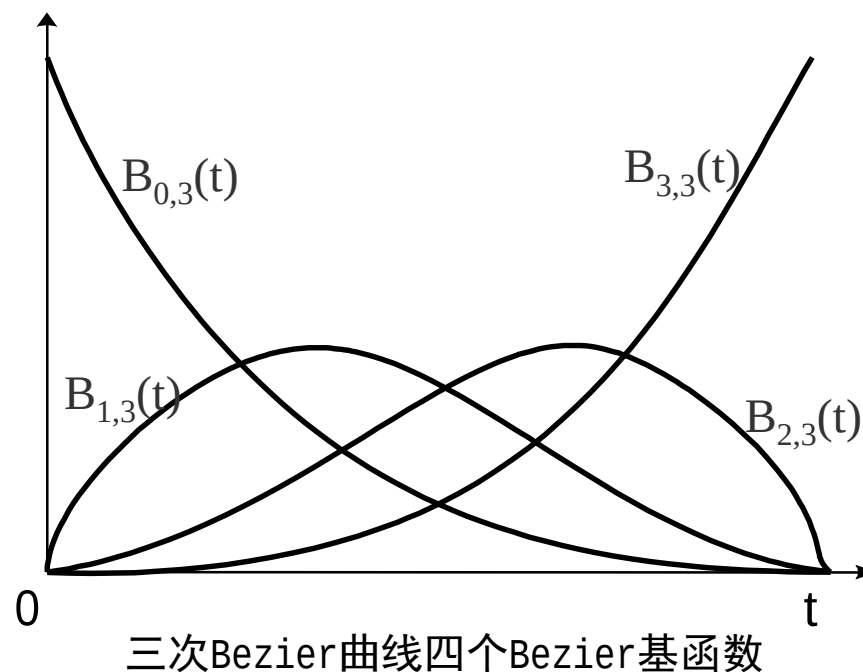
这四条曲线均是三次曲线，任何三次Bezier曲线都是这四条曲线的线形组合



三次Bezier曲线四个Bezier基函数

注意图中每个基函数在参数 t 的整个 $(0, 1)$ 的开区间范围内不为0

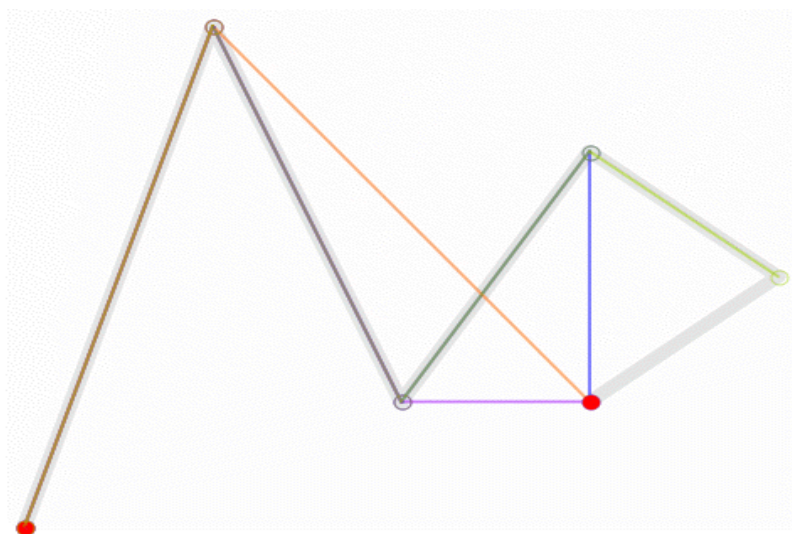
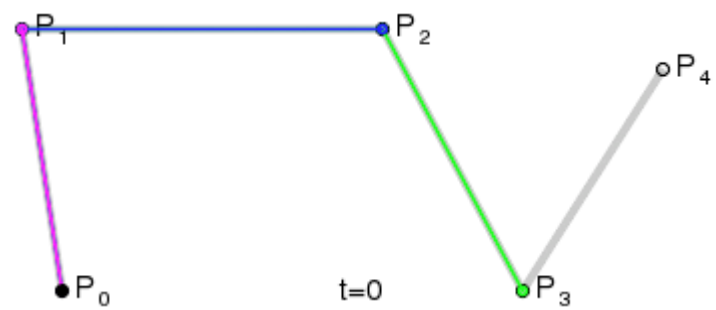
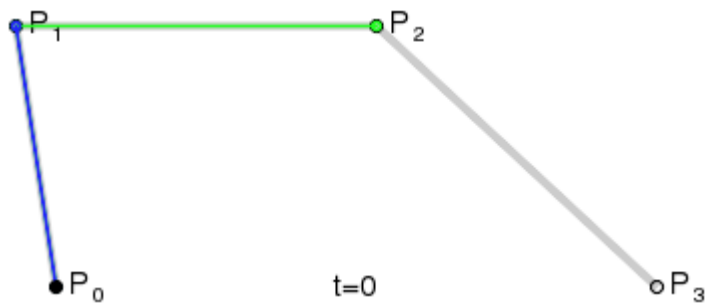
Bezier曲线不可能对曲线形状进行局部控制，如果改变任一控制点位置，整个曲线会受到影响



把Bezier三次曲线多项式写成矩阵形式：

$$p(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \quad t \in [0,1]$$
$$= T \cdot M_{be} \cdot G_{be}$$

其中， M_{be} 是三次Bezier曲线系数矩阵，为常数； G_{be} 是4个控制点位置矢量。



二、Bernstein基函数的性质

1、正性（非负性）

$$B_{i,n}(t) = \begin{cases} = 0 & t = 0, 1 \\ > 0 & t \in (0, 1), i = 1, 2, \dots, n-1; \end{cases}$$

2、权性

基函数有n+1项，n+1个基函数的和加起来正好等于1

$$\sum_{i=0}^n B_{i,n}(t) \equiv 1 \quad t \in (0, 1)$$

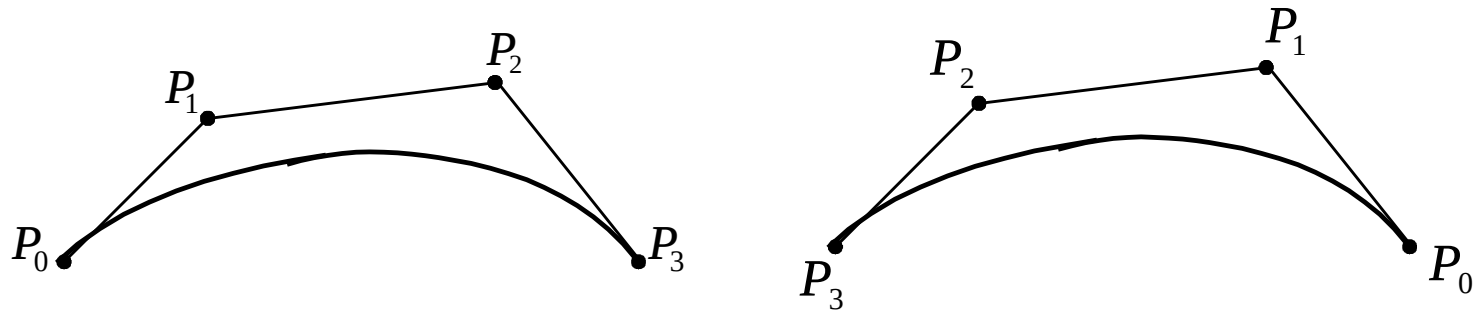
3、端点性质

$$B_{i,n}(0) = \begin{cases} 1 & (i = 0) \\ 0 & \text{otherwise} \end{cases}$$

$$B_{i,n}(1) = \begin{cases} 1 & (i = n) \\ 0 & \text{otherwise} \end{cases}$$

4、对称性

可以证明，假如保持n次Bezier曲线控制多边形的顶点位置不变，而把次序颠倒过来，则此时曲线仍不变，只不过曲线的走向相反而已



$$B_{i,n}(t) = B_{n-i,n}(1-t)$$

$$B_{n-i,n}(1-t) = C_n^{n-i} [1 - (1-t)]^{n-(n-i)} \cdot (1-t)^{n-i}$$

$$= C_n^i t^i (1-t)^{n-i} = B_{i,n}(t)$$

5、递推性

$$B_{i,n}(t) = (1-t)B_{i,n-1}(t) + tB_{i-1,n-1}(t) \quad (i = 0, 1, \dots, n)$$

即n次的Bernstein基函数可由两个n-1次的Bernstein基函数线性组合而成。因为：

$$C_n^i = (C_{n-1}^i + C_{n-1}^{i-1})$$

$$\begin{aligned} B_{i,n}(t) &= C_n^i t^i (1-t)^{n-i} = (C_{n-1}^i + C_{n-1}^{i-1}) t^i (1-t)^{n-i} \\ &= (1-t) C_{n-1}^i t^i (1-t)^{(n-1)-i} + t C_{n-1}^{i-1} t^{i-1} (1-t)^{(n-1)-(i-1)} \\ &= (1-t) B_{i,n-1}(t) + t B_{i-1,n-1}(t) \end{aligned}$$

6、导函数

$$B'_{i,n}(t) = n [B_{i-1,n-1}(t) - B_{i,n-1}(t)] \quad i = 0, 1, \dots, n;$$

7、最大值

$B_{i,n}(t)$ 在 $t = \frac{i}{n}$ 处达到最大值

8、积分

$$\int_0^1 B_{i,n}(t) dt = \frac{1}{n+1}$$

9、降阶公式

$$B_{i,n}(u) = (1-u)B_{i,n-1}(u) + uB_{i-1,n-1}(u)$$

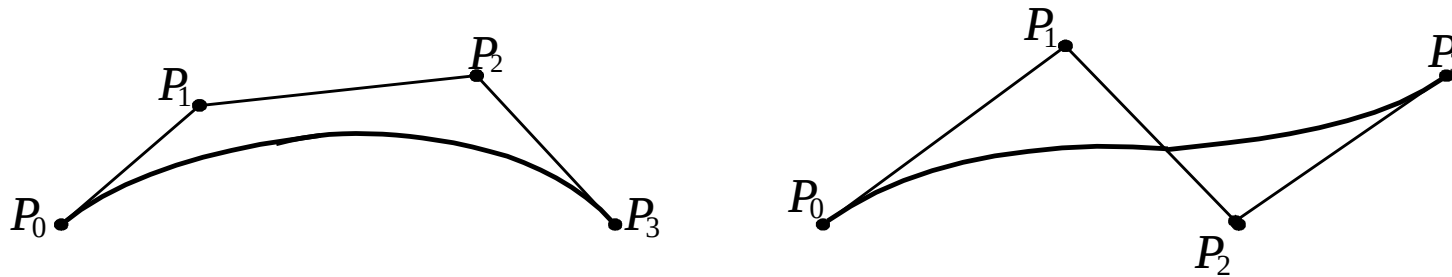
一个n次Bernstein基函数能表示成两个n-1次基函数的线性组合

10、升阶公式

$$B_{i,n}(t) = \frac{i+1}{n+1} B_{i+1,n+1}(t) + \frac{n+1-i}{n+1} B_{i,n}(t)$$

三、Bezier曲线的性质

1、端点性质



顶点 p_0 和 p_n 分别位于实际曲线段的起点和终点上

Bezier曲线段的参数方程表示如下：

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) = P_0 B_{0,n}(t) + P_1 B_{1,n}(t) + \mathbf{K} + P_n B_{n,n}(t)$$

$$p(0) = \sum_{i=0}^n P_i B_{i,n}(0) = P_0 B_{0,n}(0) + P_1 B_{1,n}(0) + \mathbf{K} + P_n B_{n,n}(0) = P_0$$

$$p(1) = \sum_{i=0}^n P_i B_{i,n}(1) = P_0 B_{0,n}(1) + P_1 B_{1,n}(1) + \mathbf{K} + P_n B_{n,n}(1) = P_n$$

2、一阶导数

Bernstein基函数的一阶导数为：

$$B'_{i,n}(t) = n[B_{i-1,n-1}(t) - B_{i,n-1}(t)] \quad i = 0, 1, \dots, n;$$

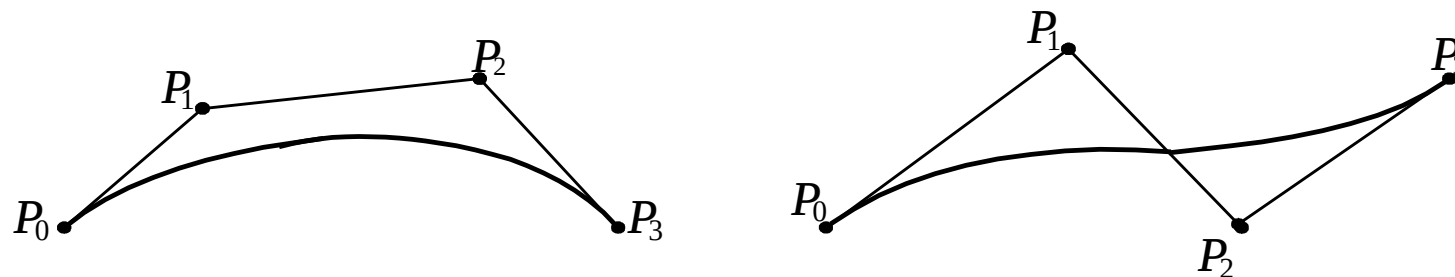
$$p'(t) = n \sum_{i=1}^n (p_i - p_{i-1}) B_{i-1,n-1}(t)$$

当 $t=0$ ：

当 $t=1$ ：

$$p'(0) = n(p_1 - p_0) \quad p'(1) = n(p_n - p_{n-1})$$

这说明Bezier曲线的起点和终点处的切线方向和特征多边形的第一条边及最后一条边的走向一致



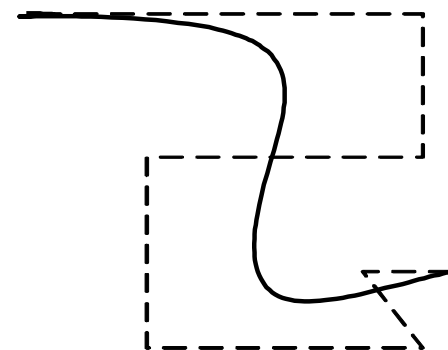
3、几何不变性

指某些几何特性不随坐标变换而变化的特性。Bezier曲线的形状仅与控制多边形各顶点的相对位置有关，而与坐标系的的选择无关

4、变差缩减性

若Bezier曲线的特征多边形是一个平面图形，则平面内任意直线与 $p(t)$ 的交点个数不多于该直线与其特征多边形的交点个数，这一性质叫变差缩减性质

此性质反映了Bezier曲线比其特征多边形的波动还小，也就是说Bezier曲线比特征多边形的折线更光顺



三、Bezier曲线的生成

生成一条Bezier曲线实际上就是要求出曲线上的点。下面介绍两种曲线生成的方法：

1、根据定义直接生成Bezier曲线

绘制Bezier曲线主要有以下步骤：

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

$$B_{i,n}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = C_n^i t^i (1-t)^{n-i} \quad (i = 0, 1, \dots, n)$$

① 首先给出 C_n^i 的递归计算式：

$$C_n^i = \frac{n!}{i!(n-i)!} = \frac{n-i+1}{i} C_n^{i-1} \quad n \geq i$$

② 将 $p(t) = \sum_{i=0}^n P_i B_{i,n}(t)$ 表示成分量坐标形式：

$$x(t) = \sum_{i=0}^n x_i B_{i,n}(t)$$

$$y(t) = \sum_{i=0}^n y_i B_{i,n}(t) \quad t \in [0,1]$$

$$z(t) = \sum_{i=0}^n z_i B_{i,n}(t)$$

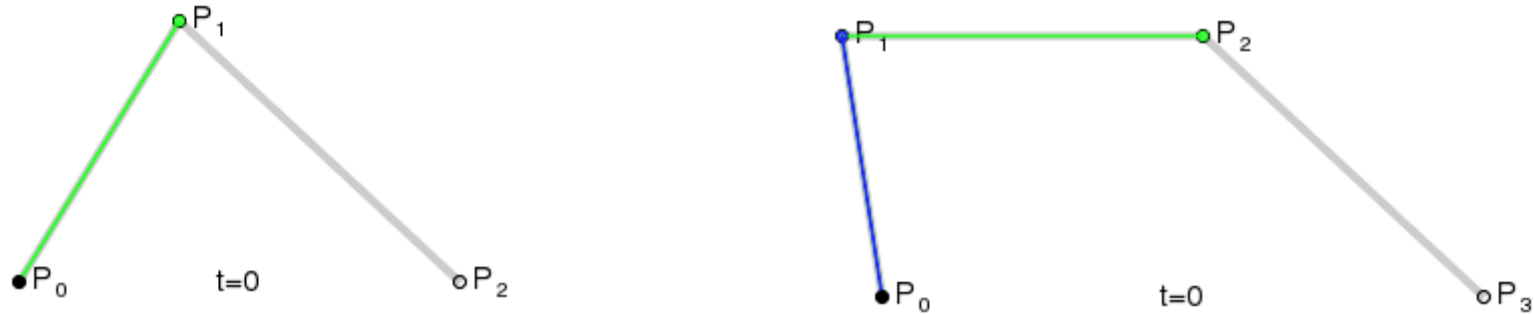
根据以上的公式可以直接写出绘制Bezier曲线的程序

$$C_n^i = \frac{n!}{i!(n-i)!} = \frac{n-i+1}{i} C_n^{i-1} \quad n \geq i$$

2、Bezier曲线的递推(de Casteljau)算法

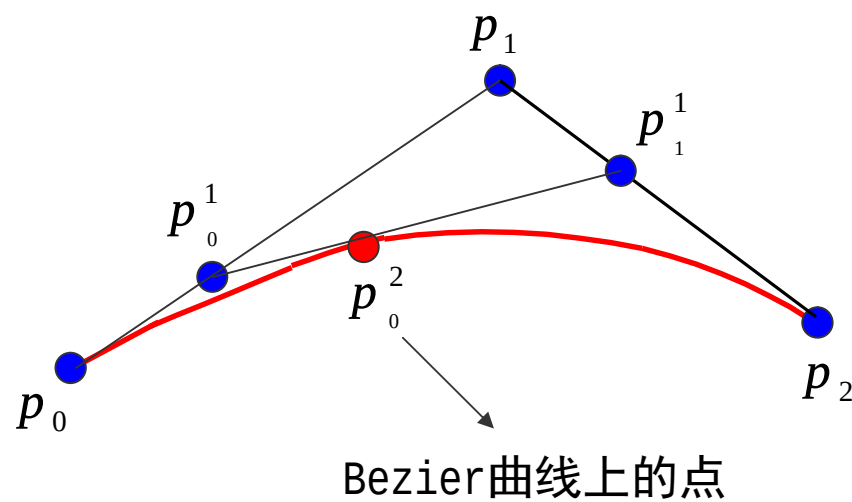
根据Bezier曲线的定义确定的参数方程绘制Bezier曲线,
因其计算量过大,不太适合在工程上使用

de Casteljau提出的递推算法则要简单得多



Bezier曲线上的任一个点(t), 都是其它相邻线段的同等比例(t)点处的连线, 再取同等比例(t)的点再连线, 一直取到最后那条线段的同等比例(t)处, 该点就是Bezier曲线上的点(t)

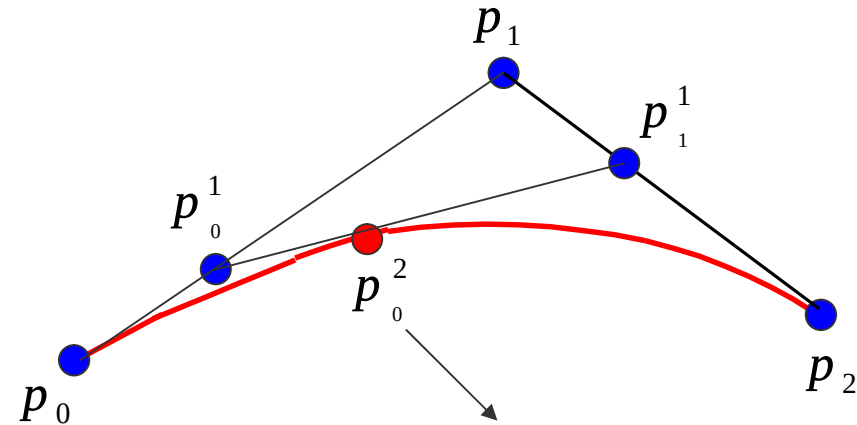
以二次Bezier曲线为例，求曲线上 $t=1/3$ 的点：



$$P_0^1 = (1 - t) P_0 + t P_1$$

$$P_1^1 = (1 - t) P_1 + t P_2$$

$$P_0^2 = (1 - t) P_0^1 + t P_1^1$$



Bezier曲线上的点

t从0变到1，第一、二式就分别表示控制二边形的第一、二条边，它们是两条一次Bezier曲线。将一、二式代入第三式得：

$$P_0^2 = (1 - t)^2 P_0 + 2t(1 - t) P_1 + t^2 P_2$$

$$P_0^2 = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2$$

当 t 从0变到1时，它表示了由三顶点 P_0 、 P_1 、 P_2 三点定义的一条二次Bezier曲线

二次Bezier曲线 P_0^2 可以定义为分别由前两个顶点(P_0, P_1)和后两个顶点(P_1, P_2)决定的一次Bezier曲线的线性组合

由 $(n+1)$ 个控制点 $P_i (i=0, 1, \dots, n)$ 定义的 n 次Bezier曲线 P_0^n 可被定义为分别由前、后 n 个控制点定义的两条 $(n-1)$ 次Bezier曲线 P_0^{n-1} 与 P_1^{n-1} 的线性组合：

$$P_0^n = (1 - t) P_0^{n-1} + t P_1^{n-1} \quad t \in [0, 1]$$

由此得到Bezier曲线的递推计算公式：

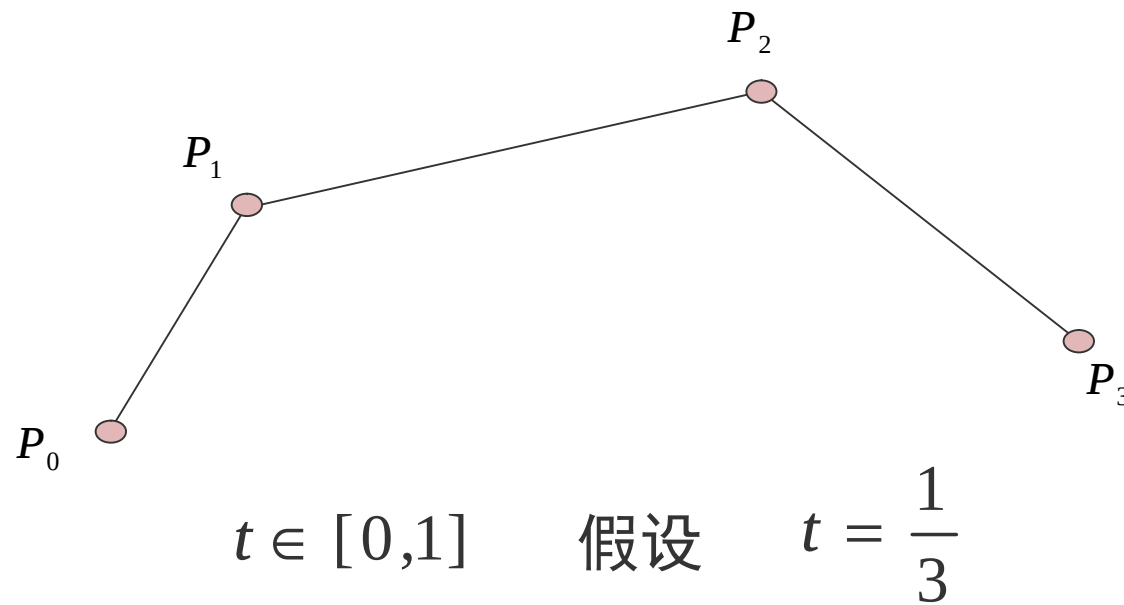
$$P_i^k = \begin{cases} P_i & k = 0 \\ (1-t)P_i^{k-1} + tP_{i+1}^{k-1} & k = 1, 2, \dots, n, i = 0, 1, \dots, n-k \end{cases}$$

这便是著名的de Casteljau算法。用这一递推公式，在给定参数下，求Bezier曲线上一点P(t)非常有效

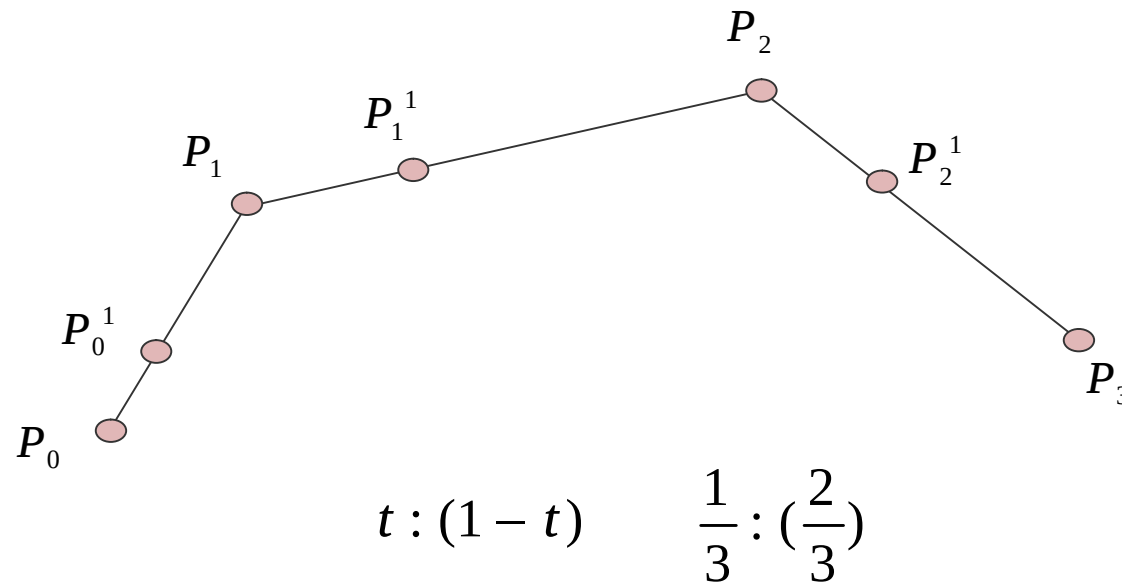
de Casteljau算法稳定可靠，直观简便，可以编出十分简捷的程序，是计算Bezier曲线的基本算法和标准算法。

3、de Casteljau算法几何作图

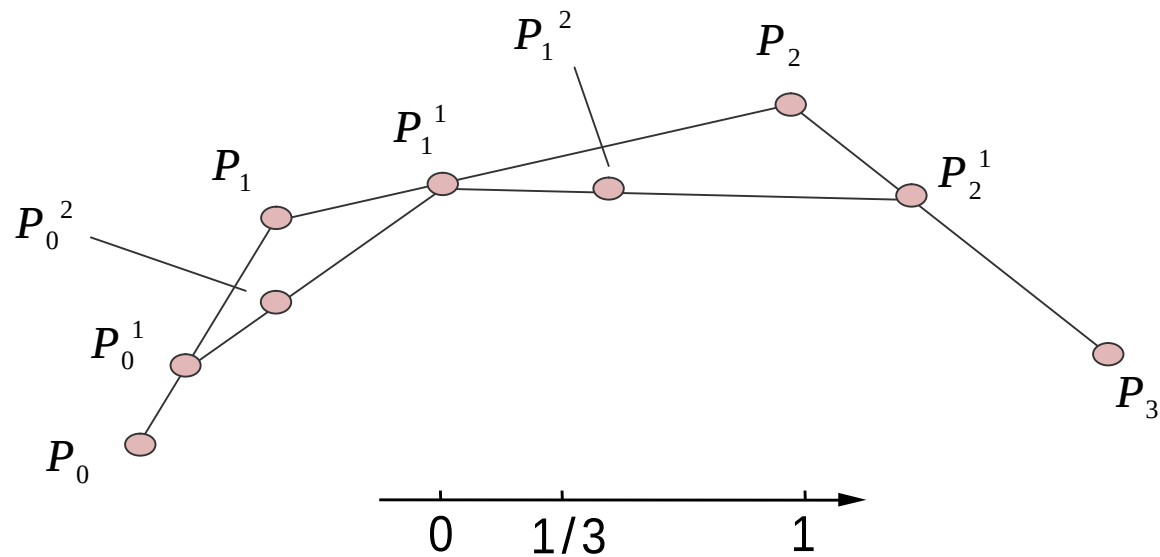
这一算法可用简单的几何作图来实现



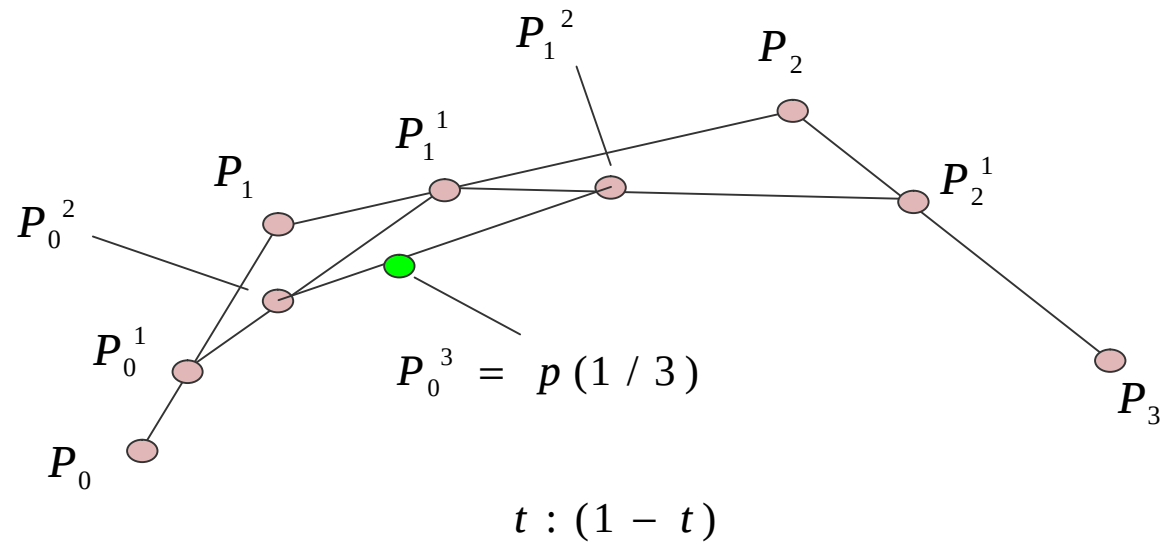
(1) 依次对原始控制多边形每一边执行相同的定比分割,
所得分点就是由第一级递推生成的中间顶点 $P_i^1 (i = 0, 1, \dots, n-1)$



(2) 对这些中间顶点构成的控制多边形再执行同样的定比分割，得第二级中间顶点： $P_i^2 (i = 0, 1, \mathbf{L}, n-2)$

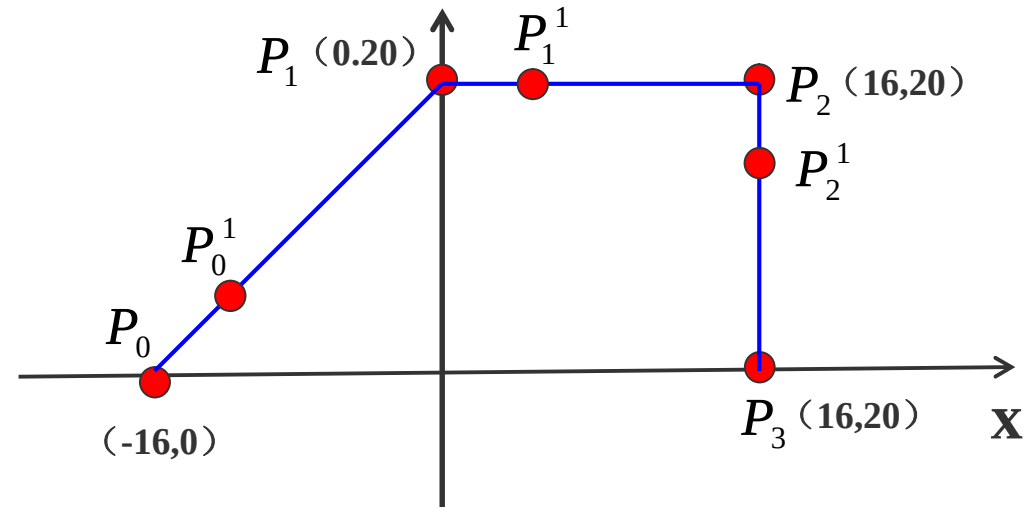


(3) 重复进行下去，直到n级递推得到一个中间顶点 $P(t)$ ，即为所求曲线上的点



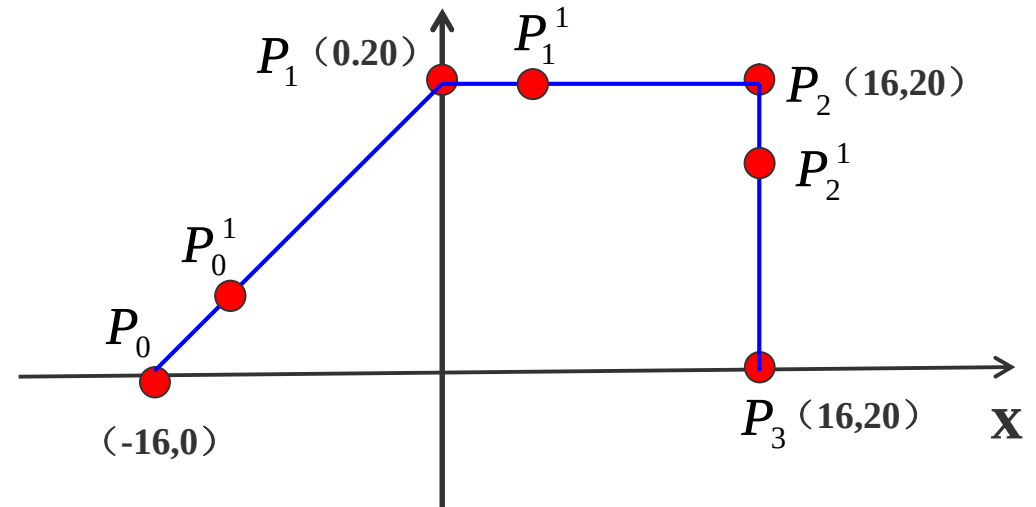
Bezier曲线计算举例

计算参数为 $t=1/4$ 的P值



$$P_i^k = \begin{cases} P_i & k = 0 \\ (1-t)P_i^{k-1} + tP_{i+1}^{k-1} & k = 1, 2, \dots, n, i = 0, 1, \dots, n-k \end{cases}$$

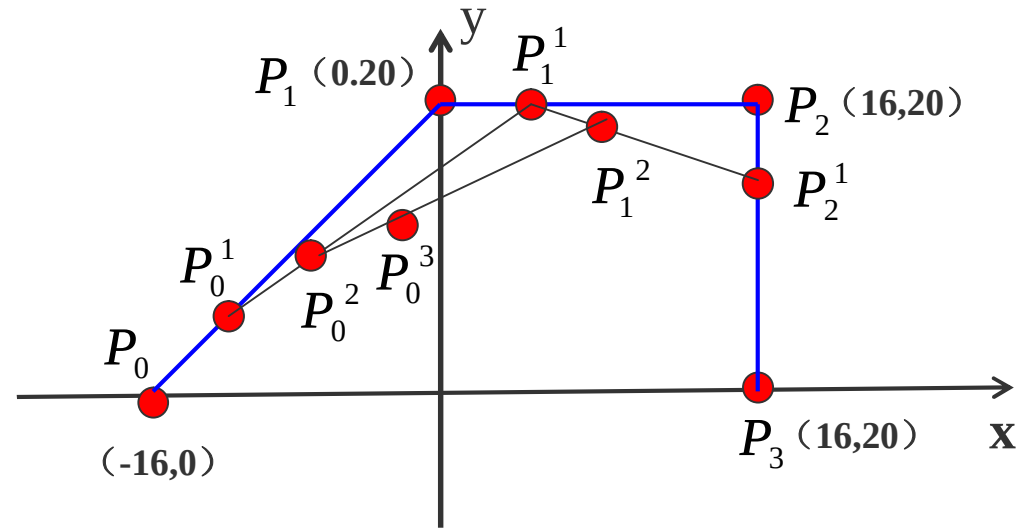
$$P_0^1 = (1-t)P_0 + tP_1 = \left(1 - \frac{1}{4}\right)P_0 + \frac{1}{4}P_1 = \frac{3}{4}[-16, 0] + \frac{1}{4}[0, 20] = [-12, 5]$$



$$P_0^1 = (1 - t)P_0 + tP_1 = \left(1 - \frac{1}{4}\right)p_0 + \frac{1}{4}p_1 = \frac{3}{4}[-16, 0] + \frac{1}{4}[0, 20] = [-12, 5]$$

$$P_1^1 = (1 - t)P_1 + tP_2 = \left(1 - \frac{1}{4}\right)p_1 + \frac{1}{4}p_2 = \frac{3}{4}[0, 20] + \frac{1}{4}[16, 20] = [4, 20]$$

$$P_2^1 = (1 - t)P_2 + tP_3 = \left(1 - \frac{1}{4}\right)p_2 + \frac{1}{4}p_3 = \frac{3}{4}[16, 20] + \frac{1}{4}[16, 0] = [16, 15]$$



$$P_0^2 = (1-t)P_0^1 + tP_1^1 = (1-\frac{1}{4})P_0^1 + \frac{1}{4}P_1^1 = \frac{3}{4}[-12, 5] + \frac{1}{4}[4, 20] = [-8, 8.75]$$

$$P_1^2 = (1-t)P_1^1 + tP_2^1 = (1-\frac{1}{4})P_1^1 + \frac{1}{4}P_2^1 = \frac{3}{4}[-12, 5] + \frac{1}{4}[4, 20] = [-8, 8.75]$$

$$P_0^3 = (1-t)P_0^2 + tP_1^2 = (1-\frac{1}{4})P_0^2 + \frac{1}{4}P_1^2 = \frac{3}{4}[-8, 8.75] + \frac{1}{4}[7, 18.75]$$

$$P_0^1 = (1 - t)P_0 + tP_1 = (1 - \frac{1}{4})p_0 + \frac{1}{4}p_1 = \frac{3}{4}[-16, 0] + \frac{1}{4}[0, 20] = [-12, 5]$$

$$P_1^1 = (1 - t)P_1 + tP_2 = (1 - \frac{1}{4})p_1 + \frac{1}{4}p_2 = \frac{3}{4}[0, 20] + \frac{1}{4}[16, 20] = [4, 20]$$

$$P_2^1 = (1 - t)P_2 + tP_3 = (1 - \frac{1}{4})p_2 + \frac{1}{4}p_3 = \frac{3}{4}[16, 20] + \frac{1}{4}[16, 0] = [16, 15]$$

$$P_0^2 = (1 - t)P_0^1 + tP_1^1 = (1 - \frac{1}{4})P_0^1 + \frac{1}{4}P_1^1 = \frac{3}{4}[-12, 5] + \frac{1}{4}[4, 20] = [-8, 8.75]$$

$$P_0^3 = (1 - t)P_0^2 + tP_1^2 = (1 - \frac{1}{4})P_0^2 + \frac{1}{4}P_1^2 = \frac{3}{4}[-8, 8.75] + \frac{1}{4}[7, 18.75]$$

$$= [-4.25, 11.25] = P(\frac{1}{4})$$

四、Bezier曲线的拼接

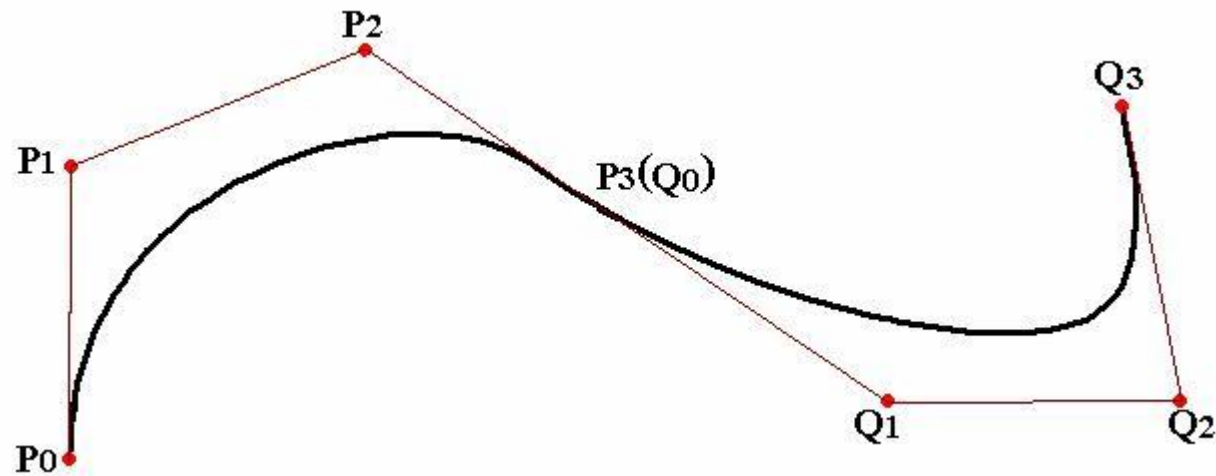
几何设计中，一条Bezier曲线往往难以描述复杂的曲线形状。这是由于增加特征多边形的顶点数，会引起Bezier曲线次数的提高，而高次多项式又会带来计算上的困难

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

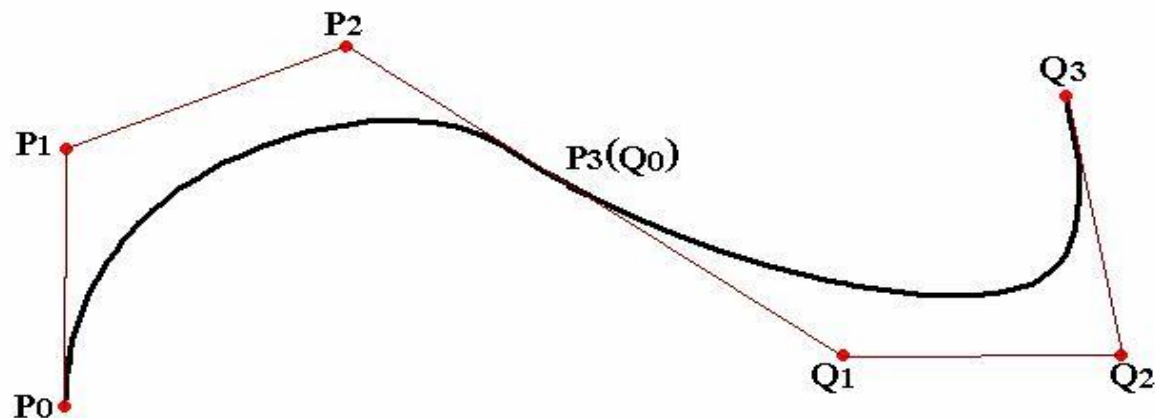
$$B_{i,n}(t) = C_n^i t^i (1-t)^{n-i} \quad (i = 0, 1, \dots, n)$$

采用分段设计，然后将各段曲线相互连接起来，
并在接合处保持一定的连续条件

给定两条Bezier曲线 $P(t)$ 和 $Q(t)$ ，相应控制点为 $P_i(i=0,1,\dots,n)$ 和 $Q_i(i=0,1,\dots,m)$



(1) 要使它们达到 G^0 连续, 则: $P_n = Q_0$



(2) 要使它们达到 G^1 连续, 只要保证 $P_{n-1}, P_n=Q, Q_1$ 三点共线就行了

Bezier曲线的起点和终点处的切线方向和特征多边形的第一条边及最后一条边的走向一致

五、Bezier曲线的升阶与降阶

假设有一个二次多项式：

$$a_0 + a_1t + a_2t^2 = 0$$

能否找一个三次多项式近似逼近这个二次多项式，或精确地转成一个三次多项式？

$$a_0 + a_1t + a_2t^2 = 0$$

$$a_0 + a_1t + a_2t^2 + 0 * t^3 = 0$$

所谓升阶就是保证曲线的形状和定向保持不变，但是要增加顶点个数

伯恩斯坦基函数不是简单的 t^2 、 t^3 ，2次Bezier基函数是 $B_{i,2}$ ，3次是 $B_{i,3}$ ，如何从 $B_{i,2}$ 转化成 $B_{i,3}$ 的形式？

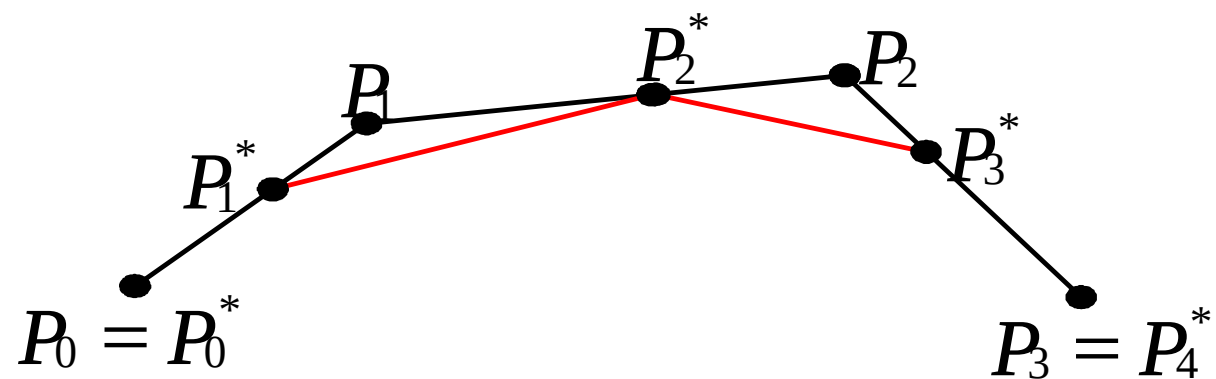
1、Bezier曲线的升阶

所谓升阶是指保持Bezier曲线的形状与方向不变，增加定义它的控制顶点数，即提高该Bezier曲线的次数

设给定原始控制顶点 $p_0, p_1 \dots p_n$ ，定义一条 n 次Bezier曲线

$$P(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

增加一个顶点后，仍定义同一条曲线的新控制顶点为 $P_0^*, P_1^*, \dots, P_{n+1}^*$ ，则有：

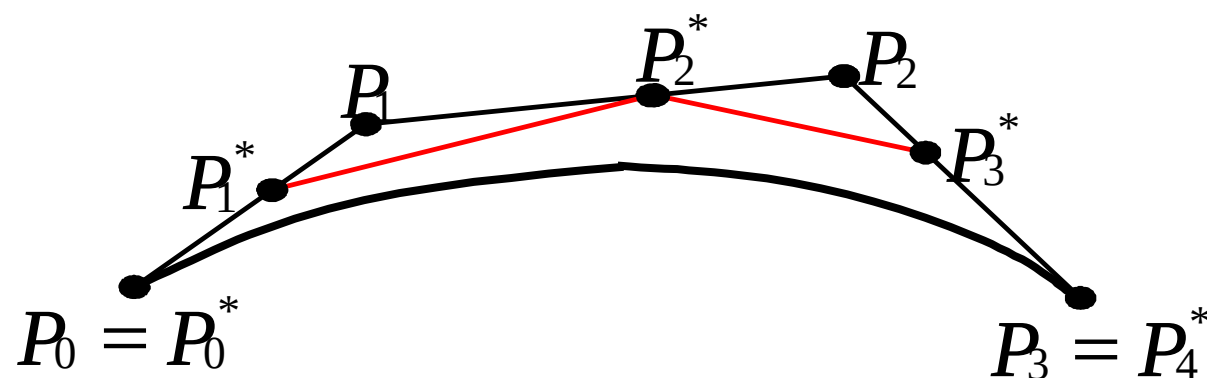


$$\sum_{i=0}^n C_n^i P_i t^i (1-t)^{n-i} = \sum_{i=0}^{n+1} C_{n+1}^i P_i^* t^i (1-t)^{n+1-i}$$

$$P_i^* C_{n+1}^i = P_i C_n^i + P_{i-1} C_n^{i-1}$$

化简即得： $P_i^* = \frac{i}{n+1} P_{i-1} + \left(1 - \frac{i}{n+1}\right) P_i \quad (i = 0, 1, \mathbf{L}, n+1)$

三次Bezier曲线的升阶实例如下图所示：



新的多边形更加靠近曲线。在80年代初，有人证明如果一直升阶升下去的话，控制多边形收敛于这条曲线

2、Bezier曲线的降阶

降阶是升阶的逆过程

$$a_0 + a_1t + a_2t^2 + a_3t^3 = b_0 + b_1t + b_2t^2$$

如果 a_3 不等于0，想找一个二次多项式精确等于它是不可能的。如果一定要降下来，那只能近似逼近

如果把 a_3t^3 扔掉，也不是不可以，但误差太大。必须想办法找到一个二次多项式，尽量逼近这个三次多项式

假定 P_i 是由 P_i^* 升阶得到，则由升阶公式有：

$$P_i = \frac{n-i}{n} P_i^* + \frac{i}{n} P_{i-1}^*$$

从这个方程可以导出两个递推公式：

$$P_i^* = \frac{nP_i - iP_{i-1}^*}{n-i} \quad i = 0, 1, \mathbf{L}, n-1$$

$$P_{i-1}^* = \frac{nP_i - (n-i)P_i^*}{i} \quad i = n, n-1, \mathbf{L}, 1$$

可以综合利用上面两个式子进行降解，但仍然不精确

Bezier曲线曲面升降阶的重要性

第一 它是CAD系统之间**数据传递与交换**的需要

第二 它是系统中分段(片)**线性逼近**的需要。通过逐次降阶，把曲面化为直线平面，便于求交和曲面绘制

第三 它是外形**信息压缩**的需要。降阶处理以后可以减少存储的信息量

六、Bezier曲面

基于Bezier曲线的讨论，可以给出Bezier曲面的定义和性质，Bezier曲线的一些算法也可以很容易扩展到Bezier曲面的情况

1、Bezier曲面的定义

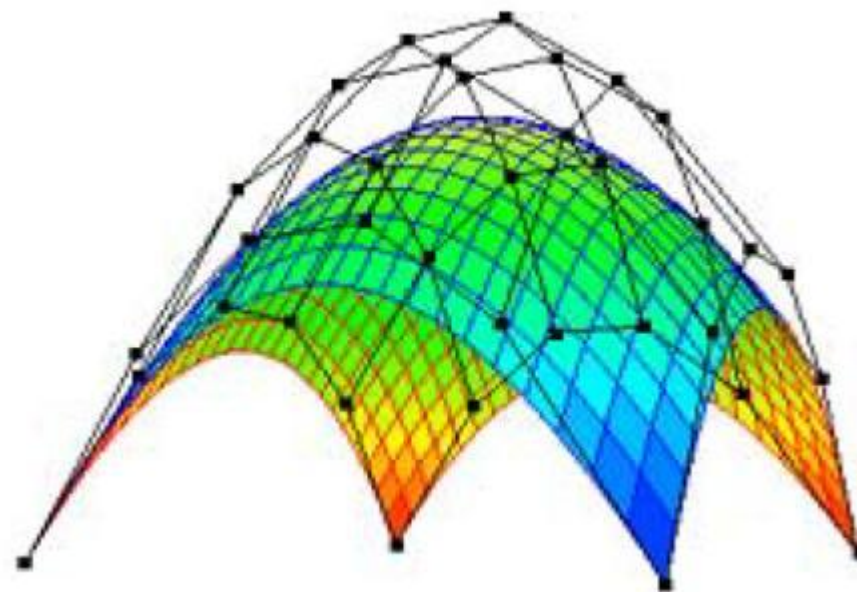
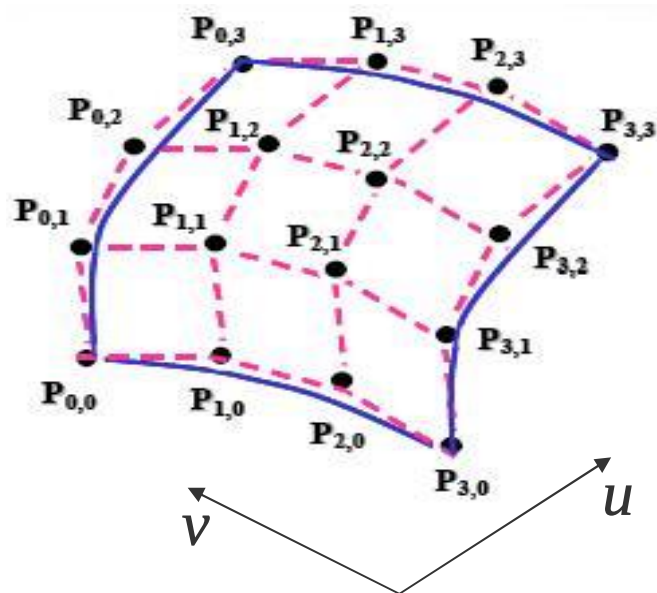
设 $p_{i,j}(0,1,\dots, n; j = 0,1,\dots, m)$ 为 $(n+1) \times (m+1)$ 个空间点, 则 $m \times n$ Bezier曲面定义为:

$$P(u,v) = \sum_{i=0}^m \sum_{j=0}^n P_{ij} B_{i,m}(u) B_{j,n}(v) \quad u,v \in [0,1]$$

$$B_{i,m}(u) = C_m^i u^i (1-u)^{m-i}$$

$$B_{j,n}(v) = C_n^j v^j (1-v)^{n-j}$$

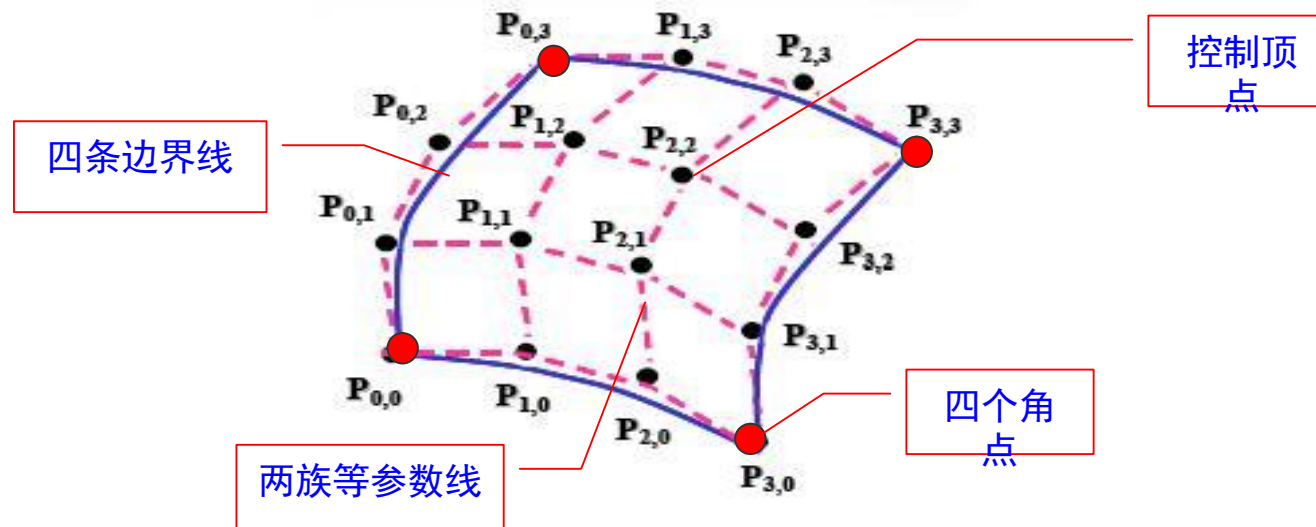
依次用线段连接点列中相邻两点所形成的空间网格，称之为特征网格



Bezier曲面的矩阵表示式是：

$$P(u, v) = \begin{bmatrix} B_{0,n}(u) & B_{1,n}(u) & \mathbf{L} & B_{m,n}(u) \end{bmatrix} \begin{bmatrix} P_{00} & P_{01} & \mathbf{L} & P_{0m} \\ P_{10} & P_{11} & \mathbf{L} & P_{1m} \\ \mathbf{L} & \mathbf{L} & \mathbf{L} & \mathbf{L} \\ P_{n0} & P_{n1} & \mathbf{L} & P_{nm} \end{bmatrix} \begin{bmatrix} B_{0,m}(v) \\ B_{1,m}(v) \\ \mathbf{L} \\ B_{n,m}(v) \end{bmatrix}$$

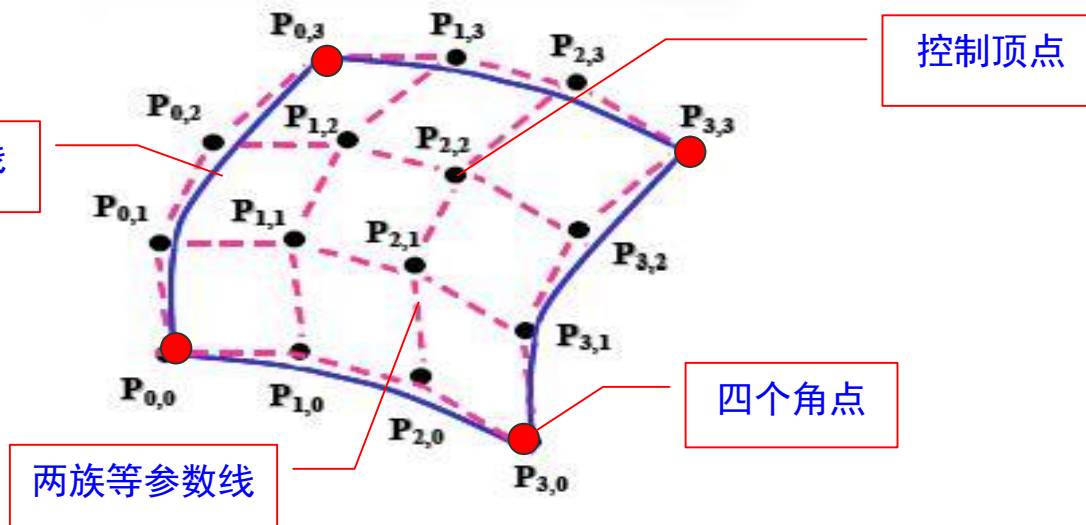
上式的曲面称为 $m \times n$ 次的，在一般应用中， n 、 m 不大于4



角点位置：四个角点分别是其控制网络的四个点

$$\begin{aligned}
 P(0,0) &= P_{0,0} & P(0,1) &= P_{0,n} \\
 P(1,0) &= P_{m,0} & P(1,1) &= P_{m,n}
 \end{aligned}$$

边界线：Bezier曲面的四条边界线是Bezier曲线



$$P(u,0) = \sum_{j=0}^m P_{j,0} BEZ_{j,m}(u)$$

$$P(v,0) = \sum_{k=0}^n P_{k,0} BEZ_{k,n}(v)$$

$$P(u,1) = \sum_{j=0}^m P_{j,3} BEZ_{j,m}(u)$$

$$P(v,1) = \sum_{k=0}^n P_{k,3} BEZ_{k,n}(v)$$

$$0 \leq u \leq 1$$

$$0 \leq v \leq 1$$

2、Bezier曲面性质

Bezier曲线的很多性质可推广到Bezier曲面

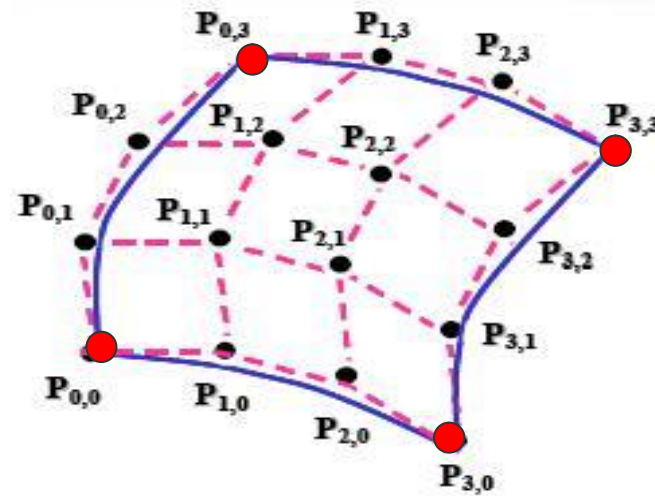
(1) Bezier曲面特征网格的四个角点正好是Bezier曲面的四个角点，即：

$$P(0,0) = P_{00}$$

$$P(1,0) = P_{m0}$$

$$P(0,1) = P_{0n}$$

$$P(1,1) = P_{mn}$$

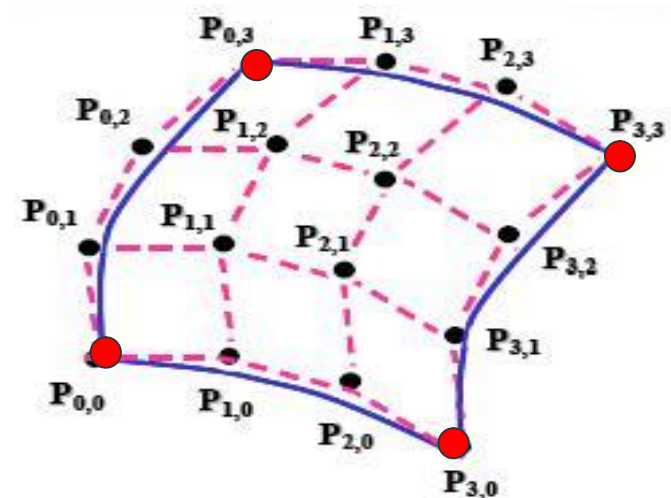


(2) Bezier曲面特征网格最外一圈顶点定义Bezier曲面的四条边界；

(3) 几何不变性

(4) 对称性

(5) 凸包性



3、Bezier曲面片的拼接

设两张 $m \times n$ 次Bezier曲面片：

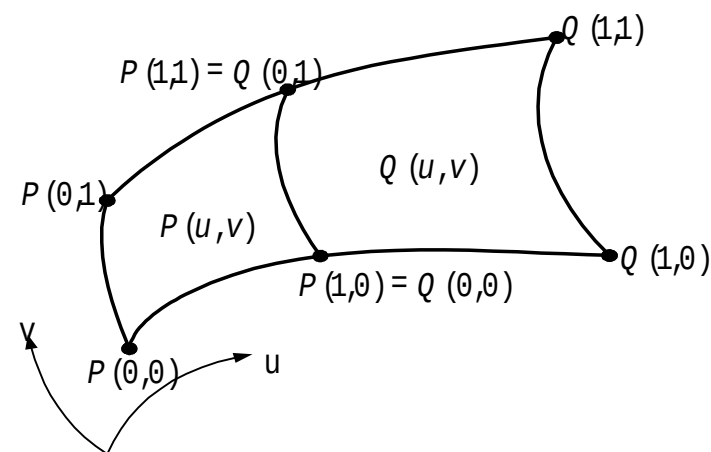
$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{ij} B_{i,m}(u) B_{j,n}(v) \quad u, v \in [0, 1]$$
$$Q(u, v) = \sum_{i=0}^m \sum_{j=0}^n Q_{ij} B_{i,m}(u) B_{j,n}(v)$$

分别由控制顶点 P_{ij} 和 Q_{ij} 定义。

如果要求两曲面片达到 G^0 连续，则它们有公共的边界，即：

$$P(1, v) = Q(0, v)$$

于是有： $P_{ni} = Q_{0i}, \quad (i = 0, 1, \dots, m)$



Bezier曲面片的拼接

如果又要求沿该公共边界达到 G^1 连续，则两曲面片在该边界上有公共的切平面，因此曲面的法向应当是跨界连续的，即：

$$Q_u(0, v) \times Q_v(0, v) = a(v) P_u(1, v) \times P_v(1, v)$$

4、递推(de Casteljau)算法（曲面的求值）

Bezier曲线的递推(de Casteljau)算法，可以推广到Bezier曲面的情形

$$P_{ij}(i=0,1,\mathbf{L},m; j=0,1,\mathbf{L},n)$$
$$P(u,v) = \sum_{i=0}^{m-k} \sum_{j=0}^{n-l} P_{i,j}^{k,l} B_{i,m}(u) B_{j,n}(v) = \mathbf{L} = P_{00}^{m,n} \quad u,v \in [0,1]$$

一条曲线可以表示成两条低一次曲线的组合，一张曲面可以表示成低一次的四张曲面的线性组合

其中：

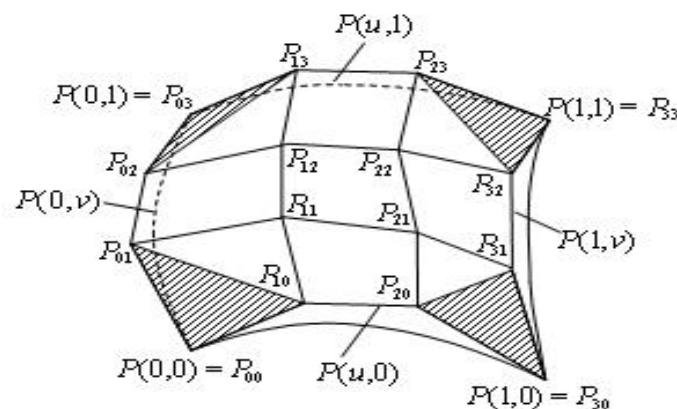
$$P_{i,j}^{k,l} = \begin{cases} P_{ij} & (k = l = 0) \\ (1 - u)P_{ij}^{k-1,0} + uP_{i+1,j}^{k-1,0} & (k = 1, 2, \mathbf{L}, m; l = 0) \\ (1 - v)P_{0,j}^{m,l-1} + vP_{0,j+1}^{m,l-1} & (k = m, l = 1, 2, \mathbf{L}, n) \end{cases} \quad (1)$$

或

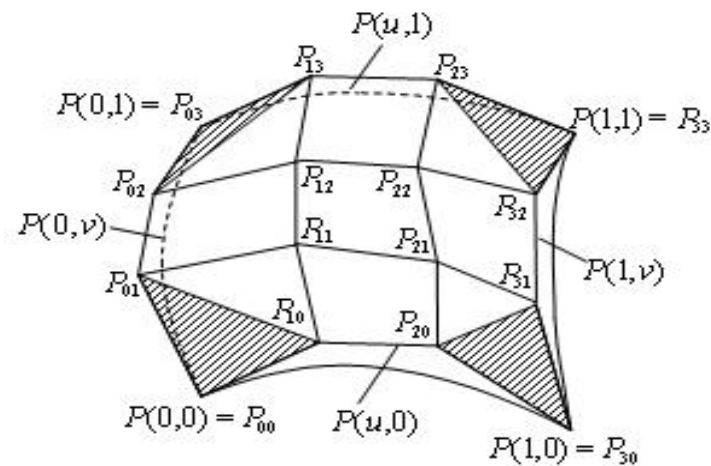
$$P_{ij}^{k,l} = \begin{cases} P_{ij} & (k = l = 0) \\ (1 - v)P_{ij}^{0,l-1} + vP_{i,j+1}^{0,l-1} & (k = 0; l = 1, 2, \mathbf{L}, n) \\ (1 - u)P_{i0}^{k-1,n} + uP_{i+1,0}^{k-1,n} & (k = 1, 2, \mathbf{L}, m; l = n) \end{cases} \quad (2)$$

$$P_{i,j}^{k,l} = \begin{cases} P_{ij} & (k = l = 0) \\ (1-u)P_{ij}^{k-1,0} + uP_{i+1,j}^{k-1,0} & (k = 1, 2, \mathbf{L}, m; l = 0) \\ (1-v)P_{0,j}^{m,l-1} + vP_{0,j+1}^{m,l-1} & (k = m, l = 1, 2, \mathbf{L}, n) \end{cases} \quad (1)$$

当按(1)式方案执行时，先以u参数值对控制网格u向的n+1个多边形执行曲线de Casteljau算法，m级递推后，得到沿v向由n+1个顶点 P_{0j}^{m0} 构成的多边形

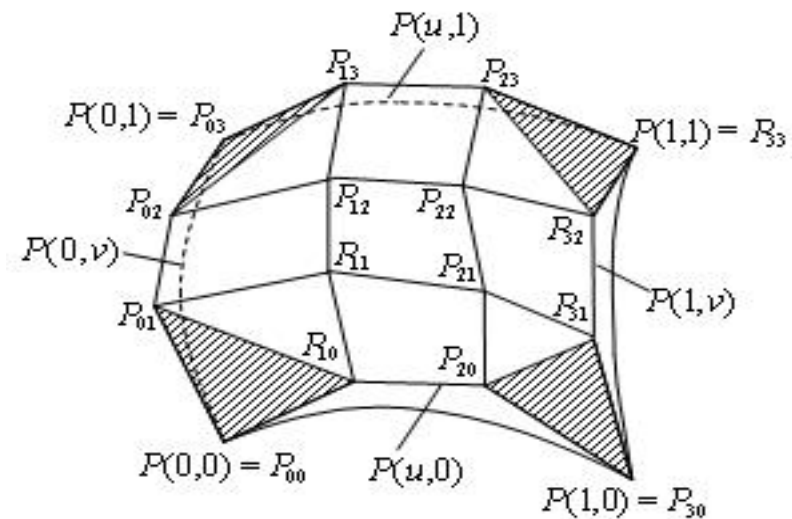


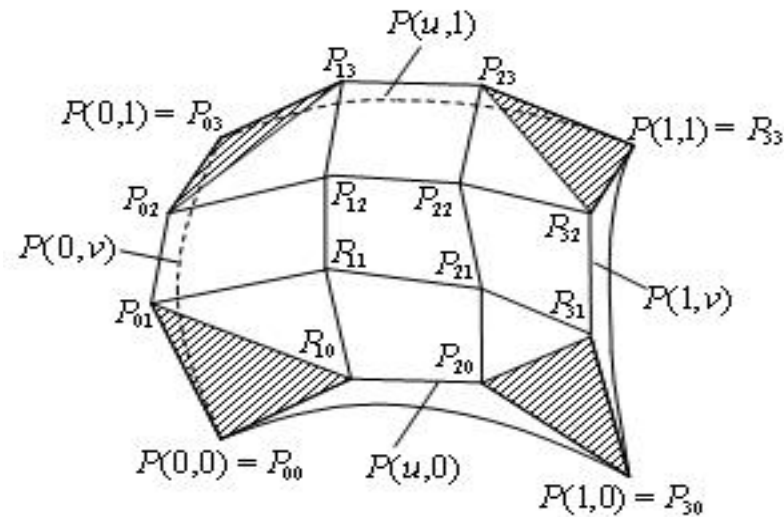
再以 v 参数值对它执行曲线的de Casteljau算法， n 级递推以后，得到一个 P_{00}^{mn} ，即所求表面上的点。



一条曲线可以表示成2条低一次的曲线的线性组合，曲面可以表示成低一次的4张曲面的线性组合。

这张曲面有16个顶点，现在要算曲面上一点 $P(1/2, 1/2)$ 的值，如何计算？





首先拿 $u=1/2$ 来计算，得到4个点。再拿这四个点做为新的Bezier曲线的控制顶点，按 $v=1/2$ 来计算，算出的这个点就是所要求的值

之所以有这样的算法，是因为：

$$\begin{aligned} P(u, v) &= \sum_{i=0}^m \sum_{j=0}^n P_{ij} B_{i,m}(u) B_{j,n}(v) \\ &= \sum_{i=0}^m \left(\sum_{j=0}^n P_{ij} B_{j,n}(v) \right) B_{i,m}(u) \end{aligned}$$

B样条曲线与曲面

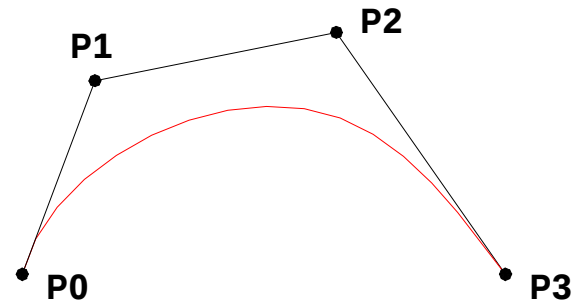
一、B样条产生的背景

Bezier曲线曲面有很多优点，可以用鼠标拖动控制顶点以改变曲线的形状，非常直观，给设计人员很大的自由度

Bezier曲线曲面是几何造型的主要方法和工具

但是Bezier曲线有几点不足：

(1) 一旦确定了特征多边形的顶点数($n+1$ 个)，也就决定了曲线的阶次(n 次)

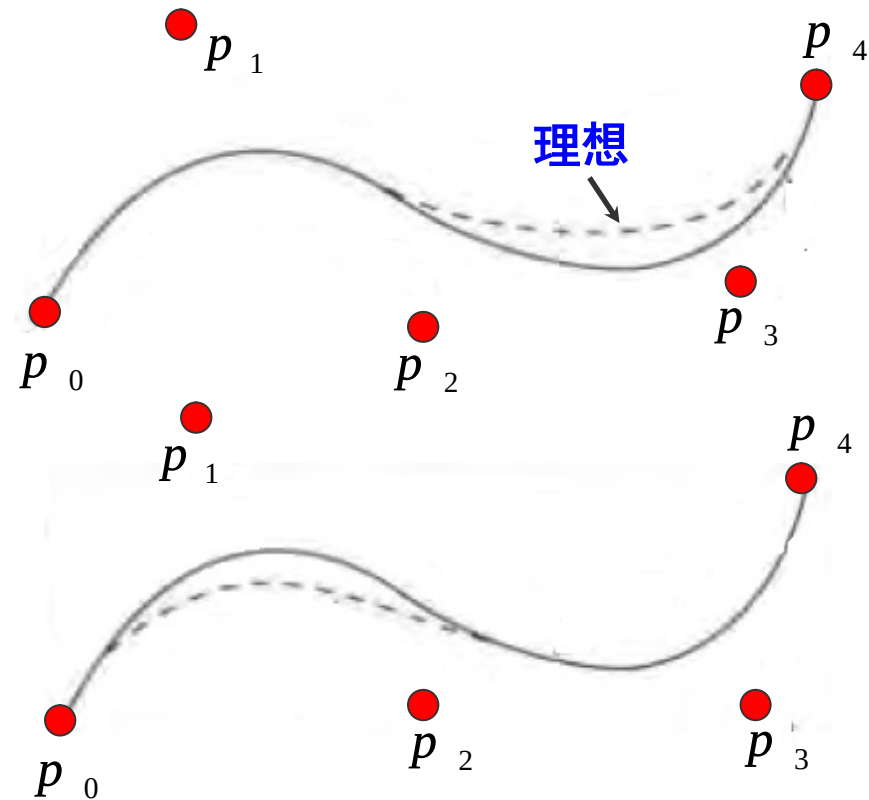


(2) Bezier曲线或曲面的拼接比较复杂

(3) Bezier曲线或曲面不能作局部修改

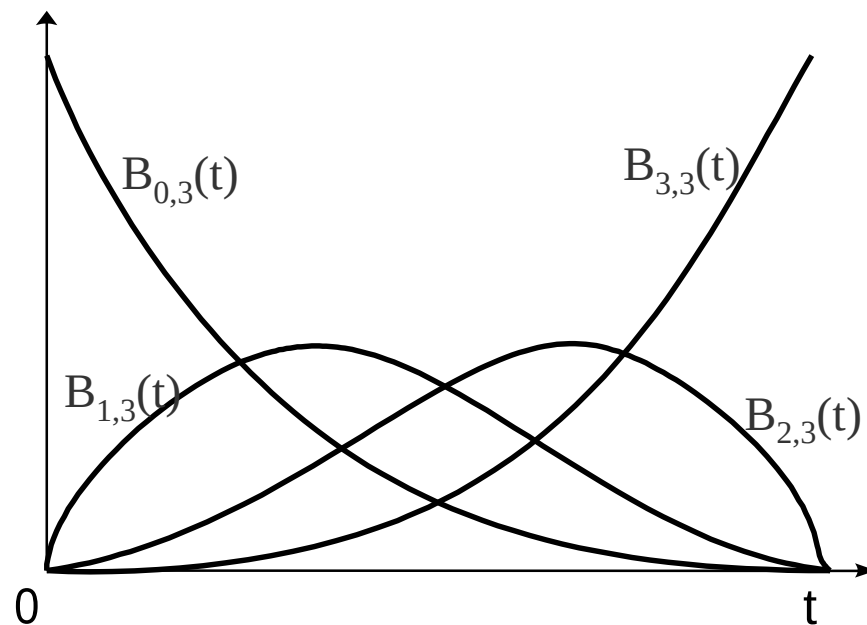
为了纠正偏离，需要向上移动 p_2 和 p_3 ，以使Bezier曲线更接近所期望的曲线

但是，这也影响了曲线的前半部分，迫使它偏离所期望的曲线



函数值不为0的区间通常叫做它的[支撑区间]

因为每个Bernstein多项式在整个区间 $[0, 1]$ 上都有支撑，且曲线是这些函数的混合，所以每个控制顶点对0到1之间的 t 值处的曲线都有影响



三次Bezier曲线四个Bezier基函数

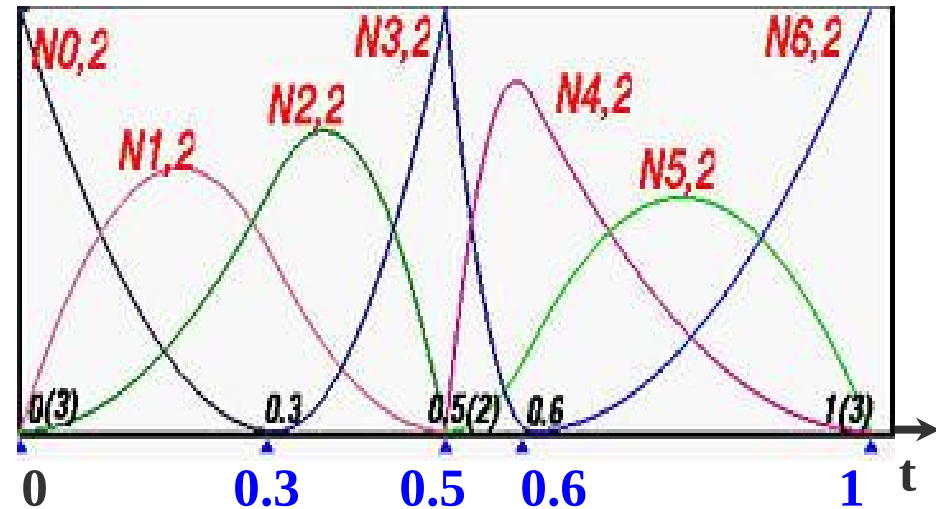
图中显示了7个混合函数

$N_{0,2}$ $N_{1,2}$ $N_{2,2}$ $N_{3,2}$

$N_{4,2}$ $N_{5,2}$ $N_{6,2}$

每个函数只在区间 $[0,1]$ 上
的一部分有支撑，如：

$N_{1,2}$ 的支撑是 $[0, 0.5]$ $N_{4,2}$ 的支撑是 $[0.5, 1]$



1972年，Gordon、Riesenfeld等人提出了B样条方法，在保留Bezier方法全部优点的同时，克服了Bezier方法的弱点

样条（spline）—— 分段连续多项式！

整条曲线用一个完整的表达形式，但内在的量是一段一段的，比如一堆的3次曲线拼过去，两条之间满足2次连续

这样既克服了波动现象，曲线又是低次的。既有统一的表达时，又有统一的算法

如何进行分段呢？

现在有 $n+1$ 个点，每两点之间构造一条多项式， $n+1$ 个点有 n 个小区间

每个小区间构造一条三次多项式，变成了 n 段的三次多项式拼接在一起，段与段之间要两次连续，这就是三次样条

如有5个点，构造一个多项式，应该是个四次多项式。现在采用样条方式构造四段曲线，每一段都是三次的，且段与段之间要 C^2 连续。

二、B样条的递推定义和性质

B样条曲线的数学表达式为：

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u) \quad u \in [u_{k-1}, u_{n+1}]$$

$P_i (i = 0, 1, \dots, n)$ 是控制多边形的顶点

Bezier曲线

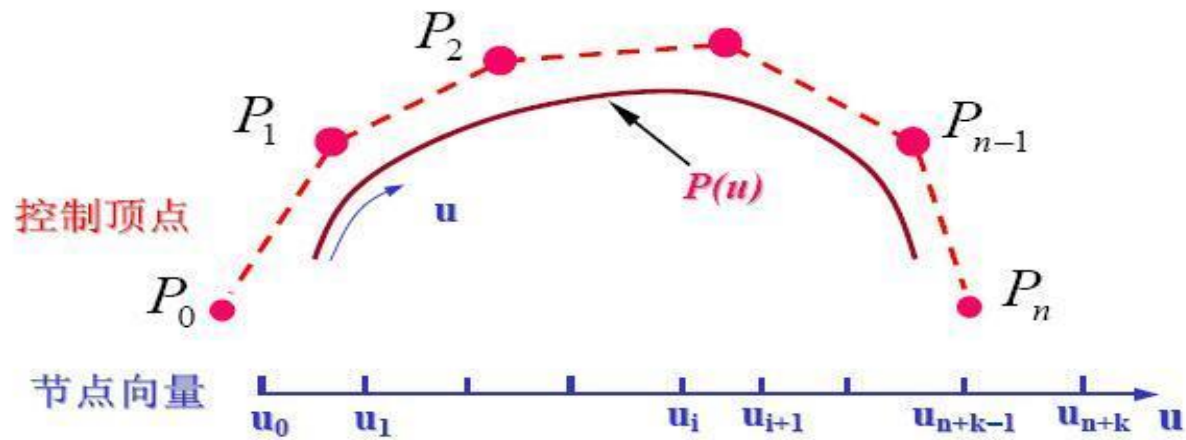
$$p(u) = \sum_{i=0}^n P_i B_{i,n}(u) \quad u \in [0,1]$$

$B_{i,k}(u)$ 称为k阶 (k-1次) B样条基函数, k是刻画次数的。
其中k可以是2到控制点个数n+1之间的任意整数

对Bezier曲线来说, 阶数和次数是一样的; 但对B样条, 阶数是次数加1

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u) \quad u \in [u_{k-1}, u_{n+1}]$$

B样条基函数是一个称为节点矢量的非递减的参数u的序列所决定的k阶分段多项式, 这个序列称为节点向量



$$u_0, u_1, \mathbf{L}, u_{k-1}, u_k, \mathbf{L}, u_n, u_{n+1}, \mathbf{L}, u_{n+k-1}, u_{n+k}$$

B样条基函数实际上就是一个多项式，一个比较复杂的、有特点的多项式而已。如何得到这个B样条基函数？

de Boor-Cox递推定义

B样条基函数可以有各种各样的定义方式，但是公认的最容易理解的是de Boor-Cox递推定义

它的原理是，只要是 k 阶（ $k-1$ 次）的B样条基函数，构造一种递推的公式，由0次构造1次，1次构造2次，2次构造3次...依次类推

$$B_{i,1}(u) = \begin{cases} 1 & u_i < u < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

并约定: $\frac{0}{0} = 0$

$$B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

该递推公式表明：若确定第*i*个*k*阶B样条 $B_{i,k}(u)$ ，需要用到 $u_i, u_{i+1}, \dots, u_{i+k}$ 共*k*+1个节点，称区间 $[u_i, u_{i+k}]$ 为 $B_{i,k}(u)$ 的支承区间

$$B_{i,1}(u) = \begin{cases} 1 & u_i < u < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

并约定: $\frac{0}{0} = 0$

$$B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

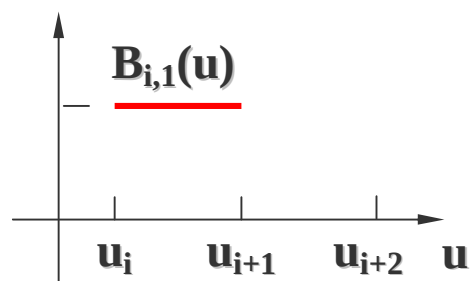
曲线方程中, $n+1$ 个控制顶点 $P_i (i=0, 1, \dots, n)$, 要用到 $n+1$ 个 k 阶B样条 $B_{i,k}(u)$ 。它们支撑区间的并集定义了这一组B样条基的节点矢量 $U = [u_0, u_1, \dots, u_{n+k}]$

$$u_0, u_1, \mathbf{L}, u_{k-1}, u_k, \mathbf{L}, u_n, u_{n+1}, \mathbf{L}, u_{n+k-1}, u_{n+k}$$

$$B_{i,1}(u) = \begin{cases} 1 & u_i < u < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

$$B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

$B_{i,1}(u)$ 是 0 次多项式



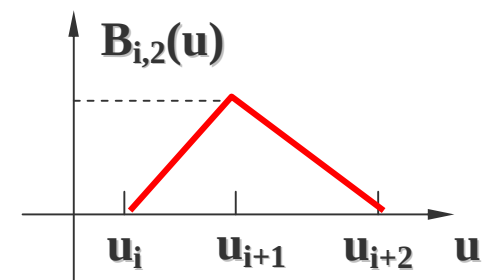
(1 阶 B 样条)

2 阶 (一次) B 样条 $B_{i,2}(u)$?

$$B_{i,2}(u) = \frac{u - u_i}{u_{i+1} - u_i} B_{i,1}(u) + \frac{u_{i+2} - u}{u_{i+2} - u_{i+1}} B_{i+1,1}(u) \quad B_{i,1}(u) = \begin{cases} 1 & u_i < u < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

$$B_{i+1,1}(u) = \begin{cases} 1 & u_{i+1} < u < u_{i+2} \\ 0 & \text{Otherwise} \end{cases}$$

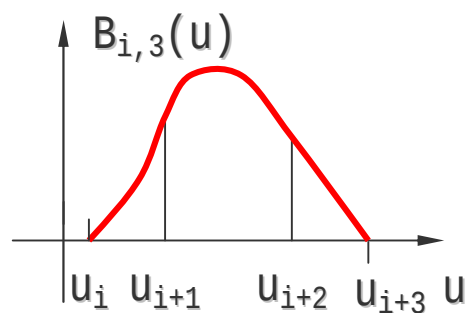
$$B_{i,2}(u) = \begin{cases} \frac{u - u_i}{u_{i+1} - u_i} & u_i \leq u \leq u_{i+1} \\ \frac{u_{i+2} - u}{u_{i+2} - u_{i+1}} & u_{i+1} \leq u \leq u_{i+2} \\ 0 & \text{其它} \end{cases}$$



(2阶B样条)

一次B样条 $B_{i,2}(u)$ 可有两个0次B样条 $B_{i,1}(u)$ 和 $B_{i+1,1}(u)$ 递推得到，是它们的凸线性组合

再有两个一次B样条 $B_{i,2}(u)$ 和 $B_{i+1,2}(u)$ 递推得到二次B样条 $B_{i,3}(u)$



$B_{i,3}(u)$ 的图像

每个 p_i 都有一个 $B_{i,k}(u)$ 与之匹配，有 $n+1$ 个 $B_{i,k}(u)$ ，曲线的次数是 $k-1$ 次，问题是这条曲线的定义区间是什么？

Bezier曲线的定义区间是 $[0, 1]$ ？

第二个问题是对 $n+1$ 个顶点， k 阶的B样条曲线需要多少个节点向量 (u_i) 与之匹配？

三、B样条基函数定义区间及节点向量

1、B样条曲线定义区间是什么？

Bezier曲线的定义区间是 $[0, 1]$

2、第二个问题是对 $n+1$ 个顶点， k 阶的B样条曲线需要多少个节点向量 (u_i) 与之匹配？

1、K阶B样条对应节点向量数

$$B_{i,1}(u) = \begin{cases} 1 & u_i < u < u_{i+1} \\ 0 & \text{Otherwise} \end{cases} \quad B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

对于 $B_{i,1}$ （1阶0次基函数）来说，涉及 u_i 到 u_{i+1} 一个区间，即一阶的多项式涉及一个区间两个节点

$B_{i,2}$ 是由 $B_{i,1}$ 和 $B_{i+1,1}$ 组成，因此 $B_{i,2}$ 涉及2个区间3个节点；

$B_{i,3}$ 涉及3个区间4个节点 $\dots$ ， $B_{i,k}$ 涉及k个区间k+1个节点

$$u_0, u_1, \mathbf{L}, u_{k-1}, u_k, \mathbf{L}, u_n, u_{n+1}, \mathbf{L}, u_{n+k-1}, u_{n+k}$$

2、B样条函数定义区间

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u)$$

$$B_{i,1}(u) = \begin{cases} 1 & u_i < x < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

$$u \in [u_{k-1}, u_{n+1}]$$

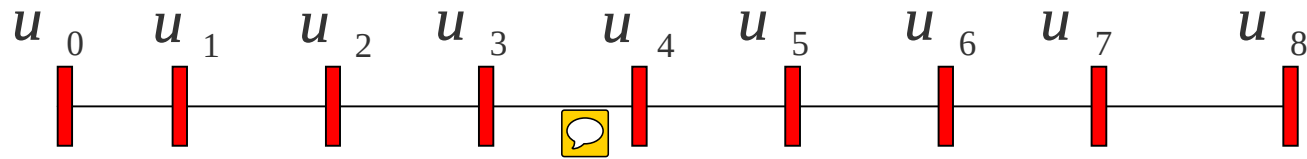
$$B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

$$u_0, u_1, \mathbf{L}, u_{k-1}, u_k, \mathbf{L}, u_n, u_{n+1}, \mathbf{L}, u_{n+k-1}, u_{n+k}$$

以k=4, n=4为例 节点矢量为:

$$U = \{ u_0, u_1, u_2, u_3, u_4, u_5, u_6, u_7, u_8 \}$$

$$U = \{u_0, u_1, u_2, u_3, u_4, u_5, u_6, u_7, u_8\}$$

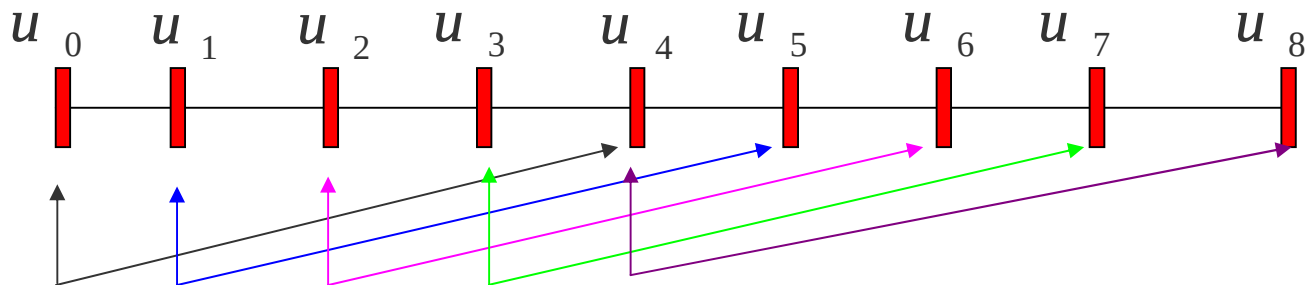


$$n=4 \quad k=4 \quad B_{i,k}(u) = \frac{u-u_i}{u_{i+k-1}-u_i} B_{i,k-1}(u) + \frac{u_{i+k}-u}{u_{i+k}-u_{i+1}} B_{i+1,k-1}(u)$$

第一项是 $p_0 B_{0,4}(u)$ ，涉及哪些节点？ u_0 到 u_4

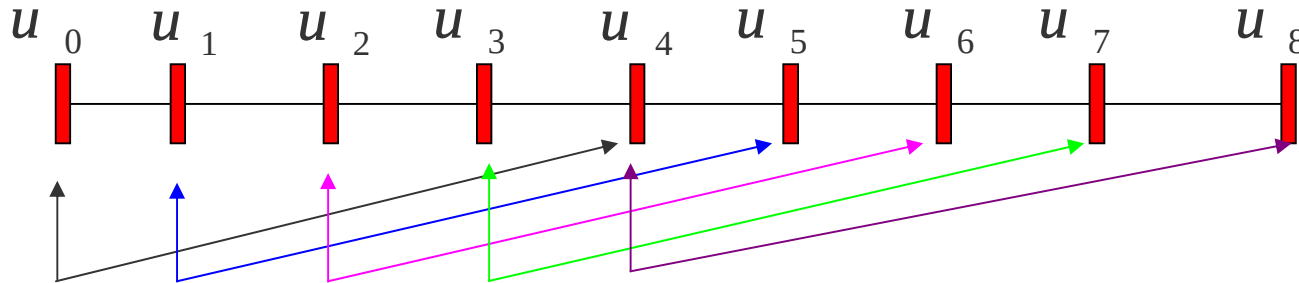
第二项是 $p_1 B_{1,4}(u)$ ，涉及哪些节点？ u_1 到 u_5

第五项是 $p_4 B_{4,4}(u)$ ，涉及哪些节点？ u_4 到 u_8



$P_0B_{0,4}(u)$, 涉及 u_0 到 u_4 个节点 $P_3B_{3,4}(u)$, 涉及 u_3 到 u_7 个节点
 $P_1B_{1,4}(u)$, 涉及 u_1 到 u_5 个节点 $P_4B_{4,4}(u)$, 涉及 u_4 到 u_8 个节点
 $P_2B_{2,4}(u)$, 涉及 u_2 到 u_6 个节点

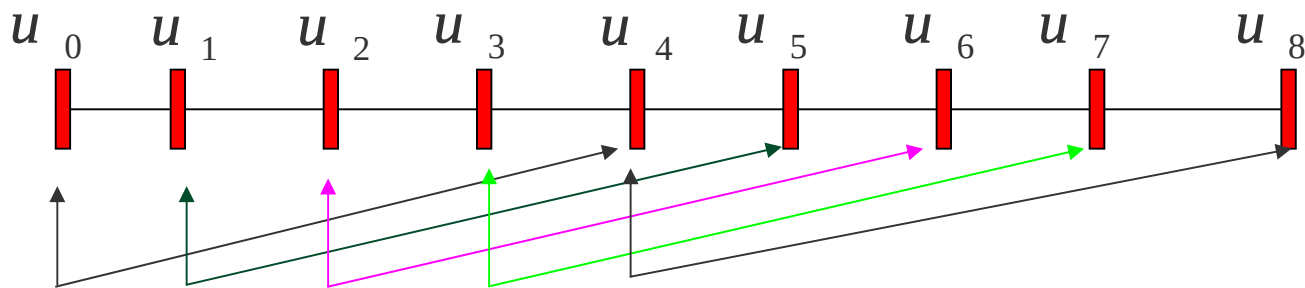
“阶数+顶点” 等于节点向量的个数。问函数的定义区间？



区间要合法，区间里必须要有足够的基函数与顶点配对

哪个区间是第一个开始有意义的区间呢？

上图区间是 u_3 到 u_5 （从 u_{k-1} 到 u_{n+1} ），B样条基函数严重依赖于节点向量的分布



上面的曲线被分成两段： u_3u_4 ， u_4u_5 。如有5个顶点 P_0 、 P_1 、 P_2 、 P_3 、 P_4 ，B样条是一段段过渡过去

哪些基函数是在 u_3u_4 区间里有定义？正好是 $P_0P_1P_2P_3$

$P_1P_2P_3P_4$ 在 u_4u_5 区间里有定义，两端之间有三个顶点是一样的，这样就保证了两段拼接的效果非常好

B样条曲线定义：

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u) \quad B_{i,1}(u) = \begin{cases} 1 & u_i < x < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$
$$u \in [u_{k-1}, u_{n+1}] \quad B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

u_i 是节点值， $U = (u_0, u_1, \dots, u_{n+k})$ 构成了k阶（k-1次）B样条函数的节点矢量

B样条曲线所对应的节点向量区间： $u \in [u_{k-1}, u_{n+1}]$

四、B样条基函数的主要性质

1、局部支承性：

$$B_{i,k}(u) \begin{cases} \geq 0 & u \in [u_i, u_{i+k}] \\ = 0 & otherwise \end{cases} \quad \text{而Bezier在整个区间非0}$$

反过来，对每一个区间 (u_i, u_{i+k}) ，至多只有k个基函数在其上非零

2、权性：

$$\sum_{i=0}^n B_{i,k}(u) \equiv 1 \quad u \in [u_{k-1}, u_{n+1}]$$

3、连续性

$B_{i,k}(u)$ 在 r 重节点处的连续阶不低于 $k-1-r$

4、分段参数多项式：

$B_{i,k}(u)$ 在每个长度非零的区间 $[u_i, u_{i+1})$ 上都是次数不高于 $k-1$ 的多项式，它在整个参数轴上是分段多项式

五、B样条函数的主要性质

1、局部性：

k阶B样条曲线上的一点至多与k个控制顶点有关，与其它控制顶点无关

移动曲线的第i个控制顶点 P_i ，至多影响到定义在区间上那部分曲线的形状，对曲线其余部分不发生影响

2、变差缩减性：

设平面内 $n+1$ 个控制顶点 构成B样条曲线 $P(t)$ 的特征多边形。在该平面内的任意一条直线与 $P(t)$ 的交点个数不多于该直线和特征多边形的交点个数

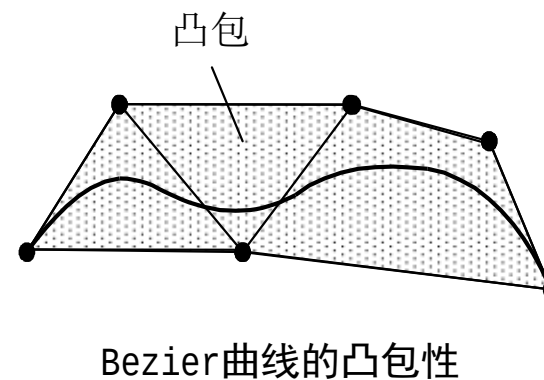
3、几何不变性：

B样条曲线的形状和位置与坐标系的选择无关

4、凸包性：

B样条曲线落在 P_i 构成的凸包之中。其凸包区域小于或等于同一组控制顶点定义的Bezier曲线凸包区域

凸包就是包含右边这6个顶点的最小凸多边形。凸多边形是把多边形的每条边延长，其它边都在它的同一侧



该性质导致顺序 $k+1$ 个顶点重合时，由这些顶点定义的 k 次B样条曲线段退化到这一个重合点；顺序 $k+1$ 个顶点共线时，由这些顶点定义的 k 次B样条曲线形状？

六、B样条曲线类型的划分

1、均匀B样条曲线 (uniform B-spline curve)

当节点沿参数轴均匀等距分布，即 $u_{i+1}-u_i = \text{常数} > 0$ 时，表示均匀B样条函数

$$\{ 0,1,2,3,4,5,6 \}$$

$$\{ 0,0.2,0.4,0.6,0.6,0.8,1 \}$$

均匀B样条的基函数呈周期性。即给定n和k，所有的基函数有相同形状。每个后续基函数仅仅是前面基函数在新位置上的重复：

$$B_{i,k}(u) = B_{i+1,k}(u + \Delta u) = B_{i+2,k}(u + 2\Delta u)$$

其中， Δu 是相邻节点值的间距，等价地，也可写为：

$$B_{i,k}(u) = B_{0,k}(u - k\Delta u)$$

下面以均匀二次（三阶）B样条曲线为例来说明均匀周期性B样条基函数的计算：

假定有四个控制点，取参数值 $n=3$ ， $k=3$ ，则 $n+m=6$

$$u = (0,1,2,3,4,5,6)$$

根据de Boor-Cox递推公式：

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u)$$

$$B_{i,1}(u) = \begin{cases} 1 & u_i < x < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

$$B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

$$B_{0,1}(u) = \begin{cases} 1 & 0 \leq u < 1 \\ 0 & \text{其它} \end{cases} \quad u = (0, 1, 2, 3, 4, 5, 6)$$

$$B_{0,2}(u) = uB_{0,1}(u) + (2 - u)B_{1,1}(u) = uB_{0,1}(u) + (2 - u)B_{1,1}(u)$$

$$= \begin{cases} u & 0 \leq u < 1 \\ 2 - u & 1 \leq u < 2 \end{cases}$$

$$\begin{aligned}
 B_{0,3}(u) &= \frac{u}{2} B_{0,2}(u) + \frac{3-u}{2} B_{1,2}(u-1) \\
 &= \begin{cases} \frac{1}{2} u^2 & 0 \leq u < 1 \\ \frac{1}{2} u(2-u) + \frac{1}{2} (u-1)(3-u) & 1 \leq u < 2 \\ \frac{1}{2} (3-u)^2 & 2 \leq u < 3 \end{cases}
 \end{aligned}$$

$$\begin{aligned}
 B_{1,3}(u) &= \begin{cases} \frac{1}{2} (u-1)^2 & 1 \leq u < 2 \\ \frac{1}{2} (u-1)(3-u) + \frac{1}{2} (u-2)(4-u) & 2 \leq u < 3 \\ \frac{1}{2} (4-u)^2 & 3 \leq u < 4 \end{cases}
 \end{aligned}$$

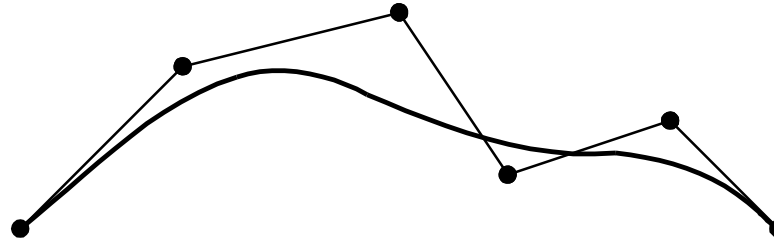
$$B_{3,3}(u) = \begin{cases} \frac{1}{2}(u-3)^2 & 3 \leq u < 4 \\ \frac{1}{2}(u-3)(5-u) + \frac{1}{2}(u-4)(6-u) & 4 \leq u < 5 \\ \frac{1}{2}(6-u)^2 & 5 \leq u < 6 \end{cases}$$

2、准均匀B样条曲线 (Quasi-uniform B-spline curve)

与均匀B样条曲线的差别在于两端节点具有重复度 k , 这样的节点矢量定义了准均匀的B样条基

均匀: $u = (0,1,2,3,4,5,6)$

准均匀: $u = (0,0,0,1,2,3,4,5,5,5)$

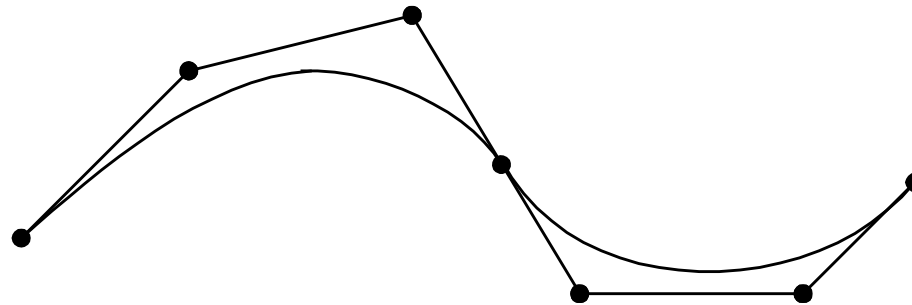


准均匀三次B样条曲线

均匀B样条曲线没有保留Bezier曲线端点的几何性质，采用准均匀的B样条曲线解决了这个问题

3、分段Bezier曲线 (Piecewise Bezier Curve)

节点矢量中两端节点具有重复度 k ，所有内节点重复度为 $k-1$ ，这样的节点矢量定义了分段的Bernstein基



三次分段Bezier曲线

B样条曲线用分段Bezier曲线表示后，各曲线段就具有了相对的独立性

另外，Bezier曲线一整套简单有效的算法都可以原封不动地采用

缺点是增加了定义曲线的数据，控制顶点数及节点数

4、非均匀B样条曲线 (non-uniform B-spline curve)

当节点沿参数轴的分布不等距，即 $u_{i+1} - u_i \neq \text{常数}$ 时，
表示非均匀B样条函数

七、B样条曲面

给定参数轴u和v的节点矢量

$$U = [u_0, u_1, \mathbf{L}, u_{m+p}]$$

$$V = [v_0, v_1, \mathbf{L}, v_{n+q}]$$

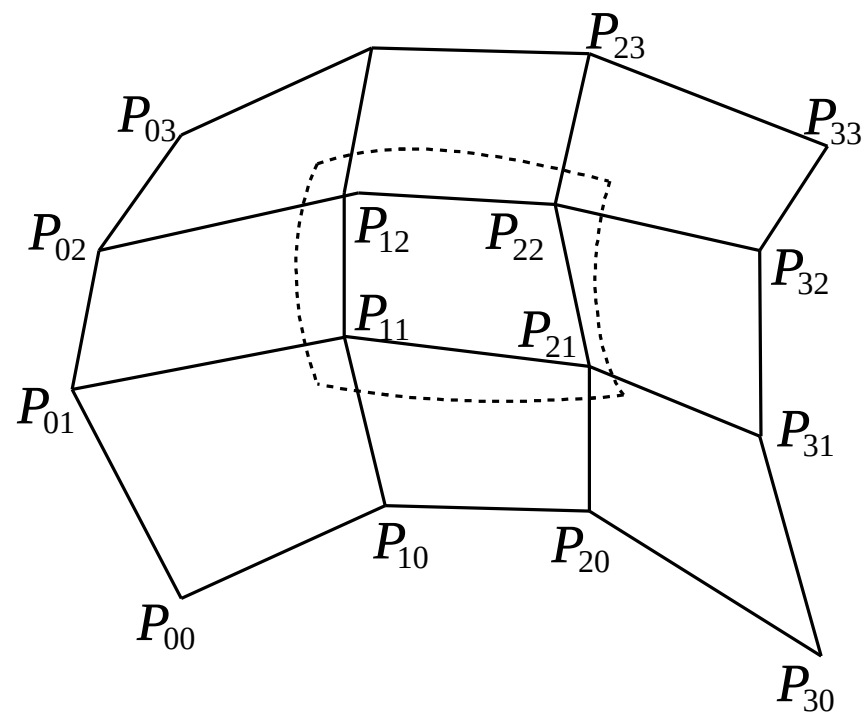
$p \times q$ 阶B样条曲面定义如下：

$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{i,j} N_{i,p}(u) N_{j,q}(v)$$

$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{i,j} N_{i,p}(u) N_{j,q}(v)$$

$P_{i,j}$ 构成一张控制网格，称为B样条曲面的特征网格

$N_{i,p}(u)$ 和 $N_{j,q}(v)$ 是B样条基，分别由节点矢量U和V按deBoor-Cox递推公式决定



双三次B样条曲面片

颜色模型

真实感图形学

真实感图形学研究什么？简单地说，就是希望用计算机生成像照相机拍的照片一样逼真的图形图像。要实现这个目标，需要三部曲：

第一步：建立三维场景（建模）；

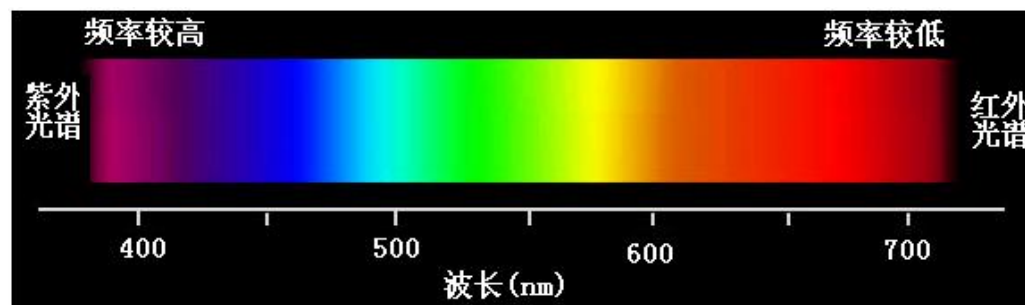
第二步：消隐解决物体深度的显示及确定物体之内的相互关系；

第三步：在解决了消隐问题之后，在可见面上进行明暗光泽的处理，然后进行绘制（渲染）。



一、颜色模型概述

1、什么是颜色？



- Ø 颜色是人的视觉系统对可见光的感知结果，感知到的颜色由光波的波长决定。
- Ø 人眼对于颜色的观察和处理是一种生理和心理现象，其机理还没有完全搞清楚。
- Ø 视觉系统能感觉的波长范围为380~780nm。

二、颜色模型概述

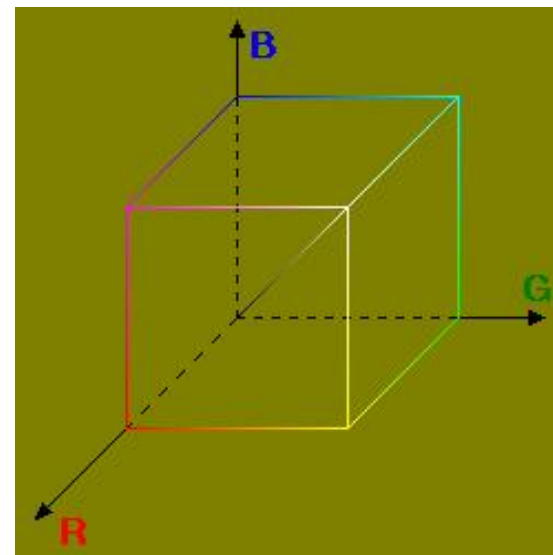
2、什么是颜色模型？

- Ø 颜色模型（空间），是表示颜色的一种数学方法，人们用它来指定颜色和标定产生的颜色。通常用三个参数表示。
- Ø 几乎所有的颜色模型都是从RGB颜色模型导出。
- Ø 目前现有颜色模型还没有一个完全符合人的视觉感知特性、颜色本身的物理特性或发光物体和光反射物体的特性。

二、常用颜色模型

1、RGB颜色工业模型

- Ø 如图所示，单位立方体中的三个角对应红色(R)、绿色(G)、蓝色(B)三基色，而其余三个角分别对应于三基色的补色——青色(C)、黄色(Y)、品红色(M)
- Ø 从RGB单位立方体的原点即黑色(0, 0, 0)到白色顶点(1, 1, 1)的主对角线被称为灰度线，线上所有的点具有相等的分量，产生灰度色调。



二、常用颜色模型

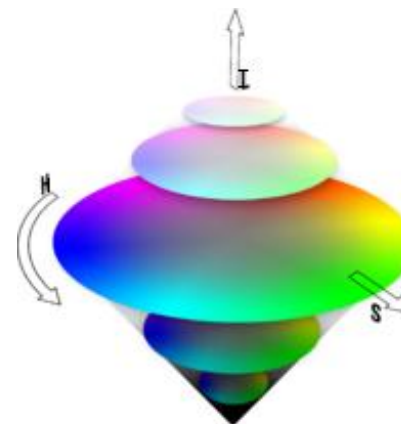
2、其它颜色工业模型

主要用于彩色电视信号传输标准，主要有YIQ、YUV、YCbCr彩色模型。三种彩色模型中，Y分量均代表黑白亮度分量，其余分量用于显示彩色信息。这样，只需利用Y分量进行图像显示，彩色图像就转换为灰度图像。

二、常用颜色模型

3、颜色视觉模型

Ø 以上彩色模型是从色度学或硬件实现的角度提出的，但用色调(Hue)、饱和度(Saturation，也称彩度)、亮度(Illumination)三要素来描述彩色空间能更好地与人的视觉特性相匹配。



二、常用颜色模型

3、颜色视觉模型

颜色的三个基本属性(也称人眼视觉三要素) ——

①色调(Hue): 由物体反射光线中占优势的波长决定的, 是彩色互相区分的基本特性。

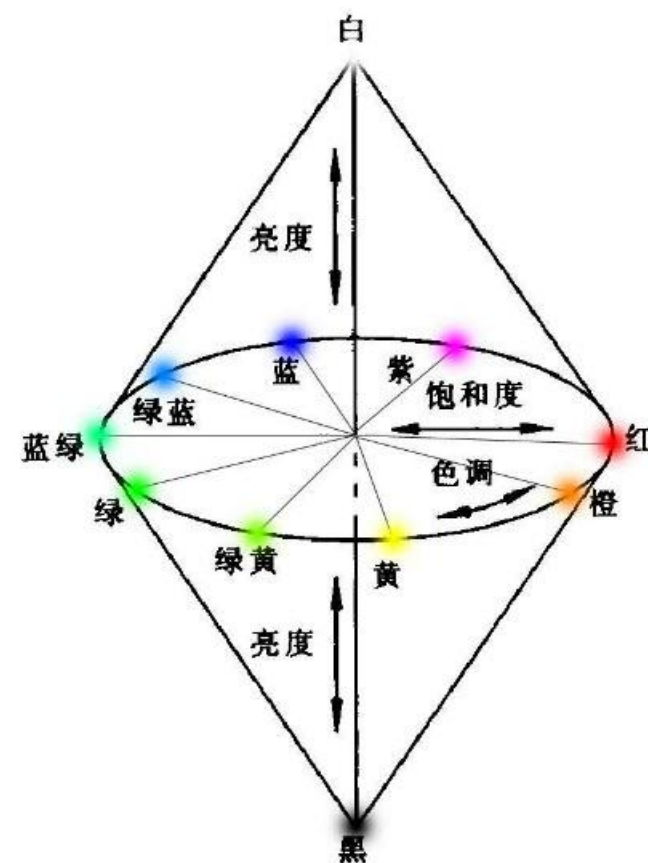
②饱和度(Saturation)或彩度: 彩色的深浅程度, 它取决于彩色中白色的含量。饱和度越高, 彩色越深, 白色光越少。

③亮度(Illumination): 光波作用于感受器所发生的效应, 它取决于物体的反射系数。反射系数越大, 物体亮度越大。

二、常用颜色模型

3、颜色视觉模型

HSI彩色模型是截面为三角形或圆形的锥体模型。

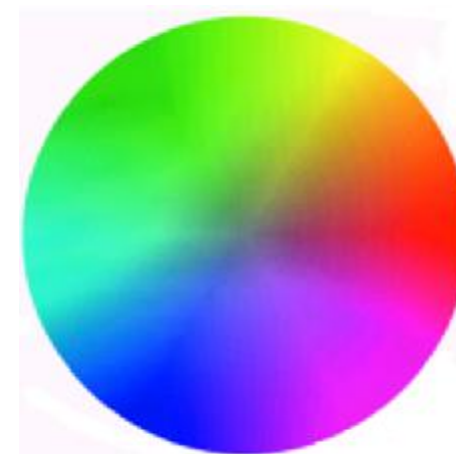


二、常用颜色模型

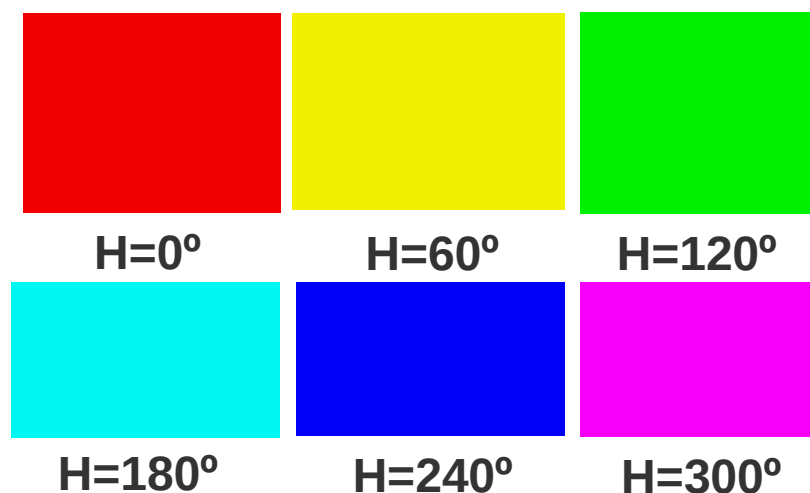
3、颜色视觉模型

色调(H)也称为色相, 指颜色的外观,
色调H用角度表示:

如赤橙黄绿青蓝紫, 角度从(红)→(绿)→(蓝)→(红)。



ü色调(H)
效果示意
图



二、常用颜色模型

3、颜色视觉模型

饱和度，分成：

ü 低(0%~20%)，不管色调如何而产生灰色；

ü 中(40%~60%)，产生柔和的色泽(pastel)；

ü 高(80%~100%)，产生鲜艳的颜色(vivid color)。

ü 饱和度(S)效果示意图：



S=0



S=1/4



S=1/2



S=1

二、常用颜色模型

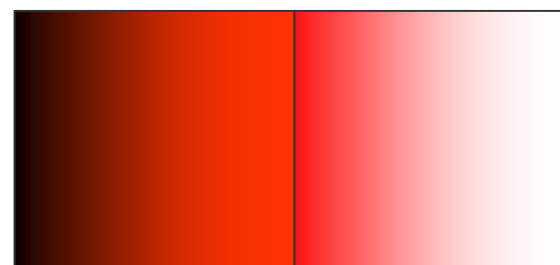
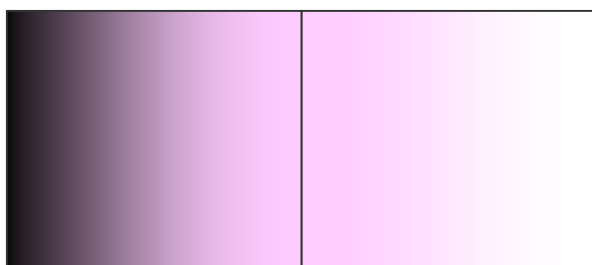
3、颜色视觉模型

强度(Intensity)是颜色的亮度 (Illumination) ；

ü 取值范围从0%(黑)~100%(最亮)；

ü 强度也指明度(value)或光亮度(lightness或Brightness) 。

ü 亮度(I)效果示意图：



二、常用颜色模型

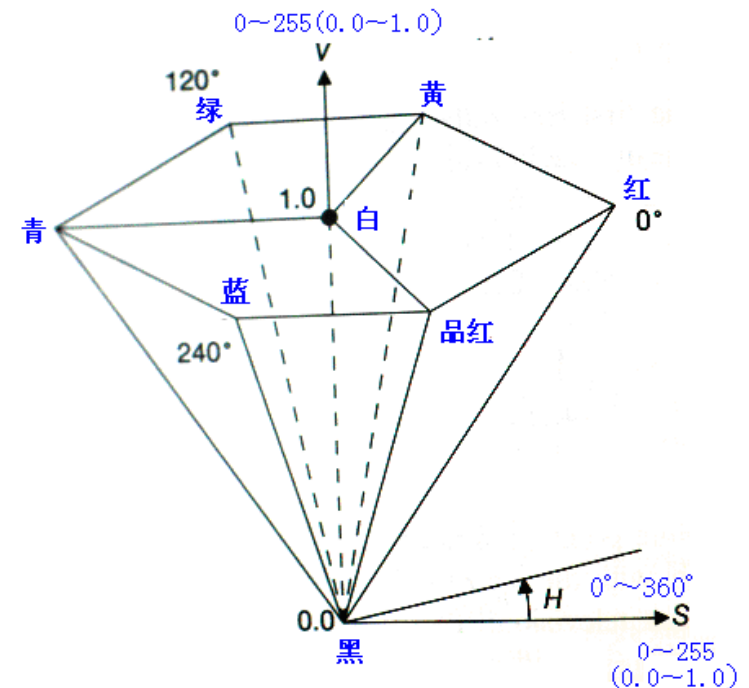
4、其他颜色视觉模型

▼ HSV(hue, saturation and value)
彩色模型——

ü A. R. Smith根据颜色的直观特性
于1978年创建的，是颜色的倒六角
锥体模型。

▼ HSL 与HSV——

ü HSL采用亮度L(lightness)、HSV采
用明度V(value)作为坐标。



简单光照模型

一、什么是光照模型？

1、光照模型

当物体的几何形态确定之后，光照决定了整个场景的显示结果。因此，真实感图形的生成取决于如何建立一个合适的光照模型（illumination model）。

光照明模型：模拟物体表面的光照明物理现象的数学模型。

简单光照明模型只考虑光源对物体的直接光照





二、光照模型的发展演化

1、早期发展

1967年，Wylie等人第一次在显示物体时加进光照效果，认为光强与距离成反比。

1970年，Bouknight提出第一个光反射模型：Lambert漫反射+环境光（第一个可用的光照模型）。这篇文章发表在 *Communication of ACM* 上。

1971年，Gouraud提出漫反射模型加插值的思想（漫反射的意思是光强主要取决于入射光的强度和入射光与法线的夹角）发表在 *IEEE Transactions on Computers* 上。

1975年，Phong提出图形学中第一个最有影响的光照明模型。在漫反射模型的基础上加进了高光项。

Graphics and
Image Processing

W. Newman
Editor

Illumination for Computer Generated Pictures

Bui Tuong Phong
University of Utah

The quality of computer generated images of three-dimensional scenes depends on the shading technique used to paint the objects on the cathode-ray tube screen. The shading algorithm itself depends in part on the

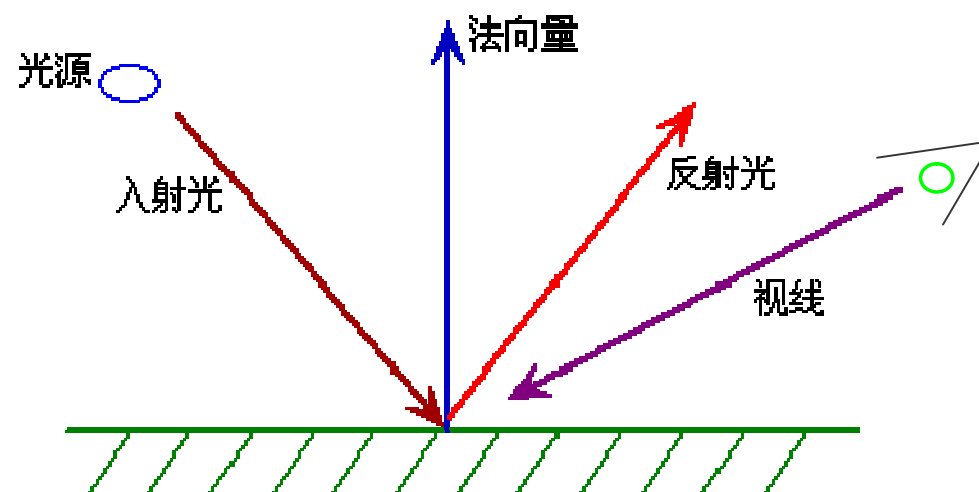
parts and the shading of the objects. Until now, most effort has been spent in the search for fast hidden surface removal algorithms. With the development of these algorithms, the programs that produce pictures are becoming remarkably fast, and we may now turn to the search for algorithms to enhance the quality of these pictures.

In trying to improve the quality of the synthetic images, we do not expect to be able to display the object exactly as it would appear in reality, with texture, over-cast shadows, etc. We hope only to display an image that approximates the real object closely enough to provide a certain degree of realism. This involves some understanding of the fundamental properties of the human visual system. Unlike a photograph of a real world scene, a computer generated shaded picture is made from a numerical model, which is stored in the computer as an objective description. When an image is then generated from this model, the human visual system makes the final subjective analysis. Obtaining a close image correspondence to the eye's subjective interpretation of the real object is then the goal. The computer system can be compared to an artist who paints an object from its description and not from direct observation of the object. But unlike the artist, who can

三、背景物理知识

1、光的传播规律

反射定律：入射角等于反射角，而且反射光线、入射光线与法向量在同一平面上。



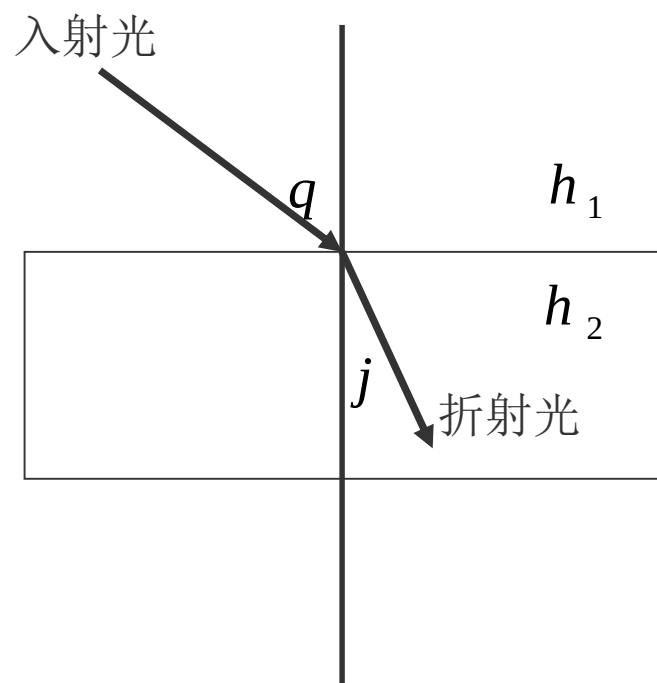
三、背景物理知识

2、折射定律

折射定律：折射线在入射线与法线构成的平面上，折射角与入射角满足：

$$\frac{h_1}{h_2} = \frac{\sin j}{\sin q}$$

其中： η_1 、 η_2 分别是入射光线在空气，物体中的折射率， θ 和 ψ 分别是入射角和折射角。



三、背景物理知识

3、能量关系

在光的反射和折射现象中的能量分布（满足能量守恒）：

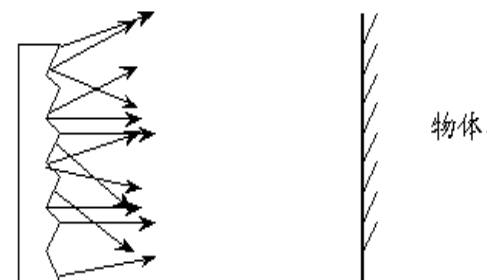
$$I_i = I_d + I_s + I_t + I_v$$

- I_i is the energy of incident light 入射光强
- I_d is the energy of diffuse reflection 漫反射光强
- I_s is the energy of specular reflection （镜面反射）
- I_t is the energy of refraction （折射）
- I_v is the energy that is absorbed 吸收光强

三、背景物理知识

漫反射光：光线射到物体表面上后（比如泥塑物体的表面，没有一点镜面效果），光线会沿着不同的方向等量的散射出去，这种现象称为漫反射。漫反射光在不同方向都是一样的。

漫反射光均匀向各方向传播，与视点无关，它是由表面的粗糙不平引起的。



三、背景物理知识

镜面反射光：一束光照射到一面镜子上或不锈钢的表面，光线会沿着反射光方向全部反射出去，这种叫镜面反射光。

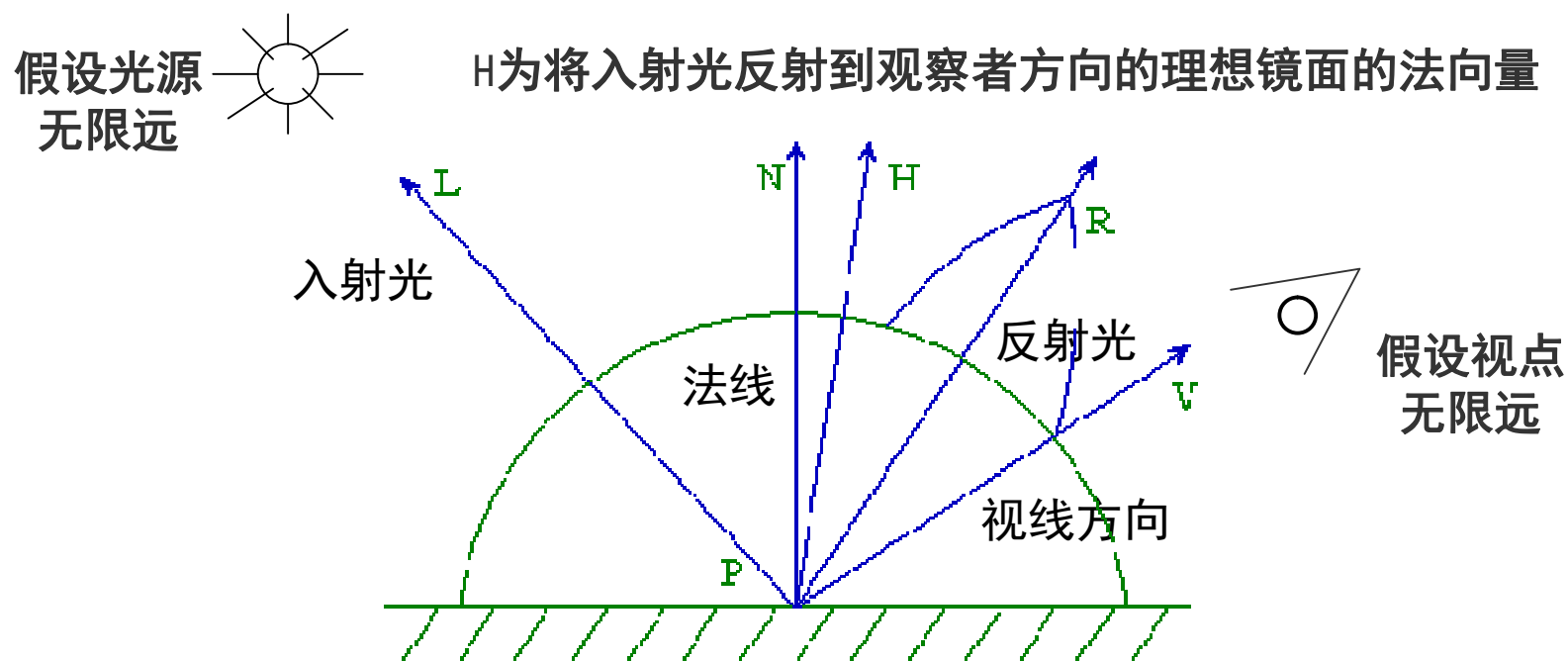
折射光：比如水晶、玻璃等，光线会穿过去一直往前走。

吸收光：比如冬天晒太阳会感觉到温暖，这是因为吸收了太阳光。

Phone光照模型

一、Phong光照模型

1、Phong光照模型（环境光+漫反射光+镜面反射光）



一、Phong光照模型

环境光

§ 邻近各物体所产生的光的多次反射最终达到平衡时的一种光。可近似认为同一环境下的环境光，其光强分布是均匀的。

$$I_{ambient} = I_a K_a$$

I_a 环境光强度 K_a 环境光反射系数

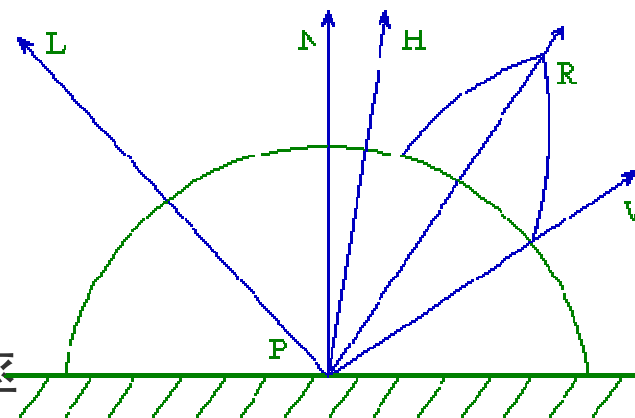
一、Phong光照模型

漫反射光

§ 光照射到比较粗糙的物体表面，物体表面某点的明暗程度不随观测者的位置变化，这种等同地向各个方向散射的现象称为光的漫反射。漫反射光强近似服从 Lambert 定律：

$$I_{\text{diffuse}} = I_p K_d (L \cdot N)$$

I_p 点光源光强 K_d 物体表面漫反射率



一、Phong光照模型

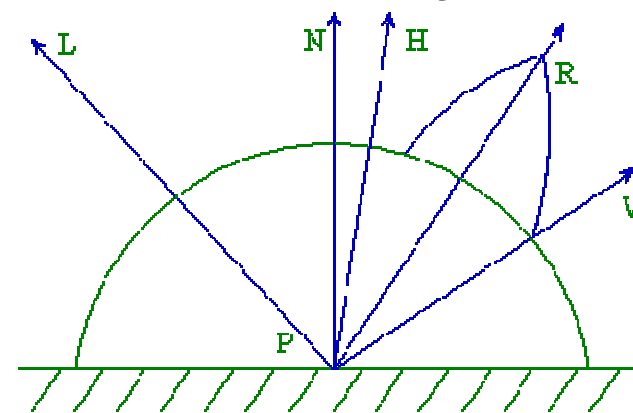
镜面反射光

§ 光照射到相当光滑的物体表面，就产生镜面反射光，其特点是在光滑表面会产生高光区域。一般用Phong提出的经验模型表达：

$$I_{\text{spec}} = I_p K_s (R \cdot V)^n$$

I_p 点光源光强 K_s 物体表面某点的高光亮系数

n 镜面反射指数，1~2000，反映光滑程度



一、Phong光照模型

1、Phong光照模型

$$I = I_a K_a + I_p K_d (L \cdot N) + I_p K_s (R \cdot V)^n$$

环境光 理想漫反射光 镜面反射光

这就是经典的Phong模型。 I_a 、 I_p 都是常数， k 也是已知的， L 是光源的方向也是已知的， N 是物体表面的法向可以算出来的， v 是视线的方向， R 也可以算出来。

一、Phong光照模型

Phong光照模型

单一光源照射下Phong光照模型的表达式: $I = I_a k_a + k_d I_l \cos\theta + k_s I_l \cos^n \alpha$

I	景物表面在被照射点处的光亮度
I_a	入射的泛光光强
k_a	景物表面对泛光的反射系数
k_d	景物表面的漫反射率
I_l	点光源所发出的入射光强度
θ	入射光与表面法向之间的夹角
k_s	景物表面的镜面反射率
n	镜面高光指数
α	镜面与反射光线之间的夹角

代替为 $L \cdot N$ 和 $V \cdot R$ 替换为 $N \cdot H$

H为将入射光反射到观察者方向的理想镜面的法向量

假设光源无限远

假设视点无限远

入射光

法线

反射光

视线方向

P

效果实例

Replay

The diagram illustrates the Phong lighting model. It shows a surface point P with a normal vector N. Incident light L comes from a source at infinity, forming an angle θ with N. The reflected light R is shown, and the view direction V is shown at an angle α from R. The diagram also shows a sphere with a bright spot, labeled '效果实例' (Effect Example), and a 'Replay' button.

一、Phong光照模型

1、Phong光照模型

结合RGB颜色模型， Phong光照明模型最终有如下的形式：

$$\begin{cases} I_r = I_{ar} K_{ar} + I_{pr} K_{dr} (L \cdot N) + I_{pr} K_{sr} (R \cdot V)^n \\ I_g = I_{ag} K_{ag} + I_{pg} K_{dg} (L \cdot N) + I_{pg} K_{sg} (R \cdot V)^n \\ I_b = I_{ab} K_{ab} + I_{pb} K_{db} (L \cdot N) + I_{pb} K_{sb} (R \cdot V)^n \end{cases}$$

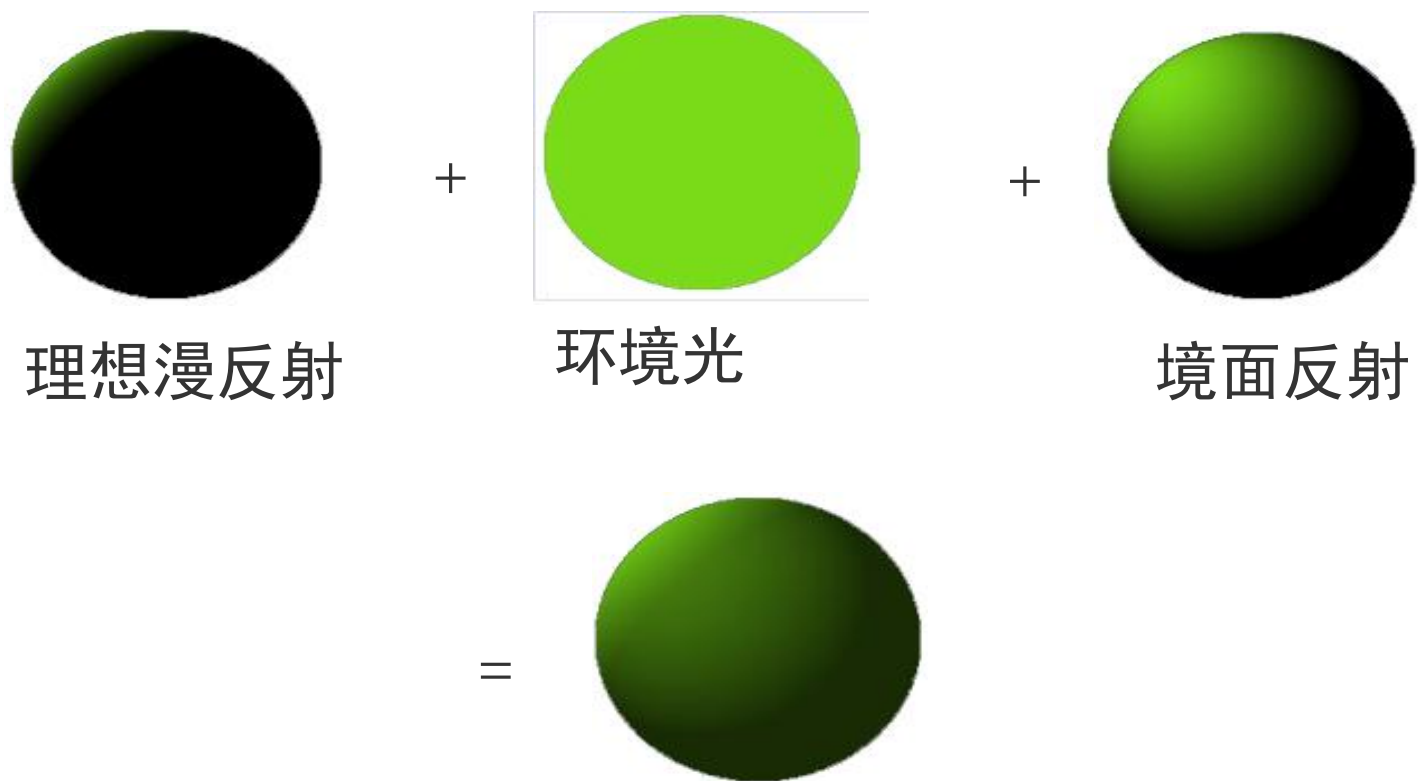
一、Phong光照模型

Phong模型扫描线算法

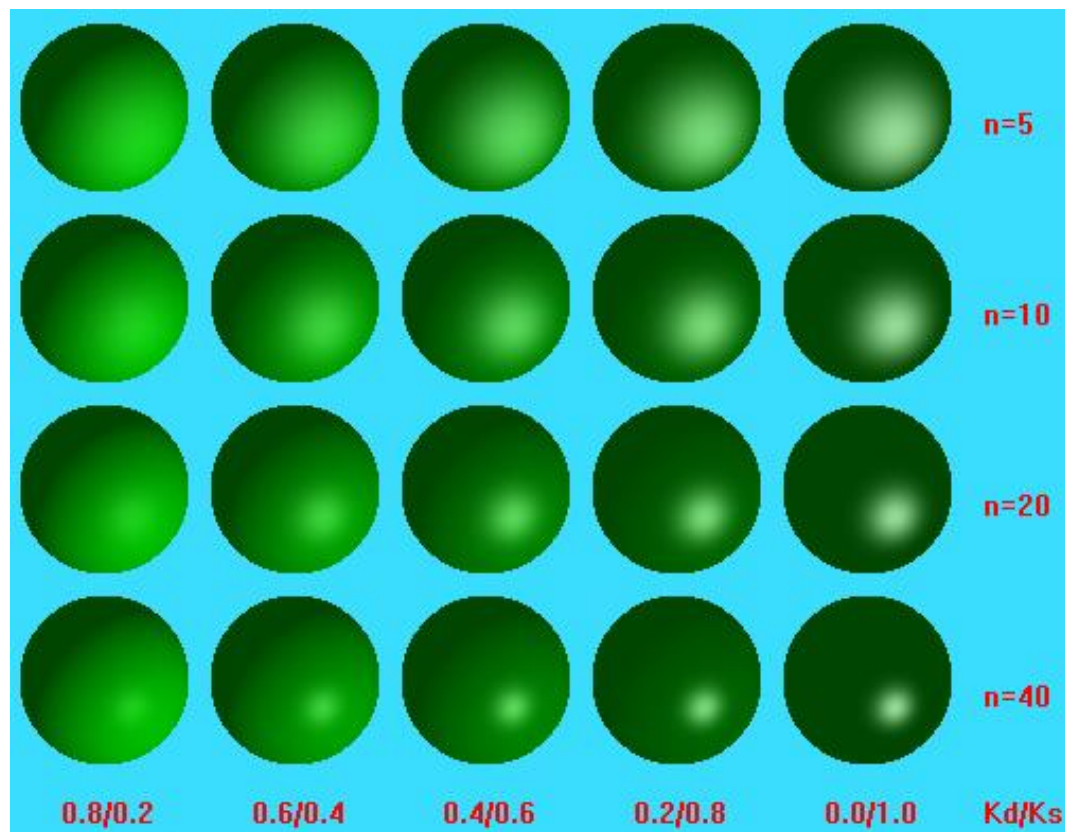
```
1.   for 屏幕上的每一条扫描线 y do
2.   begin
3.   将数组Color初始化成为y扫描线的背景颜色值;
4.   for y扫描线上的每一可见取间段s中的每个点 (x,y) do
5.       begin
6.       设 (x,y) 对应的空间可见点为P;
7.       求出P点处的单位法向量N, P点的单位入射光向量L和单位视线向
           量V, 求出L在P点的单位镜面反射向量R;
8.       
$$(r, g, b) = k_a(r_{pa}, g_{pa}, b_{pa}) + \sum [ k_d(r_{pd}, g_{pd}, b_{pd}) \cos \theta + k_s(r_{ps}, g_{ps}, b_{ps}) \cos^n \alpha ]$$

           置Color(x,y) = (r, g, b)
           end;
       显示Color;
       end;
```

Phong模型示例_1



Phong模型示例_2



一、Phong光照模型

Phong光照明模型是真实感图形学中提出的第一个有影响的
光照明模型，生成图象的真实度已经达到可以接受的程度。

Phone模型用来模拟光从物体表面到观察者眼睛的反射。尽管这种方法符合一些基本的物理法则，但它更多的是基于对现象的观察，所以被看成是一种经验式的方法。

一、Phong光照模型

在实际的应用中，由于Phong光照模型是一个**经验模型**，因此还具有以下的一些问题：

- 显示出的物体象塑料，无质感变化
- 没有考虑物体间相互反射光
- 镜面反射颜色与材质无关
- 镜面反射入射角大，会产生失真现象

增量式光照模型

- Gouraud明暗处理

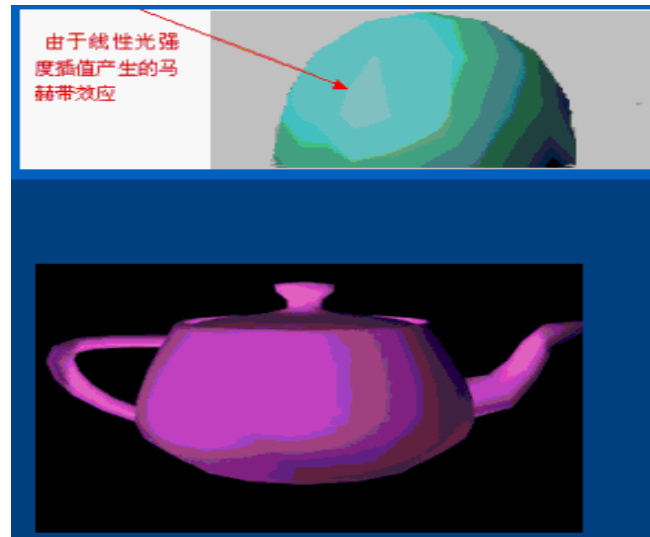
一、为什么要进行明暗处理？

- § 三维物体通常用多边形（三角形）来近似模拟。
- § 由于每一个多边形的法向一致，因而多边形内部的像素的颜色都是相同的，而且在不同法向的多边形邻接处，光强突变，使具有不同光强的两个相邻区域之间的光强不连续性(马赫带效应)。



用简单光照模型显示

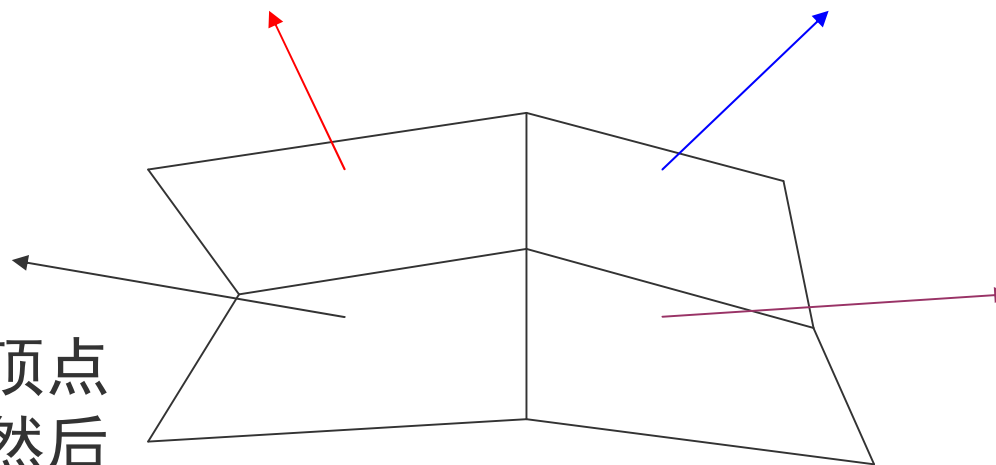
一、更多示例



解决办法? → 增量式光照模型

二、如何进行明暗处理？

基本思想： 每一个多边形的顶点处计算出光照强度或参数，然后在各个多边形内部进行均匀插值



常用方法：

- ü Gouraud明暗处理(双线性光强插值算法)
- ü Phong明暗处理(双线性法向插值算法)

三、Gouraud明暗处理

- n** 1971年，由Gouraud提出，又称双线性光强插值。
- n** 基本步骤由四步构成。

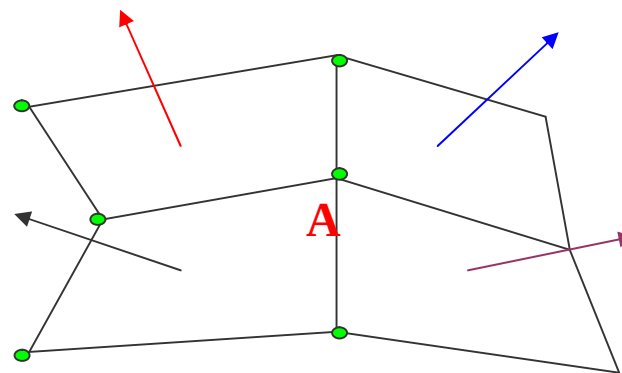
三、Gouraud明暗处理

ü 第一步

n 计算多边形顶点的平均法向。

与某个顶点相邻的所有多边形的法向平均值近似作为该顶点的近似法向量，顶点A相邻的多边形有k个，它的法向量计算为：

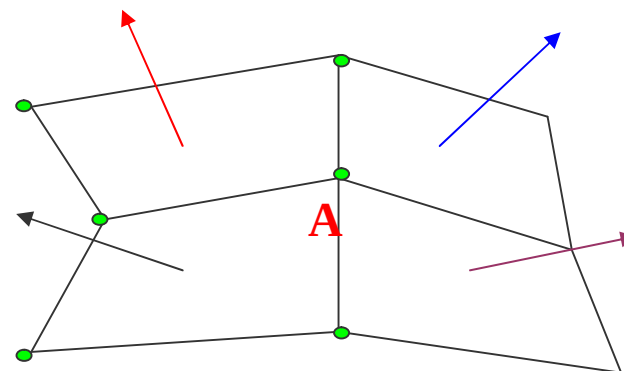
$$N_a = \frac{1}{k} (N_1 + N_2 + \dots + N_k)$$



三、Gouraud明暗处理

ü 第二步

n 用Phong光照模型计算顶点的光强。



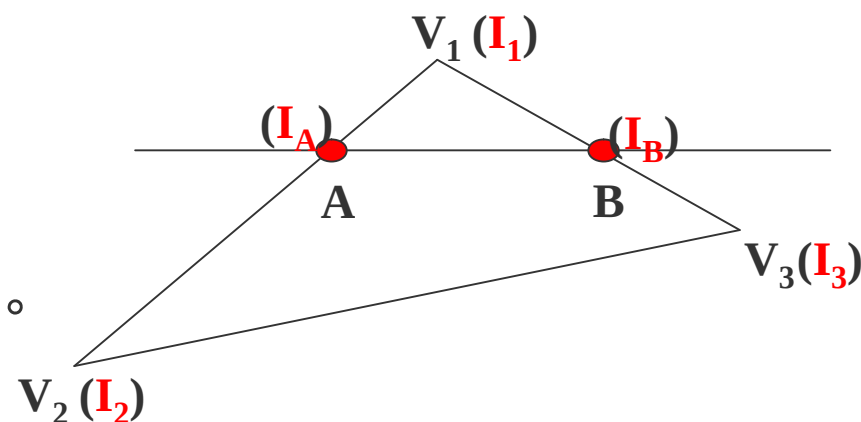
Phong光照模型出现前，采用如下光照模型计算：

$$I = I_a K_a + I_p K_d (L \cdot N_a) / (r + k)$$

三、Gouraud明暗处理

ü 第三步

n 插值计算离散边上个点的光强。

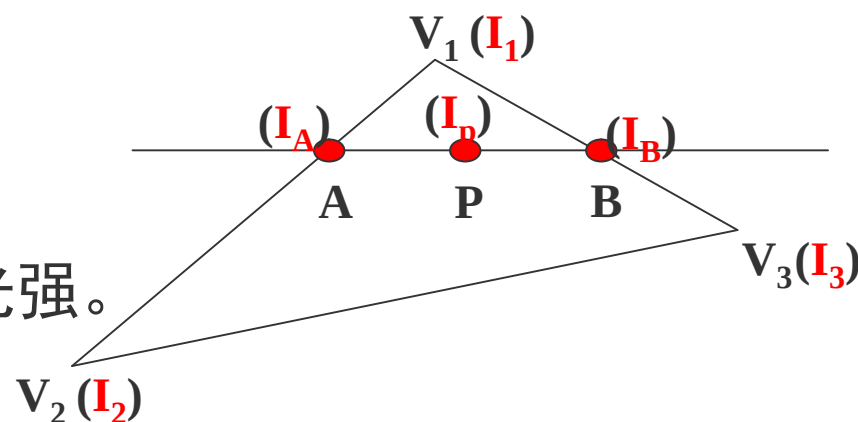


$$I_A = uI_1 + (1 - u)I_2 \quad u = \frac{AV_2}{V_1V_2}$$
$$I_B = vI_1 + (1 - v)I_3 \quad v = \frac{BV_3}{V_1V_3}$$

三、Gouraud明暗处理

ü 第四步

n 插值计算多边形内域中各点的光强。



$$I_p = tI_A + (1 - t)I_B \quad t = \frac{PB}{AB}$$

求任一点的光强需进行两次插值计算。

三、Gouraud明暗处理

ü 增量计算

n 为减少计算量，采用增量计算方法。

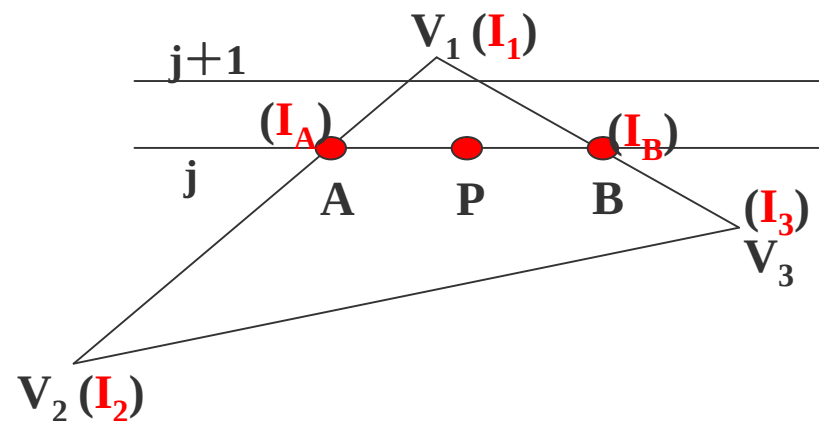
新扫描线：

$$I_{A, j+1} = I_{A, j} + \Delta I_A$$

$$I_{B, j+1} = I_{B, j} + \Delta I_B$$

$$\Delta I_A = (I_1 - I_2) / (y_1 - y_2)$$

$$\Delta I_B = (I_1 - I_3) / (y_1 - y_3)$$



扫描线内部：

$$I_{i+1, p} = I_{i, p} + \Delta I_p$$

$$\Delta I_p = (I_B - I_A) / (x_B - x_A)$$

增量式光照模型

-Phong明暗处理

一、 Gouraud明暗处理的不足

- § Gouraud明暗处理有一个最大的缺点，就是不能有镜面反射光（高光）。
- § 双线性插值是把能量往四周均匀，平均的结果就是光斑被扩大了，本来没有光斑的地方一插值反而出现了光斑。

解决办法？  Phong明暗处理

一、 Phong明暗处理

§ 与Gouraud明暗处理有何不同？

双线性光强插值？  双线性法向插值

§ 以时间为代价，引入镜面反射，解决高光问题

一、 Phong明暗处理

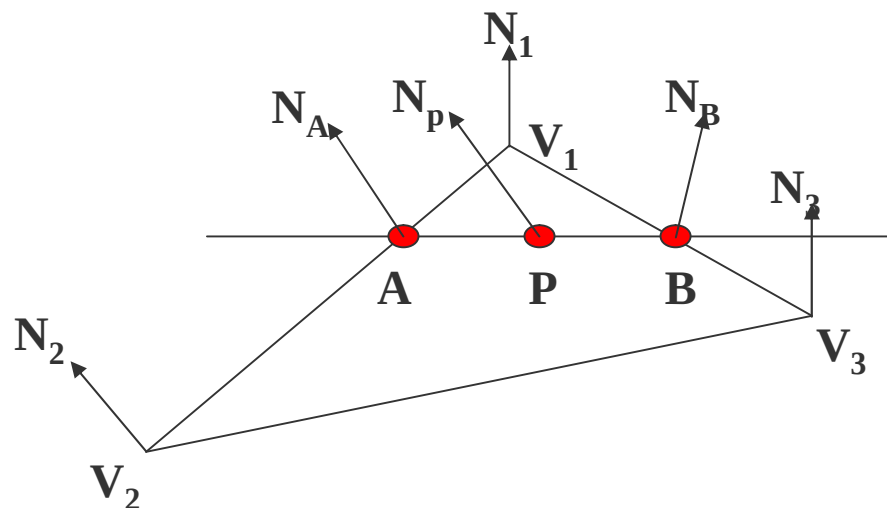
Phong明暗处理基本步骤：

- n** 计算每个多边形顶点处的平均单位法矢量，这一步骤与Gouraud明暗处理方法的第一步相同。
- n** 用双线性插值方法求得多边形内部各点的法矢量。
- n** 最后按光照模型确定多边形内部各点的光强。

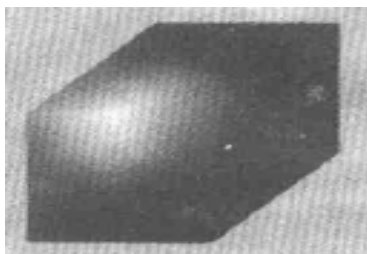
一、 Phong明暗处理

Phong明暗处理是先算角点的法向量，再算内部点的法向量，最后再用新的光照模型算内部点的颜色值。

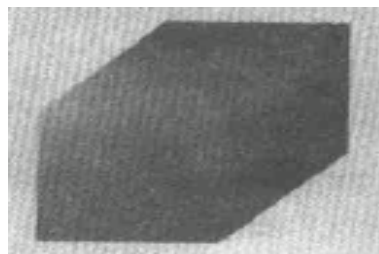
点A的法向量 N_A 为 N_1 与 N_2 的线性插值，点B的法向量 N_B 为 N_1 与 N_3 的线性插值，点P的法向量 N_P 为 N_A 与 N_B 的线性插值。



二、 两种增量式光照模型比较



Phong方法



Gouraud方法

Phong方法	Gouraud方法
1、产生的效果高光明显； 2、高光多位于多边形内部； 3、明暗变化缺乏层次感。	1、效果并不明显； 2、多边形内部无高光； 3、光强度变化均匀，与实际效果更接近。

三、 增量式光照模型总结

n双线性光强插值（Gouraud模型）能有效的显示漫反射曲面，计算量小，速度快。

n双线性法向插值（Phong模型）可以产生正确的高光区域，但是计算量要大的多。

n 增量式光照明模型的不足

- 物体边缘轮廓是折线段而非光滑曲线
- 等间距扫描线会产生不均匀效果
- 插值结果取决于插值方向

四、与简单光照模型区别

牛的三角网格模型



用简单光照模型显示



用增量式光照模型显示

局部光照模型

一、什么是局部光照模型？

- n 局部光照模型：仅处理光源直接照射物体表面的光照模型。
- n 简单光照模型是一个比较粗糙的经验模型，不足之处：**镜面反射项与物体表面的材质无关**。
- n 从光电学知识和物体微平面假设出发，介绍镜面反射与物体材质有关的普遍局部光照模型。

二、局部光照模型

n 自然光反射率系数可用Fresnel公式计算

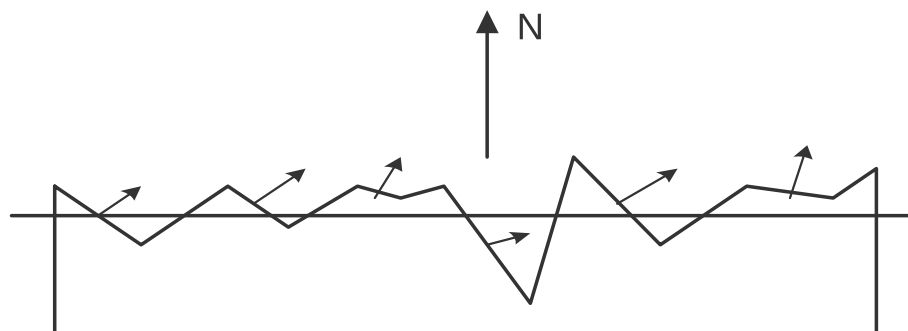
$$r = \frac{1}{2} \left(\frac{\tan^2(q - y)}{\tan^2(q + y)} + \frac{\sin^2(q - y)}{\sin^2(q + y)} \right)$$

q 是入射角，若发生反射的物体表面两侧折射率分别为 n_1, n_2 那么 y 满足这样的式子： $\sin y = \frac{n_1}{n_2} \sin q$

n 反射率与折射率有关，是波长的函数 $r(q, l)$

二、局部光照模型

n 微观情况下，物体表面粗糙不平。



微平面示意图

宏观上看，这是一个平面，法向朝上。实际上它是由许多微小平面构成的，微小平面的法向是各异的。

二、局部光照模型

n 反射率计算

微平面是理想镜面，反射率可用Fresnel公式计算，而粗糙表面的反射率与表面的粗糙度有关。

实际物体反射率： $D G r(q, l)$

D 为微平面法向的分布函数

G 为由于微平面的相互遮挡或屏蔽而使光产生的衰减因子

二、局部光照模型

§ Torrance和Sparrow采用Gauss分布函数模拟法向分布：

$$D = ke^{-(a/m)^2}$$

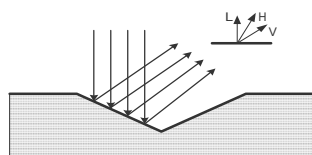
- | k 为常系数
- | a 为微平面的法向与平均法向的夹角，即 $(N \cdot H)$
- | m 为微平面斜率的均方根，表示表面的粗糙程度

$$m = \sqrt{\frac{m_1^2 + m_2^2 + \mathbf{L} + m_n^2}{n}}$$

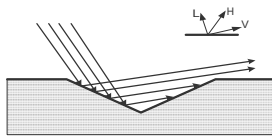
也可采用Berkmann分布函数

二、局部光照模型

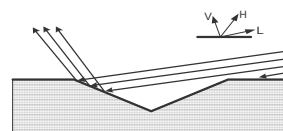
- § 衰减因子G在局部光照模型中也可以反映物体表面的粗糙程度。
- § 衰减因子是由于微平面的相互遮挡或屏蔽而产生的
- § 微平面相互遮挡的光衰减因子G，有三种情况：



$$G=1$$



$$G_m = \frac{2(N \cdot H)(N \cdot V)}{(V \cdot H)}$$



$$G_s = \frac{2(N \cdot H)(N \cdot L)}{(V \cdot H)}$$

$$G = \text{Min}\{1, G_m, G_s\}$$

二、局部光照模型

Cook和Torranace于1981年提出了局部光照模型。

R_{bd} 表示物体对入射光的反射率系数

$$R_{bd} = \frac{I_r}{E_i}$$

I_r — 反射光的光强

E_i — 单位时间内单位面积上的入射光能量

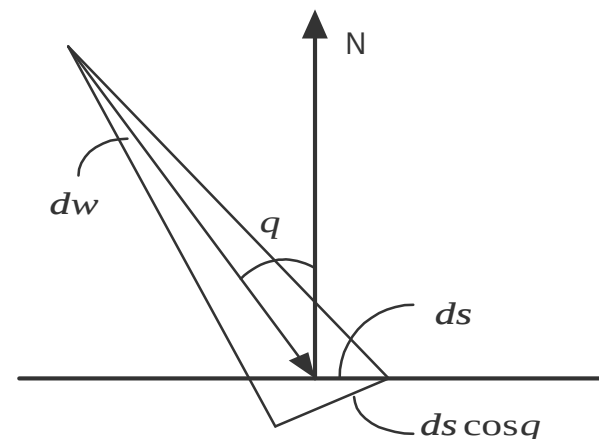
二、局部光照模型

§ 入射光能量 E_i , 可用入射光的光强 I_i 和单位面积向光源所张的立体角 $d\mathbf{v}$ 表示为:

$$E_i = I_i \cos q \cdot d\mathbf{v} = I_i (N \cdot L) d\mathbf{v}$$

§ 于是有反射光光强:

$$I_r = R_{bd} I_i (N \cdot L) d\mathbf{v}$$



二、局部光照模型

反射率系数可表示为漫反射率与镜面反射率的代数和：

$$R_{bd} = K_d R_d + K_s R_s$$

$$K_d + K_s = 1 \quad \text{漫反射与镜面反射系数}$$

物体表面的漫反射率： $R_d = R_d(l)$

物体表面的镜面反射率： $R_s = \frac{DGr(q, l)}{p(N \cdot L)(N \cdot V)}$

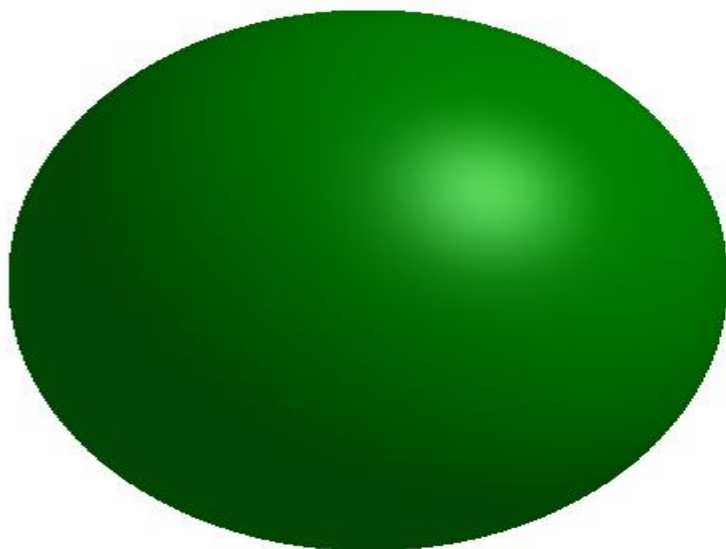
二、局部光照模型

§ 局部光照模型表示

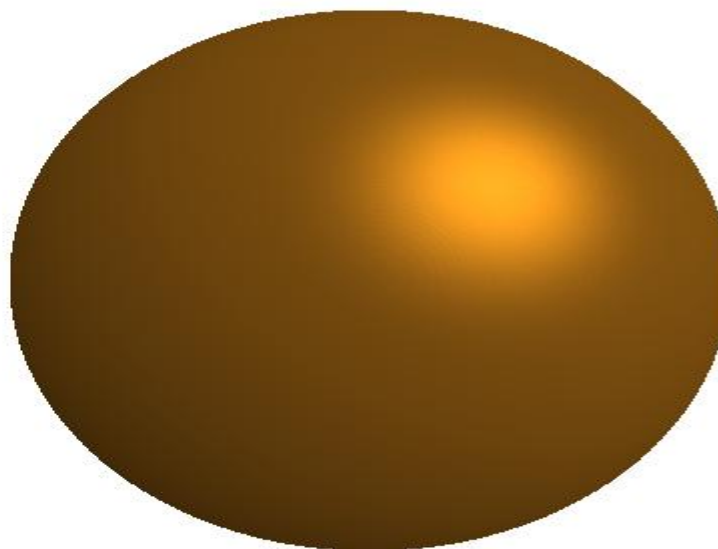
$$I_r = I_a K_a + I_i (N \cdot L) d v (K_d R_d + K_s R_s)$$

- | I_r 物体表面反射光强
- | $I_a K_a$ 表示环境光的影响
- | 最后一项是考虑了物体表面性质后的反射光强度量，是该局部光照模型的复杂性与普遍性所在。

简单与局部模型比较



简单光照模型(Phong)

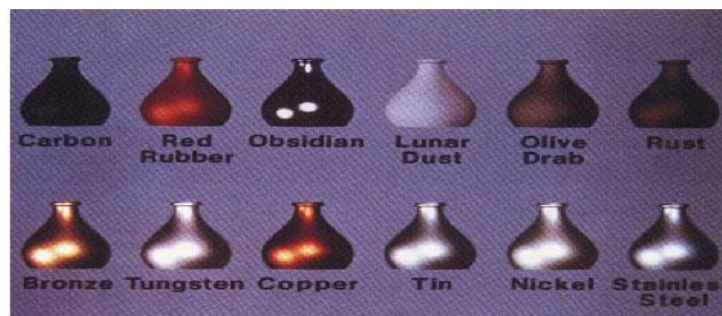


局部光照模型

三、局部光照模型的优点：

§ 相对于简单光照模型而言

- | 基于入射光能量导出的光辐射模型
- | 反映表面的粗糙度对反射光强的影响
- | 高光颜色与材料的物理性质有关
- | 改进入射角很大时的失真现象
- | 考虑了物体材质的影响，可以模拟磨光的金属光泽



光透射模型

一、 为什么考虑光透射模型？

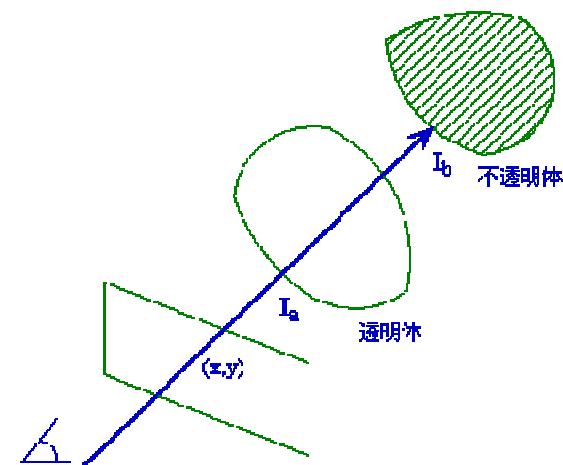
- n 简单和局部光照模型没有考虑光的透射现象。
- n 适用于场景中有透明或者半透明的物体的光照处理。
- n 早期用颜色调和法进行模拟。

二、 光透射模型

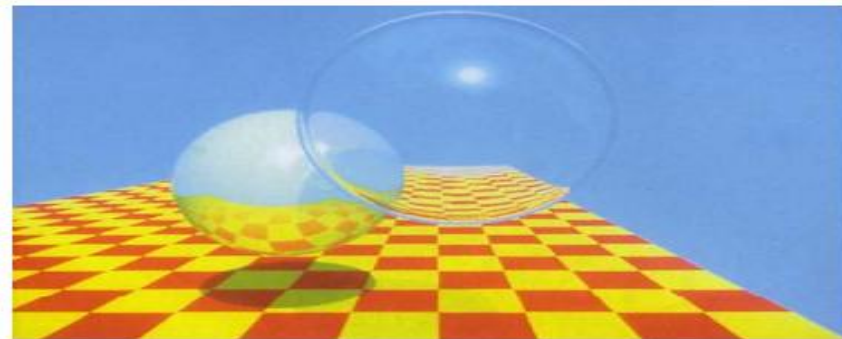
n 颜色调和法

$$I = t * I_b + (1 - t) * I_a$$

不考虑透明体对光的折射以及透明物体本身的厚度，光通过物体表面是不会改变方向的，可以模拟平面玻璃。



1980年Whitted提出了第一个整体光照模型，并给出了一般光线跟踪算法的范例，综合考虑了光的反射、折射、透射和阴影等。被认为是计算机图形领域的一个里程碑。



Turner Whitted , An improved illumination model for shaded display, Communications of the ACM, v.23 n.6, p.343-349, June 1980.

2003年Whitted当选为美国工程院院士。

三、Whitted 光透射模型

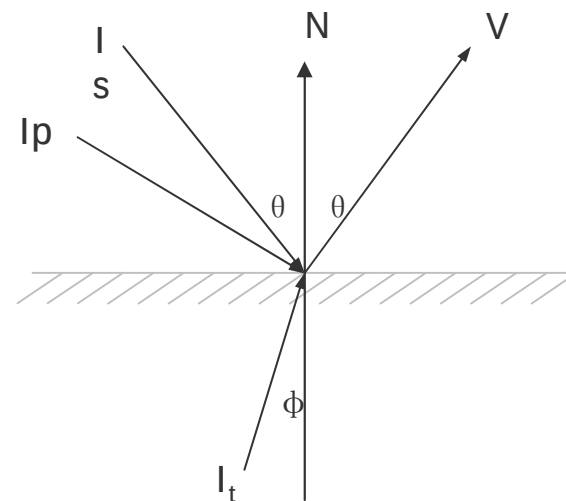
n 1980年由Whitted提出，因此命名。

在简单光照明模型的基础上，加上透射光项就得到Whitted光透射模型：

$$I = I_a \cdot K_a + I_p \cdot K_d \cdot (L \cdot N) + I_p \cdot K_s \cdot (R \cdot V)^n + I'_t \cdot K'_t$$

再加上镜面反射光项，就得到Whitted 整体光照模型：

$$I = I_a \cdot K_a + I_p \cdot K_d \cdot (L \cdot N) + I_p \cdot K_s \cdot (R \cdot V)^n + I'_t \cdot K'_t + I'_s \cdot K'_s$$



整体光照模型

一、 为什么需要整体光照模型？

- n 简单和局部光照模型不能很好地模拟光的折射、反射和阴影等，也不能用来表示物体间的相互光照影响。
- n 整体光照模型是更精确的光照模型，主要有光线跟踪和辐射度两种方法。

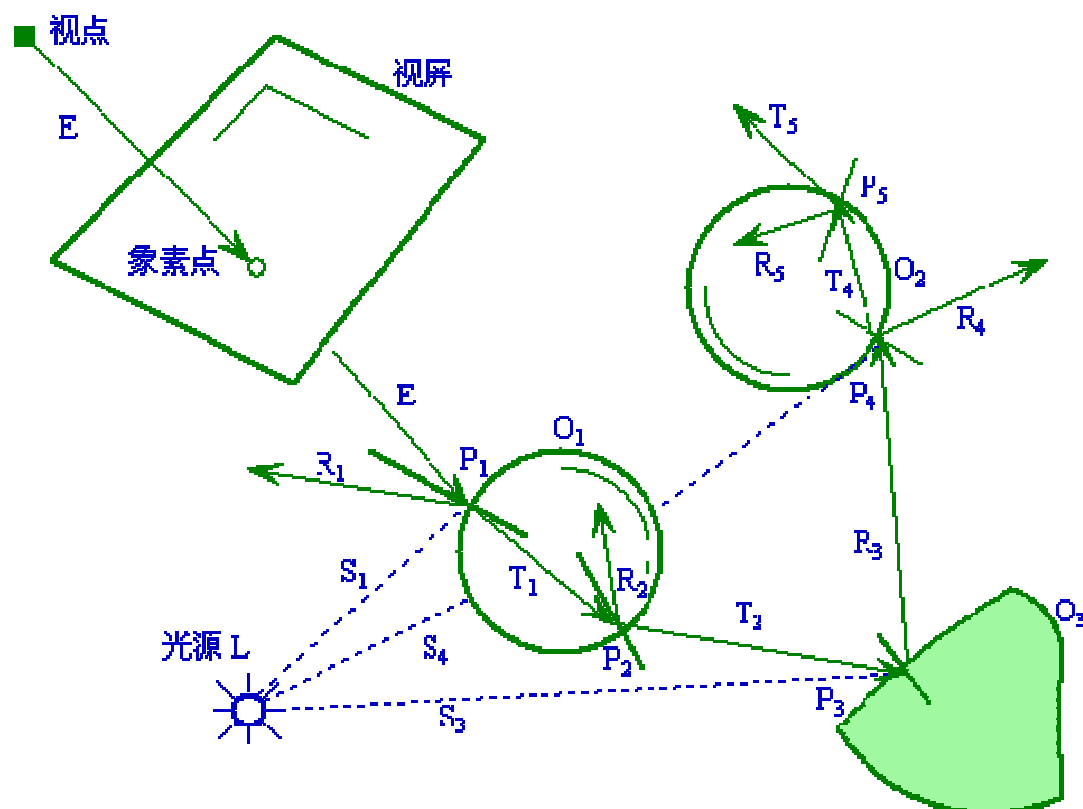
一、 为什么需要整体光照模型？

- n 简单和局部光照模型不能很好地模拟光的折射、反射和阴影等，也不能用来表示物体间的相互光照影响。
- n 整体光照模型是更精确的光照模型，主要有光线跟踪和辐射度两种方法。

二、光线跟踪基本原理 (Ray Tracing)

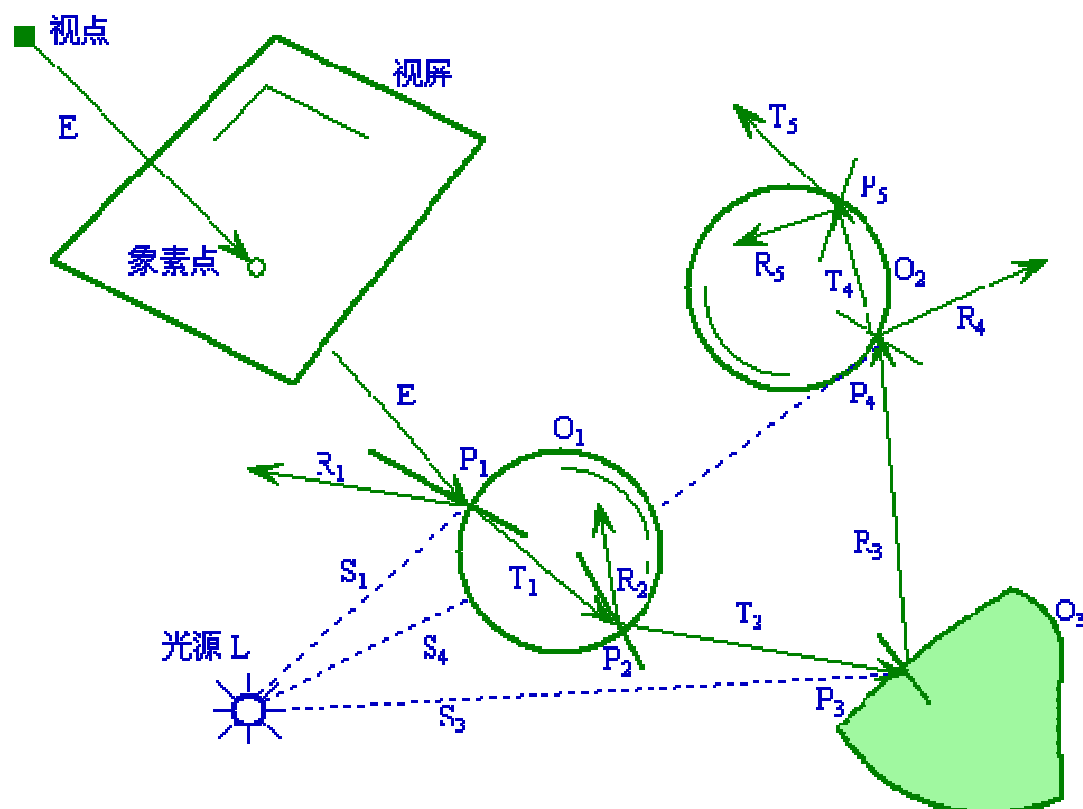
- n 光线跟踪算法是真实感图形学中的主要算法之一，该算法具有原理简单、实现方便和能够生成各种逼真的视觉效果等突出的优点，综合考虑了光的反射、折射、阴影等。

二、光线跟踪基本过程



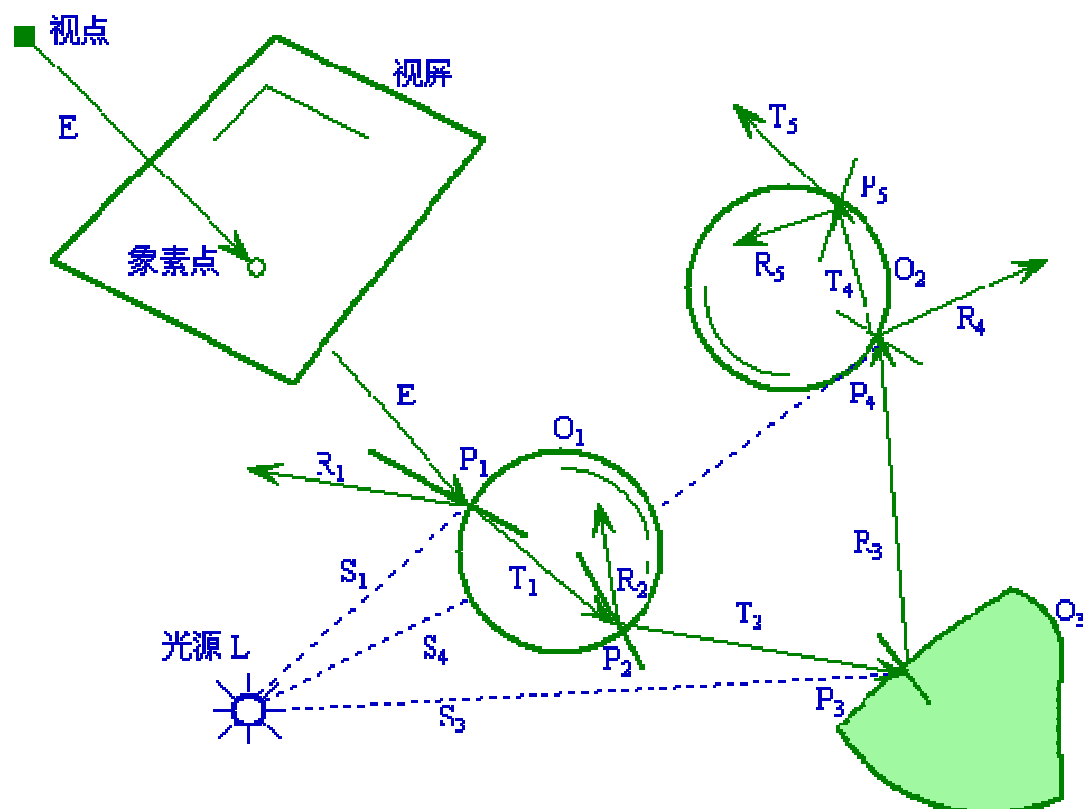
在这个场景中，
有一个点光源 L ，两
个透明体 O_1 与 O_2 ，
一个不透明体 O_3 。

二、光线跟踪基本过程



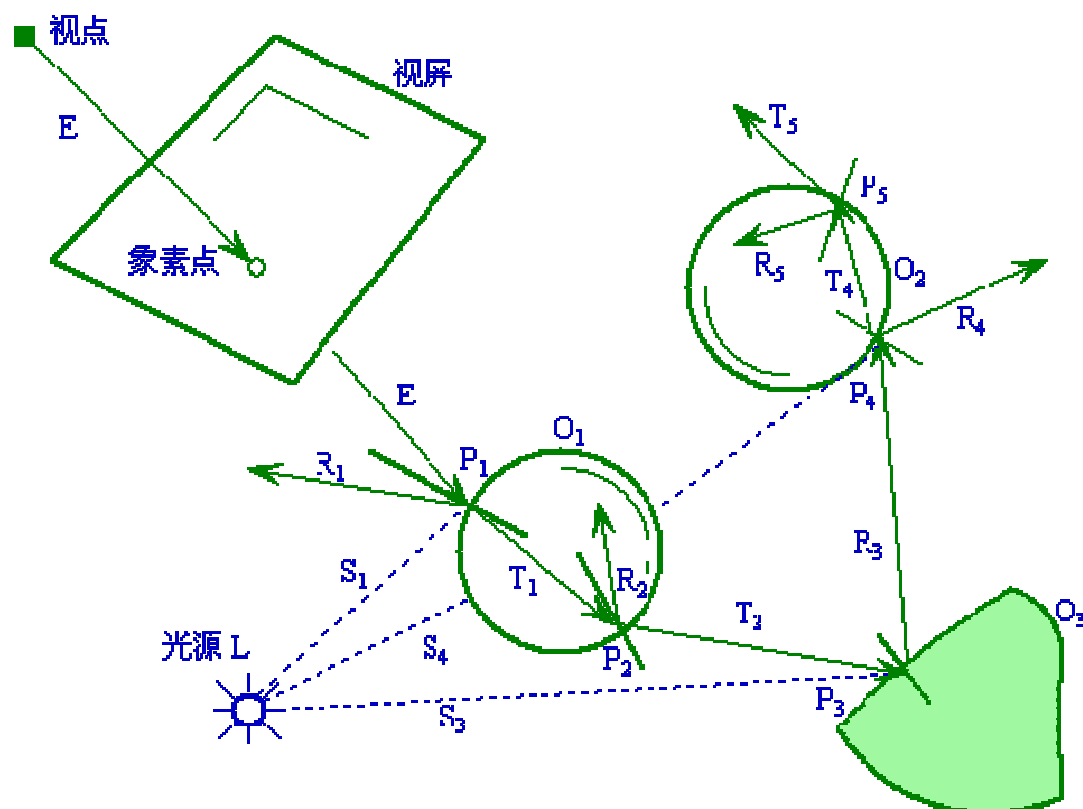
首先，从视点出发经过视屏一个像素点的视线E传播到达球体 O_1 ，交点为 P_1 。从 P_1 向光源L作一条阴影测试线 S_1 ，可以发现其间没有遮挡的物体，那么就用局部光照模型计算光源对 P_1 在其视线E方向上的光强，作为该点的局部光强；

二、光线跟踪基本过程



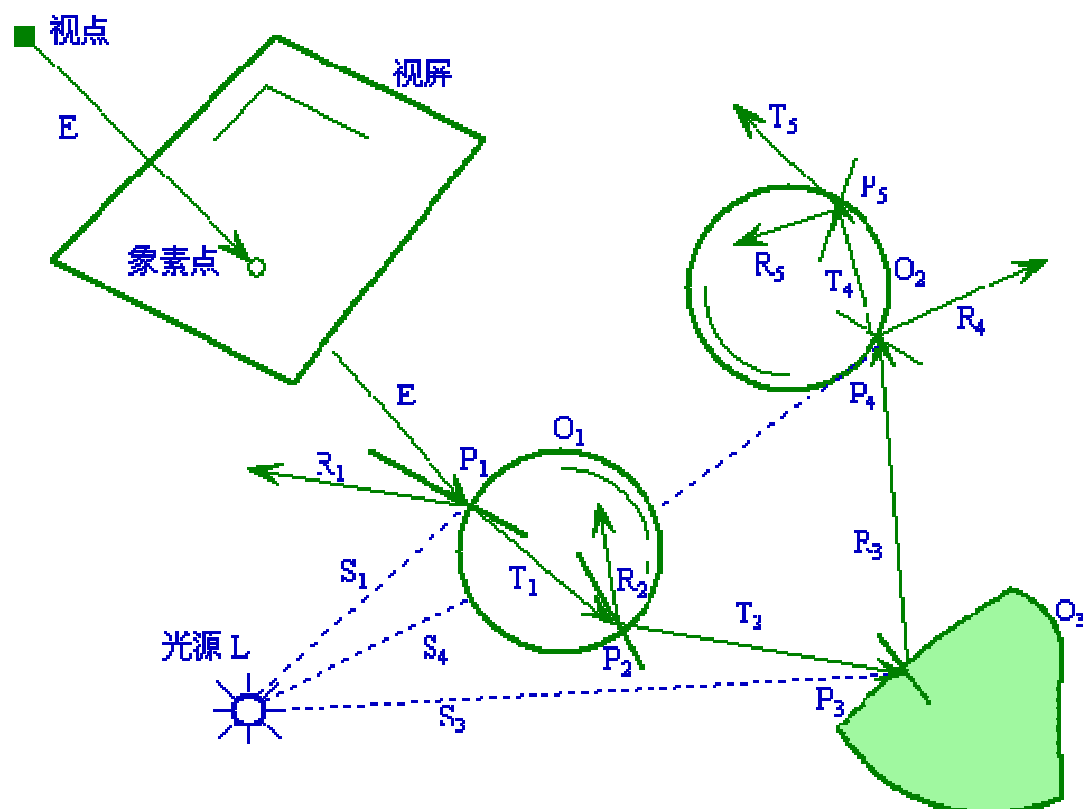
在反射光线 R_1 方向上，没有再与其他物体相交，那么就设该方向的光强为0，并结束这条光线方向的跟踪。然后对折射光线 T_1 方向进行跟踪，计算该光线的光强贡献。

二、光线跟踪基本过程



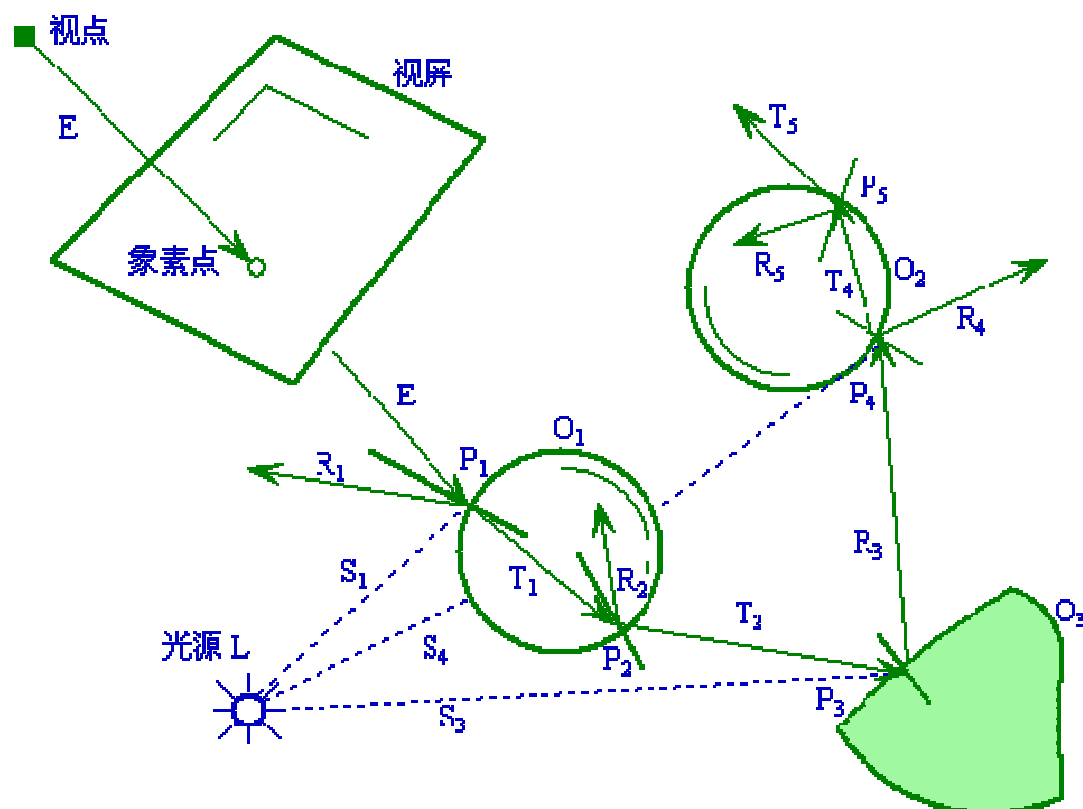
折射光线 T_1 在物体 O_1 内部传播，与 O_1 相交于点 p_2 ，由于该点在物体内部，假设它的局部光强为0。该点处同时产生了反射光线 R_2 和折射光线 T_2 ，在反射光线 R_2 方向，可以继续递归跟踪下去计算它的光强。而对折射光线 T_2 则继续进行跟踪。

二、光线跟踪基本过程



T_2 与物体 O_3 交于点 p_3 ,
作 P_3 与光源 L 的阴影测试线 S_3 , 没有物体遮挡, 正常计算该处的局部光强。由于该物体是非透明的, 可以只继续跟踪反射光线 R_3 方向的光强, 结合局部光强得到 P_3 处的光强。

二、光线跟踪基本过程



反射光线 R_3 的跟踪与前面的过程类似，算法可以递归地进行下去。重复上面的过程，直到光线满足跟踪终止条件。这样最终可以得到视屏上一个像素点的光强，也就是它相应的颜色值。

光线跟踪 (Ray Tracing)

一、光线跟踪怎么停止？

n 在算法应用的意义上，可以有以下几种终止条件。

- ü 该光线未碰到任何物体
- ü 该光线碰到了背景
- ü 光线在经过许多次反射和折射以后，就会产生衰减，光线对于视点的光强贡献很小
- ü 光线反射或折射次数即跟踪深度大于一定值

二、光线跟踪伪代码

光线跟踪算法的函数名为RayTracing（），光线的起点为start，方向为direction，光线的衰减权值为weight，初始值为1，算法最后返回光线方向上的颜色值color。

对于每一个像素点，第一次调用RayTracing（），可以设起点start为视点，而direction为视点到该像素点的射线方向。

```

RayTracing (start, direction, weight, color)
{
    if (weight < MinWeight)
        color = black
    else
    {
        计算光线与所有物体的交点中离start最近的点;
        if (没有交点)
            color = black
        else
        {
             $I_{local}$  = 在交点处用局部光照模型计算出的光强;
            计算反射方向R;
            RayTracing (最近的交点, R, weight* $W_r$ ,  $I_r$ )
            计算折射方向T;
            RayTracing (最近的交点, T, weight* $W_t$ ,  $I_t$ ) ;
            color =  $I_{local}$  +  $K_r I_r$  +  $K_t I_t$ ;
        }
    }
}

```

三、光线跟踪缺点

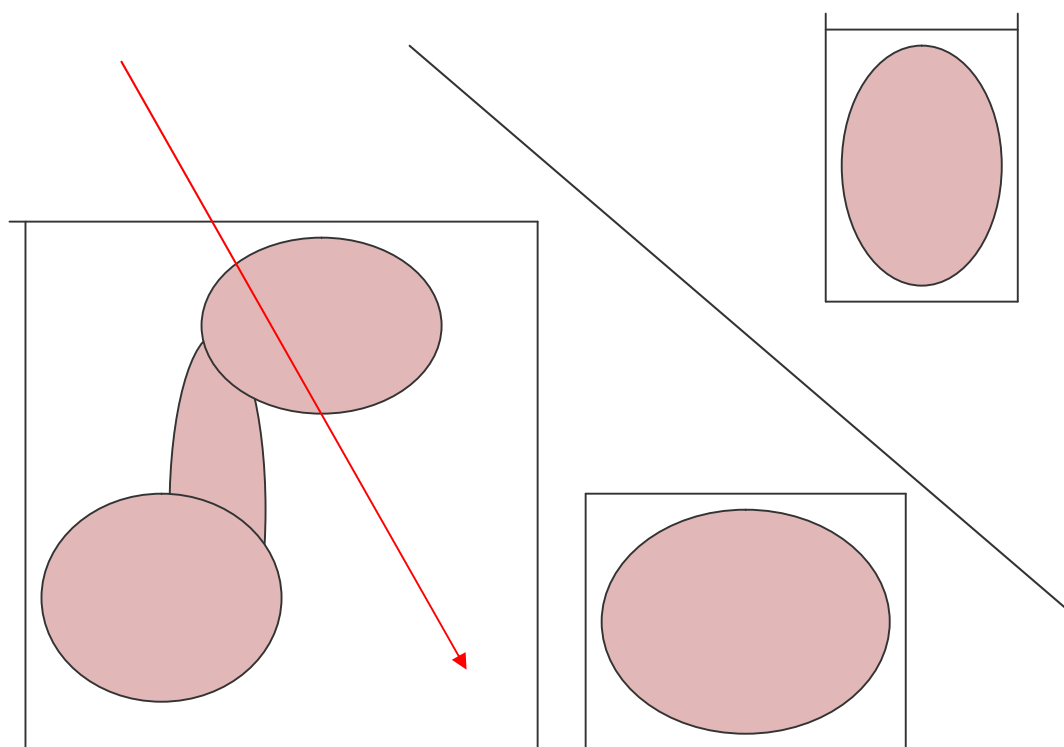
- n 光线跟踪方法由于要进行大量的求交运算，且每一条射线都要和所有的物体求交，因此效率很低，需要耗费大量的计算时间。
- n 光线跟踪方法可以进行加速。

四、光线跟踪加速

- ü 提高求交速度：针对性的几何算法、...
- ü 减少求交次数：包围盒、空间索引、...
- ü 减少光线条数：颜色插值、自适应控制、...
- ü 采用广义光线和采用并行算法等

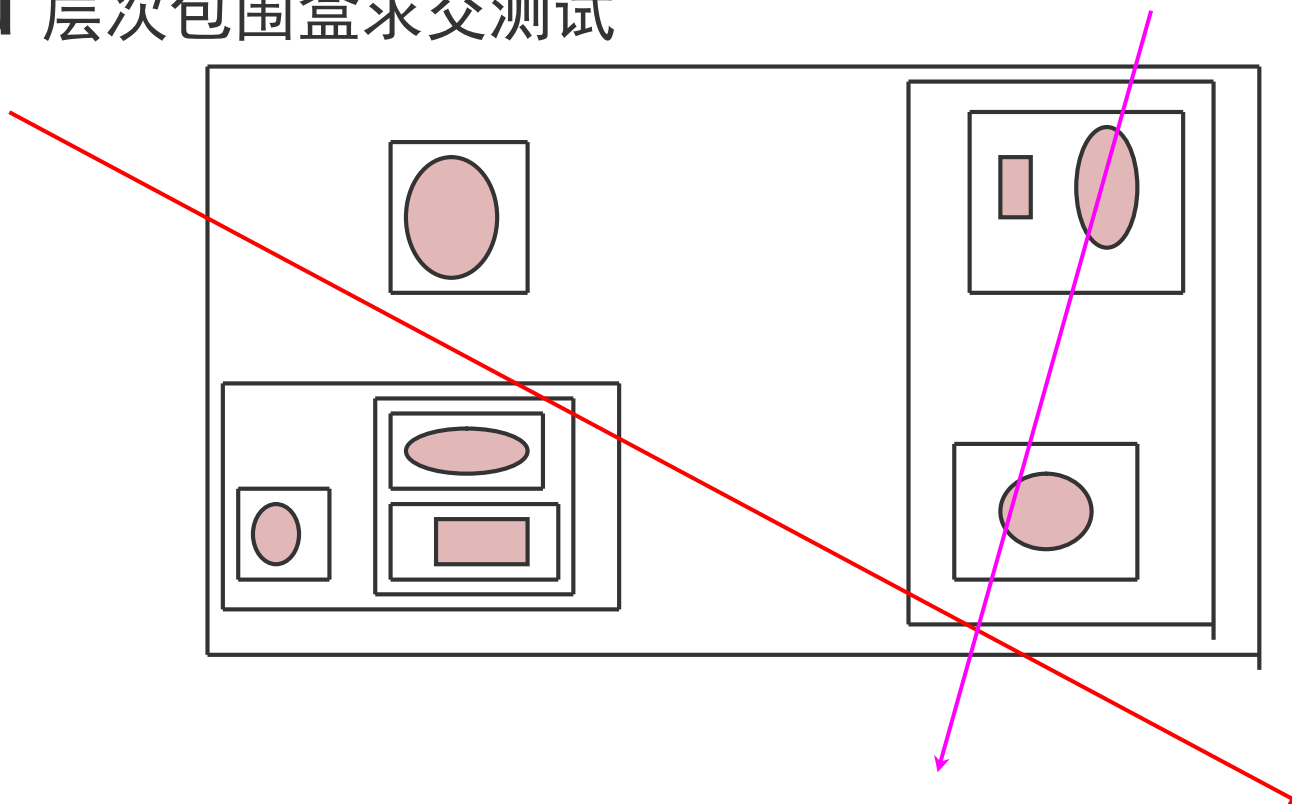
四、光线跟踪加速

ü 包围盒求交测试



四、光线跟踪加速

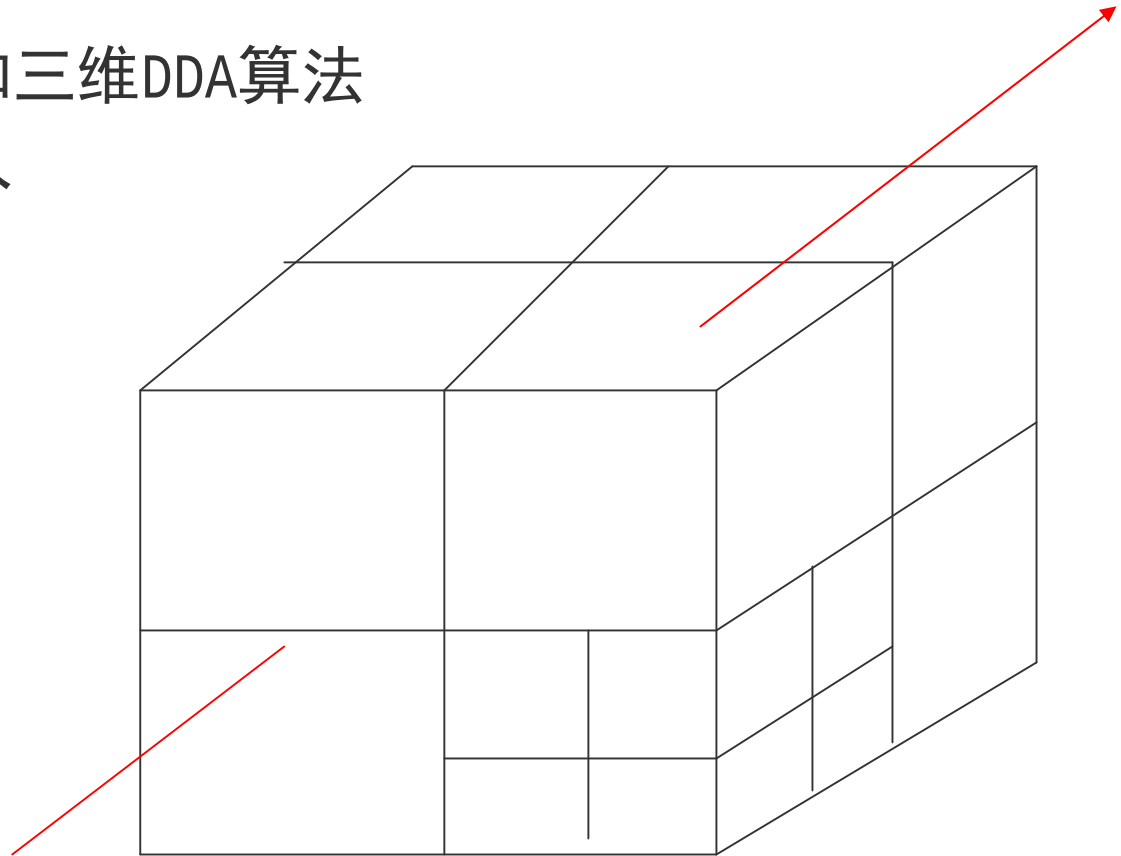
ü 层次包围盒求交测试



四、光线跟踪加速

ü 空间网格剖分和三维DDA算法

ü 空间八叉树剖分



五、光线跟踪场景渲染

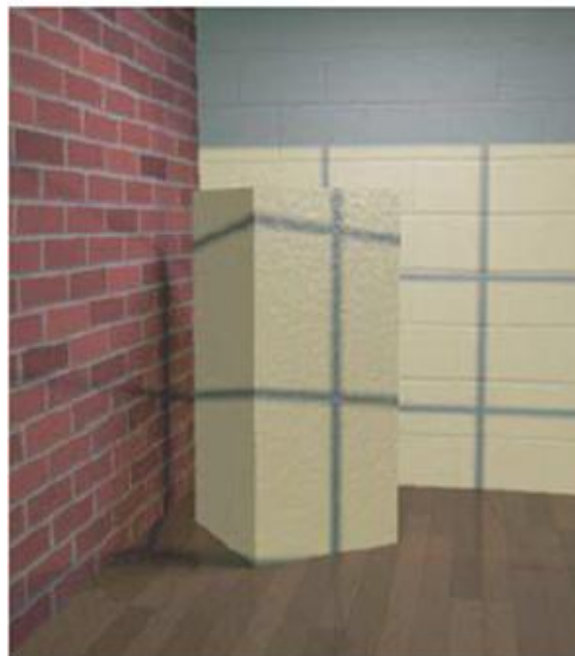


纹理映射

一、 什么是纹理和纹理映射？



没有纹理



有纹理

二、 纹理有什么用？

- n** 表面可以用纹理来代替，不用痛苦地构造模型和材质细节，节省时间和资源，让用户做其他更重要的东西。
- n** 可以用一个粗糙的多边形和纹理来代替详细的几何构造模型，节省时间和资源。

二、 纹理作用



+



=



三、 纹理分类

颜色纹理：颜色或明暗度变化体现出来的表面细节，如刨光木材表面上的木纹。



几何纹理：由不规则的细小凹凸体现出来的表面细节，如桔子皮表面的皱纹。

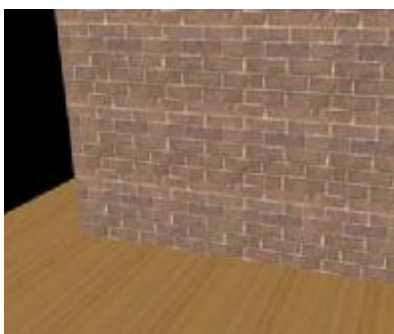


四、图形学中纹理定义

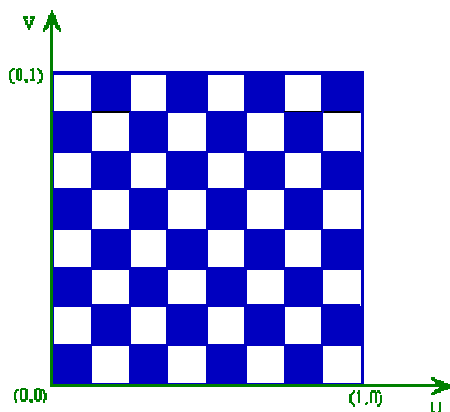
在真实感图形学中，可以用下列两种方法来定义纹理：

(1) 图象纹理：将二维纹理图案映射到三维物体表面，绘制物体表面上一点时，采用相应的纹理图案中相应点的颜色值。

(2) 函数纹理：用数学函数定义简单的二维纹理图案，如方格地毯；或用数学函数定义随机高度场，生成表面粗糙纹理即几何纹理。



图像纹理



函数纹理

$$g(u,v) = \begin{cases} 0 & [u \times 8] + [v \times 8] \text{ 为奇数} \\ 1 & [u \times 8] + [v \times 8] \text{ 为偶数} \end{cases}$$

其中： $\lfloor x \rfloor$ 表示小于x的最大整数

五、 纹理映射

纹理映射 (Texture Mapping)：通过将数字化的纹理图像覆盖或投射到物体表面，而为物体表面增加表面细节的过程。

1974年Catmull首次提出了纹理映射的概念，其主要思想是通过寻找一种从纹理空间 (u, v) 到三维曲面 (s, t) 之间的映射关系，将点 (u, v) 对应的彩色参数值映射到相应的三维曲面 (s, t) 上，使三维曲面表面得到彩色图案。

五、 纹理映射

颜色纹理坐标转换通常使用下列两种方法：

(1) 在绘制一个三角形时，为每个顶点指定纹理坐标，三角形内部点的纹理坐标由纹理三角形的对应点确定。

即指定：

$$(x_0, y_0, z_0) \rightarrow (u_0, v_0)$$

$$(x_1, y_1, z_1) \rightarrow (u_1, v_1)$$

$$(x_2, y_2, z_2) \rightarrow (u_2, v_2)$$

(2) 指定映射关系：

$$u = a_0x + a_1y + a_2z + a_3$$

$$v = b_0x + b_1y + b_2z + b_3$$

五、 纹理映射

几何纹理使用一个称为扰动函数的数学函数进行定义。

扰动函数通过对景物表面各采样点的位置作微小扰动来改变表面的微观几何形状。

设景物表面由下述参数方程定义： $Q = Q(u, v)$

则表面任一点 (u, v) 处的法线为： $N = N(u, v) = \frac{Q_u(u, v) \times Q_v(u, v)}{|Q_u(u, v) \times Q_v(u, v)|}$

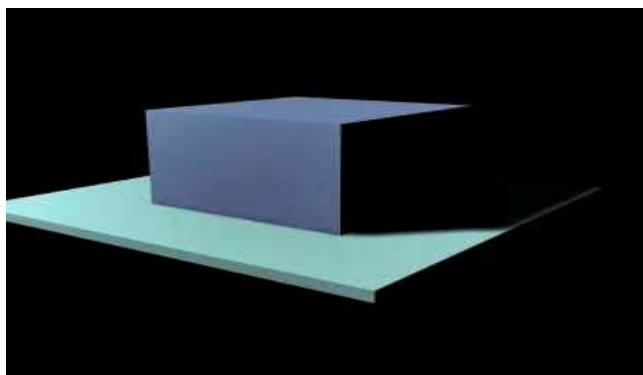
设扰动函数为： $P(u, v)$

扰动后的表面为： $Q' = Q(u, v) + P(u, v)N$

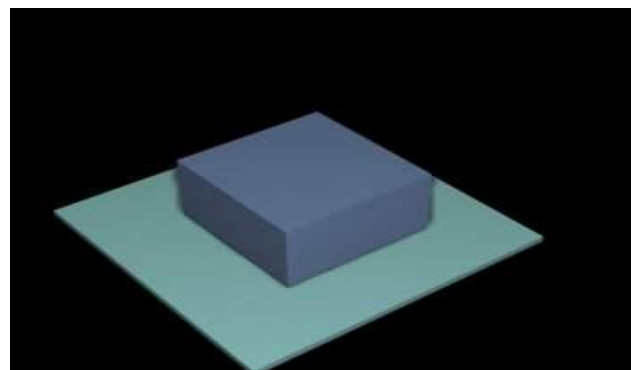
阴影处理

一、 什么是阴影？

- n 阴影是由于观察方向与光源方向不重合而造成的；
- n 阴影使人感到画面上景物的远近深浅，从而极大地增强画面的真实感。

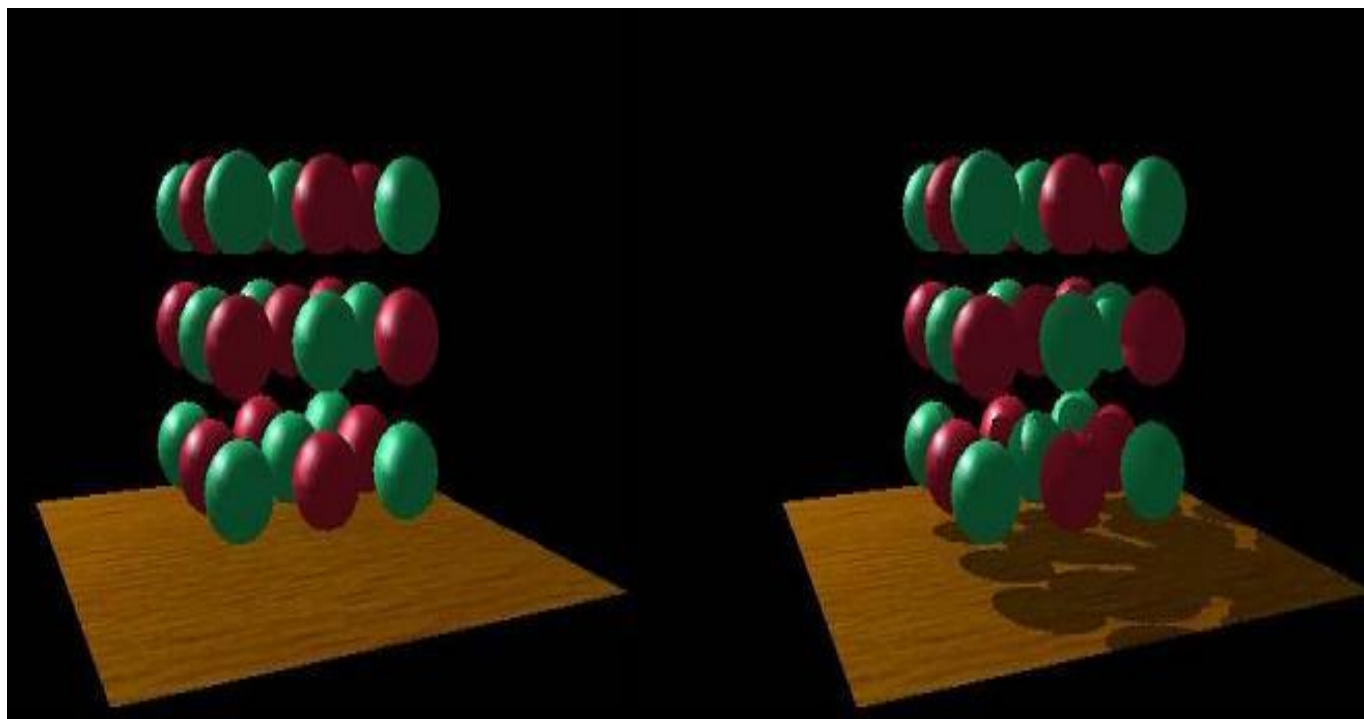


观察方向与光源方向不重合

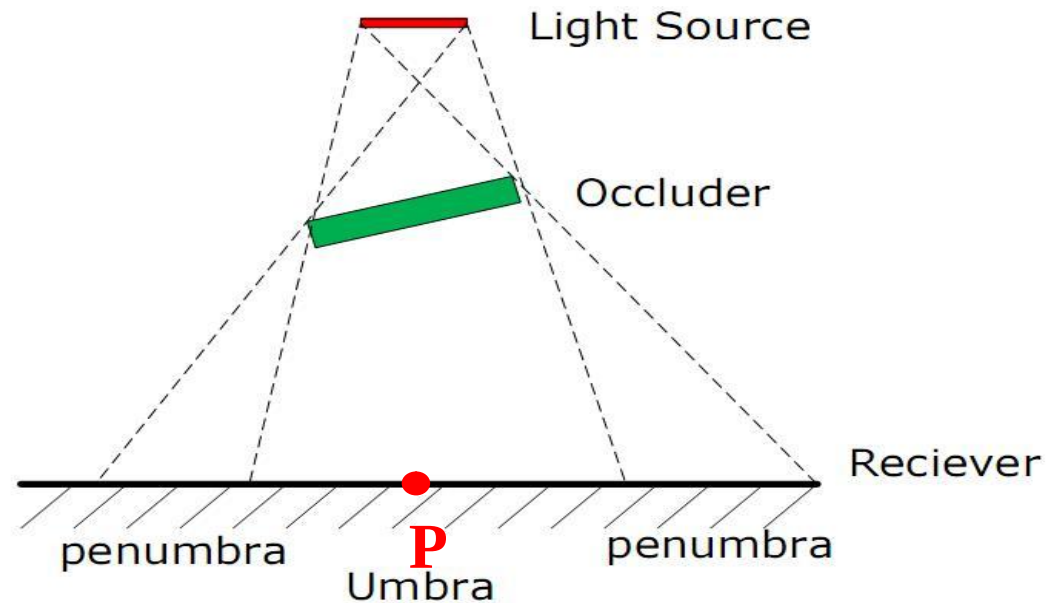


观察方向与光源方向重合

一、 阴影有什么用？

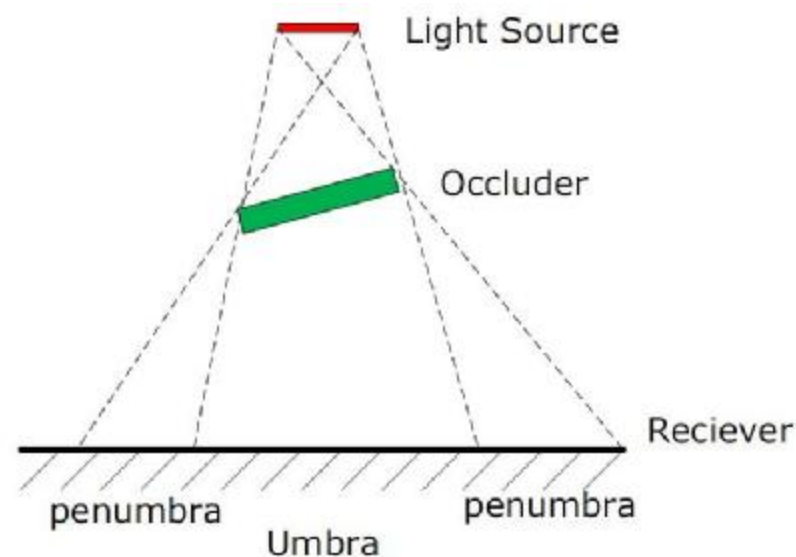


二、 什么是本影



umbra-本影区-场景中的一个点P，如果它不被光源的任何一部分所照射到，就称为在本影区里。本影就是不被任何光源所照到的区域。

二、什么是半影



Occluder-遮挡物：阴影的生成是因为空间有遮挡物。这些遮挡物把光源挡住了，所以在receivers上有些部分就很阴暗。

阴影是本影和半影的组合。求出本影和半影的并集（union）来绘出阴影。

二、 阴影

- n 自身阴影：** 由于物体自身的遮挡而使光线照射不到它上面的某些面；
- n 投射阴影：** 由于物体遮挡光线，使场景中位于它后面的物体或区域受不到光照射而形成的。

三、 阴影算法（1）

阴影体法（ **Shadow Volume** ）

Frank Crow1977年提出来的，可以在任意的物体上生成阴影。

Crow, Franklin C: "**Shadow Algorithms for Computer Graphics**",
Computer Graphics (SIGGRAPH '77
Proceedings), vol. 11, no. 2, 242-
248.

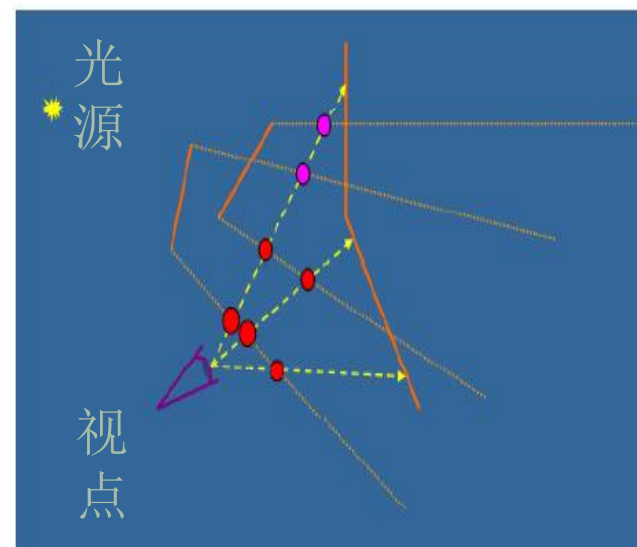


三、 阴影算法（1）

阴影体法（ **Shadow Volume** ）

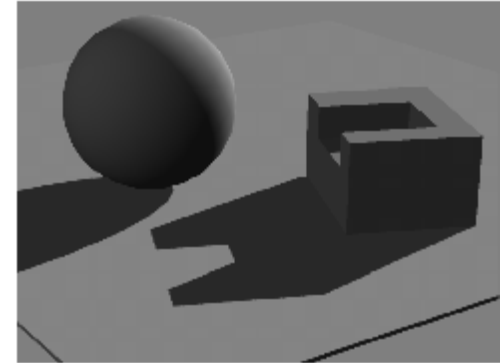
由一个点光源和一个三角形可以生成一个无限大的阴影体。落在这个阴影体中的物体，就处于阴影中。

在对光线进行跟踪的过程中，如果这条射线穿过了阴影体的一个正面（朝向视点的一个面），则计数器加1。如果这条射线穿过了阴影体的一个背面（背向视点的一个面），则计数器减1。如果最终计数器的数值大于0，则说明这个像素处于阴影中，否则处于阴影之外。



三、 阴影算法（2）

阴影图法（ **Shadow Mapping** ）

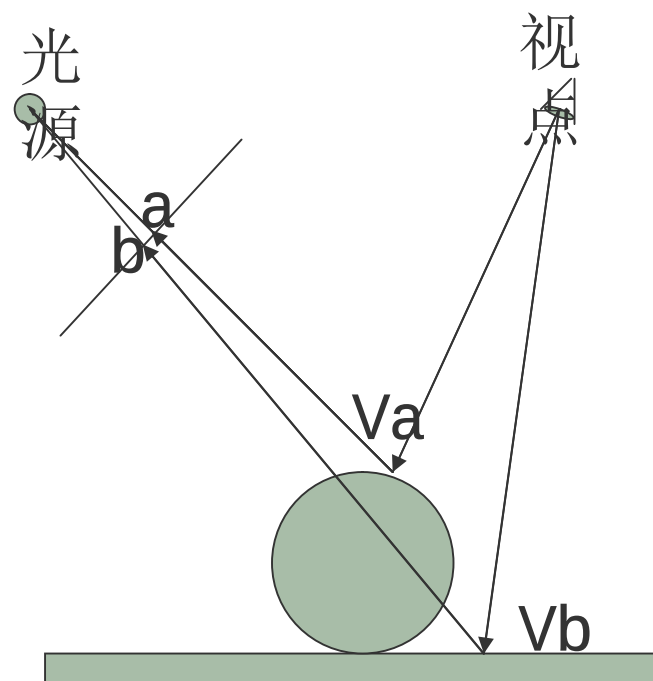


- n 这种方法的主要思想是使用Z缓冲器算法，从投射阴影的光源位置对整个场景进行绘制。
- n 这时，对于Z缓冲器的每一个像素，它的z深度值包括了
这个像素到距离光源最近点的物体的距离。一般将Z缓冲器中的整个内容称为阴影图（Shadow Map），有时候也称为阴影深度图。

三、 阴影算法（2）

阴影图法（ **Shadow Map** ）

- n** 为了使用阴影图，需要对场景进行二次绘制，不过这次是从视点的角度来进行的。
- n** 在对每个图元进行绘制的时候，将它们的位置与阴影图进行比较，如果绘制点距离光源比阴影图中的数值还要远，那么这个点就在阴影中，否则就不在阴影中。



基于图像的绘制技术

IBR (Image-Based Rendering)

传统图形绘制与基于图像绘制（IBR）比较

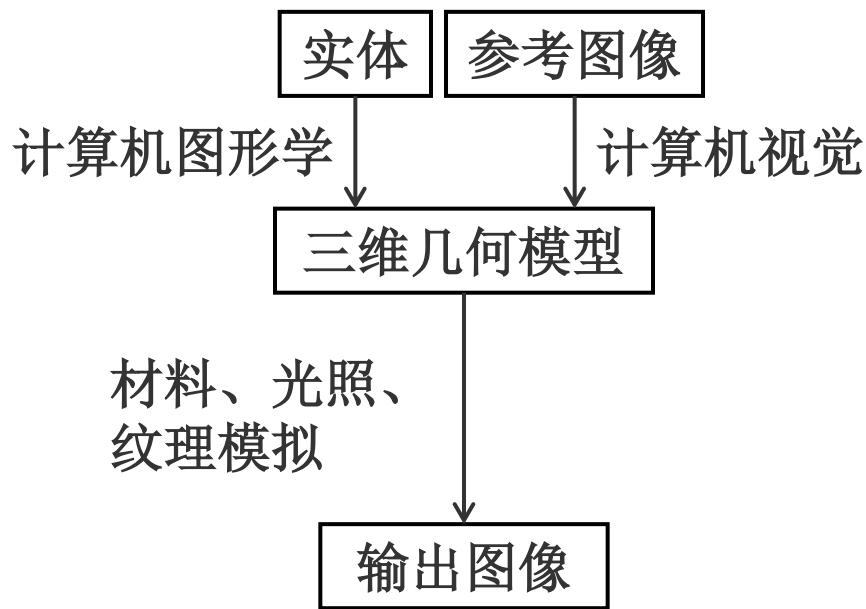


图1 传统的图像绘制过程

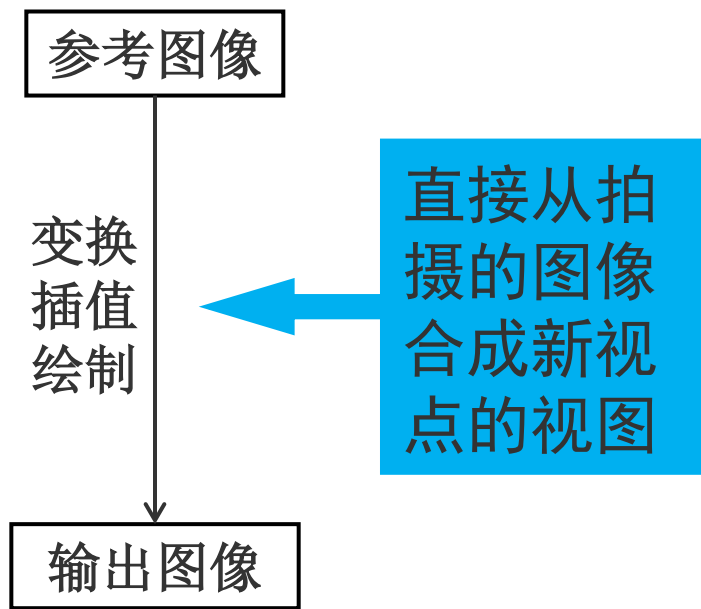


图2 IBR绘制过程

传统图形绘制与基于图像绘制（IBR）比较

传统图形绘制特点：

- 建模复杂、困难。
- 计算和显示开销大，绘制速度慢。
- 难以达到真实感效果。

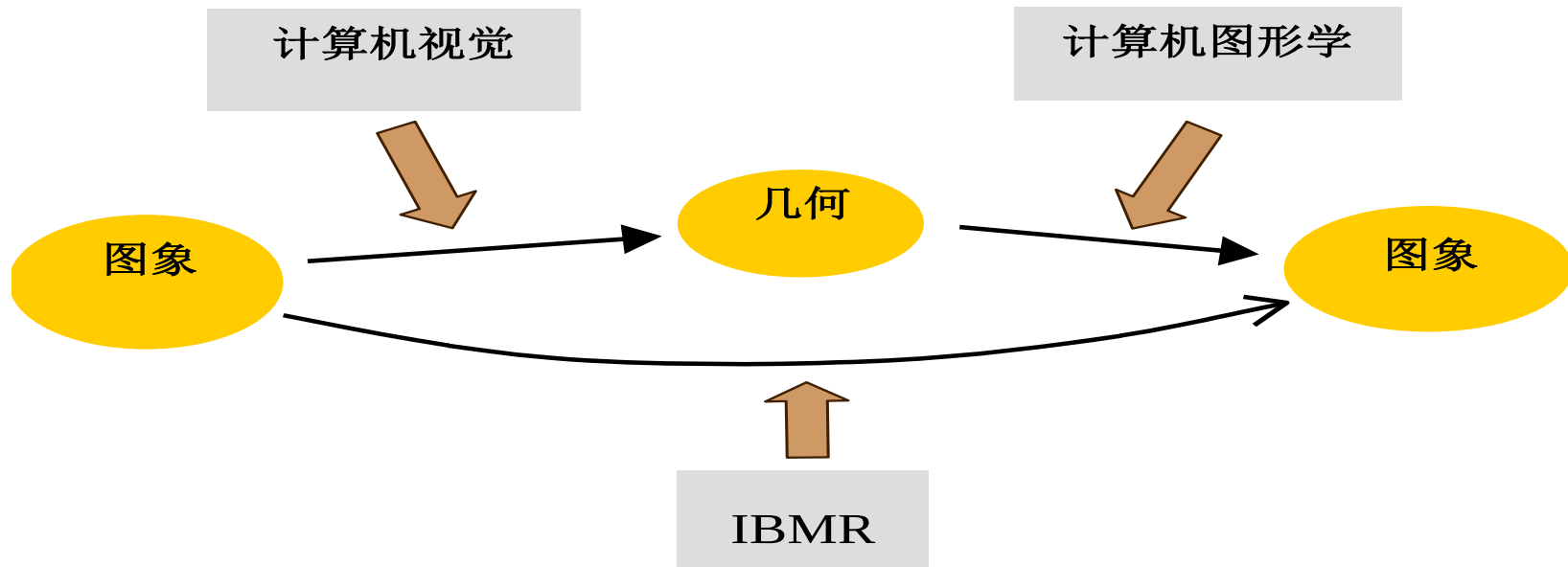
IBR特点：

- 建模简单
- 显示速度快
- 真实感强

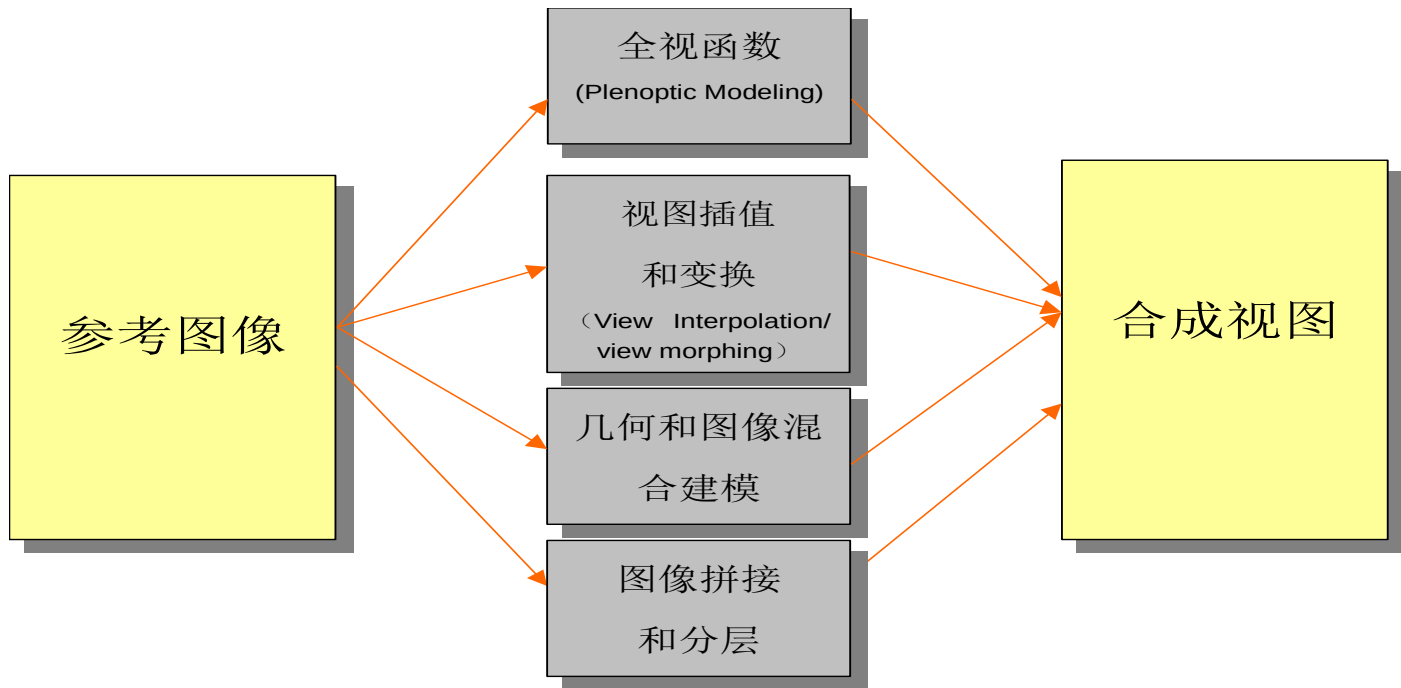
相关领域

- 基于图像的光照 (Image Based Lighting)
 - 从图像中获取场景物体的光照属性
- 计算机视觉
 - 从图像中获取场景物体的几何信息

基于图像的绘制与计算机图形学和视觉的关系



基于图像的绘制过程



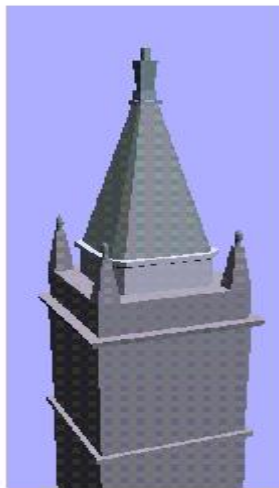
基于图像的绘制中的重要方法

- 几何和图像混合建模 (Hybrid Geometry- and Image-based Approach [DTM96])
- 视图插值和变形 (View Interpolation [CW93] / View Morphing [SD96])
- 全视函数 (Plenoptic Function-based)
- 图象拼接 (mosaic)
- 立体视觉 (Stereo Vision)

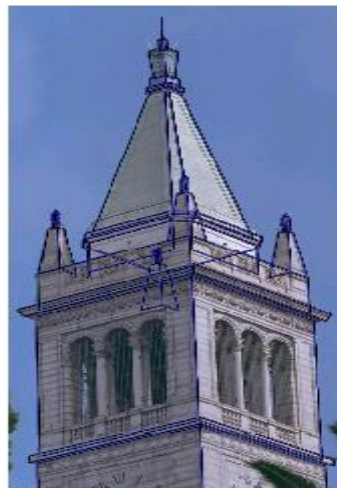
几何和图像混合建模过程（图示）



(a)



(b)



(c)



(d)

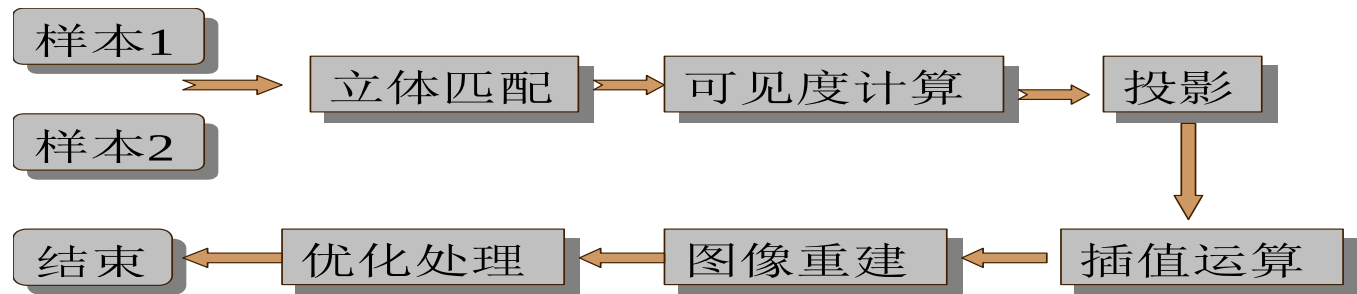
几何和图像混合建模过程

- a. 拍摄照片，交互指定建筑物边缘
- b. 生成建筑物粗模型
- c. 利用基于模型的立体视觉算法精化模型
- d. 利用基于视点的纹理映射合成新视图

几何和图像混合建模特点

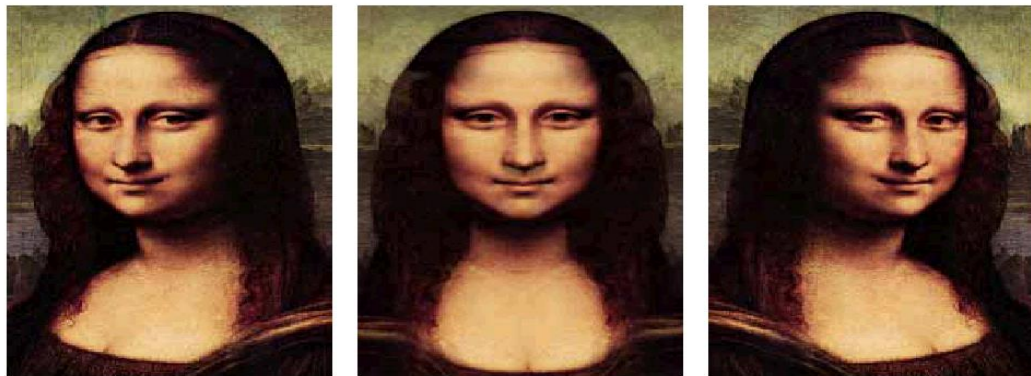
- 可以通过拍摄的几张照片合成逼真的新视图，
简单快捷
- 只能适用于普通建筑物等外形规整的景物

视图插值方法 (View Interpolation)



- 要求新视点位于两参考图象视点所决定的直线(基线, baseline)上。由参考图线性插值产生新视图。
- 一般情况下不能产生正确的透视投影结果, 而只生成近似的中间视图。

视图变换方法 (View Morphing)



$$\text{Warp}_{01}(w, P) = (1 - w)P_0 + wF_0P_0$$

$$\text{Warp}_{10}(w, P) = (1 - w)F_1P_1 + wP_1$$

- 利用参考图像上像素点重投影生成新视图
- 利用投影知识决定的变形位置

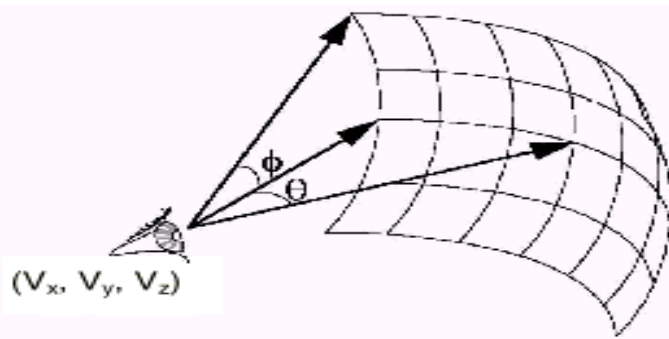
视图插值/变换方法特点

- 简单方便，只要求几幅参考图像
- 漫游范围受限，只能在几幅参考图像的视点连线之间作有限运动
- 常用于加速图形学中的绘制速度

基于全视函数的IBR

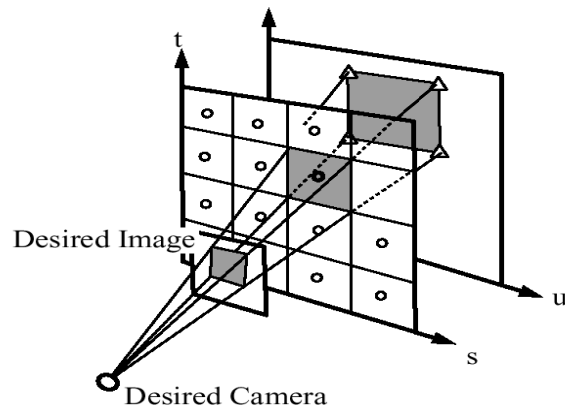
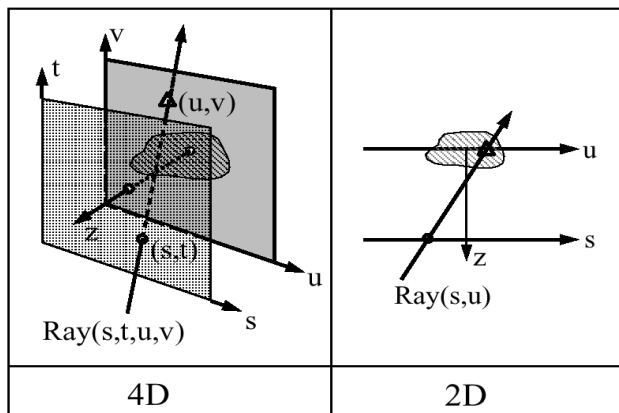
- 全视函数原型[AB91] Adelson&Bergen

$$\mu = \text{Plenoptic}(\theta, \phi, \lambda, V_x, V_y, V_z, t)$$



基于光场的显示

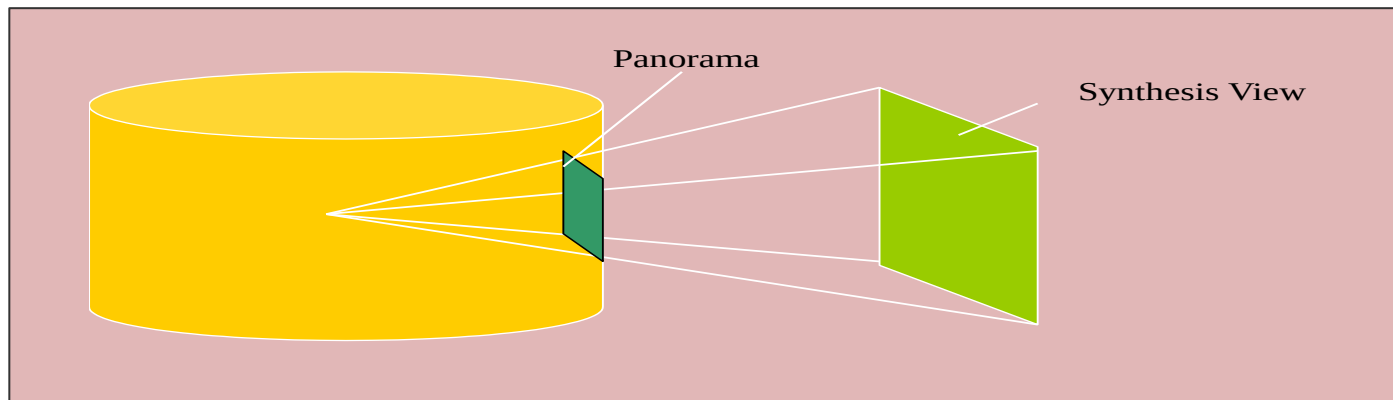
■ Light Field Rendering[LH96] and Lumigraph [GGSC96]



基于光场的显示：特点

- 基于“自由空间中沿一条光线传递的辐射能不变”的假设，把全视函数简化为描述离开或进入一封闭自由空间(如空立方体)的完全光流分布
- 由于只考虑视流信息，因此不必对反射属性作假设，不需要立体对应关系
- 数据量大，采样困难

全景图 (Panorama)

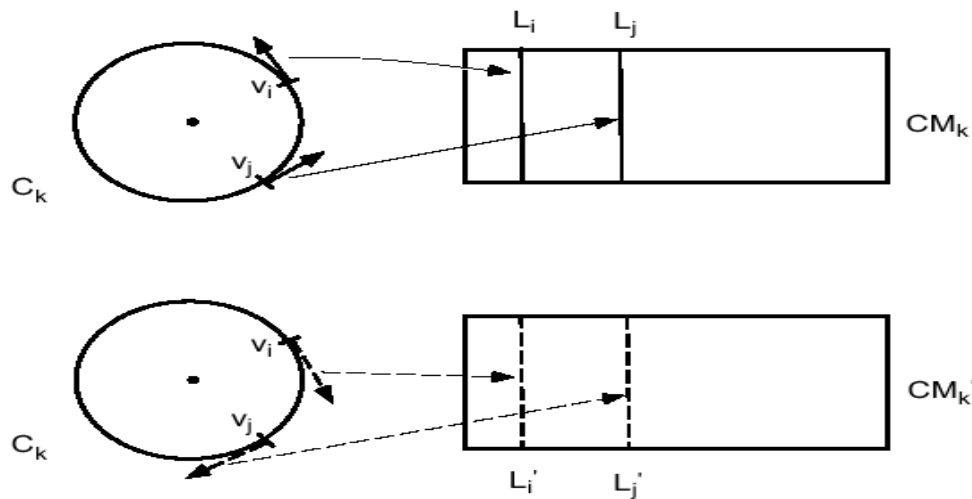


- 在一个视点拍摄的几幅图像，通过整合拼接成一个视点周围的场景视图，投影在圆柱面或者球面上成为全景图。

全景图方法特点

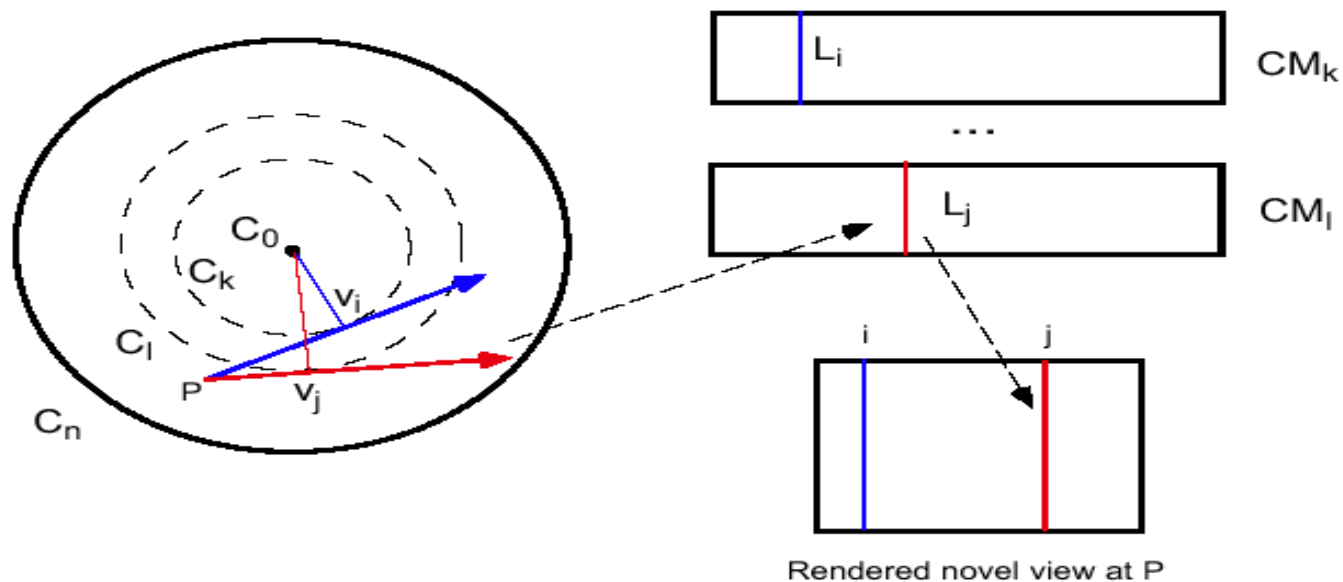
- 只需在一个视点拍摄的几张照片，数据量小，采样方便。
- 视点不能移动，但是可以转动观察方向，通过放大缩小（zoom in/zoom out）进行近似的前后运动

同心拼接 (Concentric Mosaics)

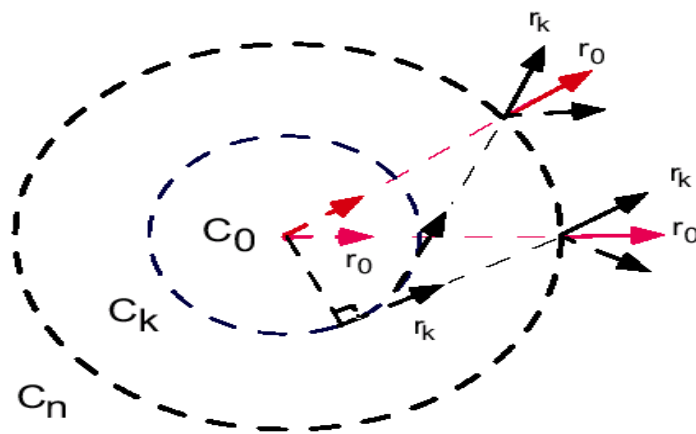


- 通过沿一系列同心圆切线方向拍摄的狭缝图像拼合成同心拼接

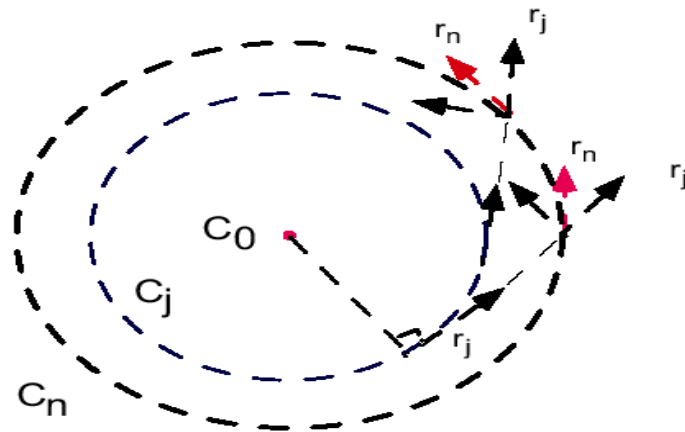
利用同心拼接合成视图原理



使用普通照相机获取狭缝图像



(a)

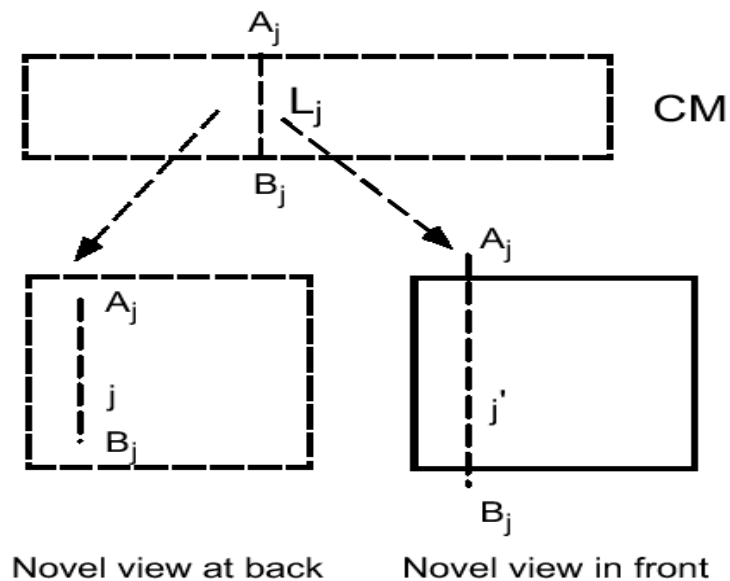
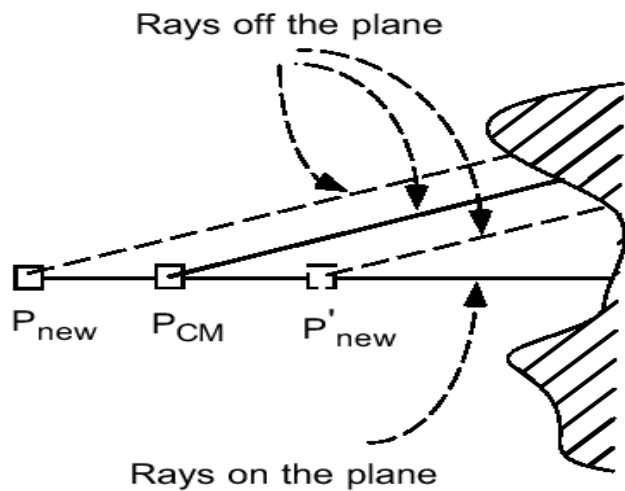


(b)

同心拼接方法特点

- 只需在一个圆上拍摄的一系列图像，采样方便。
- 视点可以在采样圆内做平面移动，生成场景真实感强
- 数据量较光场等全视函数方法为小
- 移动范围受限，基于狭缝图象的绘制：有场景畸变现象

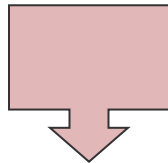
同心拼接的畸变



基于狭缝图象段的绘制

- 将同心拼接中使用的狭缝图象进一步分为狭缝图象段
- 测定不同距离上对应狭缝图象段的伸缩比例
- 利用狭缝图象段的伸缩进行正确的绘制

基于狭缝图象段的绘制：绘制



基于全视函数的方法比较

数据维数	视点移动范围	名称	提出年份
7	自由	plenoptic function	1991
5	自由	plenoptic modeling	1995
4	一个长方体内	Lightfield/Lumigraph	1996
(3)	一个二维圆内	concentric mosaics	1999
2	固定视点上	panorama	1994

基于图像的绘制发展方向

- 与传统集合绘制方式有效结合
- 算法固化提高图像生成速度，达到实时绘制
- 分层绘制
- 提高Morphing中特征提取的效率和质量
- 图像拼接中的快速配准

基于点的建模与绘制

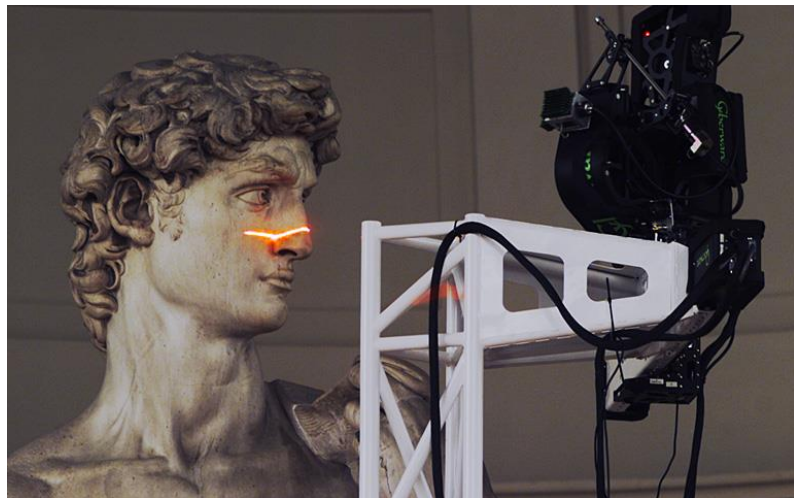
(Point based Rendering)

概述

- 随着模型多边形复杂度的剧增，点模型的优势越发明显。
- 点模型的数字几何处理流程可以概括为点的获取、点的处理和建模以及点模型的绘制3个阶段。

一、点模型获取与预处理技术

- 3D扫描仪（深度照相机生成的深度图和激光三维扫描仪等设备得到的大量空间三维点位置）。
- 点模型的另一个来源是现有模型，大部分几何模型如多边形网格模型、隐式曲面等都能方便地转化为点模型。



Digitizing David

- 点模型要预处理：主要包括配准、去噪、修补等。

1.1、配准

两大类：

- ✓ 机器配准
- ✓ 自动配准

常用方法：

- ✓ 标签法
- ✓ 特征提取法

1.2、去噪

- 移动最小二乘 (MLS) 曲面逼近原始点集模型
- 基于三维Meanshift过程的各向异性点模型去噪算法
- MLS曲面重建方法
- 点模型的非局部去噪方法
- 基于采样保真性的点模型去噪算法

1.3、修补



Original model

Marked are

Smooth Filling Context-based Filling

基于上下文的点模型修复

二、点模型建模技术

- 目标：从原始点云中构造出一个连续的表面模型。
- 点模型建模所涉及的技术较多：
 - 曲面重建
 - 曲面简化
 - 几何属性分析
 - 特征提取
 - 重采样

2. 1、曲面重建技术

- 曲面重建是指根据离散扫描点数据重建三维模型的过程。

常用方法：

- ✓ RBF (Radial Basis Function) 曲面重建方法
- ✓ MLS (Moving Least-Square) 曲面

2. 2、曲面简化

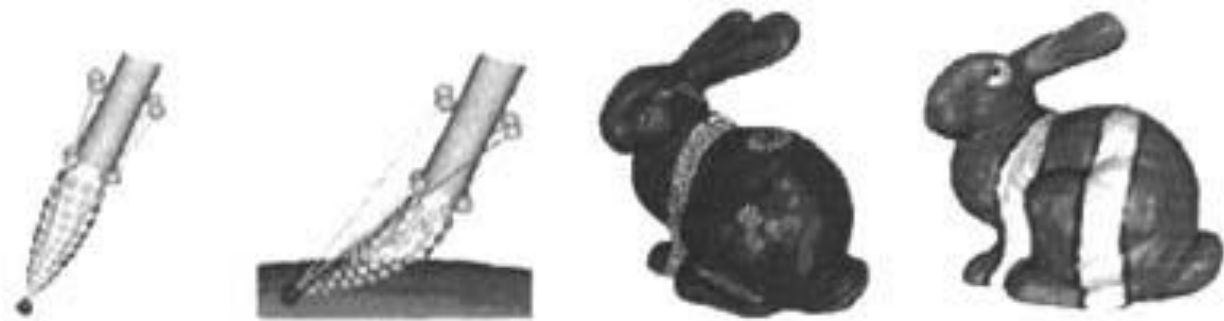
- 利用三维扫描仪获得的点模型通常具有很高的复杂度，为了使大规模数据模型符合几何处理和绘制，必须对数据模型进行简化。

三、模型编辑与几何造型技术

- 点模型的后期处理则是对点模型作进一步的造型处理
 - 编辑
 - 变形
 - 布尔运算

3. 1、点模型编辑技术

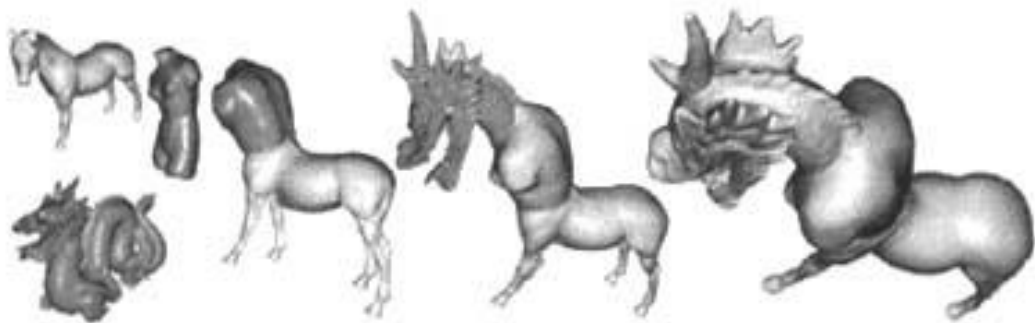
- 点模型的编辑是指对点模型的颜色、纹理等外观属性以及法向量的处理。例如下图是点模型的画刷着色效果图。



基于点模型的画刷及看色效果

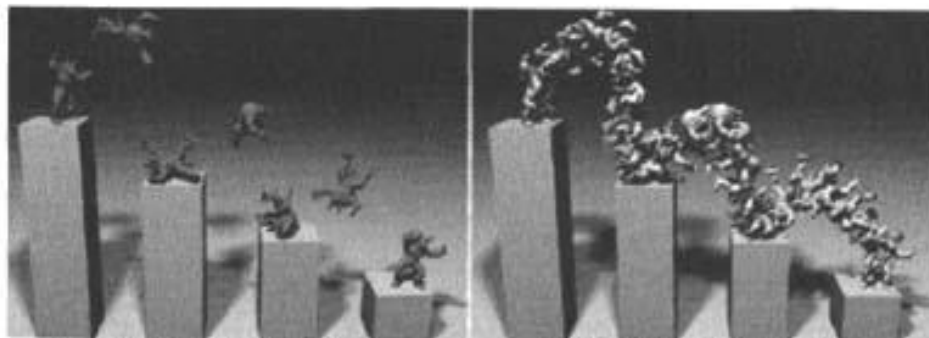
3. 2、几何造型技术

- 实体几何造型 (CSG: Constructive Solid Geometry): 是一种基于简单实体的布尔运算构造复杂模型的技术, 即通过对多个简单的点模型进行布尔运算后生成复杂的点模型。



利用布尔运算构造几何实体

- 自由变形：这种变形技术首先生成一个置换函数 $d: R^3 \rightarrow R^3$ ，然后对表面上的每个点 P_i ，实施变换 $P_i \rightarrow d(P_i)$ 。
- 点模型动画(MFM: Mesh Free Method): 是力学分析的新方法，近年来研究者将无网格方法和点模型相结合，提出了基于点的动画。例如：



(a)模型关键帧姿态

(b)算法插值获取中间动作

模型形变

四、模型渐变 (Morphing) 技术

点模型自身的特点决定了在模型渐变中，它比基于网格模型的渐变有明显的优势，但由于点模型没有提供表面的解析表达式和参数化信息，从另一方面又增加了它渐变的难度。

分为基于物理的渐变和基于几何的渐变。



点模型渐变

4. 1、基于物理的渐变

基于物理的渐变需要建立相关物理模型，中间过渡点模型根据能量方程求解，这种渐变在一定程度上模拟了真实的物理现象。

4. 2、基于几何的渐变

基于几何的渐变是通过对源和目标模型进行几何变换来获得中间过渡模型，其计算量没有基于物理的渐变大。

五、点模型绘制技术

Qsplat技术：由斯坦福大学开发的具有代表性的点绘制技术。它利用树状层次包围球数据结构，树中每个结点包含球的位置和半径、每点处的法向量、法锥面的宽度、颜色值。在进行绘制时，层次树按深度优先方法递归遍历。对每个中间结点，首先判断该球是否完全在屏幕外或者是完全背向的，以进行可见性选择。如果该结点至少有一部分子结点是可见的，则将该结点在屏幕上的投影大小同个阈值进行比较。如果大于阈值，则继续向下递归；如果小于阈值或者已经到达叶结点，则按该结点的球位置及半径确定的屏幕上的位置和大小绘制一个小区域。

六、发展趋势

- 基于点的自然场景建模。
- 基于点模型的动画。
- 点模型的简化、压缩和传输。
- 基于GPU的大规模点模型绘制。